

# Vitis 统一软件平台文档

## 嵌入式软件开发

UG1400 (v2026.1) 2026 年 7 月 31 日

本文档为英语文档的翻译版本，若译文与英语原文存在歧义、差异、不一致或冲突，概以英语文档为准。译文可能并未反映最新英语版本的内容，故仅供参考，请参阅最新版本的英语文档获取最新信息。





# 目录

- 第一部分：Vitis 入门.....6
  - 第 1 章：按设计进程浏览内容.....7
  - 第 2 章：Vitis 软件平台安装.....8
    - 安装 Vitis 软件平台..... 8
  - 第 3 章：Vitis 软件平台入门..... 10
    - Vitis 统一软件平台概述..... 10
    - 从传统 Vitis IDE 移植到 Vitis Unified IDE..... 14
    - Unified IDE 功能特性对比传统 IDE 功能特性..... 16
- 第二部分：使用 Vitis Unified IDE..... 18
  - 第 4 章：启动 Vitis Unified IDE..... 19
    - Vitis Unified IDE 启动选项..... 20
  - 第 5 章：Vitis Unified IDE 视图和功能特性..... 22
    - Vitis 资源管理器视图..... 23
    - 终止后端进程..... 25
    - 搜索视图..... 26
    - 源代码控制..... 27
      - “Debug”视图..... 35
    - 示例视图..... 36
    - 代码视图和智能编辑器..... 38
    - 问题和源代码交叉探测..... 40
    - 并行编译..... 40
    - 文件更改通知..... 40
    - 新功能特性预览..... 41
    - 工作空间日志..... 43
    - 外部 Lopper 支持..... 43
    - 扩展..... 43
    - 分段配置..... 45
    - 首选项..... 46
  - 第 6 章：Vitis Unified IDE 中的 AI 辅助..... 49
    - AI 辅助的用例..... 51

|                                          |     |
|------------------------------------------|-----|
| 第 7 章：开发.....                            | 53  |
| 管理平台 and 平台存储库.....                      | 53  |
| 目标平台.....                                | 54  |
| 应用程序.....                                | 69  |
| 创建库工程.....                               | 78  |
| 第 8 章：运行、调试和优化.....                      | 81  |
| 启动配置.....                                | 81  |
| 目标连接.....                                | 84  |
| 运行应用组件.....                              | 86  |
| 调试应用组件.....                              | 87  |
| 导出存储器函数.....                             | 108 |
| 交叉触发.....                                | 109 |
| 剖析/分析.....                               | 119 |
| 创建启动镜像.....                              | 122 |
| 闪存烧录.....                                | 124 |
| 多线缆和多器件支持.....                           | 126 |
| 经身份验证的 JTAG 访问.....                      | 128 |
| XEN 感知调试.....                            | 130 |
| 第 9 章：用户管理模式.....                        | 131 |
| 设置用户工具链.....                             | 133 |
| 第 10 章：Vitis 实用工具.....                   | 134 |
| 软件调试器.....                               | 134 |
| 器件烧录.....                                | 134 |
| Vitis 终端.....                            | 135 |
| 工程导出和导入.....                             | 135 |
| 生成设备树.....                               | 136 |
| 第 11 章：Zephyr IDE 扩展.....                | 139 |
| 第三部分：Vitis Python API.....               | 144 |
| 第 12 章：Python Vitis 命令.....              | 159 |
| 在 Python ENV 中启用 Vitis API.....          | 144 |
| Python API：用于在 Vitis 中创建和管理工程的命令行工具..... | 146 |
| 通过 Python API 管理 Vitis IDE 组件.....       | 148 |
| 修改配置文件.....                              | 151 |
| 工作空间日志覆盖范围.....                          | 157 |
| 第 13 章：Python XSDB 命令.....               | 160 |
| 第 14 章：Python XSDB 使用示例.....             | 161 |



第四部分：XSDB 命令用法..... 174

第五部分：从 XSCT 移植到 Python API..... 175

    app..... 175

    bsp..... 176

    createdts..... 178

    domain..... 180

    getaddrmap..... 183

    getperipherals..... 183

    getprocessors..... 183

    getws..... 183

    lscript..... 184

    platform..... 185

    repo..... 187

    setws..... 189

    sysproj..... 189

    importprojects..... 190

    importsources..... 190

    library..... 191

第六部分：GNU 编译器工具..... 192

    第 15 章：概述..... 193

    第 16 章：编译器框架..... 194

    第 17 章：常用编译器用法和选项..... 196

        用法..... 196

        输入文件..... 196

        输出文件..... 197

        文件类型和扩展名..... 197

        库..... 197

        语言方言..... 198

        常用编译器选项：快捷参考..... 199

        常规选项..... 199

        库搜索选项..... 200

        头文件搜索选项..... 201

        默认搜索路径..... 201

        连接器选项..... 202

        存储器布局..... 202

        对象文件节..... 203

        连接器脚本..... 205

    第 18 章：MicroBlaze 编译器的使用和选项..... 206

        MicroBlaze 编译器..... 206



|                               |     |
|-------------------------------|-----|
| 处理器特性选择选项.....                | 206 |
| 常规程序选项.....                   | 208 |
| MicroBlaze 应用二进制接口.....       | 209 |
| MicroBlaze 汇编程序.....          | 209 |
| MicroBlaze 连接器选项.....         | 210 |
| MicroBlaze 连接器脚本的各节.....      | 211 |
| 编写或自定义连接器脚本的技巧.....           | 211 |
| 启动文件.....                     | 212 |
| 修改启动文件.....                   | 214 |
| 编译器库.....                     | 215 |
| 线程安全性.....                    | 216 |
| 命令行实参.....                    | 216 |
| 中断处理程序.....                   | 216 |
| <br>第 19 章：Arm 编译器的使用和选项..... | 218 |
| 用法.....                       | 218 |
| <br>第 20 章：其他注释.....          | 220 |
| C++ 语言代码大小.....               | 220 |
| C++ 标准库.....                  | 220 |
| 位置无关代码（可重定位代码）.....           | 220 |
| 其他开关和功能特性.....                | 221 |
| <br>第七部分：嵌入式设计教程.....         | 222 |
| <br>第八部分：驱动程序和库.....          | 223 |
| 多实例 IP 的 Tcl 生成调用.....        | 223 |
| <br>附录 A：附加资源与法律声明.....       | 226 |
| 查找其他文档.....                   | 226 |
| 支持资源.....                     | 226 |
| 参考资料.....                     | 227 |
| 修订历史.....                     | 227 |
| 请阅读：重要法律声明.....               | 228 |

## 第一部分

# Vitis 入门

**注释：**\$XILINX\_VITIS 表示 AMD Vitis™ 安装目录。它是在您运行 `source <VITIS_INSTALL_DIR>/<VERSION>/settings.sh` 后设置的。

本章提供了 AMD Vitis™ 统一软件平台的概述，并描述了安装和运行该工具的安装要求及过程。

# 按设计进程浏览内容

AMD 自适应计算文档按一组标准设计进程进行组织，以便帮助您查找当前开发任务相关的内容。您可以在[设计中心](#)页面上访问 AMD Versal™ 自适应 SoC 设计进程。您还可以使用[设计流程助手](#)来更深入地了解设计流程，并找到特定于预期设计需求的内容。本文档涵盖了以下设计进程：

- 嵌入式软件开发：基于硬件平台来创建软件平台，并使用嵌入式 CPU 开发应用代码。还涵盖 XRT 和计算图 API。本文档中适用于此设计进程的主题包括：
  - [基于 XSA 创建平台组件](#)
  - [自定义预构建平台](#)
  - [创建应用组件](#)
  - [运行、调试和优化](#)

如需了解其他设计进程，请参阅《Vitis 统一软件平台文档登陆页面》([UG1416](#))

# Vitis 软件平台安装

如需了解有关 AMD Vitis™ 软件平台版本说明的信息，请参阅《Vitis 软件平台版本说明》(UG1742)。

## 安装 Vitis 软件平台

请确保您的系统满足《Vitis 软件平台版本说明》(UG1742) 中的[安装要求](#)描述的所有要求。



**提示：**为了缩短安装时间，请禁用杀毒软件并关闭所有已打开的非必要程序。

1. 访问 [AMD 自适应计算下载网站](#)。
2. 下载对应您的操作系统的安装程序。
3. 运行安装程序 `xsetup`（在 Linux 上）或 `xsetup.exe`（在 Windows 上）即可打开“Welcome”（欢迎）屏幕。解压安装程序包。
4. 单击“Next”。
5. 在“Select Install Type”（选择安装类型）屏幕上输入您的 AMD 用户帐户凭证，然后选择“Download and Install Now”（下载并立即安装）。
  - a. 单击“Next”打开安装程序的“Accept License Agreements”（接受许可协议）屏幕。
  - b. 单击每个“I Agree”（我同意）复选框接受条款和条件。
  - c. 单击“Next”打开下一个屏幕（仅限使用 Web 安装程序时才需要）。
6. 在“Select Product to Install”屏幕上，选中“Select Product to Install”页面中的选项之一。
  - a. 要安装完整的 Vitis 软件平台用于嵌入式软件 and 应用程序加速开发，请选择“Vitis”，然后单击“Next”（下一步）。此完整 Vitis 安装包括 Vivado、v++ 和 AI 引擎工具链。
  - b. 如果您只需为嵌入式处理器执行软件开发，并且要求安装占用空间较小，请选择“Vitis Embedded Development”（Vitis 嵌入式开发），然后单击“Next”。如果您已下载 Vitis 嵌入式安装程序，那么这是安装程序上可用的唯一选项。
7. 请通过选择设计工具和器件来自定义您的安装（仅限完整 Vitis 安装才需执行此步骤）。

默认“Design Tools”（设计工具）选项用于 Vitis 统一软件平台标准安装，包括 Vitis、Vivado、Vitis HLS 和 Vitis Model Composer。您无需单独安装 Vivado 工具。

虽非必需，但您可启用“Vitis IP Cache”（Vitis IP 高速缓存）来为该版本中提供的示例设计安装高速缓存文件。Vitis 工具会将这些文件安装在 `<install_dir>/<release>/Vitis/data/cache/xilinx`。

您可以启用开发所需的器件。



**重要提示！**对于 Vitis 加速流程，安装时必须选择下列器件选项：“Devices” → “Install devices for Alveo and Edge acceleration platforms”（器件 > 为 Alveo 和边缘加速平台安装器件）

8. 单击“Next”打开安装程序的“Accept License Agreements”（接受许可协议）屏幕并接受相应的条款。
9. 单击“Next”打开安装程序的“Select Destination Directory”（选择目标目录）屏幕。
10. 指定安装目录并复查位置汇总信息。确保有足够的磁盘空间，然后单击“Next”以打开“Installation Summary”（安装汇总）屏幕。
11. 单击“Install”（安装）开始安装软件。

成功安装 Vitis 统一软件后，会显示一条确认消息，并提示您运行 `installLibs.sh` 脚本。

1. 在以下位置找到此脚本：`<install_dir>/<release>/Vitis/scripts/installLibs.sh`，其中 `<install_dir>` 为安装位置，`<release>` 是安装版本。

**注释：**Windows 无需该脚本。

2. 使用 `sudo` 权限运行此脚本，如下所示：

```
sudo installLibs.sh
```

此命令会根据操作系统来为 Vitis 工具安装若干必要的软件包。



**重要提示！** 请留意此脚本返回的任何消息。请务必手动安装任何缺失的软件包。例如，如果您的 Linux 安装不包含 `zip` 命令行实用工具，那么您需要手动安装该工具。该实用工具是部分 Vitis 工具所必需的，`installLibs.sh` 脚本不会为您安装该工具。

# Vitis 软件平台入门

## Vitis 统一软件平台概述

AMD Vitis™ 统一软件平台将 AMD 软件开发的方方面面全部组合到单一统一的环境内。Vitis 软件平台支持 Vitis 嵌入式软件开发流程和 Vitis 应用加速开发流程。该软件平台是专为希望使用基于 AMD FPGA 的最新软件加速技术的软件开发者而设计的。本文档旨在探讨嵌入式软件开发流程与 Vitis 核心开发套件的使用。

Vitis 统一软件平台包含许多工具和实用工具，作为后端来为嵌入式软件开发提供支持。该平台提供 Vitis 统一集成设计环境 (IDE) 作为前端图形用户界面 (GUI)，为嵌入式软件开发者提供支持。Vitis Unified IDE 适用于使用 AMD Vivado™ Design Suite 创建的硬件设计。此 IDE 使开发者能够高效工作，为 AMD 嵌入式处理器开发软件应用。

相较于基于 Eclipse 的传统 Vitis IDE，Vitis Unified IDE 提供了焕然一新的 GUI。它采用来自 Eclipse Foundation 的最新技术，并使用 Eclipse Theia 作为其基础框架。此全新框架可实现更快的图形用户界面响应、社区驱动的多样化开源插件和灵活的配置。

Vitis Unified IDE 为嵌入式软件开发提供了下列功能特性：

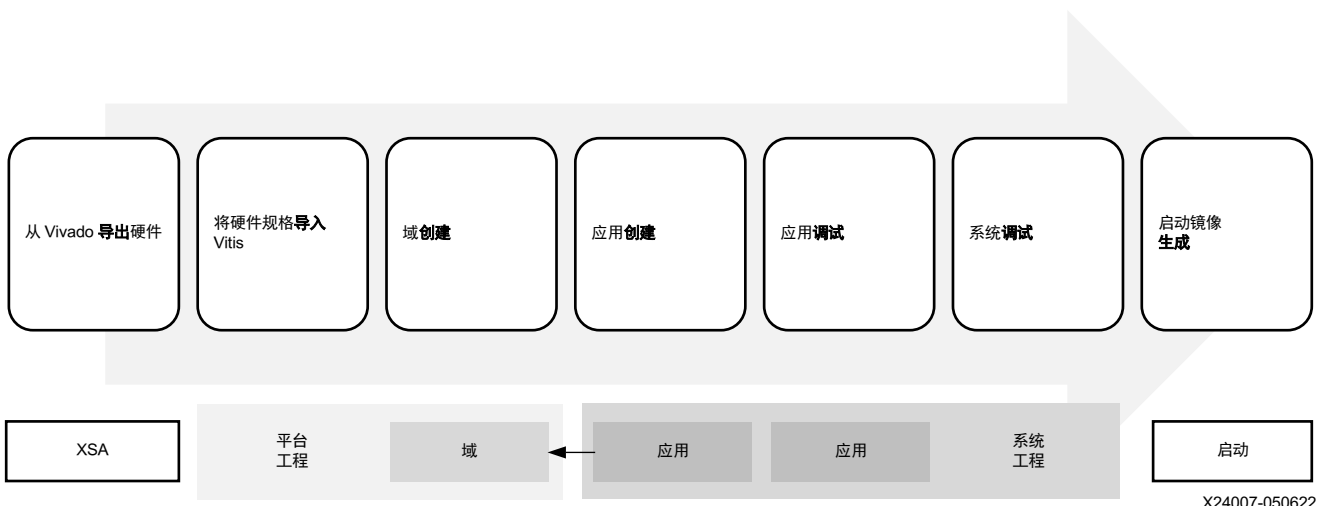
- 从 AMD Vivado™ 生成的硬件设计创建平台并生成 BSP 用于软件开发
- 从示例设计或空模板创建应用
- 配置和构建平台和应用
- 在硬件上运行、调试或剖析应用
- 利用目标连接管理器来管理多个本地或远程硬件连接
- 丰富的器件和处理器支持，包括 MicroBlaze™、Zynq 7000、Zynq AMD UltraScale+™ MPSoC 和 AMD Versal 自适应 SoC
- 创建启动镜像
- 创建静态库工程和动态库工程
- 配置器件
- 闪存烧录
- 根据组件类型按 IDE 和运行操作来管理工程
- 按用户脚本来管理工程，使用 IDE 来辅助调试过程
- 利用集成 Git 进行源码版本控制

GUI 和命令行接口 (CLI) 均支持以上功能特性。

## Vitis 软件开发工作流程

下图显示了 Vitis 统一软件平台的嵌入式软件应用开发工作流程。

图 1：嵌入式软件应用开发工作流程



- 硬件工程师负责设计逻辑，并将软件开发所需的信息从 AMD Vivado™ Design Suite 导出到 XSA 存档文件。
- 软件开发者负责通过创建平台来将 XSA 导入 Vitis 软件平台。应用加速工程大量使用该平台。为了统一各种应用的 Vitis 工作空间架构，软件开发工程现在已移植到平台和应用架构。平台包含硬件规格和软件环境设置。
- 域是软件环境设置的名称。域也是平台的一部分。
- 软件开发者基于平台和域来创建应用。
- 您可以在 Vitis Unified IDE 中调试应用。
- 在复杂系统中，有多个应用同时运行并彼此通信。因此也需要执行系统级别验证。
- 全部就绪后，Vitis Unified IDE 即可帮助创建启动镜像，用于初始化系统和启动应用。

**注释：**

Vitis Unified IDE 会对两种类型的 XSA 和对应平台提供支持。其中一种称为固定 XSA，含固定平台。另一种称为可扩展 XSA，含可扩展平台。

- 固定 XSA 包括在 Vivado 中生成的完整器件镜像，用于建立为嵌入式开发量身定制的固定平台。
- 可扩展 XSA 用于加速开发。在 v++ 的链接阶段，系统会创建一个综合的器件镜像，用于生成 BOOT.bin。此 BOOT.bin 用于加速开发的启动操作。

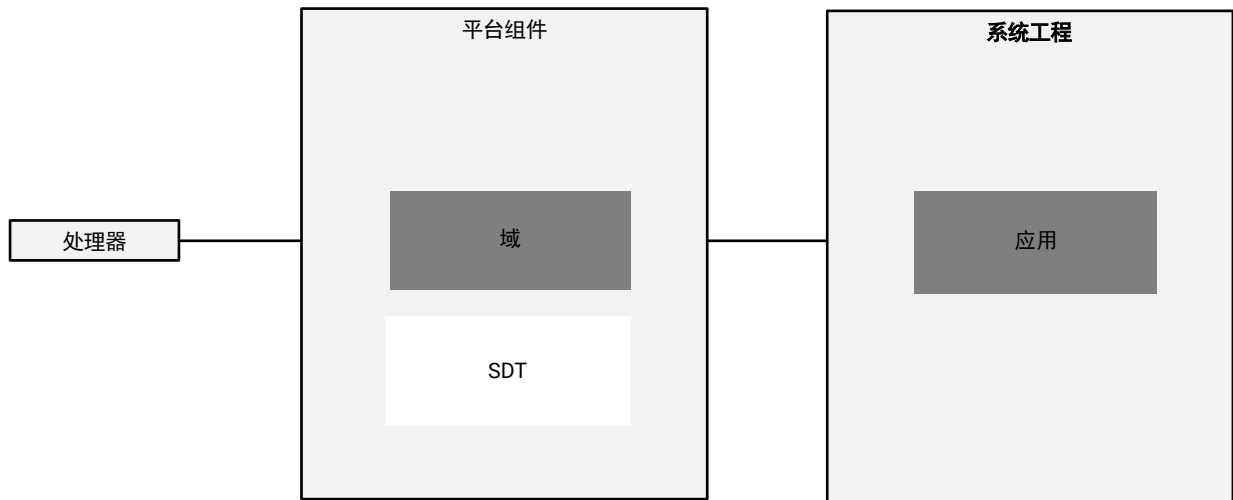
如果您使用可扩展平台创建裸机应用，可能会遇到“cannot find corresponding devices”（找不到对应器件）问题。

如需了解有关固定平台和可扩展平台的更多信息，请参阅《使用 Vitis 进行嵌入式设计开发》(UG1701) 中的[固定平台对比可扩展平台](#)。

## Vitis 软件平台中的工作空间结构

在 Vitis 统一工作空间内有两种类型的工程：

图 2：Vitis 软件平台工程类型



X50651-091525

- 工作空间：打开 Vitis 统一软件平台时，需创建工作空间。工作空间即供 Vitis 统一软件平台用于存储工程数据和元数据的目录位置。您必须在 Vitis 软件平台启动时提供初始工作空间位置。
- XSA：该工具会从 Vivado Design Suite 导出 XSA。此 XSA 包含各种硬件规格，例如，处理器配置属性、外设连接信息、地址映射和器件初始化代码等。创建平台工程时，必须提供 XSA。
- SDT：系统设备树 (SDT)：
  - 是在平台创建时使用 SDTGEN 实用工具根据 XSA 生成的。
  - 包含所有处理器和每个处理器各自的存储器映射 IP。
  - 用于捕获和维护来自 XSA 的硬件元数据。不会在目标上部署。

Lopper 实用工具用于提取硬件元数据。这些元数据可包括处理器列表、生成的 `xparameters.h`、生成的连接器脚本和创建的 BSP 等。

- 平台：目标平台是由硬件组件 (XSA) 和软件组件（域/BSP、FSBL 之类的启动组件等）组合而成的。存储库内的平台不可编辑。工作空间内的平台可编辑，称为平台组件。
- 平台组件：平台组件是 Vitis Unified IDE 中用于定义平台的工程。
- 域：
  - 域即板级支持包 (BSP) 或操作系统 (OS)，其中包含软件驱动程序集合。
  - 您可在域上构建自己的应用。
  - 创建的软件镜像仅包含 AMD 库中的部分内容，即您在自己的嵌入式设计中使用的部分。
  - 您可创建多个应用并在同一个域上运行。
  - 在平台中，每个域都绑定到单个处理器或者一个由同构处理器组成的集群（例如：A53\_0 或 A53）。
- 系统工程：
  - 系统工程用于将任一器件上同时运行的应用组合在一起。
  - 在任一系统工程中，同一个处理器不能同时存在 2 个独立应用。

- 在任一系统工程中，可以同时存在 2 个 Linux 应用。
- 每个工作空间均可包含多个系统工程。
- 应用（软件工程）：应用包含用于编译和生成二进制输出 (ELF) 文件的源文件和头文件。每个系统工程均可包含多个应用。每个应用都必须有一个对应的域。

Vitis 平台具有多种不同配置以支持不同用例，如下所述：

- 嵌入式：嵌入式平台是固定平台。此平台仅支持 Arm® 处理器和 MicroBlaze™ 处理器的嵌入式软件开发。Vitis 工具不能修改硬件设计。
- 嵌入式加速：此类型的平台支持嵌入式软件开发与应用加速。此平台可为加速内核提供时钟、总线接口和中断控制器。
- 数据中心加速：您可在此平台上开发加速内核与 x86 主机应用。此内核使用 PCIe® 总线。

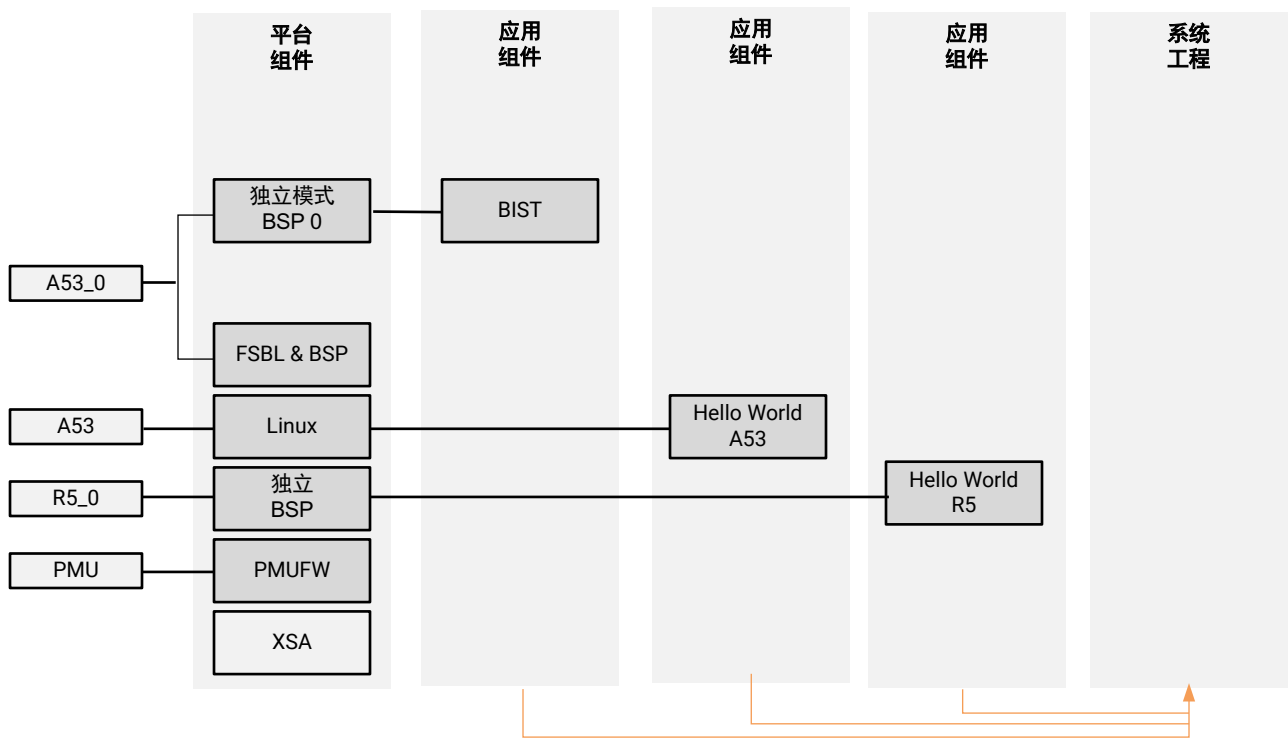
**注释：** Vitis 软件平台可以：

- 对可扩展平台的硬件设计进行扩展。
- 给原有硬件设计添加加速内核（PL 或 AI 引擎）。

您可使用此类型的平台进行软件开发。

以下是 AMD Zynq™ UltraScale+™ MPSoC 的典型 Vitis 软件开发工作空间示例。

图 3：Zynq UltraScale+ MPSoC 的 Vitis 软件开发工作空间示例



X50650-091525

- 您可以为 Arm® Cortex®-A53 SMP 集群创建 Linux 域。您可以编译 Linux 应用并将其与各库相链接。Linux 域的 `sysroot` 提供了这些库。
- Arm Cortex-A53 核 0 与 Arm Cortex®-R5F 核 0 可以同时运行 hello world 应用程序。您可以将这些应用组合到一个系统工程中。
- Arm Cortex-A53 核 0 上的裸机内置自检应用可以在自有系统工程内运行。该裸机内置自检应用可以有自己的 BSP 设置。
- 该系统工程会管理多个应用组件。在单个系统工程中添加多个应用组件意味着这些应用会同时运行。如果每次运行一个应用，则不需要系统工程。
- 您可以在平台工程中自动创建 FSBL 和 PMU 固件等启动组件。启动组件具有自己的 BSP 设置。

## 从传统 Vitis IDE 移植到 Vitis Unified IDE

Vitis Unified IDE 工作空间可确保版本控制优化。因此，工作空间和工程元数据与传统 IDE 并不兼容。为了帮助您过渡，以下提供了两种方法供您用于移植自己的工程。

### 手动移植（从 2025.1 起）

从 2025.1 版开始，您无法再使用 Vitis 传统 IDE。该版本不提供移植实用工具。如果您未曾在先前版本中移植自己的工作空间，则需要在 Vitis Unified IDE 中手动重新创建您的工作空间。

要手动移植工程，请遵循以下步骤进行操作：

- 使用最新 Vivado 工具生成新的 XSA 文件。
- 根据更新后的 XSA 文件，在 Unified IDE 中重新创建平台工程。
- 使用新平台重新创建应用工程。
- 将应用源码导入新工程。
- 构建应用并解决构建过程中出现的任何问题。

**注释：**如果您遇到任何与移植有关的问题，请参阅 [独立应用组件移植详情](#)



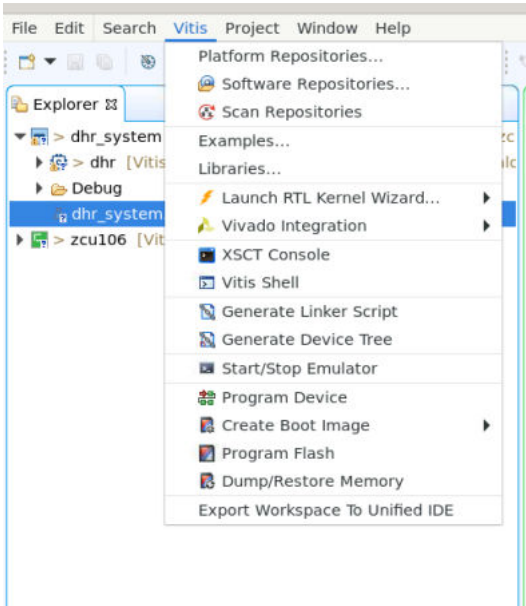
**提示：**为了使移植流程更简洁、更稳健，请使用手动移植，而不是移植实用工具。

### 使用前版本的实用工具来移植工程

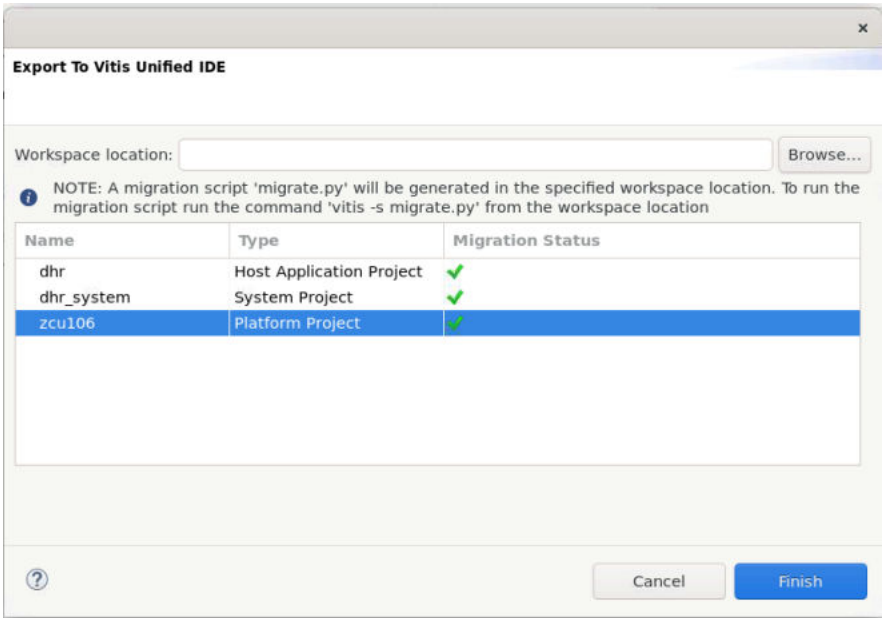
在传统 IDE 中为 2023.2、2024.1 和 2024.2 版本提供了一个移植实用工具。在传统 IDE 中移植您的工程后，您即可将自己的硬件平台和应用更新至最新版本。

要使用此方法，请遵循以下步骤进行操作。

1. 使用传统 Vitis IDE 打开该工作空间。
2. 选中“Vitis” → “Export Workspacce to Unified IDE”（Vitis > 将工作空间导出到 Unified IDE）。选中后就会出现一个对话框。



3. 在对话框设置页面中，指定移植目标工作空间位置。



- 4. 单击“Finish”（完成）。Vitis 工具会在指定位置生成移植脚本。
- 5. 打开含 Vitis 环境设置的终端，输入以下命令并按 Enter 键。

```
vitis -s migrate.py
```

Vitis 工具会运行移植脚本，并将工程资源复制到新位置。运行该流程需要一些时间。完成后您即可在工作空间目录中看到已移植的组件和工程。您可在新位置找到工程资源。

- 6. 使用 Vitis Unified IDE 打开移植后的工作空间。
- 7. 在 IDE 中打开平台工程，找到并打开平台 JSON 文件。
- 8. 单击“Switch/Re-read XSA”（切换/重新读取 XSA），选择用最新版本的工具生成的更新 XSA 文件。

**注释：**此操作会为您的应用重新生成系统设备树 (SDT) 与板级支持包 (BSP)。

9. 重新构建您的平台与应用。

表 1：使用该实用工具时受支持的工程与限制

| 工程类型  | 限制                                                                                                                       | 变通方法                                                        |
|-------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| 平台    | 对 BSP 源文件进行的本地更改不会移植到新工作空间。创建新的 BSP，并对新 BSP 应用设置。                                                                        | 将源文件手动复制到新 BSP。                                             |
|       | 添加到 Vitis IDE 的任何嵌入式软件存储库都无法移植。如果您有任何本地软件存储库，那么会在向导中显示一条警告。                                                              | 所有软件存储库都需要首先移植到 Lopper。可将移植的存储库路径添加到移植脚本中（请参阅以下截屏示例）以便移植工程。 |
|       | 使用 2023.1 或更低版本创建的 XSA 中包含的 IP 驱动程序可能会遇到编译错误。                                                                            | 需使用 2024.2 版本重新生成 XSA。                                      |
| 嵌入式应用 | 如有应用引用位于当前工作空间外的平台，那么这类应用无法移植。                                                                                           | 请首先移植平台并将应用更新为使用新平台，然后再移植该应用。                               |
|       | 如果您的应用依赖于器件 ID，那么可能需要对应用源代码进行一些修改。这是因为 xparameter.h 文件中的器件 ID 已被弃用。如需了解更多详情，请参阅《embeddedsoc 组件移植指南：基于系统设备树的构建流程》(UG1647) | 请参阅 <a href="#">独立应用组件移植详情</a>                              |
|       | 调试配置无法移植到 Unified IDE                                                                                                    | 在 Vitis Unified IDE 中创建启动配置                                 |

**注释：**从 2025.1 版起移除 Vitis 传统 IDE。如需继续使用传统 Vitis IDE，可使用以下命令启动传统 Vitis IDE。

```
vitis --classic
```

**注释：**要使用传统 Vitis IDE，请安装完整的 Vitis 软件平台。

如需了解有关 Vitis Unified IDE 的更多信息，请参阅 [启动 Vitis Unified IDE](#) 和 [Vitis Unified IDE 视图和功能特性](#)。

## 独立应用组件移植详情

驱动程序 API 不再使用器件 ID。

- 根据用于硬件控制的行业标准，所有驱动程序 API 现在都使用外设 IP 的基地址。
- 系统不再在 xparameters.h 中生成 DeviceID。直接对移植的应用程序进行编译可能导致编译错误。
- 如果您的应用依靠 DeviceID 来执行 IP 驱动程序初始化，请参阅以下答复记录：[独立应用移植详细信息](#)。

## Unified IDE 功能特性对比传统 IDE 功能特性

下表展示了 Unified IDE 与传统 IDE 的支持状态。

表 2：示例表格

| 功能特性 | Vitis Unified IDE    | 传统 Vitis IDE         |
|------|----------------------|----------------------|
| 平台创建 | <a href="#">目标平台</a> | <a href="#">目标平台</a> |

表 2：示例表格 (续)

| 功能特性          | Vitis Unified IDE                     | 传统 Vitis IDE                  |
|---------------|---------------------------------------|-------------------------------|
| 应用开发          | <a href="#">应用程序</a>                  | <a href="#">应用</a>            |
| 使用客户库         | <a href="#">创建库工程</a>                 | <a href="#">在应用工程中使用客户库</a>   |
| 运行应用          | <a href="#">运行应用组件</a>                | <a href="#">运行应用工程</a>        |
| 调试应用          | <a href="#">调试应用组件</a>                | <a href="#">调试应用工程</a>        |
| 交叉触发          | <a href="#">交叉触发</a>                  | <a href="#">交叉触发</a>          |
| TCF 剖析        | <a href="#">TCF 剖析</a>                | <a href="#">TCF 剖析</a>        |
| gprof 剖析      | 不支持                                   | <a href="#">gprof 剖析</a>      |
| 操作系统感知调试      | 不支持                                   | <a href="#">操作系统感知调试</a>      |
| Xen 感知调试      | <a href="#">XEN 感知调试</a>              | <a href="#">Xen 感知调试</a>      |
| 优化：性能分析       | 不支持                                   | <a href="#">优化性能分析</a>        |
| 创建启动镜像        | <a href="#">创建启动镜像</a>                | <a href="#">创建启动镜像</a>        |
| Program Flash | <a href="#">闪存烧录</a>                  | <a href="#">Program Flash</a> |
| 多线缆和多器件支持     | <a href="#">多线缆和多器件支持</a>             | <a href="#">多线缆和多器件支持</a>     |
| 目标管理          | <a href="#">目标连接</a>                  | <a href="#">目标连接</a>          |
| 版本控制          | <a href="#">源代码控制</a>                 | <a href="#">使用 Git 进行版本控制</a> |
| 用户管理的流程       | <a href="#">用户管理模式</a>                | 不支持                           |
| 工作空间导入和导出     | <a href="#">工程导出和导入</a>               | <a href="#">工程导出和导入</a>       |
| 软件调试器         | <a href="#">《软件调试器参考指南》(UG1725)</a>   | <a href="#">软件命令行工具</a>       |
| Python CLI    | <a href="#">第三部分：Vitis Python API</a> | 不支持                           |

注释：“Vitis Classic IDE”列下的功能链接可指引您访问本文档 2023.1 版中的对应章节。

## 第二部分

# 使用 Vitis Unified IDE

本章旨在描述如何使用 AMD Vitis™ 统一集成设计环境 (IDE) 来对平台和应用进行开发、运行、调试和优化。本章节还解释了每个 IDE 视图中的选项以及有关 [Vitis 实用工具](#) 的信息。

本部分包含以下章节：

- [Vitis Unified IDE 视图和功能特性](#)
- [开发](#)
- [运行、调试和优化](#)
- [Vitis 实用工具](#)

# 启动 Vitis Unified IDE

本节旨在讲解 Vitis Unified IDE 的启动步骤。

1. 使用以下命令加载 Vitis 软件平台环境。

```
source <Vitis_Installation_Directory>/settings64.sh
```

2. 使用以下命令启动 Vitis Unified IDE。

```
vitis -w <workspace>
```

其中 <workspace> 表示用于保存设计工程的所有内容的文件夹。该工作空间将构成设计的源文件与数据文件组合在一起。该工作空间会存储该工作空间的配置。

如需了解受支持的其他启动模式，请参阅 [Vitis Unified IDE 启动选项](#)。

**注释：**启动 AMD Vitis™ Unified IDE 前，您可设置其他环境设置以确保该工具能识别这些设置。例如，您可设置 Xilinx Runtime (XRT) 用于构建和运行数据中心加速应用：

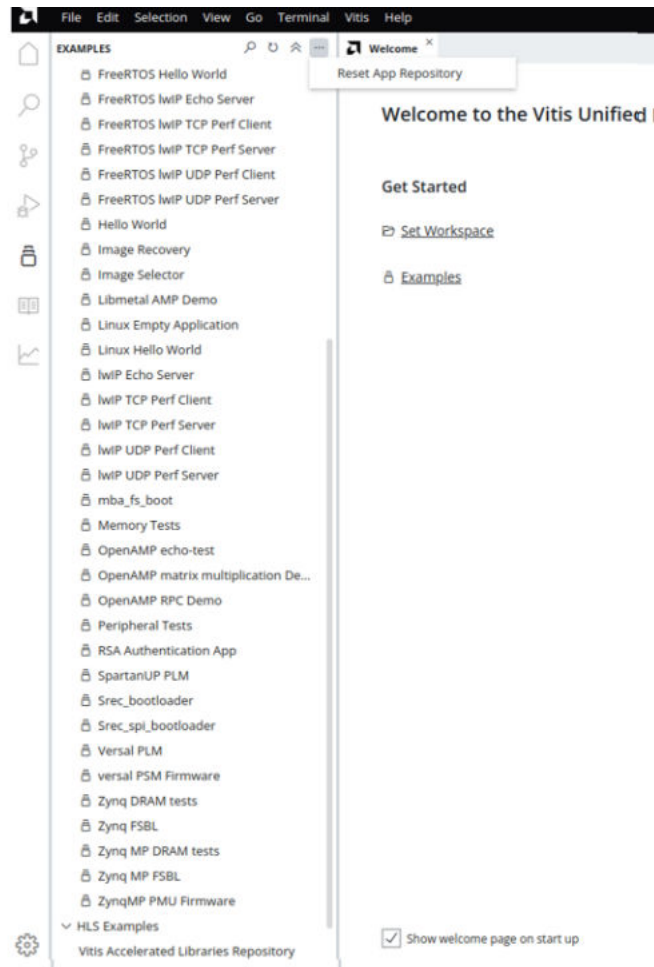
```
source <XRT_Install_Path>/setup.sh
```

您可借助下列示例来设置平台存储库路径：

```
export PLATFORM_REPO_PATHS=<platform_path>
```

vitis 命令可按您定义的选项来启动 Vitis Unified IDE。它可提供相应选项，用于指定工程的工作空间和选项。以下小节描述了 vitis 命令选项。

图 4：Vitis Unified IDE 的“Welcome”屏幕



注释：完整安装程序与嵌入式安装程序的欢迎页面 GUI 并不完全相同。

## Vitis Unified IDE 启动选项

### 启动选项

Vitis Unified IDE 支持以下模式：

- “GUI Mode”（GUI 模式）：Vitis Unified IDE 默认以 GUI 模式启动并呈现图形界面。（可选）您可以使用 `-w` 实参来指定要打开的工作空间。

```
vitis -w <workspace>
```

- “Analysis Mode”（分析模式）：分析模式会启动该工具并直接转至“Analysis”（分析）视图，以便您复查构建、运行和调试进程期间生成的汇总报告。

```
vitis -a
```

- “Interactive Mode”（交互模式）：此模式允许您在 GUI 外通过交互式 Python Shell 输入命令。欲知详情，请参阅《Vitis 参考指南》(UG1702) 中的 [Vitis 交互式 Python Shell](#)。

```
vitis -i
```

**注释：**在交互式命令提示符中输入 `help()` 即可浏览可用的命令模块。

- “Batch Mode”（批处理模式）：批处理模式用于执行指定的 Python 脚本并退出。

```
vitis -s <script>.py
```

- “Jupyter Notebook Mode”（Jupyter Notebook 模式）：使用 Vitis 环境启动 Jupyter Notebook 服务器。在默认网页浏览器中启动前端 UI。

```
vitis -j
```

您可以使用 `-h` 来打印受支持的 `vitis` 选项。

```
vitis -h
```

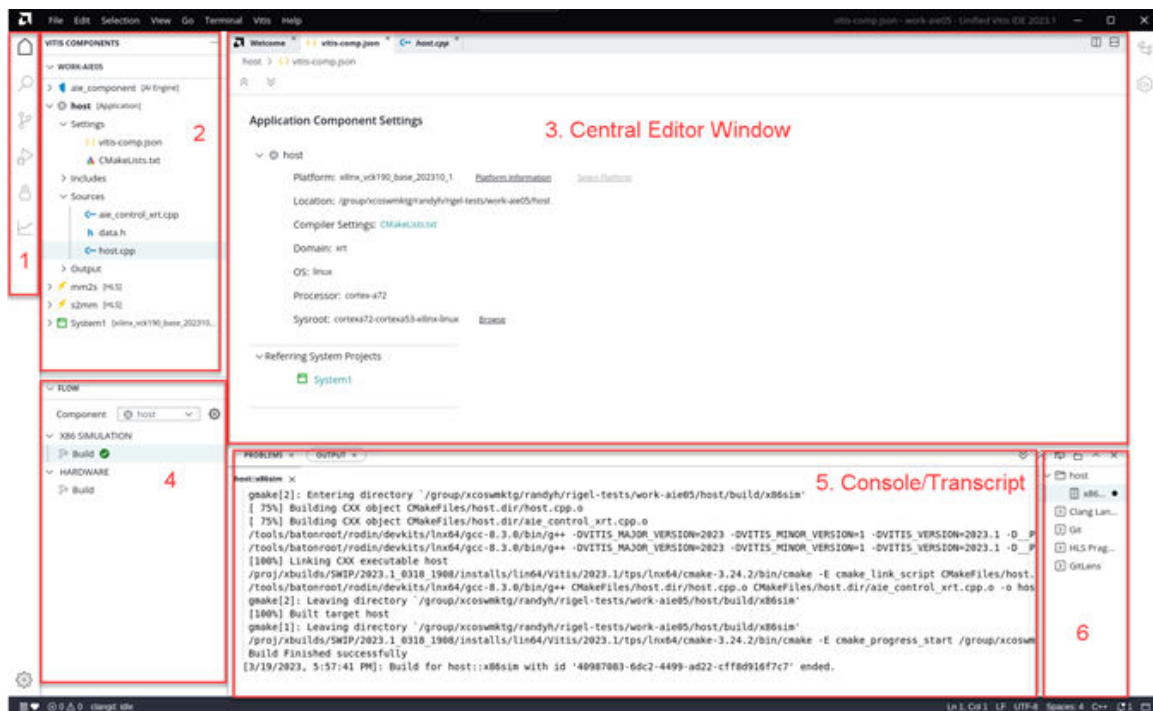
Syntax: `vitis [-a | -w | -i | -s | -j | --wait | -h | -v]`

Options:

```
    Launches New Vitis IDE (default option).
-a/--analyze [<summary file | folder | waveform file: *.*[wdb|wcfg]>]
    Open the summary file in the Analysis view.
    Opening a folder opens the summary files found in the folder.
    Open the waveform file in a waveform view tab.
    If no file or folder is specified, opens the Analysis view.
-w/--workspace <workspace_location>
    Launches Vitis IDE with the given workspace location.
-i/--interactive
    Launches Vitis python interactive shell.
-s/--source <python_script>
    Runs the given python script.
-j/--jupyter
    Launches Vitis Jupyter Web UI.
--wait/--wait
    Keeps the launching shell process attached to the Vitis Unified IDE
session and returns when the IDE exits.
-h/--help
    Display help message.
-v/--version
    Display Vitis version.
```

# Vitis Unified IDE 视图和功能特性

图 5: Vitis Unified IDE

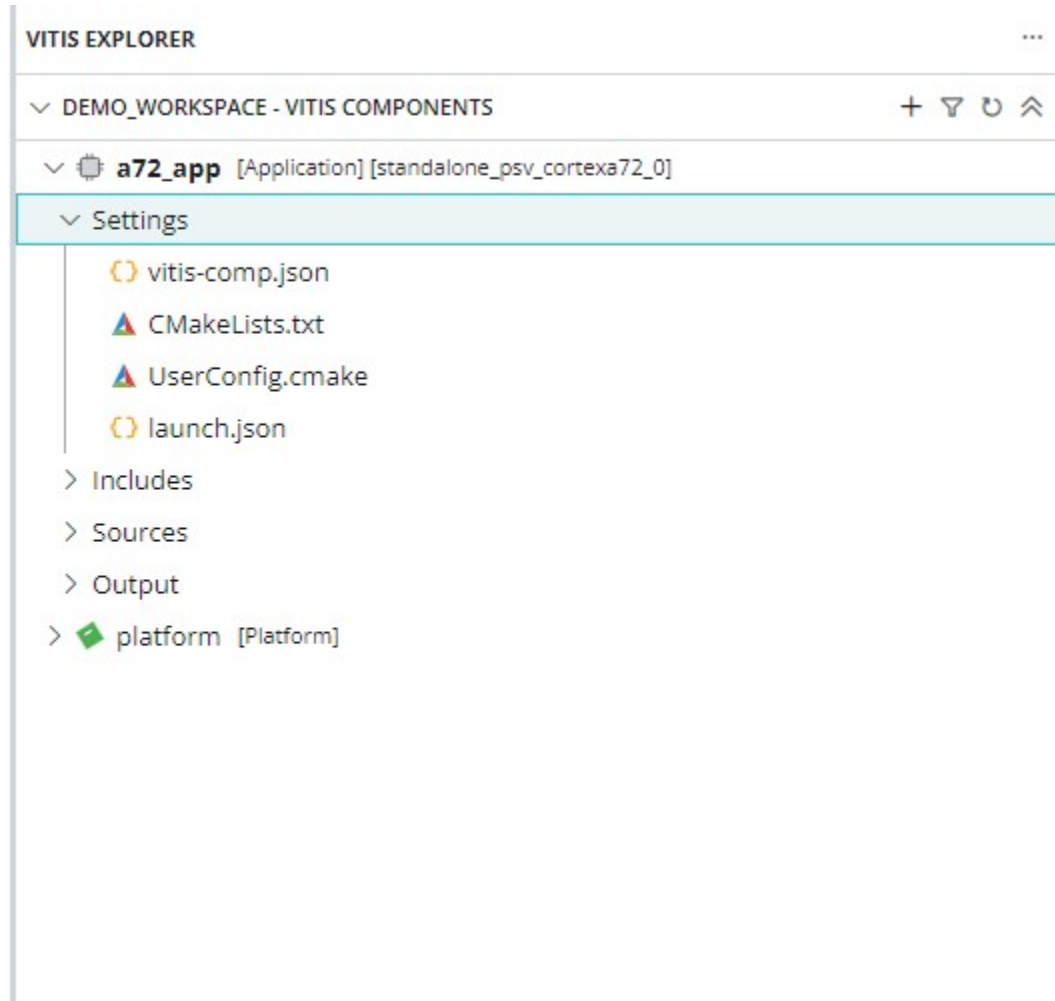


1. “Toolbar”（工具栏）菜单支持轻松访问该工具的主要功能特性。
2. 您可根据自己的偏好使用“View”（视图）下拉菜单来自定义“Toolbar”（工具栏）。
3. 您可使用 Vitis 的“Component Explorer”（组件资源管理器）来查看工作空间的虚拟层级。该工作空间可以帮助您了解组件或工程的不同要素。
4. 您可使用“Central Editor”（中央编辑器）窗口来编辑组件、配置和源文件。
5. “Flow Navigator”显示了活动组件的设计流程：
  - 组件具有不同的 workflow。
  - 在“Flow Navigator”中提供了活动组件的工作流程。
  - 您可指定激活的组件，只需在“Flow Navigator”中将其选中即可。
6. “Console/Terminal”（控制台/终端）区域会显示以下内容：
  - 该工具的输出转录文本。
  - 诸如“Terminal”（终端）窗口和“Pipeline”（流水线）视图等窗口。
  - 您可以使用工作空间的文件夹针对其中内容运行脚本。

7. “Index”（索引）会显示此会话期间构建步骤的转录文本的列表。此处显示的内容允许您重新打开已关闭的进程转录文本。

## Vitis 资源管理器视图

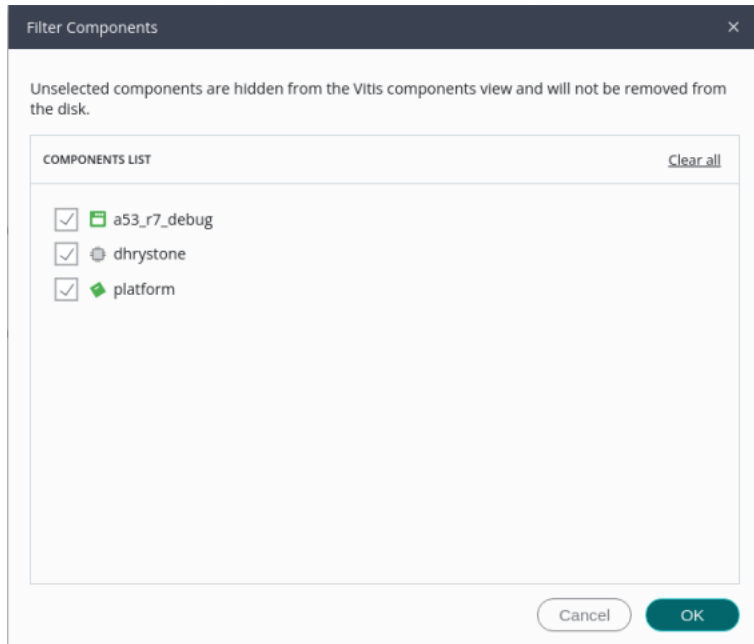
图 6：Vitis 资源管理器视图



您可以使用“Component Explorer”（组件资源管理器）执行以下操作。

您可以使用工具栏菜单执行以下操作。

- 创建“New Component”（新组件）。
- 选择“Filter Components”（筛选组件）以隐藏或显示组件。



- 刷新“Component Explorer”。
- 使用“Collapse All”（全部折叠）将显示的内容最小化。

通过使用右键单击菜单选择组件或工程，您可以执行以下操作。

- 选择“Open in Terminal”（在终端中打开）以打开终端窗口并将目录切换至组件内部。
- 选择“Show in Flow Navigator”（在 Flow Navigator 中显示）并单击组件以使其显示在“Flow Navigator”中。
- 选择“Delete”（删除）从工作空间中移除组件或工程。
- 选择“Clone Component”（克隆组件）基于现有组件创建新的组件。这对于设计探索非常有用，因为在此过程中您需要保留原始组件作为基准。然后，您即可使用克隆的组件来进行设计探索。此命令不支持系统工程。
- 选择“Reset Linker Script”（复位连接器脚本）来为独立应用组件生成连接器脚本。

通过右键单击组件或工程的 `Sources` 文件夹，您可以执行以下任务。

- 选择“Open in Terminal”（在终端中打开）以打开终端窗口并将目录切换至组件内部。
- 选择“Show in Flow Navigator”（在 Flow Navigator 中显示）并单击组件以使其显示在“Flow Navigator”中。
- 选择“Delete”（删除）从工作空间中移除组件或工程。
- 选择“Clone Component”（克隆组件）基于现有组件创建新的组件。这对于设计探索非常有用，因为在此过程中您需要保留原始组件作为基准。然后，您即可使用克隆的组件来进行设计探索。此命令不支持系统工程。
- 选择“Reset Linker Script”（复位连接器脚本）来为独立应用组件生成连接器脚本。

您可右键单击组件或工程层级中的对象来执行以下操作：

- 选择“Copy”（复制），随后选择“Paste”（粘贴）以在不同组件或工程之间复制对象。
- 选择“Copy Path”（复制路径）以复制对象的绝对路径。您可将此路径粘贴到配置文件的“Terminal”（终端）视图中。
- 选择“Copy Relative Path”（复制相对路径）以复制该对象相对于当前工作空间的相对路径。您可将此路径粘贴到配置文件的“Terminal”（终端）视图中。

- 选择 “Delete” （删除）以从工作空间中移除该对象。

“Flow” 窗口位于 “Vitis Components” （Vitis 组件）视图底部。当您选中组件时，“Flow window” 会主动显示您可对该组件执行的操作的范围。此类操作包括构建与调试。

## 基于深层 JSON 的组件显示

Vitis 统一软件平台支持在 “Component View” （组件视图）中显示来自当前工作空间的所有组件，并显示其唯一名称，与这些组件所在子目录无关。您可以直接访问、选择、编辑、编译和使用这些组件。

**注释：**系统会显示有关不受支持的组件的通知。

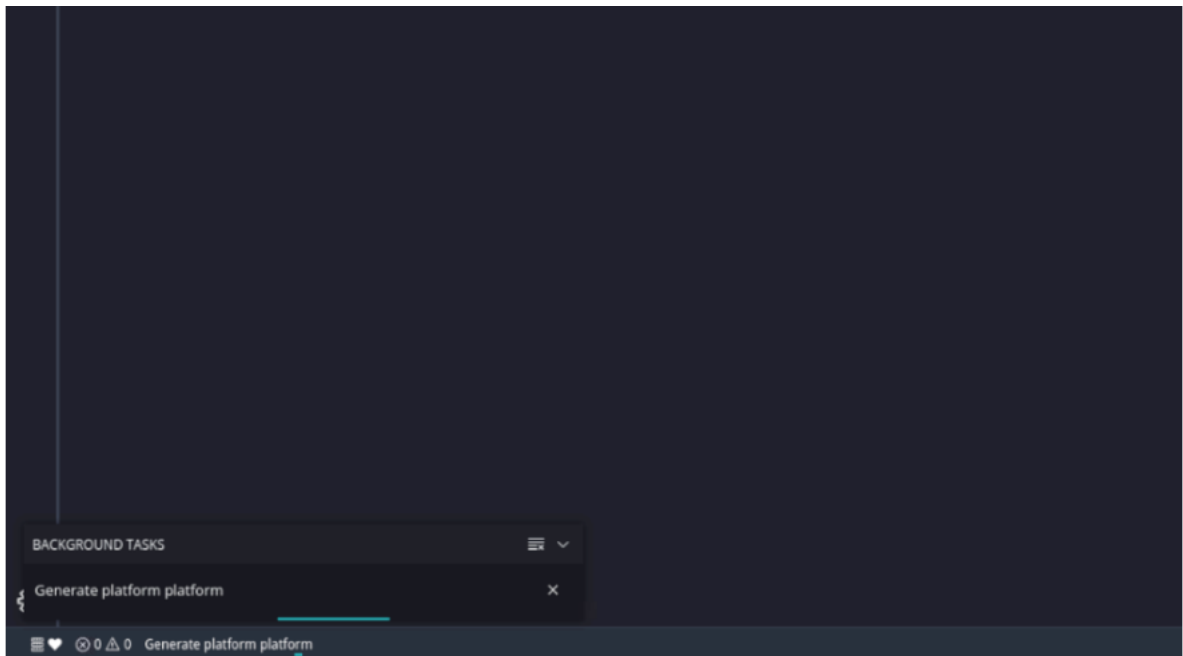
---

## 终止后端进程

在某些情况下，后端进程可能耗时过长而不产生任何结果，或者可能遇到问题。在此类情况下，终止被卡住的进程会很有用。

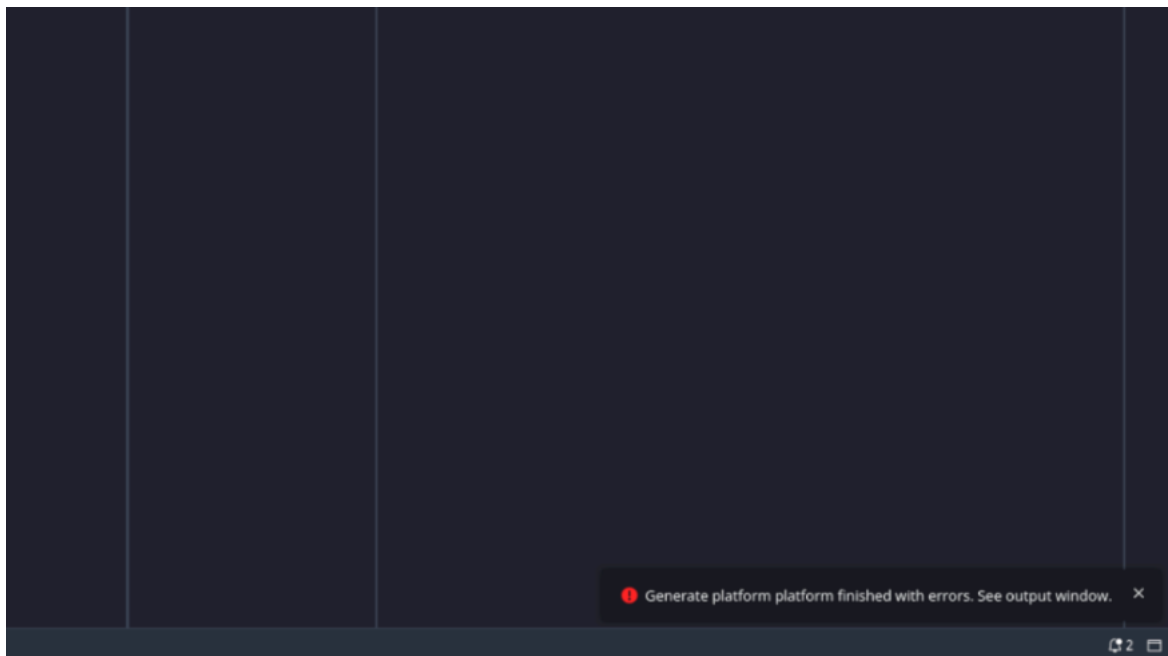
在 Vitis Unified IDE 左下角有一个进度条，会显示当前运行中的进程的进度。要终止后端进程，请首先在 Vitis Unified IDE 左下部分找到运行中的进程。

图 7：后台任务进度条



单击后台任务栏中的 “x” 按钮以终止该任务

图 8：显示进程取消或失败的消息



有多个进程可供终止：

- 构建组件
- 创建组件
- 硬件仿真
- 闪存烧录
- 更新 BSP
- 运行、调试和 XSDB
- 器件烧录

---

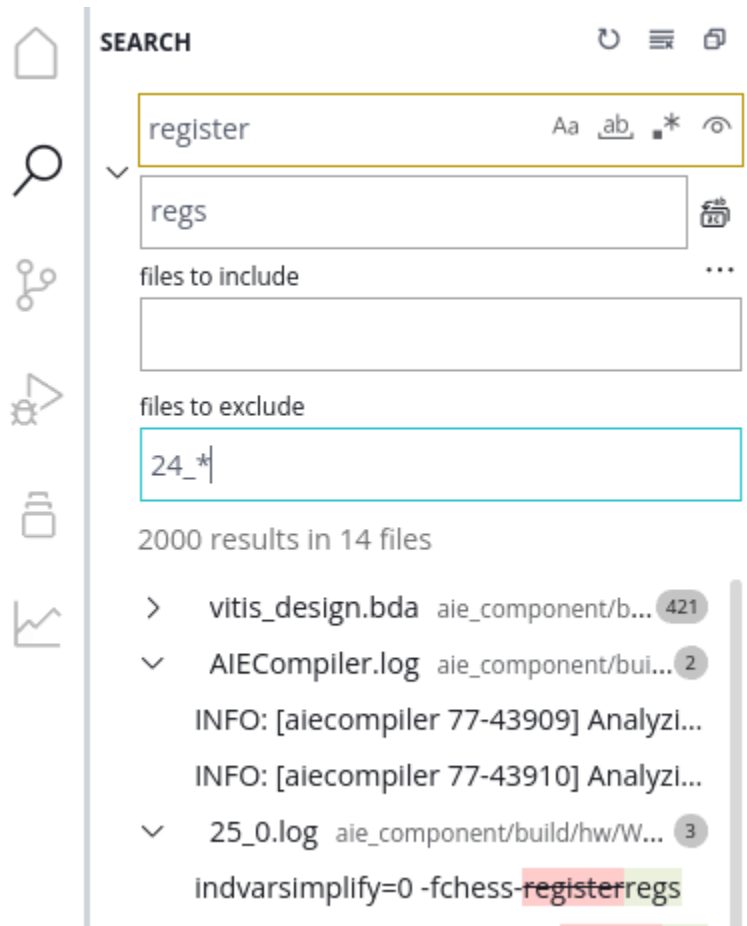
## 搜索视图

您可以使用搜索视图执行以下操作：

- 在当前工作空间中全局查找和替换文本
- 搜索文本文件，包括源文件、配置文件和 log 日志文件

系统会在文件内部执行搜索。搜索范围不包括文件名。

图 9：搜索视图



您可以匹配大小写、全字匹配、使用正则表达式或忽略某些文件。

“Toggle Search Details”（切换搜索详细信息）选项会展开“Search”（搜索）视图，以添加“Files to Include”（要包含的文件）和“Files to Exclude”（要排除的文件）字段。这些文件可包含通配符，且每个条目必须用逗号分隔。例如，以下命令表示包含（或排除）AIECompiler.log 文件以及名称包含 summary 的所有文件：

```
AIECompiler.log, *summary*
```



**提示：**请谨慎使用 Replace 功能，因为它会给您的设计引入错误。

## 源代码控制

软件开发者广泛使用源代码控制与版本控制。本节描述了如何在 Vitis Unified IDE 中使用 Git 集成。

## 跨操作系统支持

Vitis Unified IDE 支持跨操作系统场景，即，多个团队可在不同的操作系统上工作。Vitis 工作空间可从 Linux 操作系统迁移到 Windows 操作系统，反之亦然。您可以跨不同操作系统无缝打开工程或组件，并重新构建它们，而不会出现故障。

有几种在操作系统间分发 Vitis 工作空间的推荐方法：

1. 版本控制：将工程推送到共享的 GitHub 仓库，然后将其克隆到其他操作系统中。
2. 导出/导入：将工作空间导出为 .zip 文件。在其他操作系统中打开 Vitis Unified IDE 并导入此 .zip 文件。

请参阅前述章节，了解 git 用法和导出/导入功能，以获取更具体的指导信息。

## 源码控制所需的文件

Vitis 工作空间包含元数据以及源文件/构建文件；但并非所有这些文件都需要在 Git 中进行跟踪。工作空间根目录需要 .gitignore 文件。此文件指定在您将自己的工作空间提交到 Git 时忽略以下文件和文件夹：

- 构建输出目录
- 对象文件
- 依赖文件
- log 日志文件夹与 log 日志文件
- 锁定文件

如需了解有关 .gitignore 的更多信息，请访问 <https://git-scm.com/docs/gitignore>。

**注释：**忽略 “\_ide” 文件夹。在 Vitis Unified IDE 中加载新工作空间时，系统会自动生成这些文件夹。

综上，建议在 Git 中跟踪下列文件：

### 工作空间文件夹

表 3：工作空间下的文件

| 文件                       | 源码控制必需 |
|--------------------------|--------|
| _ide/version.ini         | 是      |
| _ide/setting.json        | 取决于用户  |
| _ide/worspace_journal.py | 否      |
| .gitignore               | 是      |

**注释：**version.ini 文件包含工作空间的版本信息。您可用它来管理不同 IDE 版本之间的兼容性。在源码控制中提交此文件。

**注释：**settings.json 文件存储工作空间专用设置与首选项，这些设置与首选项是根据工作空间所有者的环境量身定制的。由于它反映的是个人配置，因此请勿将此文件提交给源码控制。

**注释：**.gitignore 文件用于指定 Git 在工程中忽略的文件和目录。将此文件提交到源码控制。确保所有团队成员都遵循相同的规则，将临时文件、构建文件或用户特定文件排除在版本跟踪之外。

**注释：**工作空间日志文件用于记录在打开工作空间之后执行的操作。它可作为参考，供用户复查后台执行的 Python 命令。无需将此文件提交到源码控制。

## 平台文件夹

表 4：平台组件文件夹下的文件

| 文件                    | 源码控制必需 |
|-----------------------|--------|
| export                | 否      |
| hw/hw.xsa             | 是      |
| hw/sdt                | 否      |
| resources             | 否      |
| vitis-comp.json       | 是      |
| .gitignore            | 是      |
| psu_cortexa53_0 (BSP) | 是      |
| zynqmp_fsbl (BSP)     | 是      |

**注释：**export/ 文件夹包含平台创建期间生成的输出文件。默认情况下，通过 .gitignore 文件将该文件夹排除在版本控制之外。如果您需要提交 export/ 文件夹的内容（例如，用于共享或者存档目的），请通过 Git 使用 -f (force) 选项，如下所示：

```
git add export/* -f
```

**注释：**hw/sdt/ 文件夹包含用于 BSP 与器件树生成的系统设备树文件。您可在“Platform Settings”（平台设置）页面中单击“Switch/Re-read XSA File”（切换/重新读取 XSA）来重新生成此文件。无需将此文件提交到源码控制。

**注释：**psu\_cortexa53\_0 (BSP) 是供 Zynq UltraScale+ MPSoC A53 核使用的独立板级支持包 (BSP)。对于 Versal 器件，APU 核名称与此处名称不同，选择或创建 BSP 时请记录相应的名称。

## 应用文件夹

表 5：应用组件下的文件

| 文件                    | 源码控制必需 |
|-----------------------|--------|
| src                   | 是      |
| build                 | 否      |
| vitis-comp.json       | 是      |
| compile_commands.json | 否      |
| _ide/launch.json      | 取决于用户  |
| .gitignore            | 是      |

**注释：**build/ 文件夹包含应用构建期间生成的输出文件。默认情况下，通过 .gitignore 文件将该文件夹排除在版本控制之外。如果您需要提交 export/ 文件夹中的内容，请使用 Git 的 -f (force) 选项：

```
git add build/* -f
```

**注释：**compile\_commands.json 文件包含用于构建应用源文件的编译器命令。系统会在构建进程启动时自动生成此文件。您无需将此文件提交到源码控制。

**注释：** `launch.json` 文件位于 `_ide/` 文件夹下，用于存储用户定义的调试配置。该文件反映了个人偏好设置与特定环境的设置。请勿将此文件提交到源码控制。

系统工程文件夹

表 6：系统工程下的文件

| 文件                            | 源码控制必需 |
|-------------------------------|--------|
| <code>vitis-sys.json</code>   | 是      |
| <code>_ide/launch.json</code> | 取决于用户  |
| <code>.git ignore</code>      | 是      |

**注释：** `launch.json` 文件位于 `_ide/` 文件夹下。该文件用于存储用户定义的调试配置。它反映了个人偏好设置与特定环境的设置。请勿将此文件提交到源码控制。

# 启用 Source Control 视图

“Source Control” 视图是适用于 Git 的 GUI 帮助程序。您可以同时使用 Git 命令和源代码控制视图来处理自己的工程。“Source Control” 视图会在命令行中显示更新。



**重要提示！** 您必须已有用户 ID 和密码，并将其提供给系统，这样才能将 “Source Control” 视图与 Git 搭配使用。

要启用 “Source Control” （源代码控制）视图，请执行以下操作：

1. 将您的空工作空间初始化为 Git 仓库。
2. 创建工作空间。
3. 启动 Vitis Unified IDE 以打开此工作空间。
4. 将工作空间添加到您的 Git 仓库中。
5. 在 “Terminal” （终端）菜单中，选中 “New Terminal” （新建终端）。默认终端会打开您的工作空间所在的文件夹。
6. 在 “Terminal” 窗口中输入 `git init` 命令，然后按 Enter 键。

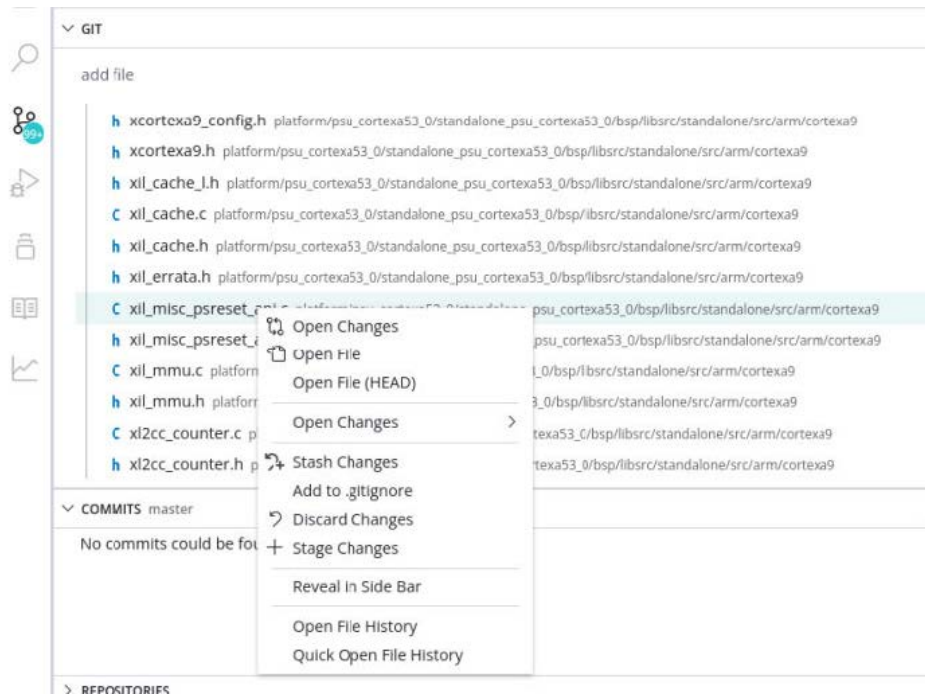
出现如下消息即表示此进程结束： `Initialized empty Git repository in /tests/temp/workVADD/.git/`

当您在该工作空间内创建新组件或工程后，Vitis Unified IDE 会生成 `.gitignore` 文件。此 `.gitignore` 文件可帮助您滤除生成的文件，以便您选取文件以执行源代码控制。您可打开此 `.gitignore` 文件，按照自己的额外要求编辑此文件。

# 添加新文件以便进行源代码控制

要添加文件以进行源代码控制，您可执行以下操作：右键单击要添加的文件，然后选中 “+ Stage Changes” （暂存更改）。

图 10：添加新文件以便进行源代码控制



此 GUI 等效于以下 Git 命令。

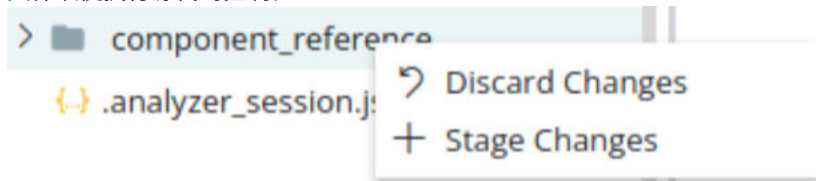
```
git add <file name>
```

## 为源代码控制添加组件

1. 切换到“Source Control”视图。
2. 切换到“View As Tree”（树状视图）。



3. 右键单击要为源代码控制添加的组件，然后选择“+ Stage Changes”（暂存更改）。此操作会添加该组件的所有文件以便执行源代码控制。



4. 右键单击组件，选择“- Unstage changes”（取消暂存更改）。

此 GUI 功能等效于以下 Git 命令：

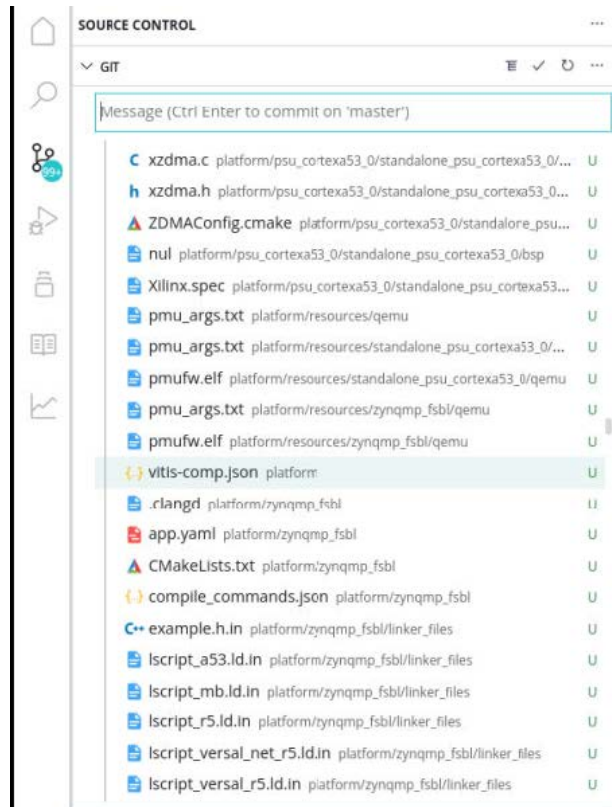
```
git add platform/
```

**注释：**如果您需要添加系统工程用于源代码控制，则需选择与系统工程捆绑的组件。

## 提交更改

要提交更改，请选择“Git”视图。输入提交消息，然后按“Ctrl + Enter”。

图 11: Git 视图



此 GUI 选项等效于以下命令：

```
git commit -m <commit message>
```

## 将工程推送到远程仓库

要将工程推送到远程仓库，可执行以下任一任务。

```
git push --set-upstream origin master
```

- origin：这是远程仓库地址。
- master：这是本地工作空间代码版本的分支。

## 相对路径支持

如果导入的文件与工作空间共享相同的路径初始节点，那么 Vitis 工具会自动选择相对路径。示例如下：

- 文件路径：/local/drive/source/app.cpp
- 工作空间路径：/local/drive/workspace

系统会选择如下相对路径：

```
$COMPONENT_LOCATION/../../sources/app.cpp
```

**注释：**将工作空间迁移或提交到源代码控制时，请谨记，务必保留相对路径关系。

## 平台 BSP 的源代码控制

要对平台 BSP 进行自定义和版本控制，您可以保留 BSP 源代码仓库的本地副本。按需修改 BSP 源代码，然后根据更新后的本地源代码来创建或重新生成平台 BSP。请遵循以下步骤进行操作。

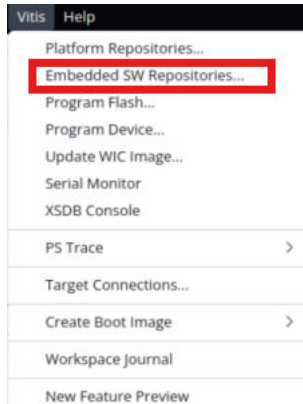
1. 下载嵌入式 BSP 仓库。

```
git clone https://github.com/Xilinx/embeddedsw.git
```

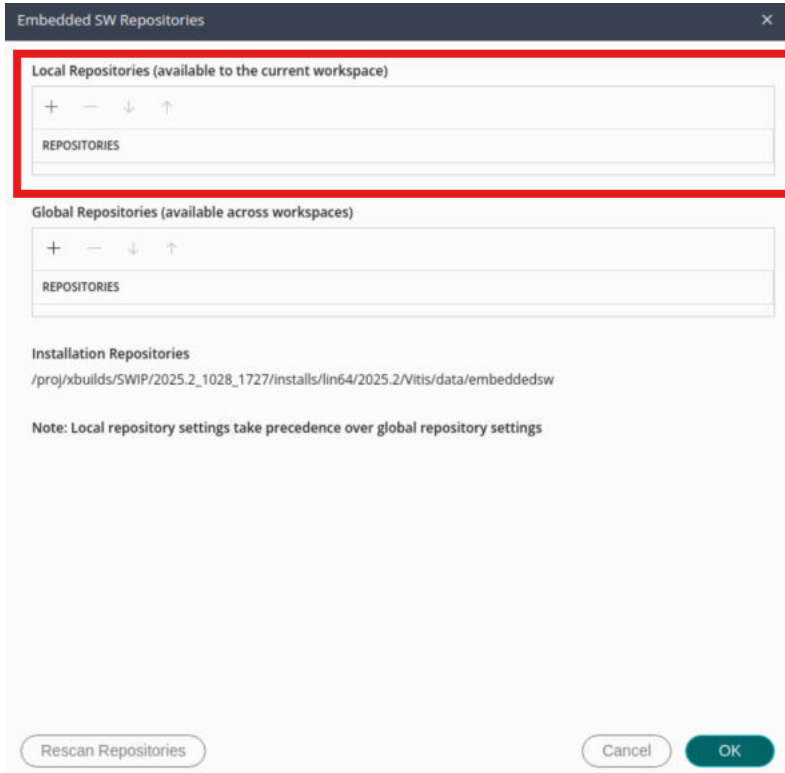
2. 切换版本。

```
cd embeddedsw & git checkout xilinx_v2025.2 # select the version you need accordingly.
```

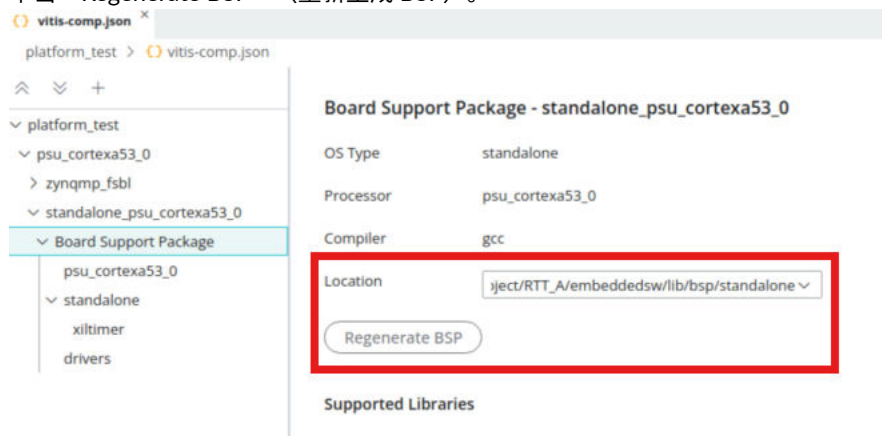
3. 启动 Vitis Unified IDE 并转至“Vitis”菜单，然后选择“Embedded SW Repositories”（嵌入式软件仓库）。



4. 单击“+”添加本地仓库，然后浏览到下载的嵌入式 BSP 仓库，并将其选中。



5. 单击“Rescan Repositories”（重新扫描仓库）确保 IDE 能检测到该仓库。
6. 单击“OK”（确定）。
7. 选择您在创建平台时需要使用的 BSP
  - a. 打开“Platform Component JSON file”（平台组件 JSON 文件），然后选择左侧面板中的“Board Support Package”（板级支持包）。
  - b. 单击“Location”（位置）框旁的下拉箭头，然后选中反映本地 BSP 仓库的选项。
  - c. 单击“Regenerate BSP”（重新生成 BSP）。



在此情况下，您可基于自己的本地仓库来创建平台 BSP，这样即可在源代码控制下轻松对其进行更新和管理。在平台目录自身内部无需对 BSP 进行维护或版本控制。

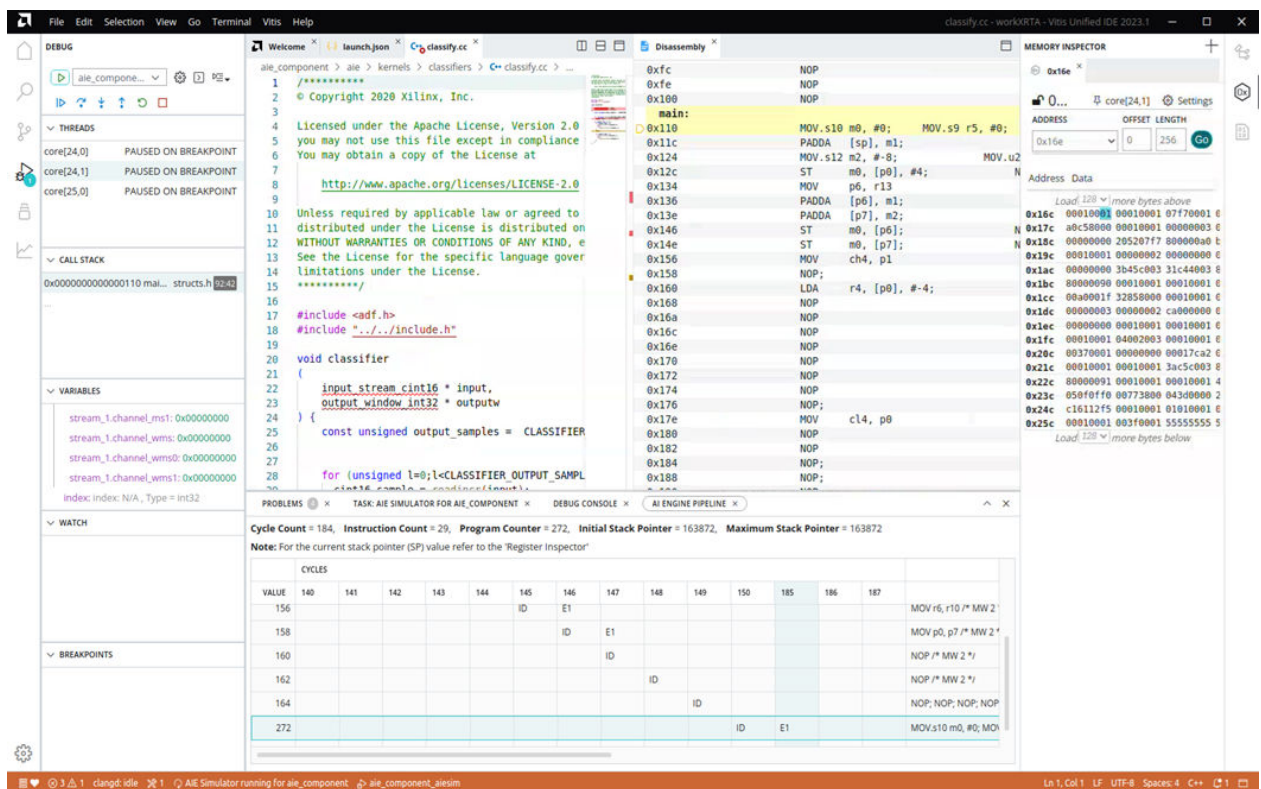
## “Debug” 视图

Vitis IDE 调试环境具有诸多基于 GUI 的传统调试环境内常见的功能特性，例如 GDB。在调试环境中，您可以执行以下任务。

- 在代码中添加断点。
- 对特定的代码行、循环或函数进行单步跳过或单步跳入。
- 检查变量状态，并将变量强制设为所需值。

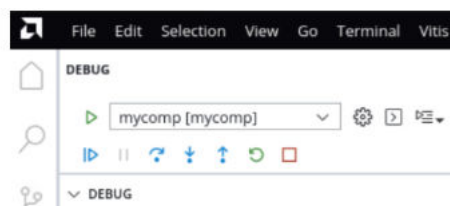
“AI Engine Debug”（AI 引擎调试）视图包含多个窗口或视图，如下图所示。

图 12：AIE 组件调试视图



- “Control Panel”（控制面板）：“Debug”视图的“Control Panel”（控制面板）位于屏幕左上角。调试期间，您可以使用“Continue”（继续）、“Step Over”（单步跳过）、“Step Into”（单步跳入）、“Step Out”（单步跳出）、“Restart”（重新启动）和“Stop”（停止）等控制按钮来控制调试进程。

图 13：调试控制面板



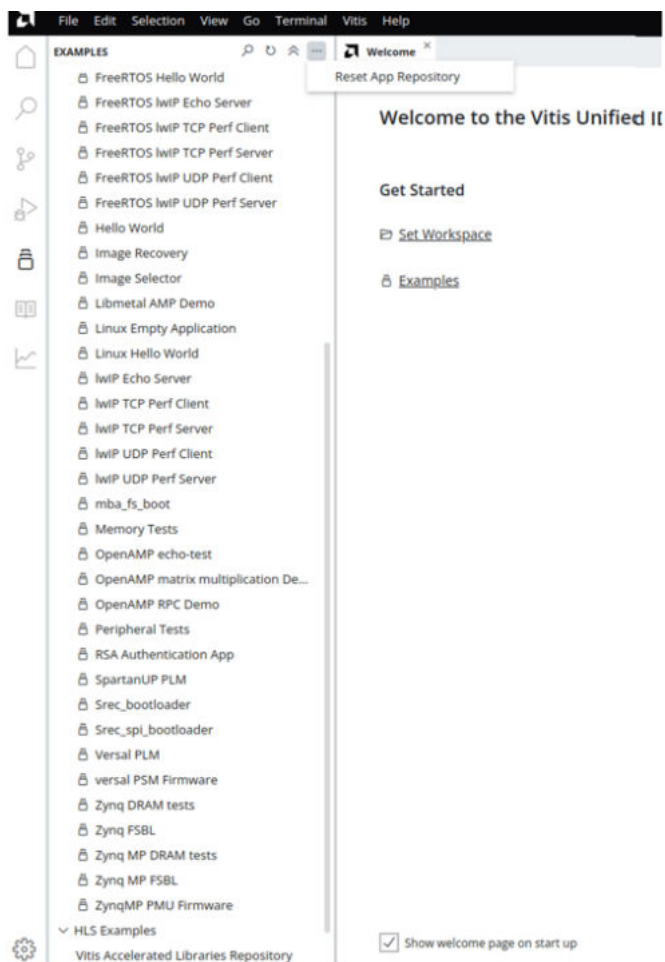
- “Threads”（线程）：“Threads”（线程）用于显示相关调试线程。调试进程会创建和销毁线程。您可在多个线程之间进行切换。
- “Call Stack”（调用栈）：“Call Stack”会显示运行应用时更新的函数调用栈。
- “Variables”（变量）：“Variables”显示了全局变量和局部变量的当前值。切换线程时，变量信息会更新。
- “Watch”（观察）：“Watch”（观察）显示了您指定要关注的变量和表达式。要添加观察点，请选择“Add Expression”（添加表达式）(+)。
- “Breakpoints”（断点）：
  - 如果主机组件的“main”函数可调试，那么系统会在该函数处设置断点。
  - 如果 PL 内核可调试，那么系统会在其 top 函数处设置断点。
  - 要添加断点，请打开源文件并单击行号左侧显示红点处。
  - 要移除断点，请单击先前添加的断点。
  - 显示红点时，可右键单击并选中“Add Conditional Breakpoint”（添加条件断点）来添加条件断点。要记录消息，请在到达断点时选择“Add Logpoint”（添加记录点）。
- “Source Code Editor”（源代码编辑器）：从“Flow Navigator”或“Launch Configuration”（启动配置）启动“Debug”后，会打开“Source Code”（源代码）视图。
- “Memory Inspector”（存储器检查器）：您可手动打开“Memory Inspector”。“Memory Inspector”会显示特定存储器地址的内容。
- “Register Inspector”（寄存器检查器）：您可手动打开“Register Inspector”。该视图会显示如下内容：
  - 位于应用程序组件源代码中触发的断点处的 Cortex-A72 的寄存器。
  - 位于 AI 引擎内核中触发的断点处的 AI 引擎的寄存器。
- “Disassembly”视图：您可从“Source Code”窗口的右键单击菜单中打开“Disassembly”视图。
- “Debug Console”（调试控制台）：此视图会显示调试进程的转录文本，以及从应用程序接收到的消息。

**注释：**在“Debug Console”中，您可看到来自 MDM（MicroBlaze/ V 调试模块）UART 的所有 print 输出。

## 示例视图

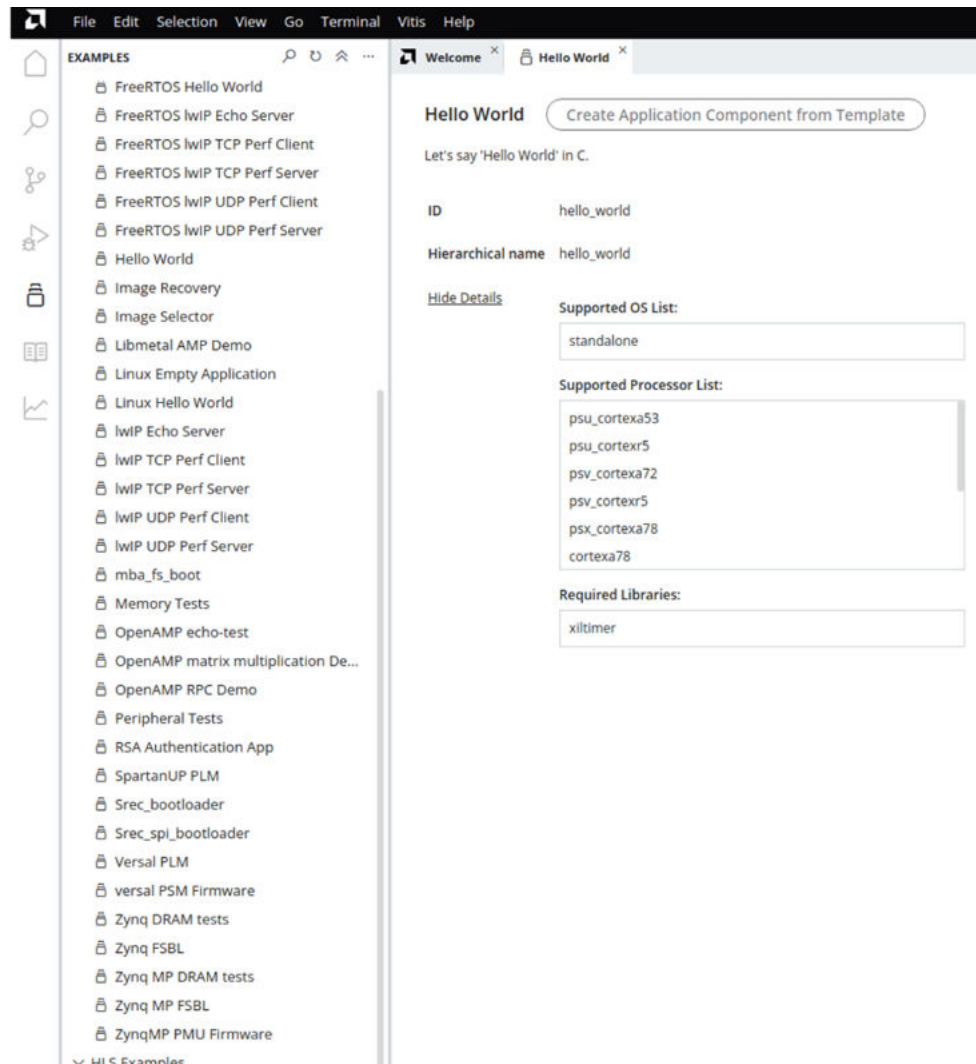
您可以通过如下方式访问“Examples”（示例）视图：从左侧面板、从“View”→“Examples”（视图 > 示例），或者使用“Ctrl + Shift + R”键盘快捷键。在此视图中，您可创建系统工程示例以便浏览该工具，还可管理示例存储库。

图 14：“Examples” 视图



要使用示例，请从“Examples”视图中将其选中，这样会打开一个模板以供您创建新的应用组件。单击“Show Details”（显示详细信息）即可显示此应用支持的操作系统和处理器。

图 15：示例视图中受支持的器件



## 代码视图和智能编辑器

Vitis Unified IDE 中的“Source Code”（源代码）编辑器支持以下功能特性：

- 适用于 C、C++、Python、Makefile 和 CMakeList.txt 文件的语法高亮功能
- 提供有关变量名称、函数名称的提示
- 跳转到变量或函数的定义
- 无需退出文件编辑操作即可查看变量或函数的定义
- 报告变量和函数的引用

您可选中右侧的  按钮或者选中“View”（视图）菜单下的“Outline View”（大纲视图）打开大纲窗口。“Outline”窗口列出源代码中的函数名称。

适用于 `launch.json` 文件和 `build.json` 文件的智能编辑器可在以下视图之间进行切换：GUI 呈现、“Text

Rendering”（文本呈现）与“Text Editor”（文本编辑器）。使用表格按钮  和代码按钮  在不同视图之间进行切换。GUI 呈现仅显示它可为上下文识别的选项。对于高级用例，请手动编辑配置文件。在任一视图内执行的修改都会立即更新另一视图。

图 16：GUI 格式

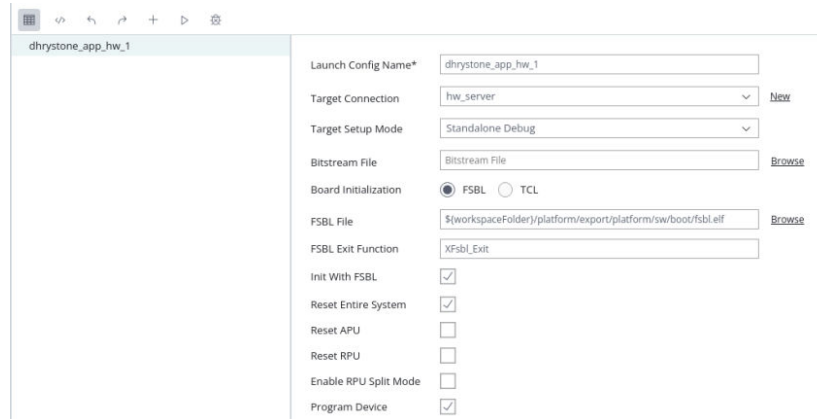


图 17：文本格式

```

1 {
2   "version": "0.2.0",
3   "configurations": [
4     {
5       "type": "tcf-debug",
6       "request": "launch",
7       "name": "dhystone_app_hw_1",
8       "debugType": "baremetal-zu",
9       "attachToRunningTargetOptions": {
10        "targetSetupMode": "standalone",
11        "executeScript": true,
12        "scriptPath": ""
13      },
14       "autoAttachProcessChildren": false,
15       "target": {
16        "targetConnectionId": "hw_server",
17        "peersIniPath": "../../peers.ini",
18        "context": "ZUPlus"
19      },
20       "targetSetup": {
21        "resetSystem": true,
22        "programDevice": true,
23        "enableRPUsplitMode": false,
24        "resetAPU": false,
25        "resetRPU": false,
26        "bitstreamFile": "",
27        "zuInitialization": {

```

启用“Auto Save”（自动保存）时，系统会自动保存更改。（可选）您可使用键盘快捷键 `Ctrl + S` 手动保存文件中的更改。

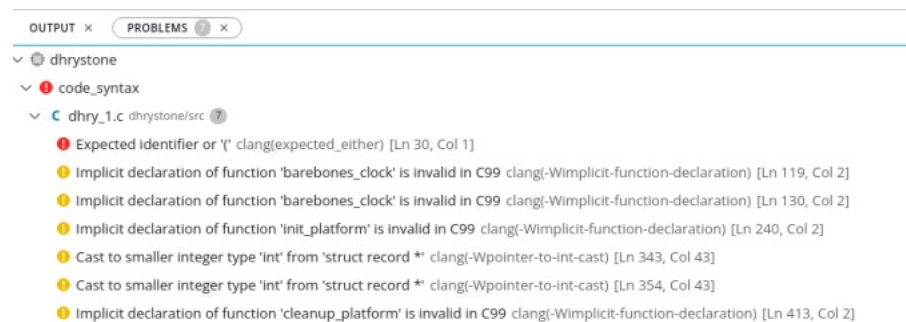
## 问题和源代码交叉探测

按住“Ctrl”键，同时左键单击任一问题，即可从输出控制台中的错误快速跳转到源代码。

### “Problem” 视图

输出通道会显示所有警告与错误消息。您单击错误消息或警告消息即可快速导航到“Problem”（问题）窗格中的错误或警告源。

图 18: “Problem” 视图



## 并行编译

Vitis Unified IDE 可以快速响应操作。它采用非阻塞构建命令，允许您继续工作，同时运行多个构建。

**注释：**如果您在性能较低的服务器上同时启动多个构建或硬件仿真作业，那么预计响应较慢。

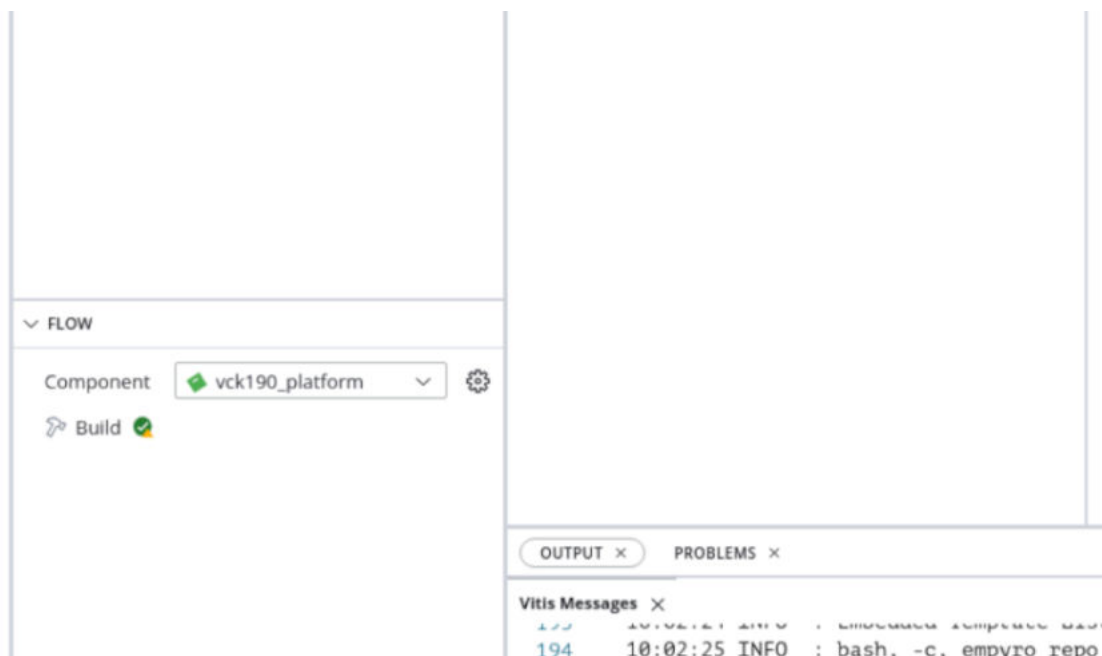
应用构建作业可并行启动多个组件。您可以为引用的组件并行构建和启动多项作业。

Vitis 工具默认禁用“Parallel Build”（并行构建）功能特性。您可通过“File”→“Preferences”→“Open Settings (UI)”（文件 > 首选项 > 打开设置 (UI)）命令并选择“Vitis”→“Build”→“Parallel Build”→“Disable”（Vitis > 构建 > 并行构建 > 禁用）设置来将其启用。

## 文件更改通知

修改文件时，构建状态图标旁的黄色指示符会提示您进行复查，并按需重新构建组件。

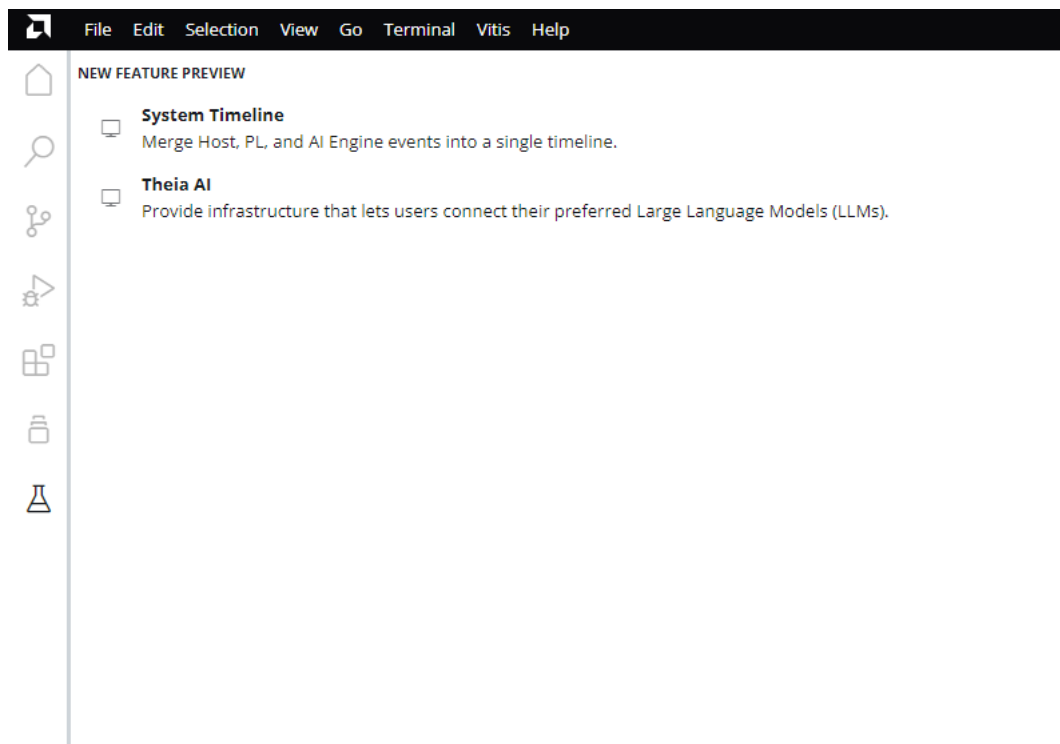
图 19：组件过期通知



## 新功能特性预览

您可以在这些新功能特性正式发布前先行探索。这些功能特性可能无法应对所有用例，并且需要进一步开发。您可单击“Vitis” → “New Feature Preview”（Vitis > 新功能特性预览）查看这些新功能特性。

图 20：新功能特性预览



您需要先启用新功能特性，然后才能使用它。

免责声明：预览功能特性目前尚处于抢先体验阶段。目前正在开发中。这些功能特性可能无法完全发挥作用或者可能不稳定。使用这些功能特性即表示您确认并接受：

- 功能受限：与完全发布的功能特性相比，抢先体验功能特性的功能性可能受限。抢先体验功能特性可能缺乏某些功能，也可能出现性能问题。
- 潜在不稳定性：抢先体验功能特性容易出现意外行为。请谨慎使用这些功能特性。
- 反馈和改进：您的反馈对于改进抢先体验功能特性至关重要。AMD 鼓励您报告在使用这些功能特性过程中遇到的任何问题、缺陷或建议。您的意见建议能帮助 AMD 增强这些功能特性的性能和稳定性。
- 不作任何保证：抢先体验功能特性按原样提供，不含任何明示或暗含的担保或保证。AMD 并不保证会以当前形式发布这些功能特性，甚至无法保证会正式发布这些功能特性。AMD 保留修改、暂停或中止这些功能特性的权利，恕不另行通知。
- 使用风险自负：使用抢先体验功能特性即表示您了解并接受其中的风险。对于因使用这些功能特性而造成的任何损害、损失或不便，AMD 概不承担任何责任。



**重要提示！** 在访问和使用抢先体验功能特性之前，请审慎考虑这些因素。

## 工作空间日志

“Workspace Journal”（工作空间日志）是一个 log 日志文件，用于记录对应于用户打开工作空间后所触发操作的后端命令。检查 GUI 使用的对应命令和重新创建工作空间是个好方法。要打开 Journal 日志文件，请转至菜单，并选择“Vitis”->“Workspace Journal”（Vitis > 工作空间日志）。

您可运行以下命令使用当前工作空间内的 journal 日志文件来重新创建工程：

```
vitis -s workspace_journal.py
```

**注释：**在 workspace\_journal.py 中，请尽可能使用相对路径，以提高可移植性。但在某些情况下，有必要使用绝对路径。执行脚本前，请确保 workspace\_journal.py 中的源路径和目标路径均已正确完成配置并且可供访问。

## 外部 Lopper 支持

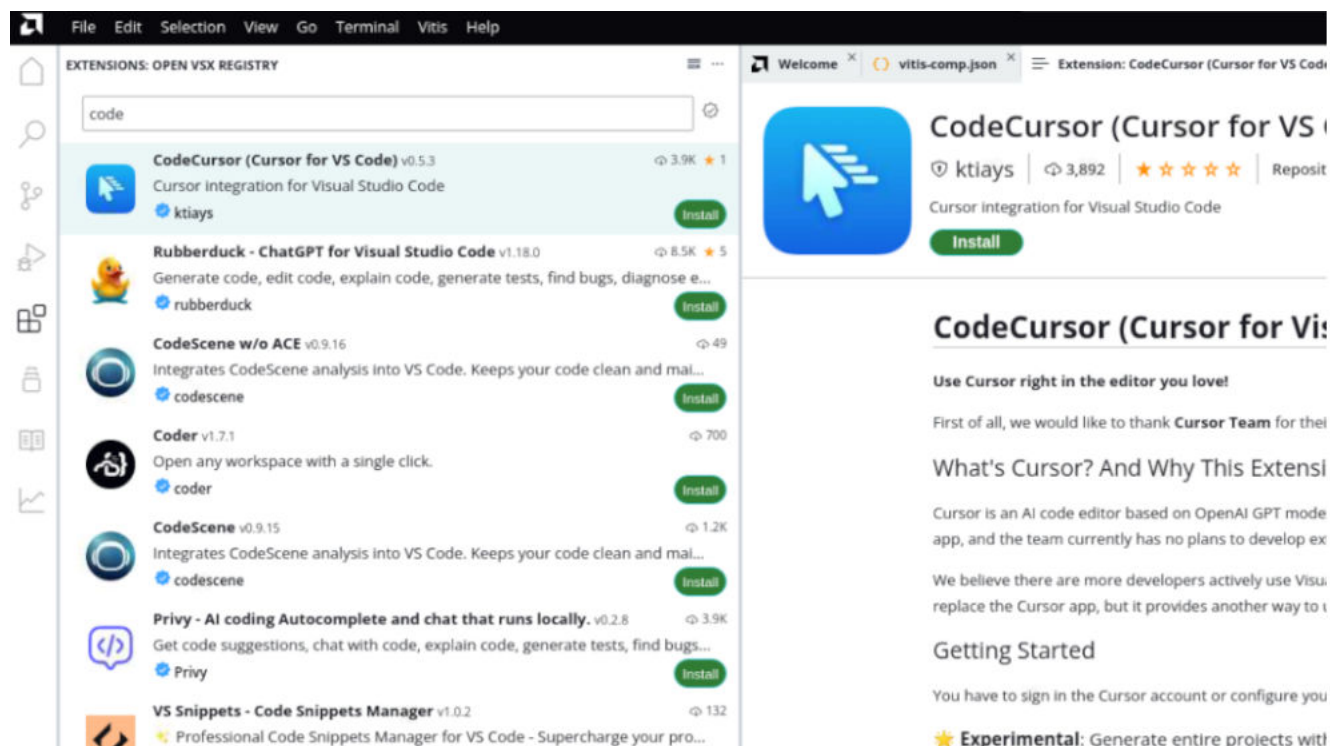
Lopper 是一款后端 Python 实用工具，供 AMD Vitis™ 用于从系统设备树抽取硬件元数据。虽然在 Vitis 安装中已包含 Lopper，但也可从 GitHub 获取。Vitis 提供了一个环境变量，以便在不修改 Vitis 安装的情况下启用外部 Lopper 的使用。要使用外部版本的 Lopper，请运行以下命令：

```
export VITIS_LOPPER_INSTALL_LOC="path/to/Lopper"
```

## 扩展

您可在 Vitis Unified IDE 中直接浏览和安装扩展。单击 Vitis Unified IDE 左侧边缘处专用的“Extensions”（扩展）图标即可访问“Extensions”视图。

图 21：扩展



列表中的每个扩展都有一段简要描述。您可选择扩展项以显示该扩展的详细信息。

**注释：**“Extension Marketplace”功能特性作为便捷的门户，可供您浏览和安装来自 [Open VSX](#) 的扩展。Open VSX 是 VS Code 扩展的开源仓库。[Eclipse Foundation](#) 提供了此扩展。对于任何扩展的质量或安全性，AMD 不开展任何筛查。



**重要提示！** 使用 Extension Marketplace 上的扩展即表示您理解并接受其中所涉及的风险。对于因使用此项功能特性而造成的任何损害、损失或不便，AMD 概不承担任何责任。在访问和使用任何扩展之前，请审慎考虑这些因素。

**注释：**这项预览功能特性目前尚处于抢先体验阶段。这意味着，该功能特性仍在开发阶段，可能不稳定。访问和使用此项功能特性即表示您承认并接受以下内容：

- 功能受限：与完全发布的功能特性相比，抢先体验功能特性的功能性可能受限。抢先体验功能特性可能缺乏某些功能、存在缺陷或出现性能问题。
- 潜在不稳定性：抢先体验功能特性仍在测试和完善中。因此，很容易出现崩溃、错误或意外行为。请谨慎使用这些功能特性。
- 反馈和改进：您的反馈对于改进抢先体验功能特性至关重要。AMD 鼓励您报告在使用这些功能特性过程中遇到的任何问题、缺陷或建议。您的意见建议能帮助 AMD 增强这些功能特性的性能和稳定性。
- 不作任何保证：抢先体验功能特性按原样提供，不含任何明示或暗示的担保或保证。AMD 并不保证会以当前形式发布这些功能特性，甚至无法保证会正式发布这些功能特性。AMD 保留修改、暂停或中止这些功能特性的权利，恕不另行通知。
- 使用风险自负：使用抢先体验功能特性即表示您了解并接受其中的风险。对于因使用这些功能特性而造成的任何损害、损失或不便，AMD 概不承担任何责任。



**重要提示！** 在访问和使用抢先体验功能特性之前，请审慎考虑这些因素。非常感谢您参与测试并提供反馈。

## Serial Monitor

Serial Monitor 扩展提供了一个串行监控器，用于查看输出以及向串行端口发送消息。在嵌入式器件上对程序进行测试或调试时，此扩展通常很有用。

您可从“Vitis 菜单”下选择“Serial Monitor”来打开此串行监控器。

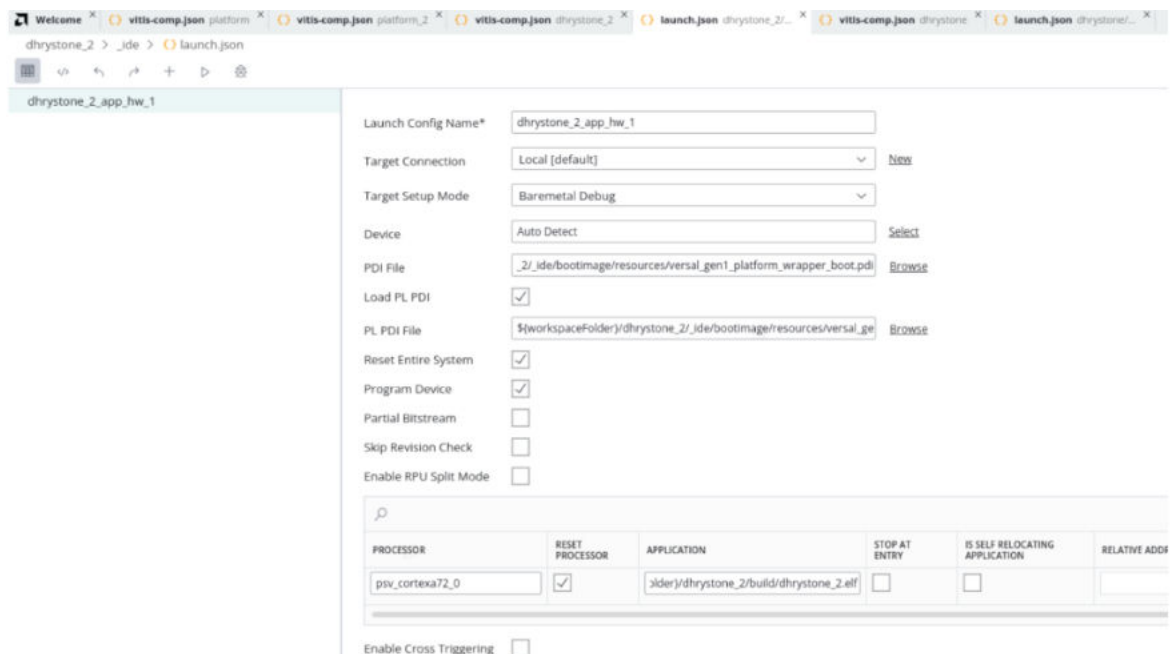
## 分段配置

“Segmented Configuration Flow”（分段配置流程）是一个 AMD Versal™ 器件开发流程，允许您分别下载或烧录启动 PDI（CIPS 和 NOC 初始化）组件和 PL PDI（FPGA 侧初始化镜像）组件。它有利于下列用途：

- 快速启动软件
- 最大程度降低本地启动闪存需求
- 通过运行时请求来加载 PL PDI 镜像

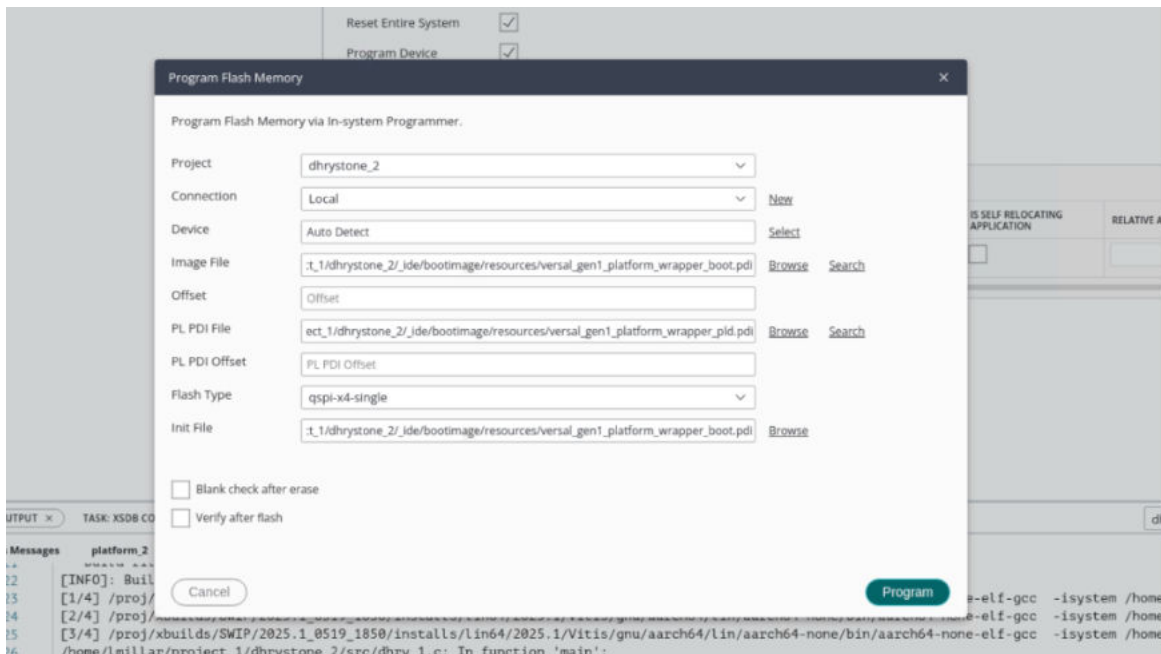
如果您用分段配置来配置自己的设计，并在编辑启动配置时选择 PL PDI，则此 GUI 功能特性允许您运行或调试独立应用程序。

图 22：选择 PL PDI



Program Flash 实用工具允许您分配 PL PDI 以将所有内容烧录到闪存。

图 23: Program Flash 实用工具



您可以按需选择烧录或下载 PL PDI。

**注释：**这项预览功能特性目前尚处于抢先体验阶段。这意味着，该功能特性仍在开发阶段，可能还不能完全发挥功能或保持稳定。访问和使用此项功能特性即表示您承认并接受以下内容：

- 功能受限：与完全发布的功能特性相比，此项功能特性的功能性可能受限。请注意，您使用此项功能特性时可能无法享受最佳体验。
- 潜在不稳定性：此项功能特性仍在测试和完善中。因此，可能容易出现崩溃、错误或意外行为。
- 反馈和改进：AMD 鼓励您报告在使用此项功能特性过程中遇到的任何问题、缺陷或建议。您的意见建议将帮助我们增强这些功能特性的性能和稳定性。
- 不作任何保证：此项功能特性按原样提供，不含任何明示或暗含的担保或保证。AMD 不保证该功能特性会以当前形式发布。我们保留修改、暂停或中止此项功能特性的权利，恕不另行通知。
- 使用风险自负：使用此项功能特性即表示您理解并接受其中所涉及的风险。对于因使用此项功能特性而造成的任何损害、损失或不便，AMD 概不承担任何责任。

## 首选项

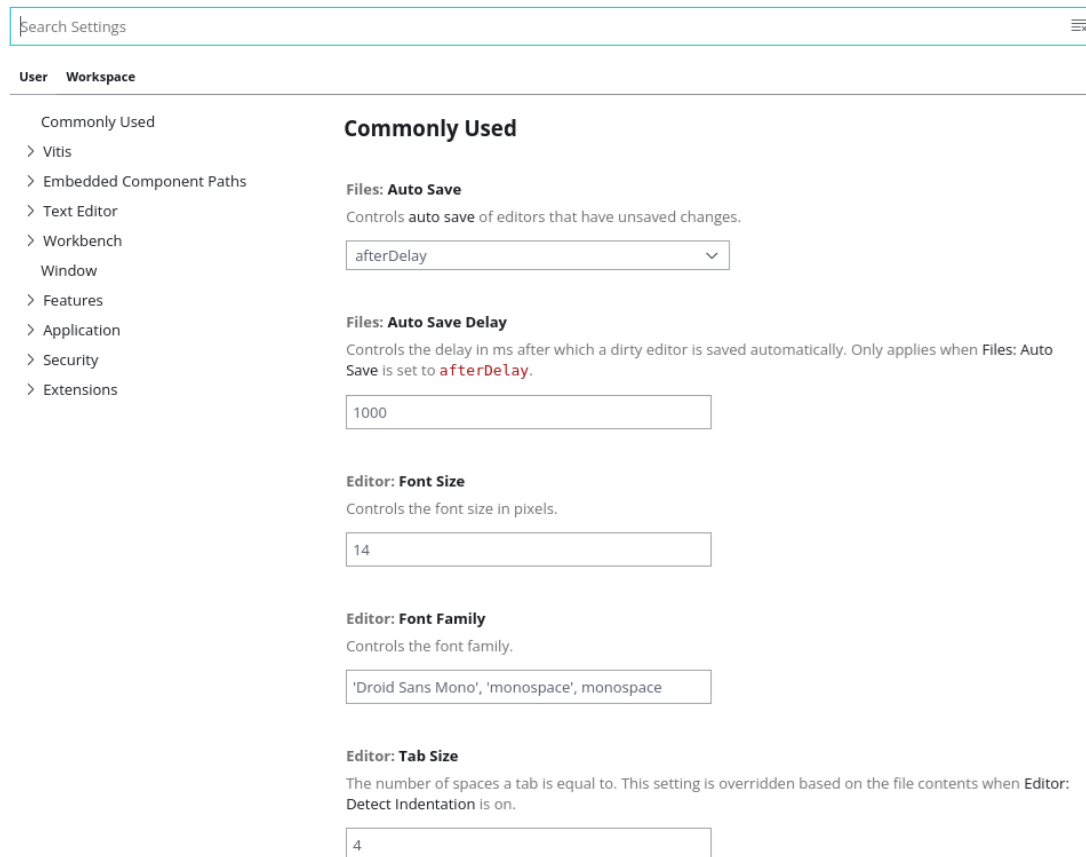
要配置 Vitis Unified IDE 的首选项，请导航到“File” → “Preferences”（文件 > 首选项）。

### 设置

- “Auto Save”（自动保存）：将更改自动保存到配置文件、源代码、CMakeLists.txt 文件等。默认开启此设置。您还可以定义“Auto Save”（自动保存）触发器和“Auto Save Delay”（自动保存延迟）值。
- “Color Themes”（颜色主题）：根据首选项，为 Vitis Unified IDE 指定“Light Theme”（亮色主题）或“Dark Theme”（暗色主题）。

- “Open Settings (UI)”（打开设置 (UI)）：打开 “Preferences”（首选项）视图，显示用于配置您的设计环境的选项。

图 24：常用首选项



- “Open Keyboard Shortcuts”（打开键盘快捷键）：为各种命令定义键盘快捷键，如 [键盘快捷键](#)、[命令选用板](#)和[快速查找](#) 中所述。
- “File Icon Theme”（文件图标主题）：为您的环境指定文件图标主题。

**注释：**您可按 “Ctrl + +” 放大显示范围，或按 “Ctrl - -” 缩小显示范围。

## 键盘快捷键、命令选用板和快速查找

### 键盘快捷键

您可从 “File” → “Preferences” → “Open Keyboard Shortcuts”（文件 > 首选项 > 打开键盘快捷键）复查和编辑键盘快捷键。另一种方法是使用快捷键 “Alt + Ctrl + 逗号”。例如：

1. 使用 “Alt + Ctrl + 逗号” 快捷键打开 “Keyboard Shortcuts”（键盘快捷键）窗口。
2. 在搜索栏中输入 `build`。
3. “Build: Hardware”（构建：硬件）的键绑定为 “Alt+Shift+BH”。

4. 要编辑用于构建硬件的键盘快捷键，请将鼠标悬停在“Build: Hardware”（构建：硬件）行上。单击左侧的“Edit Keybinding”（编辑键绑定）图标，并输入您首选的键绑定。

### 命令选用板

命令选用板允许您快速查找和执行命令，而无需牢记键盘快捷键或菜单位置。例如，要使用命令选用板运行构建硬件，请执行以下任务。

1. 在工作空间内选中您的组件或工程。
2. 输入“Ctrl + Shift + P”打开“Command Palette”（命令选用板）菜单。
3. 输入关键字 `build`、`run` 或 `debug`。例如，“Command Palette”（命令选用板）用于筛选与您输入的命令匹配的命令名称。
4. 使用鼠标或上下方向键在命令列表中滚动浏览。选中您要执行的命令。
5. 按“Enter”键执行所选命令。

例如，在“Component Explorer”（组件资源管理器）中选择一个 HLS 组件并按“Ctrl + Shift + P”，然后在“Command Palette”（命令选用板）中输入“C Syn”。“HLS: C Synthesis”（HLS: C 语言综合）是显示的选项之一。选中该选项并按 Enter 键。这样该工具就会在所选 HLS 组件上运行 C 语言综合。

### 快速查找

单击“Ctrl + P”打开“Quick Find”（快速查找）框。输入文件名，该工具就会开始查找与您的搜索字符串相匹配的文件。选择一个文件并按“Enter”键将其打开。

打开文件后，在“Quick Find”（快速查找）框中输入“@”，即可转到已打开代码中的某个函数。例如，您也可以输入 `:40` 以转至该文件的第 40 行。您可使用此方法快速查找并跳转至某个函数或行号。

打开文件后，您可在“Quick Find”框中输入“#”，以转到工作空间内的某个函数。您可使用此方法快速查找并跳转至某个函数。

按“Ctrl + T”打开“Quick Find”框。输入符号名称，该工具就会开始查找与您的搜索字符串相匹配的符号。此外，它还会搜索这些文件。

# Vitis Unified IDE 中的 AI 辅助

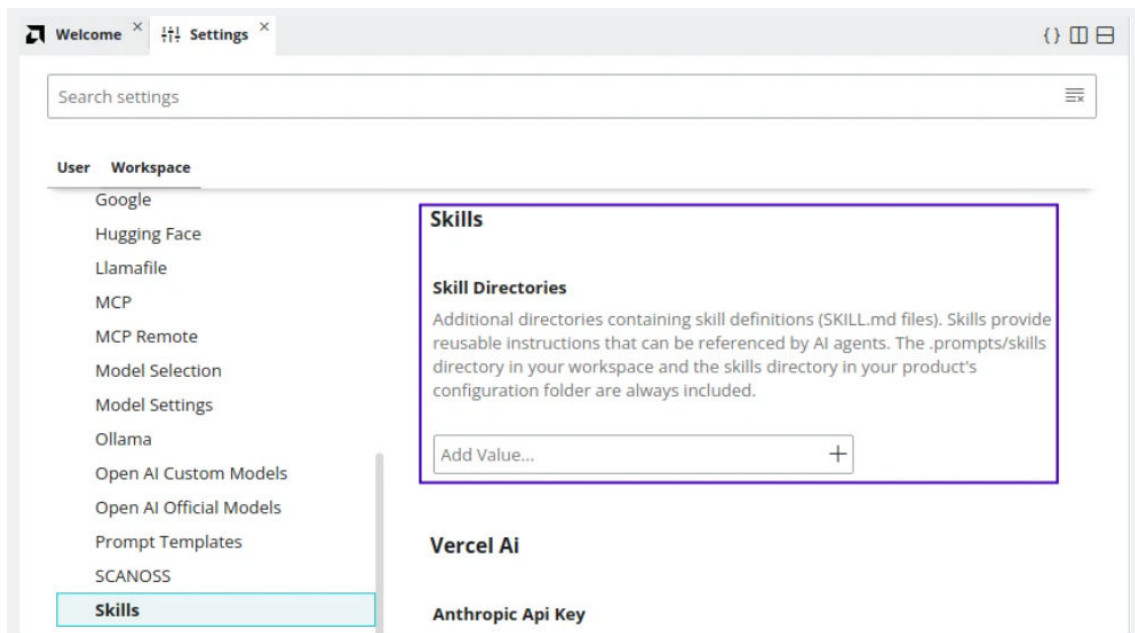
Vitis Unified IDE 包含用于 AI 辅助工作流程的 AI 聊天。AI 聊天在 IDE 内提供了一个对话式入口点，使您无需离开开发环境即可获取指导。

您可以使用 AI 聊天执行以下任务。

1. 提出文档相关问题。
2. 理解源代码和 IDE 生成的输出。
3. 对构建错误进行故障排除。
4. 获取有关工程配置更改的指导。
5. 加快从 Vitis Classic IDE 移植到 Vitis Unified IDE 的初步评估。

AI 聊天仅提供指导。它不能替代官方文档、既定设计实践或用户确认。

图 25: AI 聊天设置



要开始使用 AI 聊天：

1. 打开“Settings”（设置）。
2. 在“Settings”搜索字段中输入 AI。
3. 在筛选后的“Settings”结果中，打开与 AI 相关的设置部分。
4. 输入提供商 API 密钥。

- 5. 如果您的构建中提供了这些选项，请选择提供商和模型。
- 6. 打开“View > AI Chat”（视图 > AI 聊天）。

图 26：Anthropic API 密钥

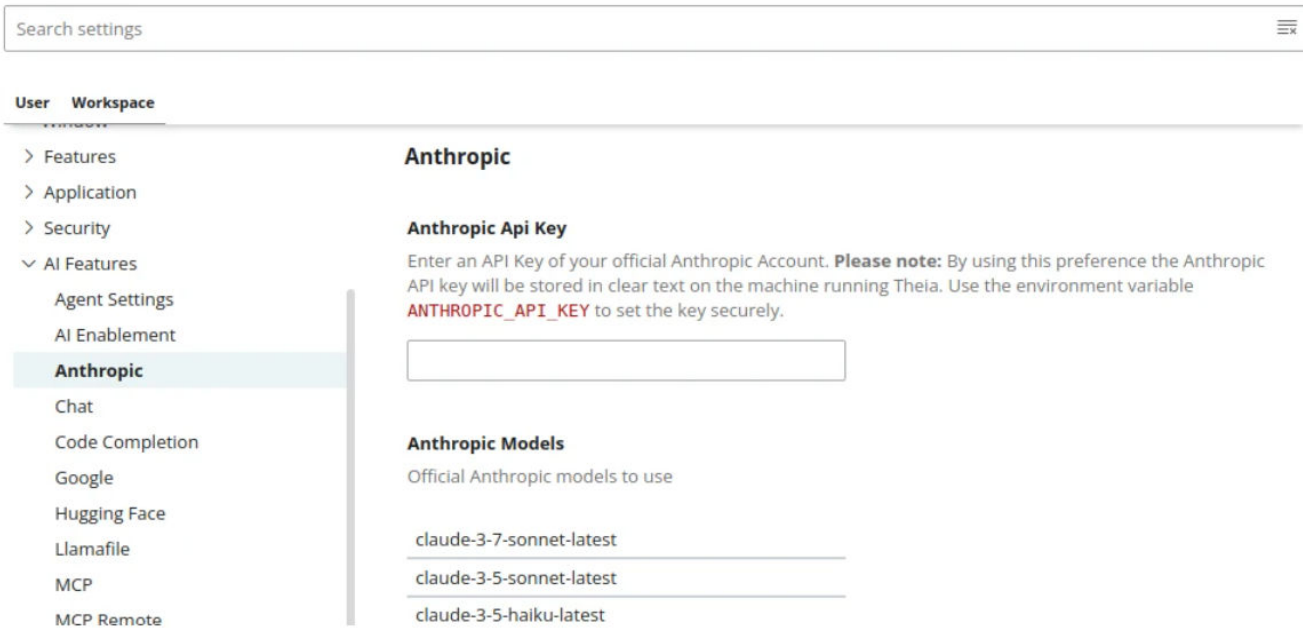
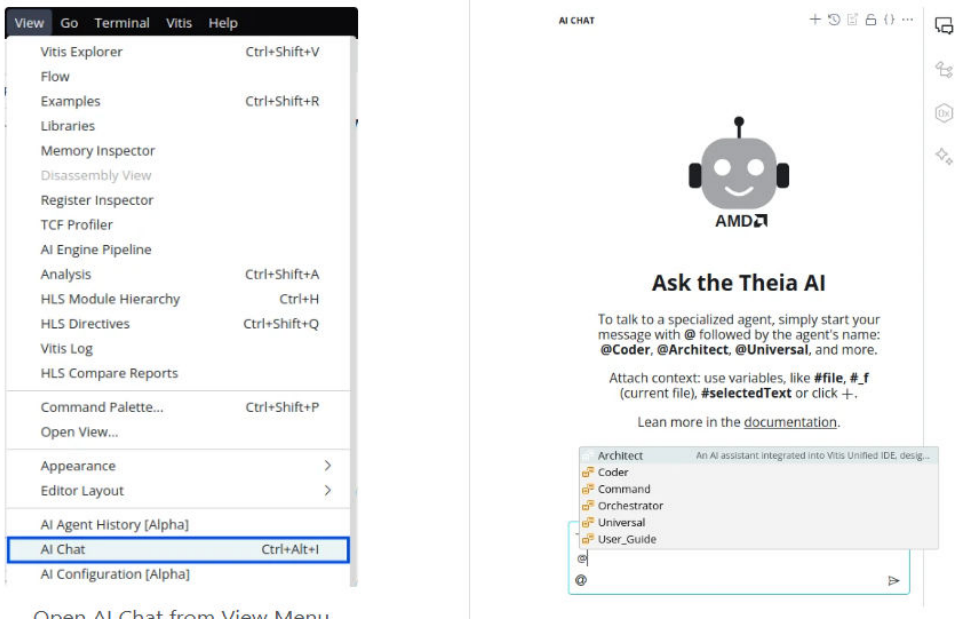


Figure 26: AI Model API Key

图 27：AI 聊天



如果您的构建提供了多个助手，请在 AI 聊天中输入 @ 以查看可用选项，并选择一个符合您任务的助手。

在使用任何 AI 生成的结果之前：

1. 检查响应。
2. 确认该建议符合您的设计意图。
3. 在应用文件更改之前先进行验证。
4. 根据需要重新构建、重新运行或重新测试。

AMD 不托管第三方 AI 服务或模型，也不会对其进行确认。第三方提供商负责模型行为、响应质量、账户访问、计费和服务正常运行时间。在使用任何 AI 提供的信息之前，您应负责数据共享决策、合规性要求以及对所有 AI 提供信息的确认。

---

## AI 辅助的用例

以下步骤是实用的 AI 辅助用例。

### 用例：诊断构建失败

1. 在“Flow Navigator”中，构建您的组件。
2. 构建完成后，在底部面板中打开“Output”（输出）视图。
3. 找到第一条有关失败的错误消息。
4. 复制错误行及其周围的上下文行。
5. 单击“AI Chat”（AI 聊天）选项卡。
6. 将错误块粘贴到消息字段中。键入如下提示词：

```
Analyze this Vitis build error. Give root cause in one sentence, then propose minimal edits and list each changed line.
```

7. 检查响应。
8. 对于每项建议的编辑操作，请打开相关文件并查看建议的更改。
9. 仅应用您批准的编辑操作。
10. 保存文件并重新构建该组件。
11. 确认原先的错误不再出现。

### 用例：更新工程配置

1. 在编辑器中打开目标配置文件。
2. 在“AI Chat”中，键入特定的编辑请求，例如：

```
Update launch.json to point program to the current ELF under the workspace and keep existing debug arguments unchanged.
```

3. 对于每项建议的编辑操作，请打开配置文件并验证更改是否与建议一致。
4. 如果建议内容表述不清晰，请让 AI Chat 解释已更改的每一行。
5. 应用已批准的编辑操作。保存此文件。
6. 重新构建或重新启动以确认更改。

**用例：解释 IDE 生成的报告**

1. 运行构建、分析或调试操作，以生成报告。
2. 打开相关的报告视图或输出面板。
3. 如果报告过于冗长或不清晰，请复制相关的小节。
4. 单击“AI Chat”（AI 聊天）选项卡。
5. 将报告内容粘贴到消息字段中。键入如下提示词：

```
Explain this build report. What do the warnings mean and what should I  
fix first?
```

6. 查看解释和建议的后续步骤。
7. 打开相关源文件并应用修复。

# 开发

本章节旨在描述您如何使用 AMD Vitis™ 集成设计环境 (IDE) 来创建和管理目标平台和应用。

## 管理平台 and 平台存储库

“Platform Repositories”（平台存储库）窗口显示了可供 Vitis Unified IDE 使用的平台。上半部分表格显示了软件安装中包含的基础平台，这些平台可供所有用户使用。下半部分表格显示了 `$PLATFORM_REPO_PATHS` 环境变量包含的指定位置。下半部分表格显示了除 Vitis 工具外，另行安装的平台。

您可转至已打开的工程的主菜单中的“Vitis” → “Platform Repositories”（Vitis > 平台存储库），来管理可供使用的平台。

1. 单击“+”或“-”图标即可在列表中添加或移除平台搜索目录。
2. 选中平台目录即可查看该目录的平台。
3. 选中平台旁的“info”（信息）链接即可查看其详细信息。

图 28：平台存储库

| Platform Repositories                   |           |                |            |                                                          |
|-----------------------------------------|-----------|----------------|------------|----------------------------------------------------------|
| NAME                                    | BOARD     | FLOW           | VENDOR     | PATH                                                     |
| ▼ vck190_platform (1)                   |           |                |            | ...u102_workspace\vck190_platform\export\vck190_platform |
| ✓ vck190_platform                       | vck190    | Embedded       | xilinx.com | ...platform\export\vck190_platform\vck190_platform.xpfm  |
| ▼ zcu102_edt (1)                        |           |                |            | ...edt\edt_zcu102_workspace\zcu102_edt\export\zcu102_edt |
| ✓ zcu102_edt                            | zcu102    | Embedded       | xilinx.com | ...orkspace\zcu102_edt\export\zcu102_edt\zcu102_edt.xpfm |
| ▼ base_platforms (5)                    |           |                |            | ..._0429_1858\installs\lin64\2025.1\Vitis\base_platforms |
| ✓ xilinx_vek280_base_202510_1           | vek280    | Embedded Accel | xilinx.com | ...vek280_base_202510_1\xilinx_vek280_base_202510_1.xpfm |
| ✓ xilinx_vmk180_base_202510_1           | vmk180    | Embedded Accel | xilinx.com | ...vmk180_base_202510_1\xilinx_vmk180_base_202510_1.xpfm |
| ✓ xilinx_vck190_base_202510_1           | vck190    | Embedded Accel | xilinx.com | ...vck190_base_202510_1\xilinx_vck190_base_202510_1.xpfm |
| ✓ xilinx_vck190_base_dfx_202510_1       | xd        | Embedded Accel | xilinx.com | ...ase_dfx_202510_1\xilinx_vck190_base_dfx_202510_1.xpfm |
| ✓ xilinx_zcu104_base_202510_1           | xd        | Embedded Accel | xilinx.com | ...zcu104_base_202510_1\xilinx_zcu104_base_202510_1.xpfm |
| ▼ internal_platforms (68)               |           |                |            | ..._0j\rdi\builds\2025.1_daily_latest\internal_platforms |
| ✓ xilinx_vek280_base_bdc_202510_1       | vek280    | Embedded Accel | xilinx.com | ...ase_bdc_202510_1\xilinx_vek280_base_bdc_202510_1.xpfm |
| ✗ xilinx_samsung_u2x4_202010_1          | samsung   | Data Center    | xilinx     | ...msung_u2x4_202010_1\xilinx_samsung_u2x4_202010_1.xpfm |
| ✗ xilinx_u55n_gen3x4_xdma_2_202110_1    | u55n      | Data Center    | xilinx     | ...ma_2_202110_1\xilinx_u55n_gen3x4_xdma_2_202110_1.xpfm |
| ✓ xilinx_vck190_base_bdc_202420_1       | xd        | Embedded Accel | xilinx.com | ...ase_bdc_202420_1\xilinx_vck190_base_bdc_202420_1.xpfm |
| ✗ xilinx_u250_gen3x16_xdma_3_1_202020_1 | u250      | Data Center    | xilinx     | ...1_202020_1\xilinx_u250_gen3x16_xdma_3_1_202020_1.xpfm |
| ✓ xilinx_vck190_base_dfx_202420_1       | xd        | Embedded Accel | xilinx.com | ...ase_dfx_202420_1\xilinx_vck190_base_dfx_202420_1.xpfm |
| ✓ xilinx_vck190_base_dfx_202510_1       | xd        | Embedded Accel | xilinx.com | ...ase_dfx_202510_1\xilinx_vck190_base_dfx_202510_1.xpfm |
| ✗ xilinx_u30_gen3x4_1_202020_1          | u30       | Data Center    | xilinx     | ...0_gen3x4_1_202020_1\xilinx_u30_gen3x4_1_202020_1.xpfm |
| ✓ xilinx_vek280_base_bdc_202420_1       | vek280_es | Embedded Accel | xilinx.com | ...ase_bdc_202420_1\xilinx_vek280_base_bdc_202420_1.xpfm |
| ✗ xilinx_u250_gen3x16_xdma_4_1_202210_1 | u250      | Data Center    | xilinx     | ...1_202210_1\xilinx_u250_gen3x16_xdma_4_1_202210_1.xpfm |
| ✗ xilinx_vck5000_gen4x8_qdma_2_202220_1 | vck5000   | Data Center    | xilinx     | ...2_202220_1\xilinx_vck5000_gen4x8_qdma_2_202220_1.xpfm |
| ✗ xilinx_vck5000_gen4x8_xdma_1_202120_1 | vck5000   | Data Center    | xilinx     | ...1_202120_1\xilinx_vck5000_gen4x8_xdma_1_202120_1.xpfm |
| ✗ xilinx_u50_gen3x4_xdma_2_202010_1     | u50       | Data Center    | xilinx     | ...dma_2_202010_1\xilinx_u50_gen3x4_xdma_2_202010_1.xpfm |
| ✓ xilinx_zcu102_base_202420_1           | xd        | Embedded Accel | xilinx.com | ...zcu102_base_202420_1\xilinx_zcu102_base_202420_1.xpfm |
| ✗ xilinx_u50lv_gen3x4_xdma_2_202010_1   | u50lv     | Data Center    | xilinx     | ...a_2_202010_1\xilinx_u50lv_gen3x4_xdma_2_202010_1.xpfm |
| ✓ xilinx_vmk180_base_bdc_202420_1       | xd        | Embedded Accel | xilinx.com | ...ase_bdc_202420_1\xilinx_vmk180_base_bdc_202420_1.xpfm |
| ✗ advantech_vega-4002_xdma_201830_2     | vega-4002 | Data Center    | advantech  | ..._xdma_201830_2\advantech_vega-4002_xdma_201830_2.xpfm |
| ✗ xilinx_u30_gen3x4_2_202020_1          | u30       | Data Center    | xilinx     | ...0_gen3x4_2_202020_1\xilinx_u30_gen3x4_2_202020_1.xpfm |
| ✗ xilinx_u380_gen3x16_xdma_1_202130_1   | u380      | Data Center    | xilinx     | ...a_1_202130_1\xilinx_u380_gen3x16_xdma_1_202130_1.xpfm |

## 目标平台

在 Vitis 统一软件平台中，应用的运行时环境即目标平台。目标平台由硬件组件 (XSA) 和软件组件（域、FSBL 或 PLM 之类的启动组件等）组合而成。

平台工程是工作空间内的可自定义目标平台。您可添加、修改或删除平台工程中的域。您也可以启用、禁用和修改启动组件。域是指板级支持包 (BSP) 或操作系统 (OS)，其目标为单一处理器或同构处理器组成的集群（例如，含 SMP Linux 的 4x Cortex®-A53 集群）。每个平台可包含的域数量无限。

本章节旨在解释如何创建硬件设计，以及如何使用该硬件设计来创建应用平台。

## 创建硬件设计（XSA 文件）

AMD 硬件设计是使用 AMD Vivado™ Design Suite 创建的，可以赛灵思支持存档 (XSA) 专属文件格式导出，以供 Vitis 软件平台使用。如需了解有关如何在 Vivado 中创建嵌入式设计以及如何生成 XSA 文件的信息，请参阅以下嵌入式设计教程：

- 《Zynq 7000 SoC：嵌入式设计教程》([UG1165](#))
- 《Zynq UltraScale+ MPSoC：嵌入式设计教程》([UG1209](#))
- 《Versal 自适应 SoC 嵌入式设计教程》([UG1305](#))

常规步骤如下所示：

1. 创建 Vivado 工程。
2. 创建块设计。
3. 生成镜像或比特流。
4. 使用“File” → “Export” → “Export Hardware”（文件 > 导出 > 导出硬件）以导出硬件，然后选择“Fixed Platform”（固定平台）选项。

## 基于 XSA 创建平台组件

要在 Vitis Unified IDE 内创建新的平台组件，请执行以下步骤。

1. 单击 Vitis Unified IDE 中的“File”（文件）选项，然后选中“New Component” → “Platform”（新建组件 > 平台）。



**提示：**您也可以从“Welcome”（欢迎）页面选择“Create Platform Component”（创建平台组件）命令。

2. 这样即可打开“Create Platform Component” Wizard（创建平台组件向导）。
  - 输入组件名称和组件位置，然后单击“Next”（下一步）。
  - 选择“Browse”（浏览）查找 XSA 文件。您也可以选择基于现有平台来创建平台，或者使用下拉菜单选择内置 XSA 文件。内置固定 XSA 文件仅包含 PS 初始化。单击“Next”。

图 29：创建平台组件

Create Platform Component

Name and Location > Flow > OS and Processor > Summary

**Select Platform Creation Flow**

Create a platform component by selecting the hardware design and add software domains.

☒ Hardware Design ☐ Existing Platform

Hardware Design (XSA) For Implementation:  [Browse](#)

Emulation

Hardware Design (XSA) For Emulation:  [Browse](#)

Advanced Options

SDT Source Repo:  [Browse](#)

Board DTSI:

User DTSI:  [Browse](#)

DT ZOCL: ☐

[Cancel](#) [Back](#) [Next](#)

**注释：**如果您选择基于现有平台来创建平台，那么它会将您指定的平台复制到您的当前工作空间内。

**注释：**如果您要支持硬件仿真，请展开“Emulation”，并选中“Browse”（浏览）以查找仿真 XSA 文件。

**注释：**输入的硬件设计类型决定了您创建的平台类型。如果输入硬件设计是固定 XSA，那么系统会为嵌入式设计创建平台。如果输入硬件设计是可扩展 XSA，那么系统会创建可扩展平台。

SDTgen 是 Vitis Unified IDE 的内置工具。

- SDTgen 工具负责读取 XSA 文件并生成 SDT（系统设备树）文件。
- 如需了解有关 SDT 的更多信息，请参阅[系统设备树](#)。
- 另一个内置工具 Lopper 会解析此 SDT 并生成对应的 BSP（板级支持包）文件。
- 为方便用户调试并发挥 SDT 流程的优势，高级选项允许您切换到自定义版本的 SDTgen 工具。高级选项的使用如下：

- “SDT Source REPO”（SDT 源码仓库）：输入客户版本的 SDTgen REPO。

**注释：**如果您遇到与系统设备树或 SDTgen 工具相关的问题，请使用更新版本的 SDTgen 工具进行调试。如需了解有关系统设备树的更多信息，请参阅[系统设备树仓库](#)。

- “Board DTS”（开发板 DTS）：输入开发板名称，以便获取对应的开发板 DTS 文件。如需了解更多信息，请参阅《PetaLinux 工具文档：参考指南》([UG1144](#)) 以获取对应的开发板名称。

- “User DTS”（用户 DTS）：

指定一个或多个自定义 .dtsi 文件或 .dts 文件，以将这些文件包含在该工具根据 XSA 生成的系统设备树 (SDT) 中。此选项同时支持 Linux 和独立平台创建。尽管此选项与 SDTgen、开发板 DTS 和 DT 叠加层选项归为一组，但它并不限于 Linux 流程。

对于独立 BSP 平台创建，Lopper 在 BSP 生成期间会使用生成的 SDT，因此您提供的 DTSI 内容可能影响 BSP 元数据和驱动程序配置。SDTgen 不会自动生成开发板专用节点或外设专用节点，使用“User DTS”（用户 DTS）添加这些节点，或者覆盖已生成的 SDT 内容。

例如，如果某个外设是在块设计之外例化的，SDTgen 可能不会将其输出到生成的 SDT 中。在这种情况下，请提供描述该外设的自定义 DTSI 文件，以使最终的 SDT 包含该外设节点，并使独立 BSP 和 Lopper 流能够将该节点与正确的驱动程序关联起来。该节点必须与硬件设计匹配，并包含关联驱动程序所需的属性，例如 `compatible`、地址范围、中断、时钟和复位（如适用）。

- 选择 XSA 后，该工具会读取 XSA 并识别可用的处理器和操作系统域。为平台指定“Operating system”（操作系统）和“Processor”（处理器），单击“Next”以继续转至“Summary”（汇总）页面。

Create Platform Component

Name and Location > Flow > OS and Processor > Summary

Select Operating System and Processor

Specify the details for the initial domain to be added to the platform component. The platform can be modified later to add new domains or change settings by opening the platform editor.

Operating system: standalone

Processor: psu\_cortexa53\_0

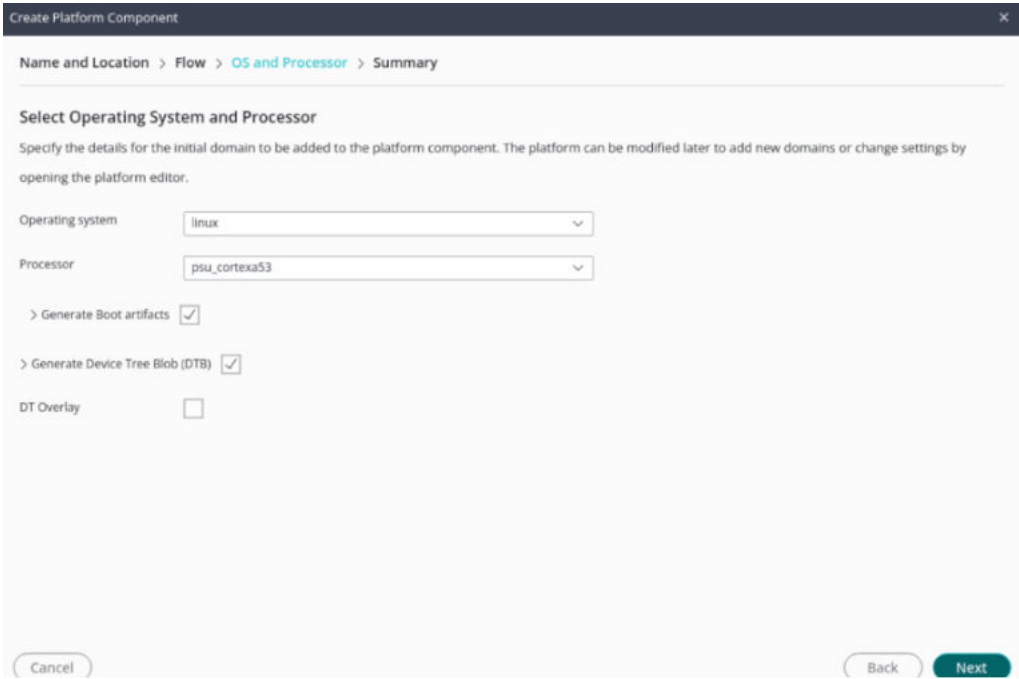
Architecture: 64-bit

Generate Boot artifacts: ☒

Generate PMU Firmware: ☐

Target processor to create FSBL: ☒ psu\_cortexa53\_0 ☐ psu\_cortexr5\_0

Cancel Back Next



**注释：**启用“Generate Boot Artifacts”（生成启动工件）选项和“Generate PMU Firmware”（生成 PMU 固件）选项：

- 当域设为“Standalone”（独立）时，Vitis 工具会自动生成您的平台的 FSBL（第一阶段启动加载程序）和 PMU（平台管理单元）固件组件。
- “Generate Boot Artifacts”选项对于 Versal 自适应 SoC 不可用。Versal 自适应 SoC 只需要 PDI 文件。

**注释：**启用“Generate Device Tree Blob”（生成设备树二进制对象 (DTB)）选项时，该系统会自动为您的平台生成“Device Tree Blob”（设备树二进制对象）。仅当域设为 Linux 时，该选项才适用。

**注释：**“DT Overlay”（DT 叠加层）仅适用于可扩展平台或由 V++ 工具生成的固定 XSA，用于生成设备树二进制对象 (dtb) 的叠加层。

如需查看选项的各项设置，请参阅下表。

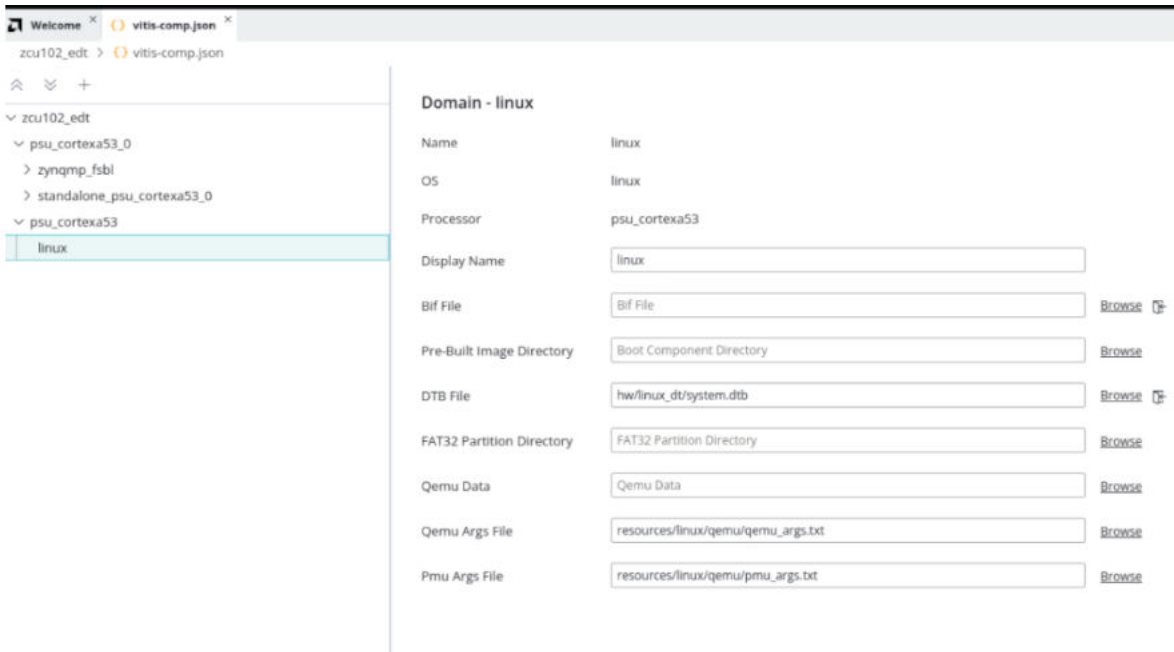
表 7：不同器件家族的选项设置

| 器件家族   | 生成启动工件           | PMU                                 | DTB                | DTB 叠加层                                       |
|--------|------------------|-------------------------------------|--------------------|-----------------------------------------------|
| Zynq   | 启用 FSBL 用于器件初始化。 | 无选项                                 | 只有 Linux 系统才需要 DTB | 系统正常启动并运行后加载某些器件时，只有 Linux 系统才需要使用 DTB 叠加层文件。 |
| ZynqMP | 启用 FSBL 用于器件初始化。 | 独立域并不需要 PMU，但 Linux 系统需要它来管理高安全性功能。 | 只有 Linux 系统才需要 DTB | 系统正常启动并运行后加载某些器件时，只有 Linux 系统才需要使用 DTB 叠加层文件。 |
| Versal | 无选项              | 无选项                                 | 只有 Linux 系统才需要 DTB | 系统正常启动并运行后加载某些器件时，只有 Linux 系统才需要使用 DTB 叠加层文件。 |

- “Summary”页面反映了您在前几个页面上的选择。请复查汇总信息并单击“Finish”（完成）以创建平台组件，或者单击“Back”（返回）以返回至先前页面并更改您的选择。

创建平台组件后，会在中央编辑器窗口中打开 `vitis-comp.json` 文件。

图 30: 域 - linux



根据您为自己的平台所选的操作系统以及您可能已选的处理器，vitis-comp.json 平台的内容可能不尽相同。

- 对于 Linux 操作系统，请指定“BIF file”（BIF 文件）、“Boot Component Directory”（启动组件目录）、“SD Card Directory”（SD 卡目录）及“QEMU Data”（QEMU 数据）。“QEMU Args”（QEMU 实参）文件会自动填充。
- 对于独立（裸机）操作系统：系统无需执行特殊操作。
- 对于 FreeRTOS：系统无需执行特殊操作。

确认设置后，您可在“Flow Navigator”中选中平台组件并选中“Build”命令来构建该组件。完成编译后，Vitis 工具会在平台组件的 Output 文件夹中填充该平台及其相关软硬件组件。

请注意以下针对第三方工具链的编译器支持：

- ARMClang 支持 Versal A72 和 R5 处理器
- ARMCC 支持 Zynq A9 处理器
- IAR 支持 Zynq MP R5 处理器和 Zynq A9 处理器



**重要提示！** Vitis Unified IDE 的安装并不包括这些编译器。因此，您需要自行安装这些编译器，并将其添加到 \$PATH 环境变量中。以下是为 vck190 创建平台时选择“ARMCLANG”编译器的示例。

图 31：操作系统和处理器摘要

Create Platform Component

Name and Location > Flow > OS and Processor > Summary

Select Operating System and Processor

Specify the details for the initial domain to be added to the platform component. The platform can be modified later to add new domains or change settings by opening the platform editor.

Operating system: standalone

Processor: psv\_cortexa72\_0

Compiler: armclang

The armclang settings must be sourced before launching the Vitis Unified IDE to properly use the armclang compiler.

Cancel Back Next

如果使用 CLI 流程，那么命令如下所示：

```
platform = client.create_platform_component(name = "platform", hw_design =
"vck190", os = "standalone", cpu = "psv_cortexa72_0", domain_name =
"standalone_psv_cortexa72_0", generate_dtb = False, advanced_options =
advanced_options, compiler = "armclang")
```

要创建一个独立平台并在其中通过“User DTS”选项包含自定义外设 DTSI，请在高级选项字典中传递 DTSI 文件，如下示例所示：

```
adv = client.create_advanced_options_dict(
    user_dtsi="/path/to/custom_peripheral.dtsi"
)

platform = client.create_platform_component(
    name="platform_standalone",
    hw_design="/path/to/design.xsa",
    os="standalone",
    cpu="psu_cortexa53_0",
    domain_name="standalone_psu_cortexa53_0",
    generate_dtb=False,
    advanced_options=adv,
)

platform.build()
```



**提示：** 打开工作空间 journal 日志以查看 Vitis Unified IDE 中执行的相应 CLI 命令。

**注释：** 所有编译器的流程都相同。

## 自定义预构建平台

仅当平台包含在工作空间内时，才可编辑。要自定义预构建平台，请将其导入工作空间。

1. 确保您要自定义的平台显示在平台存储库列表中。如未显示，请将平台路径添加到平台存储库中。
2. 使用以下任一方法启动“New Component” Wizard（新建组件向导）：
  - a. 选中“File” → “New Component” → “Platform”（文件 > 新建组件 > 平台）。
  - b. 在“Welcome”（欢迎）页面中，单击“Create Platform Component”（创建平台组件）以打开“New Platform Component” Wizard（新建平台组件向导）。
3. 在“New Platform Component” Wizard 中，在“Component Name”（组件名称）和“Component Location”（组件位置）名称字段中提供组件名称和组件位置。
4. 单击“Next”。
5. 在“Platform Component”（平台组件）页面中，选中“Existing platform”（现有平台）。选择所需平台，然后单击“Next”（下一步）复查平台组件汇总信息。

**注释：**确保要自定义的平台显示在平台存储库列表中。如未显示，请单击“+”将平台路径添加到平台存储库中。

6. 单击“Finish”（完成）。现在，您可以像任何其他平台一样，在工作空间内修改此新平台。

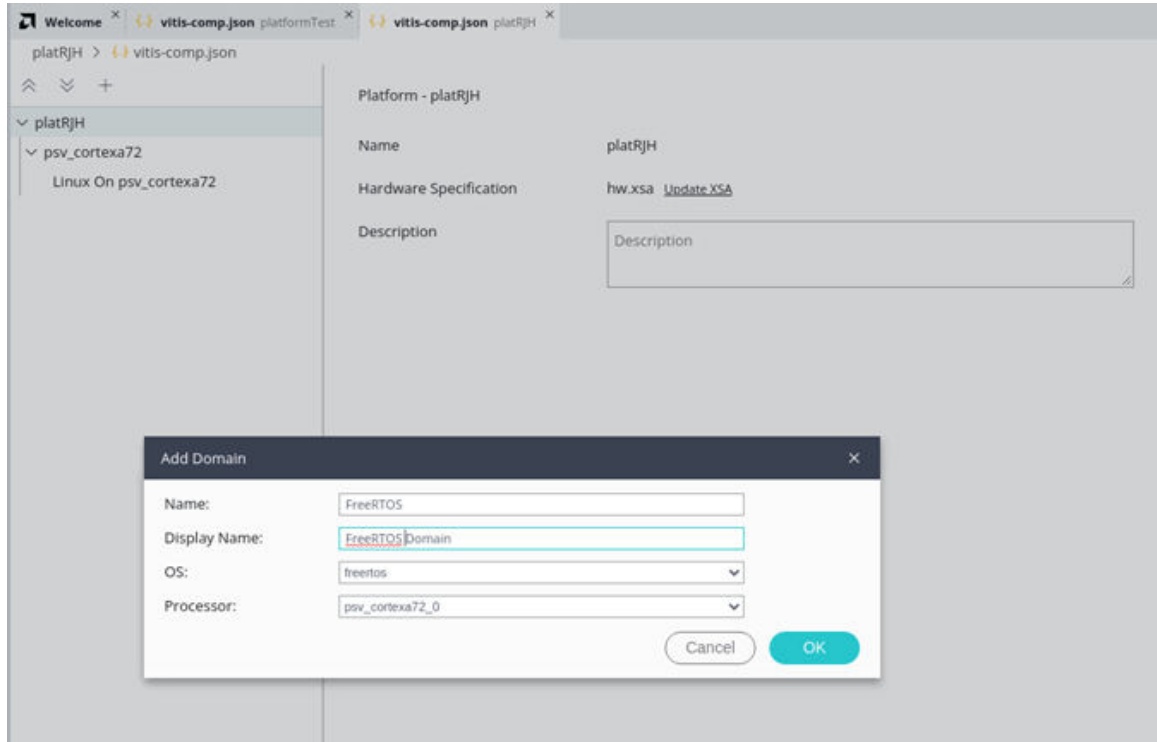
## 为现有平台添加域

每个平台均可包含多个域，以支持不同的操作系统并以不同的处理器为目标。Vitis 工具支持 Linux、FreeRTOS 与独立（或裸机）。

1. 在当前工作空间内，展开“Component Explorer”（组件资源管理器）下的平台组件以打开“Settings”（设置）文件夹并选中 `vitis-comp.json` 文件。

**注释：**要创建平台组件，请参阅 [基于 XSA 创建平台组件](#)。

2. 单击“+”按钮以“Add Domain”（添加域）。



3. 指定“Name”（名称）和“Display name”（显示名称）。
4. 对于“OS”（操作系统），请选中“Linux”、“FreeRTOS”或“Standalone”（独立）。
5. 从可用处理器中选择所需处理器。处理器选项随所选操作系统而变。对于支持多种架构的处理器，在创建域时请选择所需的处理器架构（32 位或 64 位）。
6. 单击“OK”（确定），将域添加到平台。

对于 FreeRTOS 域和 Standalone 域，会为其创建板级支持包 (BSP)。您可指定要在 BSP 中包含的库。

对于 Linux 域，您可从平台中选择新的 Linux 域来配置其附加详细信息。“Boot Components Directory”（启动组件目录）必须包含 BIF 所需的所有组件。PetaLinux 可以生成这些组件。



**提示：**生成平台时，会把 Linux 域设置中的组件复制到平台文件夹内。Windows 不支持复制符号链接，因此您无法向 Linux 域添加 sysroot。

## 配置域

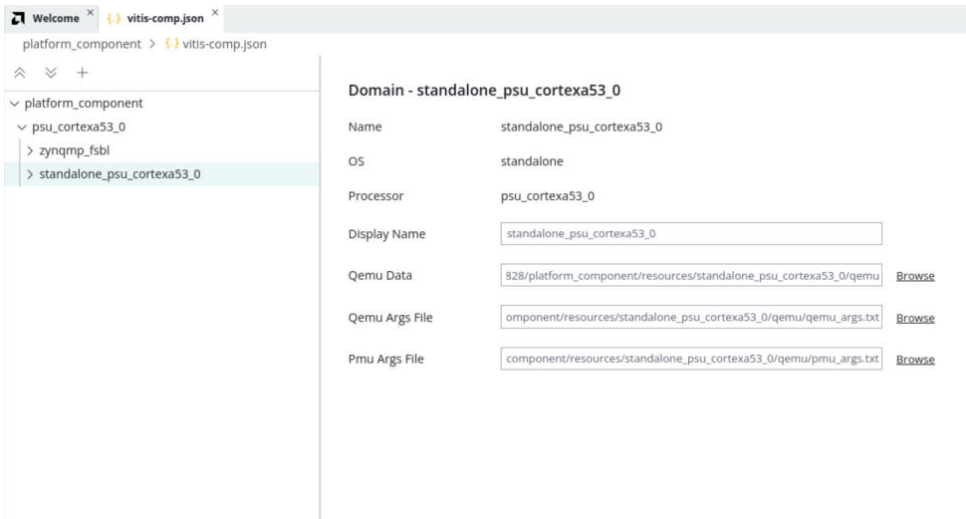
域有多种不同类型。最常见的是独立域。每个域都有关联的 BSP，可进行广泛配置。此外，域概述页面还包含该域的其他设置。

## 域概述页面

### 独立域和 FreeRTOS 域

独立域概述页面与 FreeRTOS 域概述页面完全相同，提供与 QEMU 硬件仿真平台相关的少量配置选项。这些选项是使用预定义的安装文件路径来自动填充的。

图 32：独立域概述页面

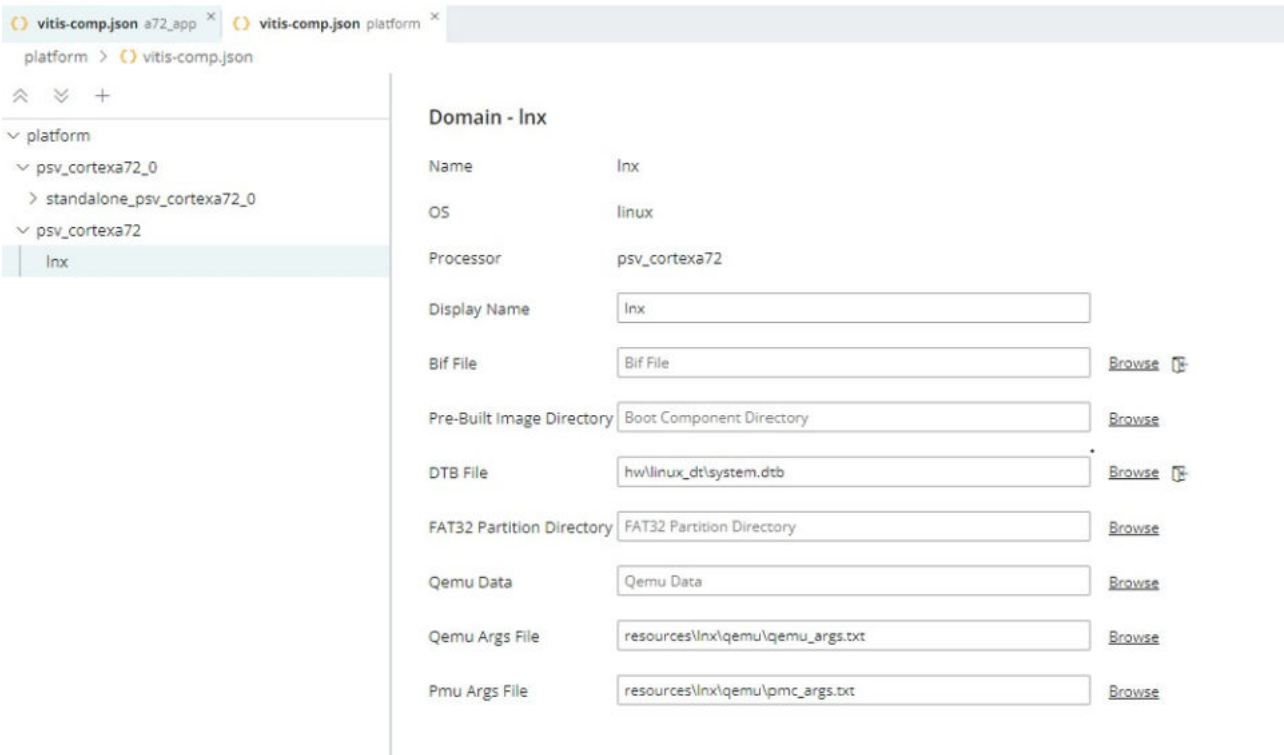


Linux 域

Linux 域概述页面类似于独立页面，但包含更多配置选项。

**注释：**对于嵌入式应用的固定平台，以下字段不适用。

图 33：Linux 域概述页面



- “BIF File”（BIF 文件）：启动镜像格式文件。单击 “Browse”（浏览）从文件系统中选择 bif 文件，或者单击 “Browse” 旁的按钮为您的平台生成 bif 文件。

- “Pre-Built Image Directory”（预构建的镜像目录）：包含 Linux 镜像文件的目录，包括启动组件、内核镜像和 rootfs。
- “DTB File”（DTB 文件）：用于启动 Linux 系统的系统设备树二进制文件。如果预构建的镜像目录包含 DTB 文件，那么 Vitis 工具会自动填充该字段中的 DTB 文件。
- “FAT32 Partition Directory”（FAT32 分区目录）：用于向 FAT32 分区添加其他文件。
- “Qemu Data”（QEMU 数据）：构建平台时，自动填充。该字段为硬件仿真提供了启动组件。

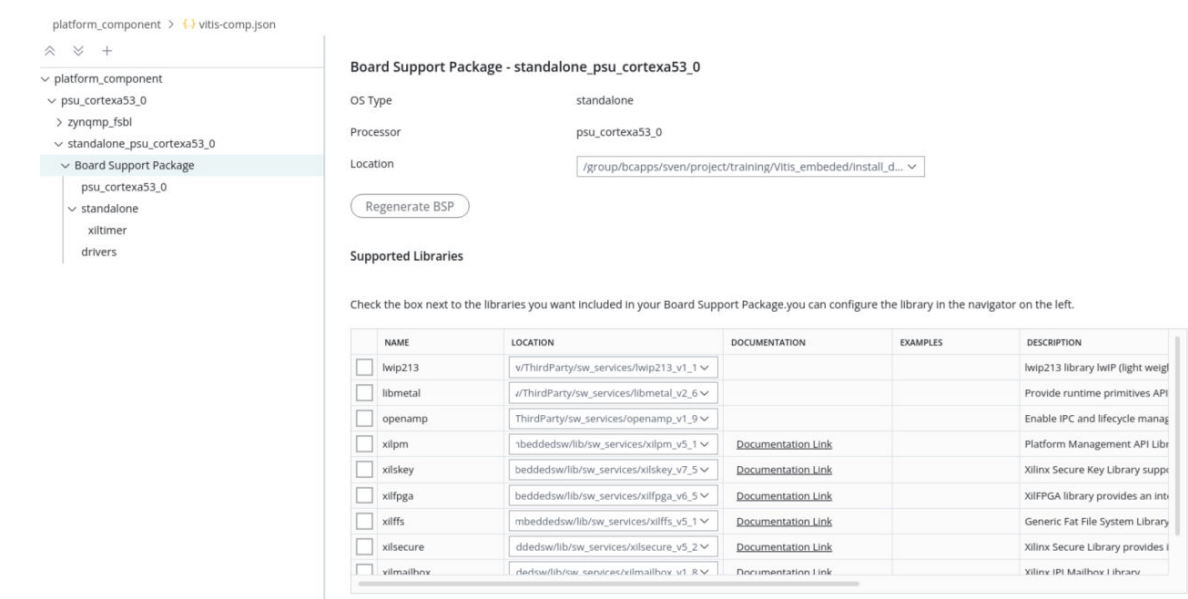
板级支持包设置页面

“Board Support Package Settings”（板级支持包设置）页面包含多个配置页面，仅适用于非 Linux 域。

**注释：**您无法更改此页面上所选的操作系统。软件平台创建步骤决定了操作系统类型。

在“Component”（组件）视图中选中平台组件，然后单击 `vitis-comp.json` 文件将其打开。下一步，展开 `standalone_psu_cortexa53_0` 并单击“Board Support Package”（板级支持包）。

图 34：概述



操作系统设置部分允许您配置操作系统的参数。

图 35：处理器参数配置

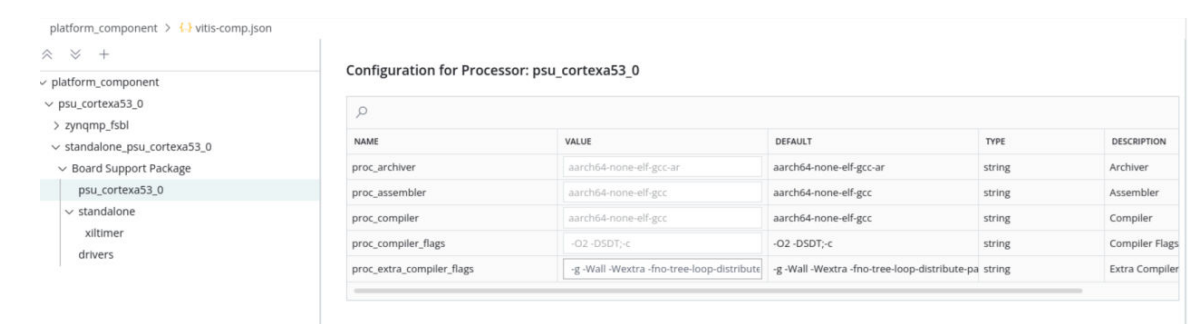
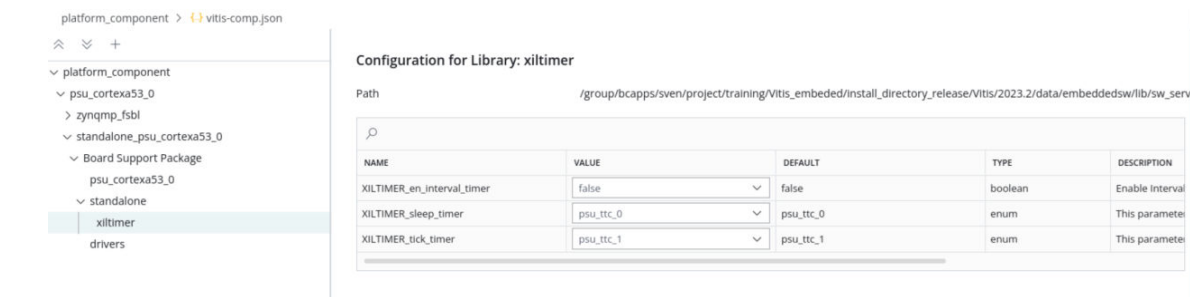


图 36：操作系统参数配置



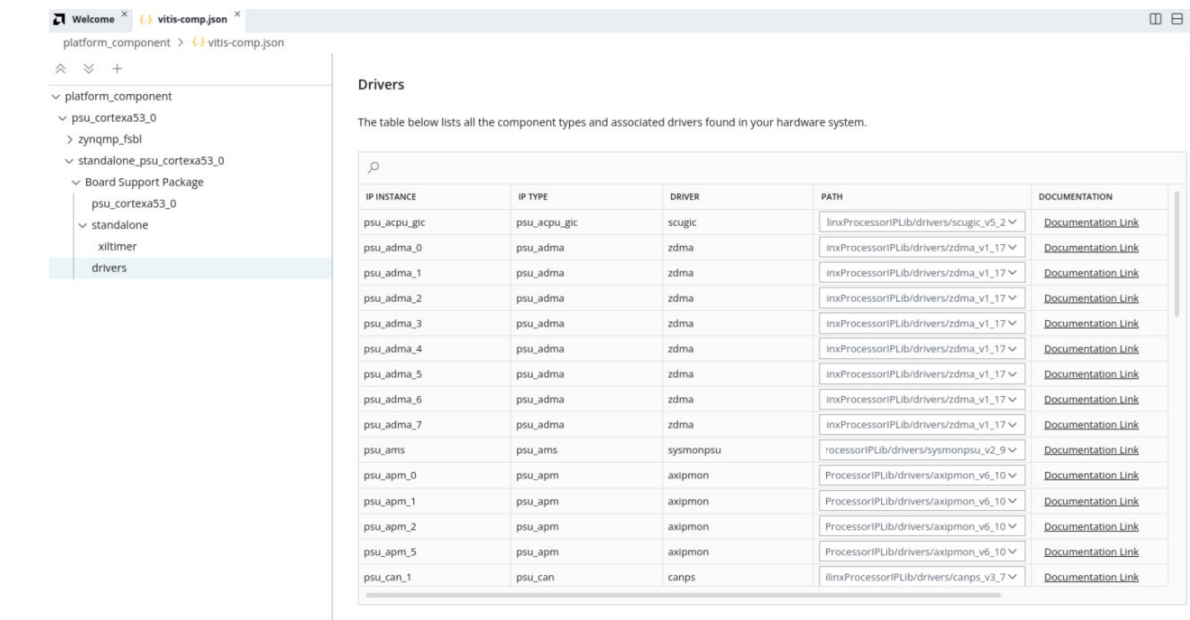
库设置页面允许您配置库页面上启用的每个库的参数。

图 37：库配置



“drivers”（驱动程序）部分列出了为系统中每个外设分配的所有器件驱动程序。您可以选中每个外设，更改其默认器件驱动程序分配及其版本。要为外设移除驱动程序，请将驱动程序分配至“none”。

图 38：驱动程序



构建设置部分列出了用于构建 BSP 的选定工具链以及一些额外的配置设置。

## 切换 FSBL 目标处理器

创建平台时，可为 FSBL 选择目标处理器。创建平台后，可以将其重新定向到 Zynq UltraScale+ MPSoC 器件上的其他处理器。要将平台组件重新定向到 Cortex-R5F，请执行以下步骤。

1. 单击平台组件，展开“Settings”（设置）并双击“vitis-comp.json”文件。
2. 选择“psu\_cortexa53\_0” → “zynqmp\_fsbl”。
3. 单击“Re-target to psu\_cortexr5\_0”（重定向至 psu\_cortexr5\_0）。
4. 重新构建平台。

### Domain - zynqmp\_fsbl

Name zynqmp\_fsbl

OS standalone

Processor psu\_cortexa53\_0

Display Name zynqmp\_fsbl

FSBL is currently targeted to 'psu\_cortexa53\_0'. use the below option to re-target to 'psu\_cortexr5\_0'. Doing this will clear the BSP and application setting one on this boot domain

Re-target to psu\_cortexr5\_0

## 修改 FSBL 源代码

如果您在平台生成中选择启动组件生成，那么 Vitis 工具就会在平台组件中生成 FSBL 应用程序。要修改这些应用的源代码，请执行以下步骤。

1. 要修改 FSBL 的源代码，请转至对应平台并展开“Source”（源代码）。
2. 展开 zynqmp\_fsbl\_bsp 文件夹，并修改内部源文件。
3. 保存更改，并以新更改来重新构建平台。

**注释：**要随时重置域/BSP 源代码，请单击“Board Support Package”（板级支持包）概述页面上的“Regenerate BSP”（重新生成 BSP）选项。

**注释：**在 [修改域源码（驱动程序和库代码）](#) 中提供了另一种更新 FSBL 和 PMUFW 源代码的方法。

## 修改域源码（驱动程序和库代码）

要使用 AMD Vitis™ 软件平台添加或修改域源码（驱动程序和库代码），您必须创建自己的存储库，并在其中包含所有必要的文件（包括 .mld/.mdd 文件和源文件）。已安装的驱动程序和库代码位于 <Vitis\_Install\_Dir>/data/embeddedsw 目录内。在驱动程序或库代码组件的 src 目录内包含源文件，在 data 目录内则包含元数据。在 .mld/.mdd 文件内，增大驱动程序/库版本号，并将此存储库添加到 Vitis 软件平台中。

Vitis 软件平台会自动推断该存储库内包含的所有组件，并使这些组件可供在其环境内使用。要执行任何修改，则必须在该存储库内完成所需的更改。构建应用即可使更改生效。

## 创建软件存储库

软件存储库是一个目录，您可在其中安装第三方软件组件以及驱动程序、库和操作系统的自定义副本。添加软件存储库时，AMD Vitis™ 软件平台会自动推断该存储库内包含的所有组件，并使这些组件可供在其环境内使用。

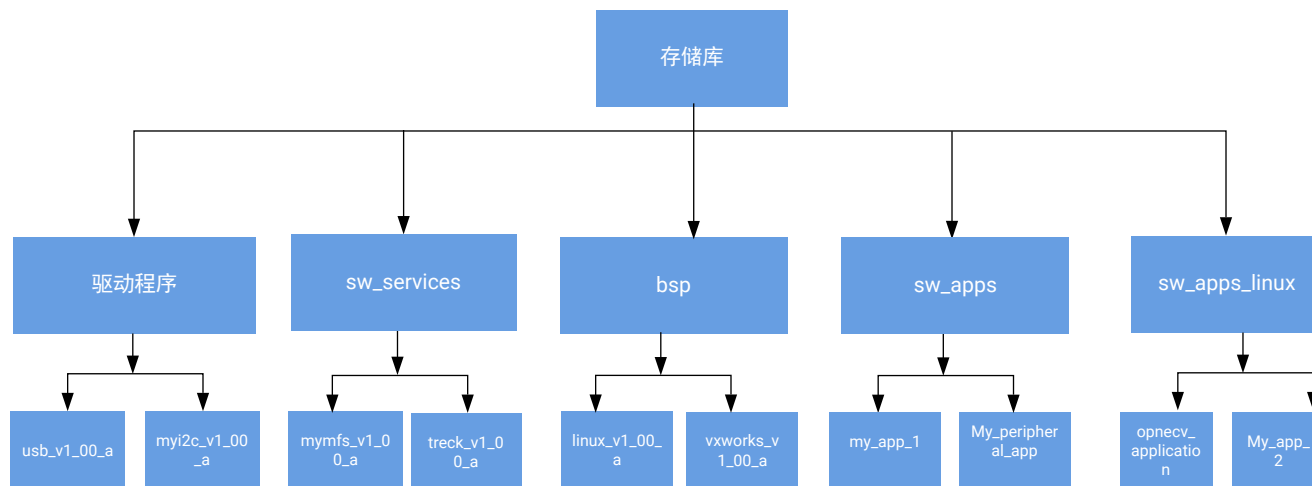
您的 Vitis 软件平台工作空间可指向多个软件存储库。软件存储库的范围可设为全局（跨所有工作空间可用）或本地（仅在当前工作空间内可用）。本地软件存储库中已添加到工作空间内的组件的优先级高于全局软件存储库中的相同组件。全局软件存储库的优先级高于 Vitis 软件平台安装中的相同组件。

Vitis 软件平台中的存储库需要以特定方式对组件进行组织。您的存储库中的软件组件必须属于以下目录之一：

- drivers：用于保存器件驱动程序。
- sw\_services：用于保存库。
- bsp：用于保存软件平台和板级支持包。
- sw\_apps：用于保存软件独立应用。
- sw\_apps\_linux：用于保存 Linux 应用。

在每个目录内，必须存在包含各软件组件的子目录。下图显示了存储库结构。

图 39：存储库结构



X50663-091725

## 添加软件存储库

1. 选择“Vitis” → “Embedded SW Repositories”（Vitis > 嵌入式软件存储库）。
2. 要添加您在 [创建软件存储库](#) 中创建的存储库，请遵循以下两个步骤中的任一步骤进行操作：
  - 要确保用于保存您的驱动程序或库的仓库仅在当前工作空间可用，请单击“+”以将其添加到“Local Repositories”（本地存储库）下。
  - 要确保您的存储库驱动程序或库存储库可在所有工作空间内使用，则请单击“+”以将其添加到“Global Repositories”（全局存储库）下。
3. 单击“OK”（确定）添加存储库。

## 为域复位 BSP 源文件

执行以下步骤即可将域的 BSP 源文件复位：

1. 选中平台组件，展开“Settings”（设置），单击 `vitis-comp.json` 文件并选中相应的域。
2. 单击“Regenerate BSP”（重新生成 BSP）。这样即可为所选的域/BSP 复位源文件。

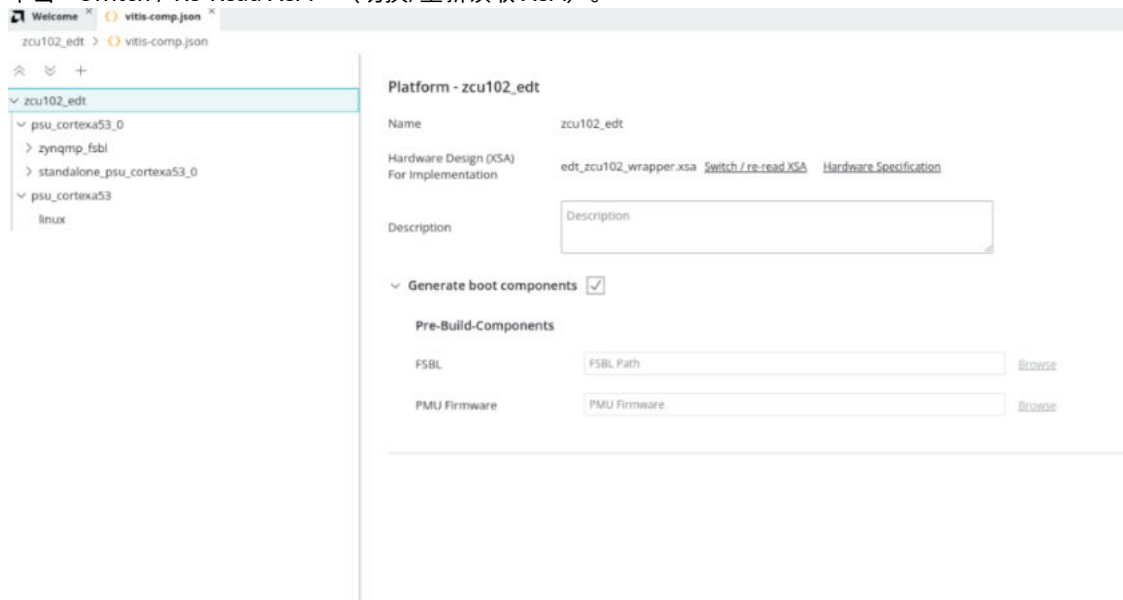
**注释：**此操作仅将源文件（而非设置）还原为原始状态。

## 更新硬件规格

AMD Vitis™ 软件平台允许您通过更新所含软件组件的方式，以新硬件来更新平台工程。如果您的 AMD Vivado™ 工程及其导出的 XSA 已更新，则需要手动执行此工作流程以便 Vitis 软件平台可获取更新后的硬件规格。您可在软件平台根据新硬件调整软件组件之后编辑设置。

要更改平台工程的硬件规格文件，请执行以下步骤：

1. 右键单击组件视图中的平台组件，然后展开“Settings”（设置），打开“vitis-comp.json”。
2. 单击“Switch / Re-Read XSA”（切换/重新读取 XSA）。



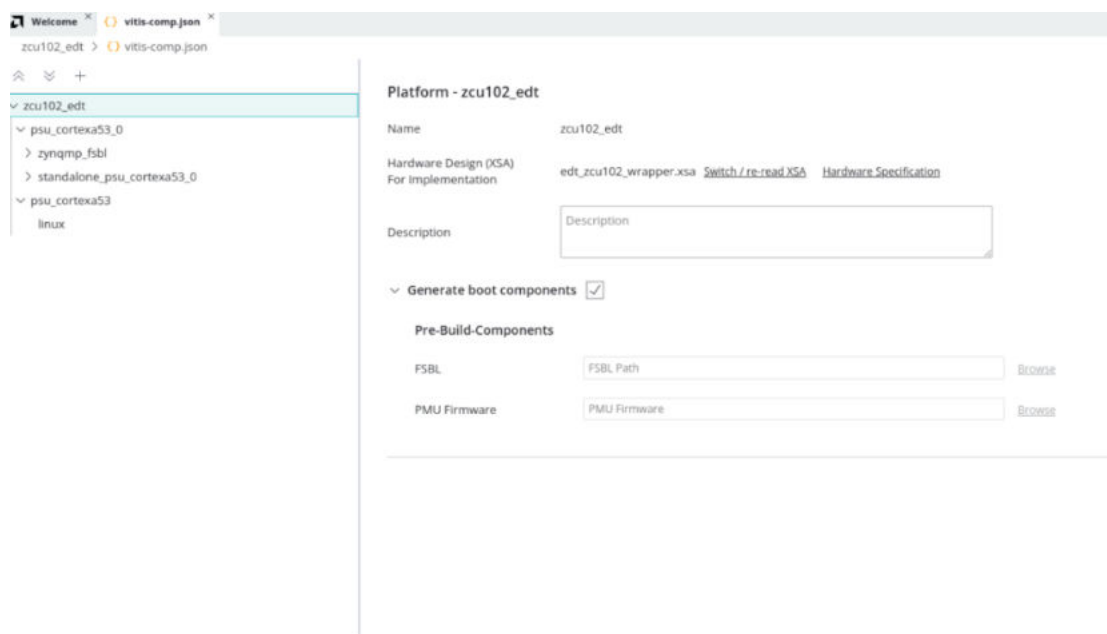
3. 在弹出窗口中指定硬件规格源文件。
4. 单击“open”（打开）以选中 XSA 文件。

## 读取硬件规格

AMD Vitis™ 软件平台允许您读取底层硬件平台信息。

执行以下步骤读取硬件信息。

1. 右键单击组件视图中的平台组件，展开“Settings”（设置），然后打开“vitis-comp.json”文件。
2. 要访问有关硬件平台的硬件信息，请单击“Hardware Specification”（硬件规格）。例如，处理器地址信息和 PL IP 地址信息。



**注释：**单击“Hardware Specification”的效果与双击 XSA 文件相同。

## 应用程序

Vitis 工具中的嵌入式应用在嵌入式系统或嵌入式平台上运行。在 Vitis 环境中，嵌入式应用具有以下特征：

- 以嵌入式系统为目标：嵌入式应用在嵌入式硬件系统上运行。嵌入式应用通常资源受限，并且是专为特定任务而设计的。这类系统覆盖范围广泛，从小型微控制器到嵌入式 FPGA 或 SoC（片上系统）等更复杂的器件都包含在内。
- 实时或资源有限：嵌入式应用经常需要满足实时约束或者在严格的资源限制下运行。它们必须高效利用可用的 CPU、存储器和 I/O 资源，以可靠和可预测的方式来执行任务。
- 多样化的用例：Vitis 中的嵌入式应用适用于多种用途。用例包括控制物联网 (IoT) 器件、管理感测器和执行器、运行实时控制算法、执行信号处理、通信等。
- 硬件集成：嵌入式应用可与硬件组件和外设紧密交互。它们通常需要特定的器件驱动程序和低层次硬件访问，以便与感测器、电机、通信接口和其他嵌入式硬件对接。
- 开发环境：Vitis 提供的开发环境允许开发者在自己的目标硬件平台上创建、编译和部署嵌入式应用。它包含用于代码开发、调试和性能优化的各种工具。
- 交叉编译：开发者经常要对嵌入式应用进行并行编译。Vitis 常用于编译应用程序，使其能在与主机架构不同的目标嵌入式系统上运行。

## 创建应用组件

### 基于示例创建应用组件

您可遵循以下步骤，基于示例模板来创建应用组件：

1. 通过以下任一方法打开“Example”（示例）视图：转至“View” → “Examples”（视图 > 示例）、单击“File” → “New” → “Component From Example”（文件 > 新建 > 基于示例创建组件）、单击左侧工具栏，或者使用“Ctrl + Shift + R”快捷键。
2. 单击示例列表中的示例。
3. 单击“Show details”（显示详细信息），了解模板要求。
4. 单击“Create Application Component from Template”。欲知详情，请参阅 [创建应用组件](#)。

创建 Linux 应用组件时，您可选择第三方 sysroot 工具链，方法是选中对应复选框并提供路径。

如果您禁用 `use sysroot toolchain`，那么 Vitis 工具会使用默认 gcc 编译器。如果您传递的路径中不含任何 gcc 编译器或 g++ 编译器，那么 Vitis 会抛出错误。

对于 CLI 流程，请给您的 `create_app_component` 命令添加以下实参：

```
use_sysroot_toolchain=True
sysroot_toolchain=<TOOLCHAIN_PATH>
```

## 创建应用组件

独立域生成的 BSP 可以链接到各种应用。应用程序也可以在 Linux 域操作系统上运行。创建应用需要平台。

要创建应用组件，请选中“File” → “New Component” → “Application”（文件 > 新建组件 > 应用）。



**提示：**或者，您可在“Welcome”（欢迎）页面上的“Embedded Development”（嵌入式开发）分组中选中“Create Embedded Application”（创建嵌入式应用）。

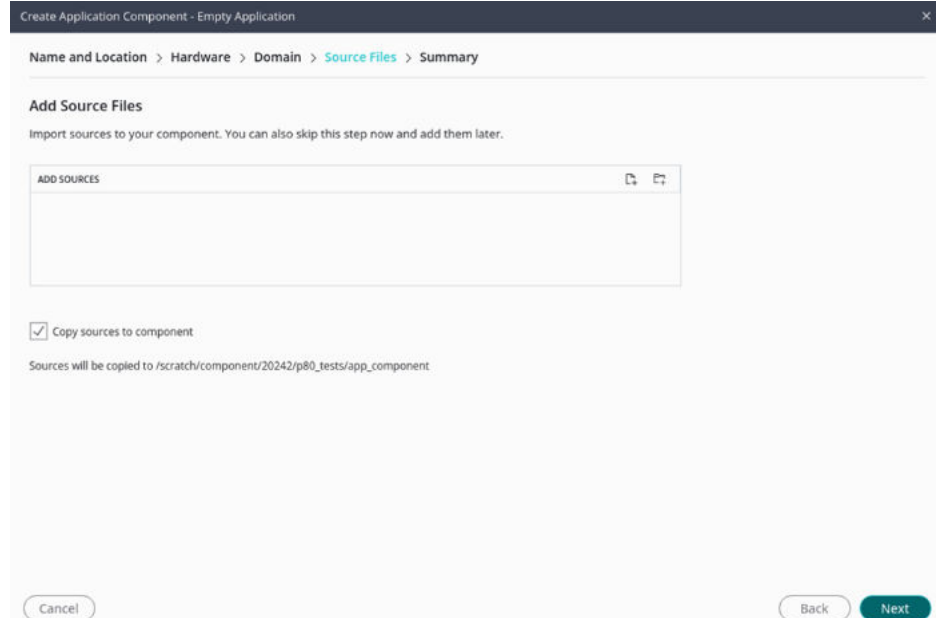
1. 输入应用组件名称并指定位置。使用长度为 3 到 100 个字符的平台名称，该名称必须以字母数字字符开头，并且仅包含字母、数字、空格、下划线 (\_)、和号 (&) 或连字符 (-)。单击“Next”（下一步）
2. 选择固定平台，然后单击“Next”。

**注释：**如果没有现有的固定平台，请参阅 [基于 XSA 创建平台组件](#) 章节了解如何创建平台。如果列表中不含您的目标平台，您可单击“+”按钮添加目标平台。

- 选择处理器域用于运行您的应用，然后单击“Next”。如果该平台包含受支持的 MicroBlaze V Linux 域，请选择该 Linux 域以创建应用组件。

**注释：**如果该域不适合您的应用，请单击“+ create new”（创建新的域）以添加新的域。根据要求指定操作系统和处理器，然后单击“Next”。

- 单击“Add files”（添加文件）或“Add folders”（添加文件夹），为应用导入源文件或文件夹。



**注释：**默认情况下，Vitis 工具会选中“Copy Source to Component”（将源文件复制到组件）。如果您倾向于直接引用外部源文件，而不是将其复制到工作空间内，那么请取消选中此选项。

- 复查应用组件的汇总页面，然后单击“Finish”（完成）以创建组件。
- 导入源文件

现在，您即可构建自己的组件或配置您的构建设置。

## 创建系统工程

遵循以下步骤创建系统工程。

- 转至“File” → “New Component” → “System Project”（文件 > 新建组件 > 系统工程）。

**注释：**您可在“Welcome”（欢迎）页面中使用“Create System Project”（创建系统工程）快捷方式来创建系统工程。

- 输入系统工程名称和系统工程位置。单击“Next”。
- 选择系统工程的平台。单击“Next”。

**注释：**如果“Repo”（存储库）列表中含您的平台，您可单击“+”按钮添加其他工作空间内的平台。或者，您可参阅 [基于 XSA 创建平台组件](#) 以了解如何创建新平台并将其添加到“Repo”列表中。

- 在“Embedded Component Path”（嵌入式组件路径）设置对话框中，执行嵌入式开发时，无需设置镜像路径。单击“Next”。

**注释：**加速应用开发会使用“Embedded Component Path”（嵌入式组件路径）设置。加速开发期间，您需要指定 rootfs、内核镜像和启动组件才能生成最终 `sd_card.img` 文件。

5. 请复查系统工程汇总信息，然后单击“Finish”（完成）。

## 在系统工程组件内管理多个应用

系统工程可包含多个应用，这些应用可在器件上同时运行。同一个处理器的两个应用不能放置在同一个系统工程内。例如，在 AMD Zynq™ UltraScale+™ MPSoC 器件上，Cortex®-A53 上运行的独立应用和 Cortex®-R5F 上的应用如果预计将同时运行，则可保持在同一个系统工程内。但不得在单个系统工程内将 Cortex-A53 上的应用与 Linux 应用相结合。这是因为这些应用使用的是相同的 Cortex-A53 处理器。

以下步骤演示了将两个应用添加到同一个系统工程的流程：

1. 单击系统工程并展开“Settings”（设置），单击“vitis-sys.json”打开该工程。

**注释：**`Vitis-sys.json` 是用于配置系统工程的系统工程设置文件。

2. 单击“System Project Settings”→“Components”→“Adding Existing Components”（系统工程设置 > 组件 > 添加现有组件）。
3. 单击“Application”（应用）弹出窗口，并选择您的工作空间内的应用组件。

## 构建工程

开发软件应用的第一步是创建板级支持包以供应用使用。下一步，您即可创建应用工程。

为此应用构建可执行文件时，Vitis 会自动执行以下操作。

1. Vitis 软件平台会构建板级支持包，也称为平台。
2. Vitis 软件平台会使用特定于平台的 `gcc/g++` 编译器来编译应用软件。
3. 随后，来自应用和板级支持包的-object 文件会链接在一起形成最终可执行文件。连接器脚本以-object 文件作为输入，指定-object 文件在存储器中的布局位置。

以下章节提供了构建应用过程中所涉及的概念的概述。

## 构建配置设置

您可手动修改构建配置：

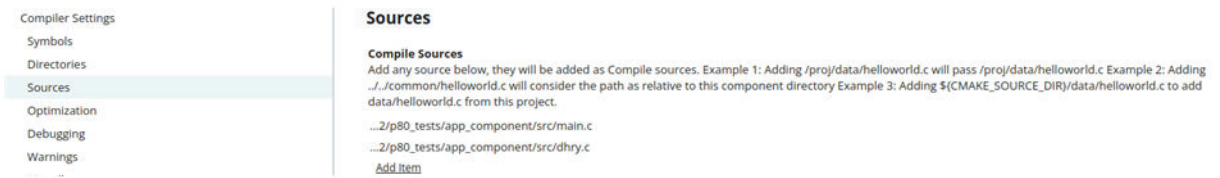
- 编译器设置：调试和优化级别
- 面向编译传递的宏
- 连接器设置

## 管理源代码

本节介绍了如何管理源代码。

1. 在“Component”（组件）视图中单击您的应用组件并展开“Settings”（设置）。打开 `vitis-comp.json` 文件，单击 `UserConfig.cmake` 以配置编译器。或者，也可将光标悬停在应用组件上，这样会出现一个设置按钮，单击该按钮即可打开 `vitis-comp.json` 文件。您也可以从“Settings”（设置）文件夹目录中打开 `UserConfig.cmake`。

- 在“Compiler Settings”（编译器设置）下选中“Sources”（源文件）部分。



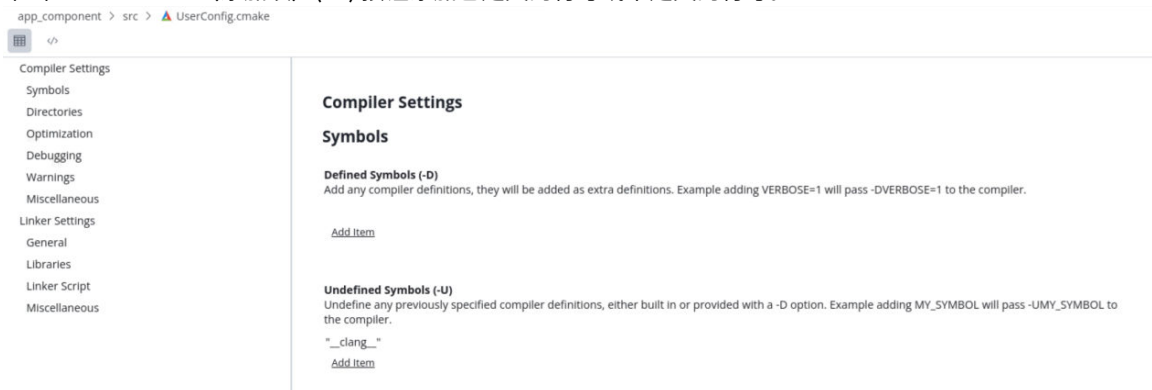
- 单击“Add Item”（添加项）来为您的应用添加源代码。
- 您还可将鼠标悬停在源文件名称上，选中并移除该文件。

**注释：** `UserConfig.cmake` 支持 GUI 格式和文本格式。您可单击“`</>`”图标访问源代码编辑器，以查看源代码并以文本格式修改设置。

## 添加符号或定义

Vitis 工具会对 `#define` 指令中的定义和符号进行处理和分词 (tokenize)。您可通过以下步骤在 Vitis IDE 中添加或移除符号：

- 在“Component”（组件）视图中单击您的应用组件并展开“Settings”（设置）。打开设置目录下的 `UserConfig.cmake` 文件。
- 在“Compiler Settings”（编译器设置）下，选中“Symbols”（符号）。
- 单击“Add Item”（添加项）(📄) 按钮添加已定义的符号或未定义的符号。



**注释：** `UserConfig.cmake` 支持 GUI 格式和文本格式。您可单击“`</>`”图标访问源代码编辑器，以查看源代码并以文本格式修改设置。

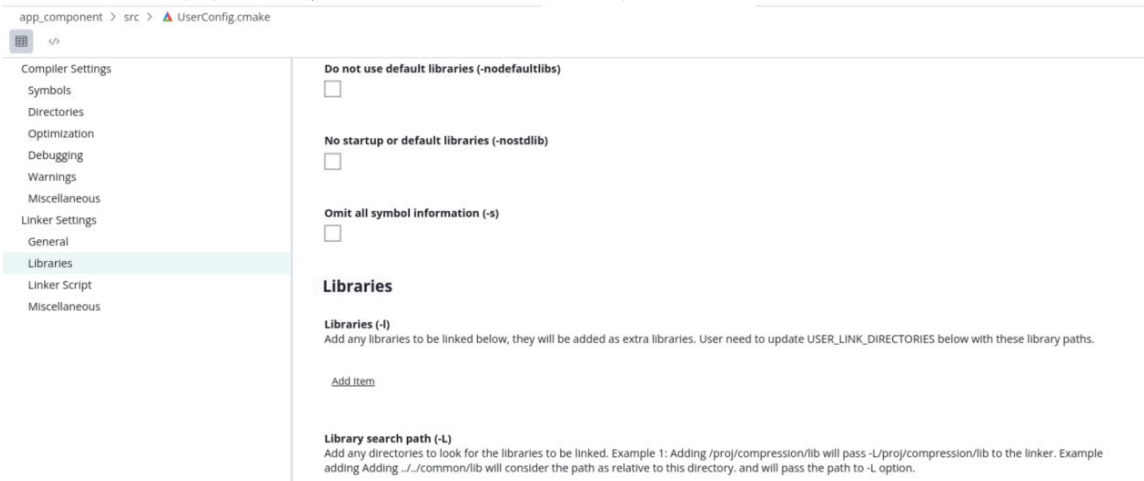
## 添加库和库路径

您可为应用工程添加库和库路径。如果有要链接的自定义库，可向该连接器指定库路径和库名称。

要为您的应用工程设置属性，请执行以下操作：

- 在“Component”（组件）视图中单击您的应用组件并展开“Settings”（设置）。在设置目录下，打开 `UserConfig.cmake` 文件。
- 在“Compiler Settings”（编译器设置）下，选中“Libraries”（库）

3. 在“Libraries”（库）部分中，单击“Add Item”（添加项）以添加库名称或库路径。



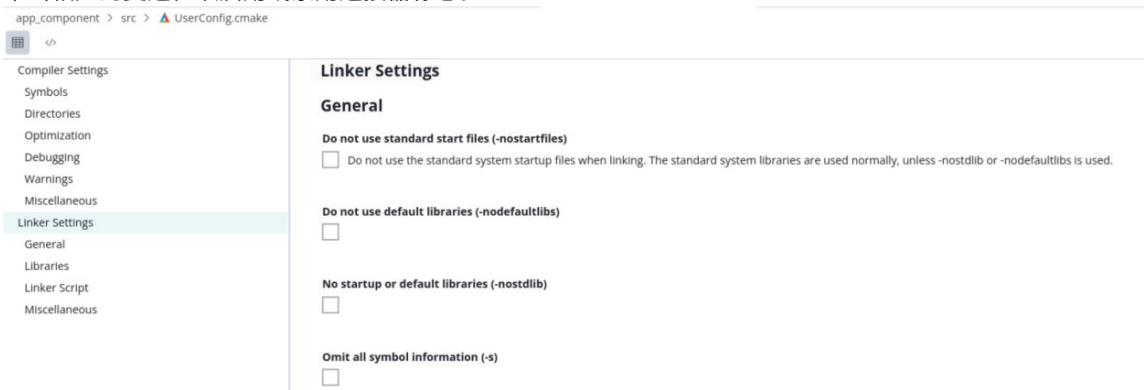
**注释：** `UserConfig.cmake` 支持 GUI 格式和文本格式。您可单击“`</>`”图标访问源代码编辑器，以查看源代码并以文本格式修改设置。

## 指定连接器选项

使用以下步骤为应用程序组件指定连接器选项。您还可指定“Tool Settings”（工具设置）中未涵盖的任何其他连接器标志。

要为您的应用组件设置属性，请执行以下操作：

1. 在“Component”（组件）视图中单击您的应用组件并展开“Settings”（设置）。打开设置目录下的 `UserConfig.cmake` 文件。
2. 在“Compiler Settings”（编译器设置）下，选中“Linker Settings”（连接器设置）。
3. 单击相应的复选框以启用或禁用连接器标志。



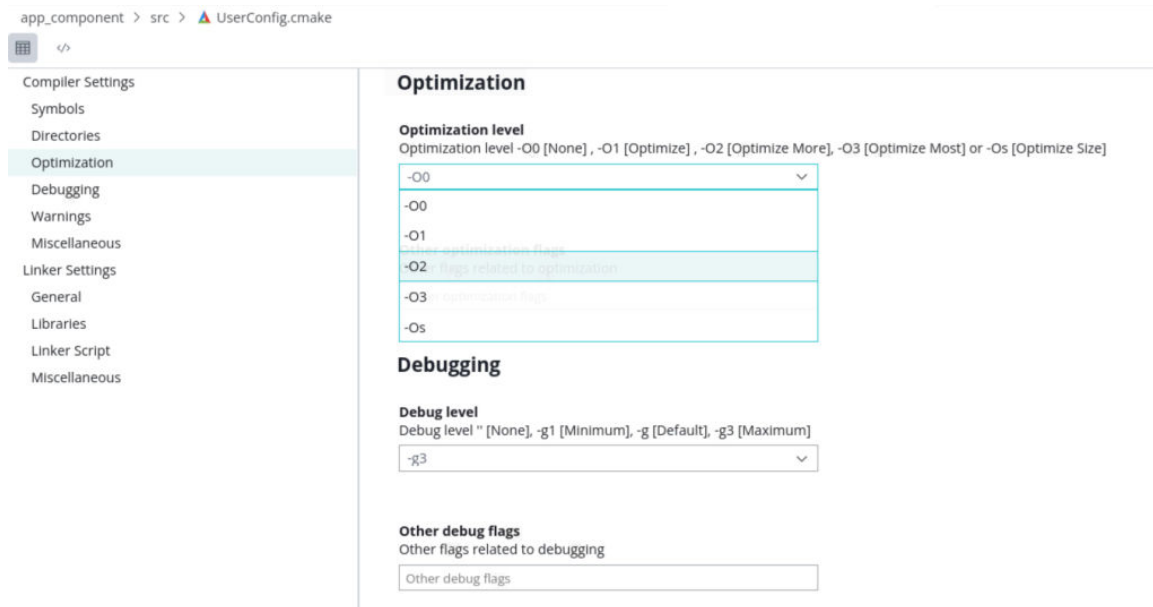
**注释：** “`UserConfig.cmake`”支持 GUI 格式和文本格式。您可单击“`</>`”图标访问源代码编辑器，以查看源代码并以文本格式修改设置。

## 指定调试和优化编译器标志

Vitis 软件平台会为应用组件分配默认优化级别和调试标志。您可为自己的应用组件更改默认值。

要为您的工程设置属性，请执行以下操作：

1. 在组件视图中单击您的应用组件并展开“Settings”（设置）。打开设定目录下的 `UserConfig.cmake` 文件。
2. 在“Compiler Settings”（编译器设置）下，选中“Optimization”（优化）。
3. 在“Optimization”部分，单击下拉列表按钮选择优化级别。“Debugging”（调试）部分就位于“Optimization”部分之下。同样，单击下拉列表按钮即可更改调试级别。您可在其他标志字段中输入其他优化标志或调试标志。



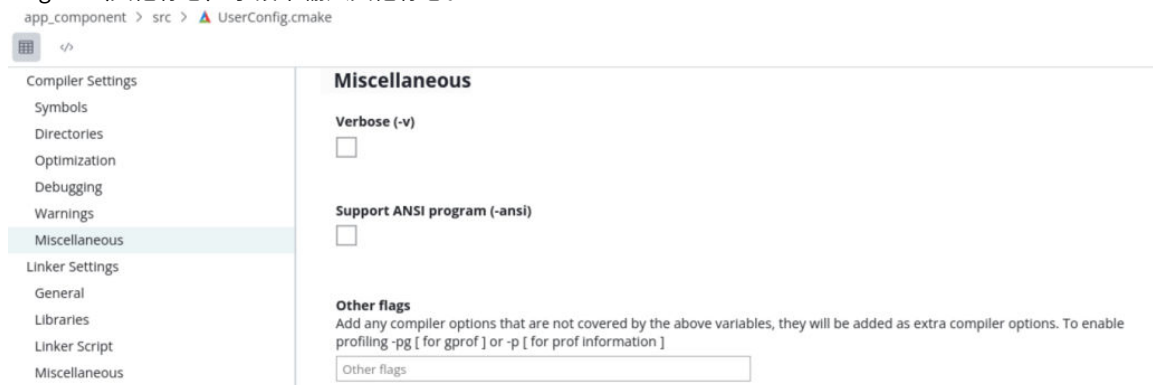
**注释：** `UserConfig.cmake` 支持 GUI 格式和文本格式。您可单击 “</>” 图标访问源代码编辑器，以查看源代码并以文本格式修改设置。

## 指定杂项编译器标志

您可在“Miscellaneous”（杂项）部分中指定任何其他编译器标志。

要为您的工程设置属性，请执行以下操作：

1. 在组件视图中单击您的应用组件并展开“Settings”（设置）。打开设定目录下的 `UserConfig.cmake` 文件。
2. 在“Compiler Settings”（编译器设置）下，选中“Miscellaneous”。
3. 您可单击相应复选框启用“Verbose”（详细）或“Support ANSI”（支持 ANSI）编程支持。您可在“Other Flags”（其他标志）字段中输入其他标志。



**注释：** `UserConfig.cmake` 支持 GUI 格式和文本格式。您可单击 “</>” 图标访问源代码编辑器，以查看源代码并以文本格式修改设置。

## 连接器脚本

编译与链接是应用可执行文件构建进程的两个主要步骤。接受连接器命令语言文件的连接器脚本会执行链接。连接器脚本描述目标机器的存储器布局以及存储器中程序节的位置。

**注释：**只有独立应用才需要连接器脚本。Linux 操作系统可帮助管理存储器分配，并且不需要连接器脚本。

Vitis 软件平台提供了连接器脚本生成器用于简化为 GCC 创建连接器脚本的任务。Vitis IDE 中的连接器脚本生成器会检查系统设备树 (SDT) 中的存储器节点，以决定可用的存储器节。

**注释：**

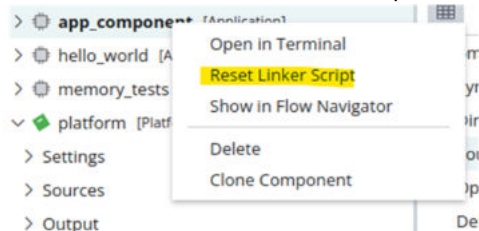
- 对于多核处理器系统，每个处理器都运行不同的 ELF 文件，并且每个 ELF 文件都需要独立的连接器脚本。请确保在存储器中不存在两个重叠的 ELF 文件。
- 默认连接器始终指向存储器中可用的 DDR 存储器地址。如果您在给定硬件/域工程下创建应用，那么对于这些应用，存储器会重叠。

## 为应用复位连接器脚本

**注释：**您可双击连接器脚本来对其进行编辑。该脚本位于 `<application_name>/sources/src/lscript.ld`。

执行以下步骤为应用组件生成连接器脚本，或者删除使用应用组件创建的连接器脚本。

- 右键单击组件视图中的“app\_component”。
- 在弹出窗口中选中“Reset Linker Script”（复位连接器脚本）。



- 单击“OK”（确定）。

请先纠正错误，然后再使用新的连接器脚本来构建您的应用程序。

**注释：**无论是否存在连接器脚本，出现消息视图时，选中相应的选项即可覆盖此文件。单击“OK”（确定）即可覆盖此文件，或者单击“Cancel”（取消）即可取消覆盖。

## 手动添加连接器脚本

如果您要为自己的应用工程手动添加连接器脚本，请执行以下操作：

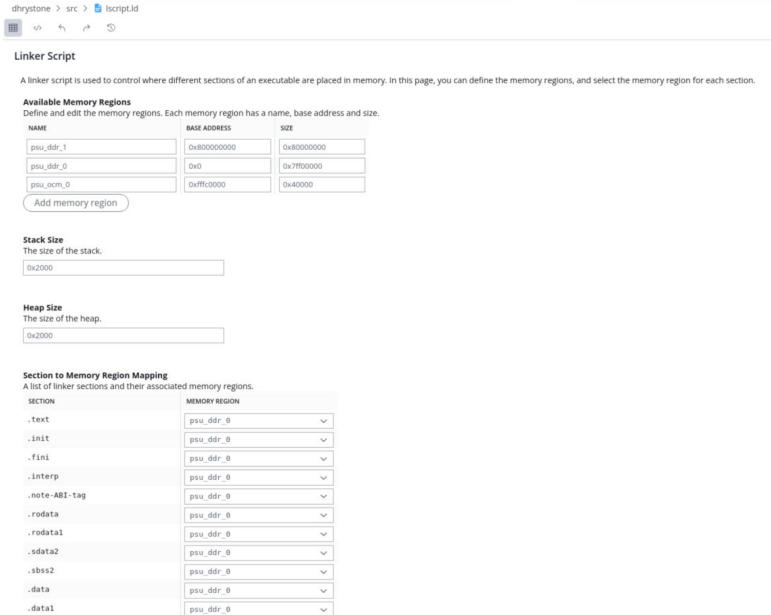
- 在“Component”（组件）视图中选中该应用组件，然后展开“Settings”（设置）。右键单击“UserConfig.cmake”将其打开。
- 右键单击“Linker Script”（连接器脚本）转至连接器脚本设置部分。
- 单击“Browse”（浏览）选择您自己的连接器脚本文件。

## 修改连接器脚本

有多种方法可用于更新连接器脚本。

应用程序组件源代码包含连接器脚本。执行以下步骤即可对该脚本进行更新。

1. 选中组件视图中的应用组件，并展开“Sources”（源代码）。右键单击“lscript.ld”将其打开。



2. 单击“</>”按钮即可查看连接器文件的源代码。
3. “Undo/Redo”（撤销/重做）：撤销或重做刚刚针对连接器脚本执行的操作。
4. 您可单击“Reset linker script”（复位连接器脚本）按钮来复位连接器脚本。

连接器脚本编辑器可提供以下功能。

**注释：**此连接器脚本也支持文本格式。单击“</>”按钮即可更改为文本格式以执行修改。

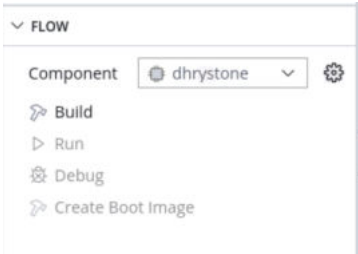
表 8：连接器脚本编辑器功能

| 名称                               | 功能                                                                                                          |
|----------------------------------|-------------------------------------------------------------------------------------------------------------|
| Add Memory Regions               | 这部分列出了连接器脚本中指定的存储器区域。您可单击右侧“Add”（添加）按钮来添加新区域。您可修改每个已定义的存储器区域的名称、基地址和大小。                                     |
| Stack and Heap Sizes             | 这部分显示了堆节和栈节的大小。只需编辑文本框中的值即可更新这些节的大小。                                                                        |
| Section to Memory Region Mapping | 这部分支持为连接器脚本中定义的任何节更改已分配的存储器区域。要更改已分配的存储器区域，只需单击该存储器区域以显示下拉菜单，即可从中选择可选存储器区域。您可选中多个节或整个节，单击下拉按钮，即可整体更改为存储器区域。 |

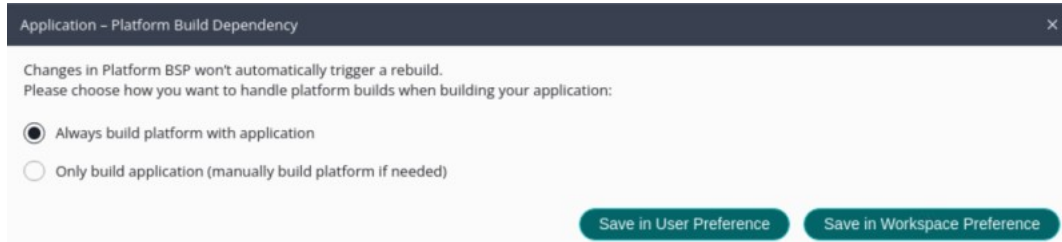
构建应用组件

设置构建配置并修改连接器后，即可开始构建应用。

1. 转至 Flow Navigator



- 单击组件字段中的下拉列表按钮选择应用组件。
- 单击“Build”（构建）以构建应用组件并选择应用构建首选项



**注释：**对平台 BSP 的更改不会自动触发重新构建。首次构建应用程序时，请在显示弹出消息后转到工作空间或首选项。选择“always build the platform before building the application”（始终构建平台后构建应用）选项，或者选择“manually build the platform”（手动构建平台）选项。

完成构建后，您可在“Component”（组件）视图中的应用组件下的输出目录中检查输出。

## 读取 ELF 反汇编

此任务可供您查看反汇编代码。

您可以在编译应用后，在输出目录中检索应用程序的 ELF 文件。您可双击“ELF file”（ELF 文件）查看代码的反汇编视图。

# 创建库工程

## 创建软件库组件

软件库组件为同一工作空间中的应用组件打包可复用的编译代码。库组件不定义 `main()` 入口点，而是生成可链接工件代替可运行的可执行文件。

Vitis Unified IDE 支持：

- 适用于裸机域的静态库组件
- 适用于 Linux 域的动态库组件

支持的处理器范围：

- 静态库（裸机）：Arm、MicroBlaze 和 RISC-V 处理器域
- 动态库：Linux 处理器域

## 创建静态库组件（裸机）

要创建静态库组件，请选择“File” → “New Component” → “Static Library”（文件 > 新建组件 > 静态库）。

- 输入组件名称和位置，然后单击“Next”（下一步）。
- 选择固定平台，然后单击“Next”。
- 选择硬件目标，然后单击“Next”。
- 选择目标域，然后单击“Next”。

5. 添加源文件或文件夹，然后单击“Next”。
6. 复查汇总信息，然后单击“Finish”（完成）。

**注释：**如果不存在合适的域，请先在平台设置中创建新的裸机域。

创建完成后，从“Flow Navigator”构建该库组件。

IDE 使用 Lopper 提取硬件配置元数据，并生成构建配置文件。IDE 生成 `CMakeLists.txt`、`UserConfig.cmake` 和 `compile_commands.json`。构建会生成 `.a` 静态库。

### 创建动态库组件 (Linux)

要创建动态库组件，请选择“File” → “New Component” → “Dynamic Library”（文件 > 新建组件 > 动态库）。

1. 输入组件名称和位置，然后单击“Next”（下一步）。
2. 选择与应用关联的目标 Linux 处理器域，然后单击“Next”。
3. 选择目标 Linux 域，然后单击“Next”。
4. 添加源文件或文件夹，然后单击“Next”。
5. 复查汇总信息，然后单击“Finish”（完成）。

**注释：**动态库流程不包含单独的硬件选择向导步骤。

IDE 使用 Linux 构建流程，不运行基于 Lopper 的硬件元数据提取。IDE 生成 `CMakeLists.txt` 和 `UserConfig.cmake`。构建会生成 `.so` 共享库。

## 在应用工程中使用自定义库

应用可以使用您在同一工作空间中创建的库组件。

在构建应用之前，请在 `UserConfig.cmake` 中配置以下条目。

- `USER_INCLUDE_DIRECTORIES`
- `USER_LINK_DIRECTORIES`
- `USER_LINK_LIBRARIES`

配置这些条目后，IDE 会在构建过程中将应用与指定的库工件进行链接。

此工作流程可在多个应用之间实现代码复用，而无需复制源文件。该工作流程将库与应用的职责分离开来。

从命令行创建库组件

您可以通过 CLI/Python 流创建库组件，以支持自动化和 CI/CD 工作流程。

使用 Vitis Python API `create_library_component` 流并配合下列操作：

- 平台选择
- 域选择（静态库选择裸机，动态库在版本支持的前提下选择 Linux）
- 源导入
- 组件构建

对于静态库流，IDE 使用 Lopper 提取硬件元数据，然后在构建配置生成期间使用这些元数据。



---

**提示：**您可以使用工作空间日志来捕获并重放适用于您环境的确切命令顺序。

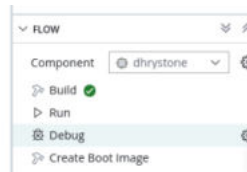
---

# 运行、调试和优化

## 启动配置

要调试、运行和剖析应用，则必须创建启动配置以捕获用于执行应用的设置。为此，请转至 Flow Navigator 并选中应用组件，右键单击“Run”（运行）或“Debug”（调试）旁的“Open Settings”（打开设置）。

图 40: “Flow” 窗口



**注释：**“Open Settings” 命令是隐藏图标。将光标悬停在 Flow Navigator 上的“Run”或“Debug”旁时，即可查看该图标。

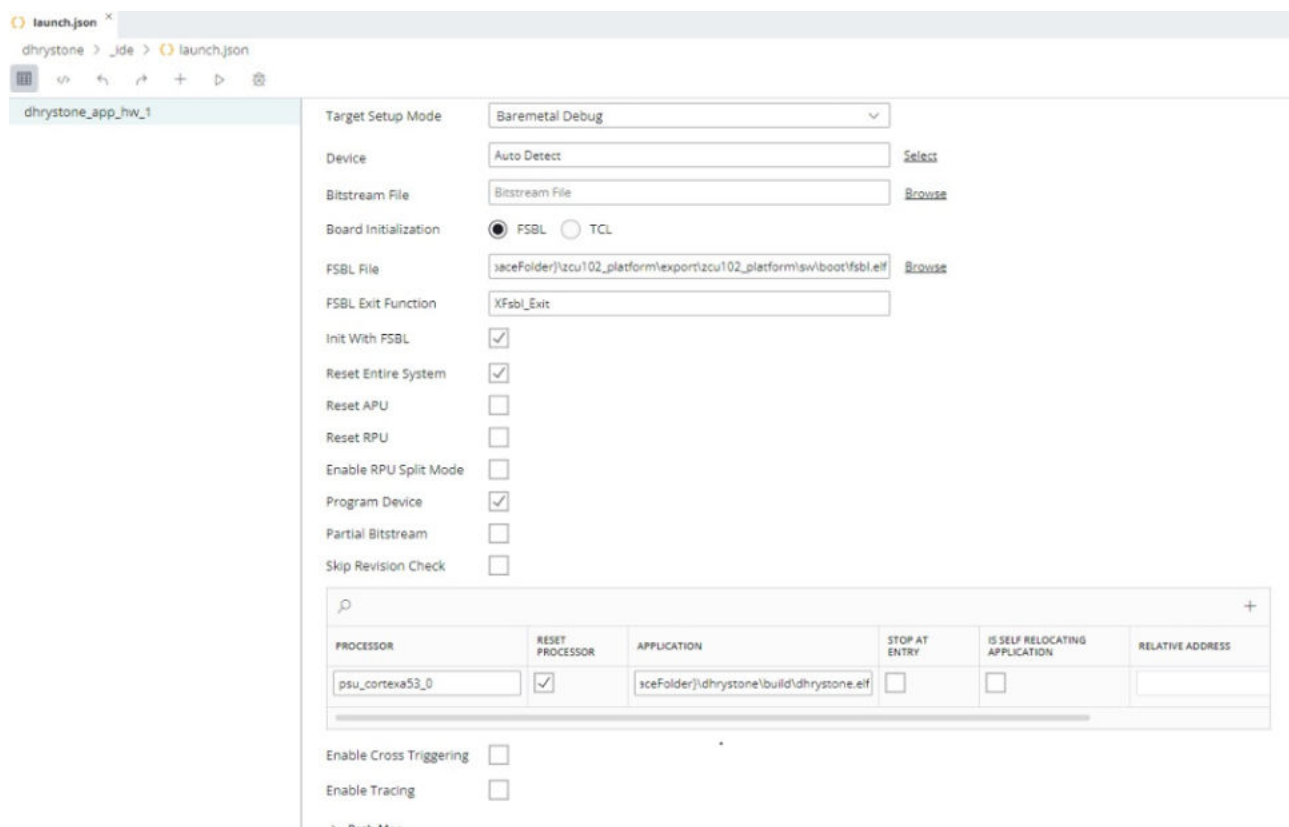
`launch.json` 文件允许您编辑启动配置。“Launch Configuration”（启动配置）编辑器顶部有一个工具栏菜单，如下图所示。启动配置的具体内容因所选组件或工程而异。

### 启动配置

“Launch Configuration” 工具栏中的命令包括：

- “Settings Form / Source Editor”（设置表单/源文件编辑器）：您可使用下列命令在 `launch.json` 文件的 GUI 视图与“Text Editor”（文本编辑器）视图之间进行切换。
- “Undo/Redo”（撤销/重做）：撤销或重做最近的操作。
- “Add Configuration”（添加配置）：您可单击“+”按钮添加配置。系统工程的默认启动配置是：Emulation SW（软件仿真）、Emulation HW（硬件仿真）和 Hardware（硬件）。
- “Run / Debug”（运行或调试）：使用当前选中的启动配置来启动运行或调试。

图 41：启动配置



将光标悬停于配置名称上，这样就会显示“Duplicate”（复制）命令和“Delete”（删除）命令。选中“Delete”即可移除启动配置，或者选中“Duplicate”即可复制启动配置。

您可在每个配置中更新设置以配置该工具，然后再运行或调试组件或工程。设置启动配置后，您即可在“Launch Configuration”（启动配置）编辑器中选择“Run”（运行）命令或“Debug”（调试）命令。您还可从“Flow Navigator”选择“Run”或“Debug”，如果有多个启动配置可用，那么您还可指定任一启动配置。

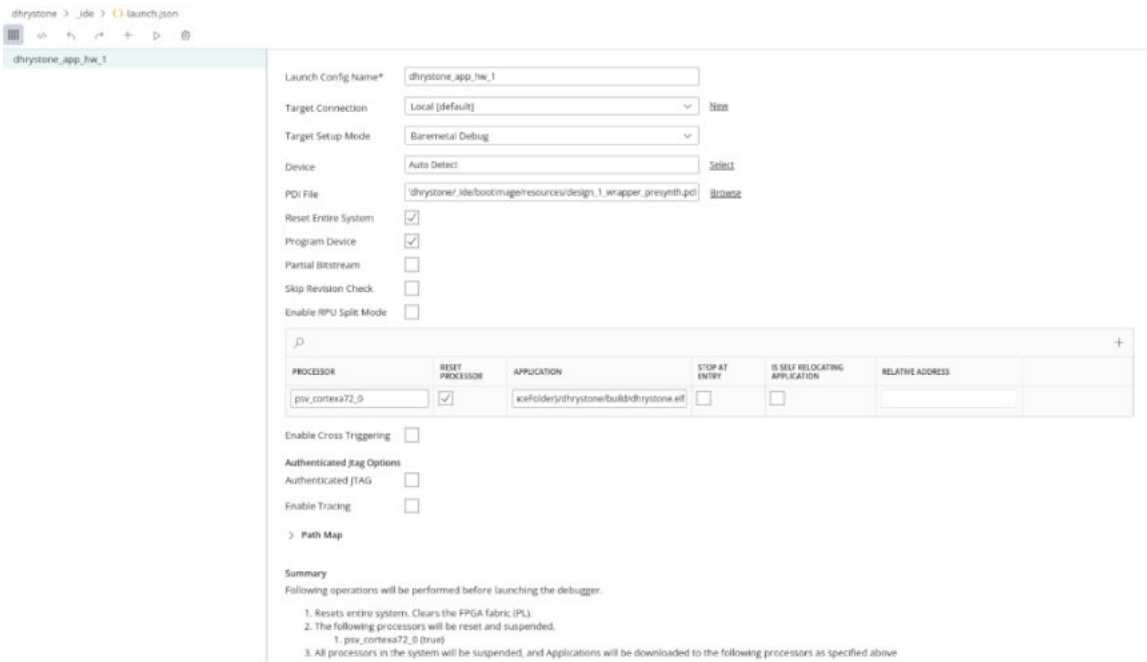
## 主页

主视图包含下列选项：

- “Launch Config Name”（启动配置名称）：您可指定配置名称
- “Target Connection”（目标连接）：在“Connection”字段中，可以通过单击“New”（新建）来选择目标或创建目标连接。
- “Target Setup Mode”（目标设置模式）：您可选择独立调试或者连接运行中的目标。
- “Bitstream File”（比特流文件）：您可浏览至对应目录以指定比特文件
- “Board Initialization”（开发板初始化）：您可使用 FSBL 或 Tcl 来初始化开发板。
- “FSBL file”（FSBL 文件）：指定用于执行初始化的 FSBL 文件
- “FSBL Exit Function”（FSBL 退出函数）：指定 FSBL 退出函数

- “Init With FSBL”（使用 FSBL 初始化）：使用 FSBL 来初始化 PS。
- “Reset Entire System”（复位整个系统）：如果系统中仅有 1 个处理器，则执行系统复位
- “Reset APU”（复位 APU）：复位所有 APU 核
- “Reset RPU”（复位 RPU）：复位所有 RPU 核
- “Enable RPU Split Mode”（启用 RPU 拆分模式）：将 RPU 核置于拆分模式下，以便使每个 RPU 核彼此独立
- “Program Device”（器件烧录）：允许工具将比特文件烧录到器件中
- “Enable Cross Triggering”（启用交叉触发）：启用交叉触发功能

图 42：启动配置窗口

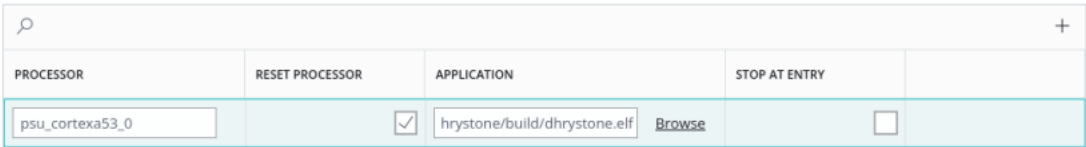


注释：屏幕底部会显示启动配置的汇总信息。

应用部分

在 “Launch configuration”（启动配置）主视图中，您可以为应用工程设置详细信息并选中 ELF 文件。

图 43：调试配置的应用部分



- “PROCESSOR”（处理器）：指定详细的处理器

- “Reset Processor”（复位处理器）：您可选择此项以复位整个硬件系统或者特定处理器，或者选择不执行复位。执行复位可确保先前调试会话不会产生副作用。
- “APPLICATION”（应用）：指定应用文件
- “STOP AT ENTRY”（在入口处停止）：应用在入口点停止。
- +：如有多个应用采用不同处理器，那么您可单击“+”添加新的处理器和应用配置。

**注释：**将光标悬停在“+”列下方的配置上时，会显示“Edit”（编辑）命令和“Delete”（删除）命令。选中“Delete”（删除）移除一组应用配置，或者选中“Edit”（编辑）以编辑配置。

## 目标连接

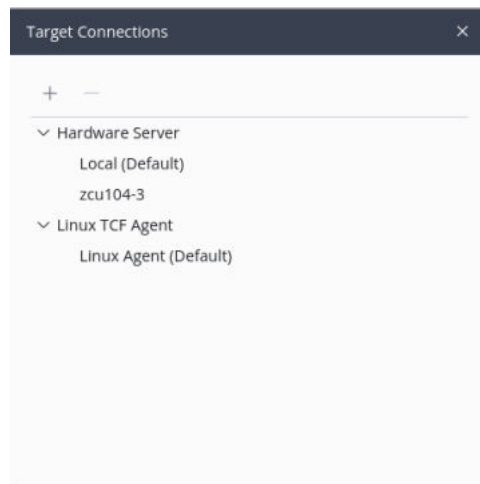
“Target Connections”（目标连接）对话框 (  ) 允许您配置多个远程目标。它可显示已连接的目标，并提供可供您添加或删除目标连接的选项。

Vitis 软件平台通过 AMD `hw_server` 来建立目标连接。硬件服务器代理必须在连接到目标硬件的远程主机上运行。

目标连接扩展到 Vitis Unified IDE 内的所有实用工具，这些实用工具会在运行时处理各目标。

从该菜单中选中“Vitis” → “Target Connections”（Vitis > 目标连接）以打开目标管理器。

图 44：目标连接



## 创建新的目标连接

您可通过在“Target Connections”（目标连接）视图添加新连接来配置远程目标详细信息。

要创建新的目标连接，请执行以下操作。

1. 右键单击要添加连接的“Hardware Server”（硬件服务器）或“Linux TCF Agent”（Linux TCF 代理）。
2. 单击工具栏上的“Add Target Connection”（添加目标连接）按钮（“+”）。
3. 这样会打开“Target Connection Details”（目标连接详情）页面。

4. 在“Target Name”（目标名称）字段中，输入新的远程连接的名称。
5. 选中“Set as default target”（设为默认目标）复选框以将此目标设置为默认目标。Vitis Unified IDE 会将此默认目标用于后续与开发板之间进行的所有交互。
6. 在“Host”（主机）字段中，输入远程主机的名称或 IP 地址。此机器连接到目标，并且 `hw_server` 在此机器上运行。
7. 在“Port”（端口）字段中，输入正在运行 `hw_server` 的端口号。默认情况下，`hw_server` 在端口 3121 上运行。
8. 如果硬件服务器在远程主机上运行，请选择“Use Symbol Server”（使用符号服务器）。
9. 单击“OK”即可创建新目标连接。

**注释：**单击“OK”前，您可单击“Test Connection”（测试连接）来对连接进行测试。

## 设置自定义 JTAG 频率

通过设置 JTAG 频率，可以采用 JTAG 线支持的其他频率来工作。

要设置 JTAG 频率，请执行以下步骤。

1. 右键单击要添加连接的“Hardware Server”（硬件服务器）或“Linux TCF Agent”（Linux TCF 代理）。
2. 单击工具栏上的“Add Target Connection”（添加目标连接）按钮（“+”）。
3. 这样会打开“Target Connection Details”（目标连接详情）页面。
4. 在“Target Name”（目标名称）字段中，输入新的远程连接的名称。
5. 选中“Set as default target”（设为默认目标）复选框以将此目标设置为默认目标。Vitis Unified IDE 会将此默认目标用于后续与开发板之间进行的所有交互。
6. 在“Host”（主机）字段中，输入远程主机的名称或 IP 地址。此机器连接到目标，并且 `hw_server` 在此机器上运行。
7. 在“Port”（端口）字段中，输入正在运行 `hw_server` 的端口号。默认情况下，`hw_server` 在端口 3121 上运行。
8. 如果硬件服务器在远程主机上运行，请选择“Use Symbol Server”（使用符号服务器）。
9. 单击“Advanced”（高级）并选中“Automatically discover devices ON JTAG chain”（自动发现 JTAG 链上的器件），以查看 JTAG 器件链详细信息。
10. 在“Set custom frequency”（设置自定义频率）下拉列表中，选择频率。

**注释：**当前频率可采用服务器设置的默认频率，也可以采用调试客户端设置的自定义频率。

11. 单击“OK”（确定）以保存配置并创建新的目标连接。Vitis 工具会将频率保存在工作空间内，并在为器件执行连接命令时使用此频率。
12. **注释：**如果服务器仅连接到一个客户端，那么一旦此连接关闭，线缆频率就会复位为默认值。请勿从不同客户端同时执行多项调试操作。

## 建立目标连接

要建立目标连接，可使用本地开发板或远程开发板。您可通过检查本地连接来确认到本地开发板的连接。

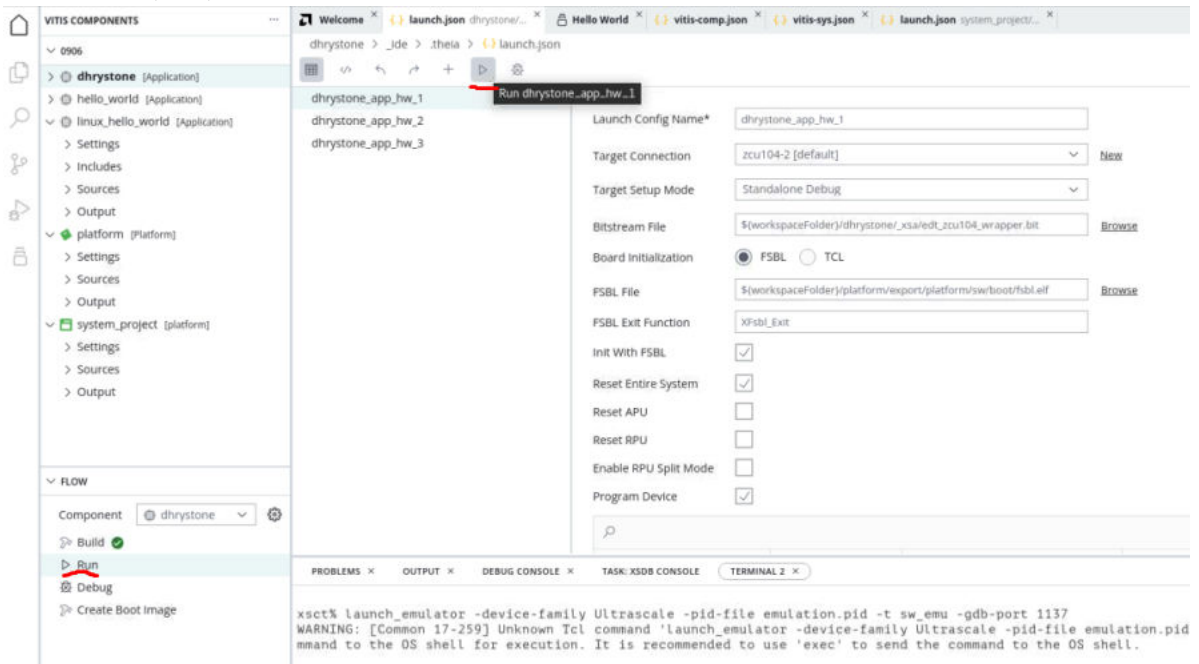
要使用远程开发板来建立目标连接，请执行以下操作：

1. 确保远程主机连接到该目标。

2. 在远程主机上手动启动 `hw_server`：
  - a. 在远程主机上打开 shell。
  - b. 使用 `C:/Xilinx/<version>Vitis/settings64.bat` 或 `<Vitis_local_install_path>/<version>Vitis/settings64.csh` 通过 `source` 命令执行设置脚本。
3. 在连接到开发板的机器上运行 `hw_server`。  
**注释：**确保远程主机连接到该目标（开发板）。
4. 选择端口号和主机名以创建到运行 `hw_server` 的主机的目标连接。
5. 右键单击新创建的目标连接，然后选择“Set As Default”（设为默认）。

## 运行应用组件

1. 转至 Flow Navigator 并选择要运行的应用组件。
2. 单击“Open Settings”启动配置。请参阅 [启动配置](#) 以了解有关启动和配置启动配置文件的信息。
3. 请参阅 [创建新的目标连接](#) 以了解有关设置目标连接的信息。
4. 单击“run”（运行）即可运行应用。



**注释：**运行该应用后，您需在“Debug”（调试）视图中单击“Terminate”（终止）按钮才能终止此会话。

## 调试应用组件

### 在 MicroBlaze V 上调试 Zephyr 应用程序

Vitis Unified IDE 支持您使用用户管理模式对 MicroBlaze V 处理器上运行的 Zephyr 应用程序进行调试

如需了解有关用户管理模式的更多详细信息，请参阅 [用户管理模式](#)。

AMD Zephyr 仓库位于：<https://github.com/Xilinx/zephyr-amd.git>

以下教程演示了如何使用 Vitis Unified IDE 在用户管理模式下调式 Zephyr 应用程序。

#### 设置 Zephyr 环境

1. 首先，遵循 Zephyr 官方指南设置 Zephyr 应用开发环境。请参阅 [Zephyr Getting Started](#)。
2. 用 AMD 克隆来替换您在上一步骤中克隆的上游仓库。

```
cd <local path>/zephyrproject/  
mv zephyr zephyr.upstream  
git clone https://github.com/Xilinx/zephyr-amd.git -b xlnx_rel_v2025.1  
zephyr  
west update  
west lopper-install
```

#### 创建硬件设计

使用基于 Vivado CED 的 MicroBlaze V 设计预设以 AC701 评估板作为目标，加载 Vivado 并生成硬件设计 (XSA)。此外，您也可以选择创建自己的硬件设计。对于您自己的设计，请确保您的硬件设计与当前发布的 zephyr-amd 仓库兼容。

您的设计定稿并导出 XSA 文件后，您即可开始与 Zephyr 集成。

#### 预设设计示例

AMD Vivado 为各种评估板提供预设设计示例。每个嵌入式硬件设计都提供一个子系统，可作为实时与微控制器配置的目标。截至目前，Zephyr 仅支持实时配置。

以下章节演示了如何实现以 AC701 评估板为目标的 MicroBlaze V 设计预设 CED。

#### 生成系统设备树

如需了解使用硬件设计 XSA 生成系统设备树 (SDT) 的步骤，请参阅《Vitis 统一软件平台文档：嵌入式软件开发》(UG1400) 中的[使用 SDTGen 生成系统设备树](#)。要使用 west lopper 命令为目标硬件设计生成 Zephyr 设备树和 Kconfig，SDT 是必需的。

```
. /tools/AMD/Vivado/2025.2/settings64.sh  
sdtgen sdt.tcl
```

## 设置 Zephyr 仓库

由于基于 FPGA 的每个硬件设计都可能有不同的硬件配置，因此您需要根据目标硬件设计来更新设备树 (boards/amd/<board\_name>/<board\_name>.dts)。基址和中断 ID 可能与默认设备树中的配置不同。您需要根据硬件设计手动更新 .dts 和 kconfigs (soc/xlnx/<board\_name>/Kconfig, soc/xlnx/kconfig.defconfig)，或者使用 west lopper 命令通过 Lopper 对其进行设置。后续章节提供了使用 west lopper 命令为目标硬件设计设置 DTS 的步骤。

```
LOPPER_DTC_FLAGS="-b 0 -@" west lopper-command -p processor_instance_name
-s output_dir/system-top.dts -w <local path>/zephyrproject/zephyr
```

### 用法示例

```
LOPPER_DTC_FLAGS="-b 0 -@" west lopper-command -p microblaze_riscv_0 -s
design_1_wrapper/system-top.dts -w ~/zephyrproject/zephyr
```

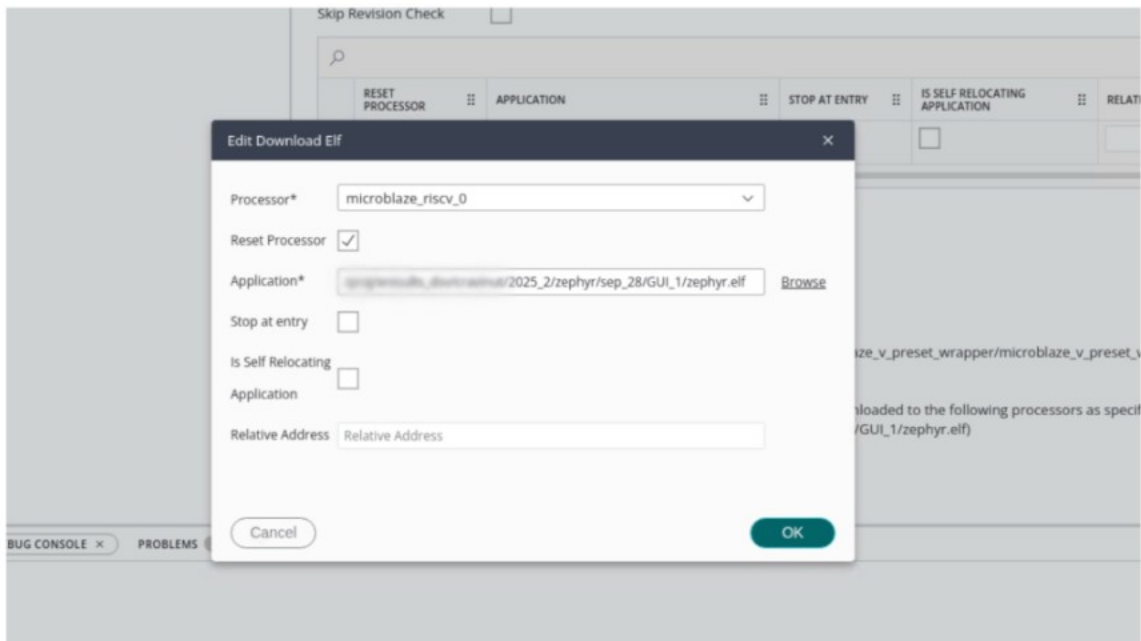
调用 lopper 命令后，Zephyr 环境即已准备就绪，可立即用于为目标硬件设计构建内核测试和应用程序。使用以下命令将评估板替换为您的目标以构建一个样本测试。

```
cd zephyr
west build -p -b board tests/misc/test_build/
# The executable would be generated in following location
<zephyr repository path>/build/zephyr/zephyr.elf
west build -p -b mbv32 tests/misc/test_build/
```

## 创建启动配置

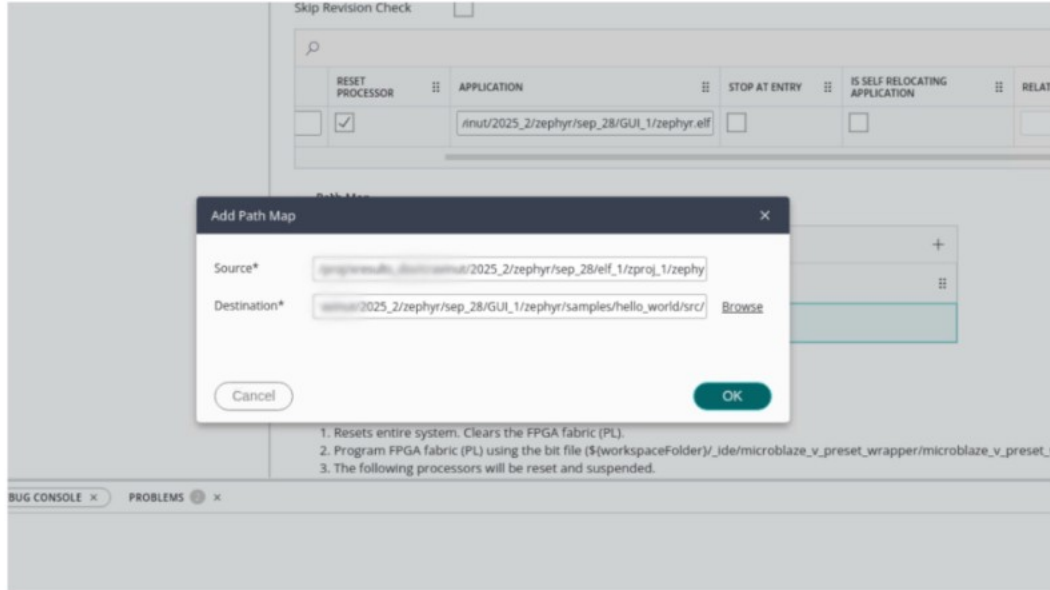
加载 Vitis 统一软件平台 2025.2 版本并打开新文件夹。此文件夹会在“User Managed Mode”（用户管理模式）下打开 Vitis。创建新的启动配置，步骤如下：右键单击“Explorer”（资源管理器）视图并选择“Edit Launch Configurations”（编辑启动配置），然后选择“Add Configuration”（添加配置）。

将目标处理器设为 microblaze\_riscv\_0 并提供 zephyr.elf 应用文件的路径。



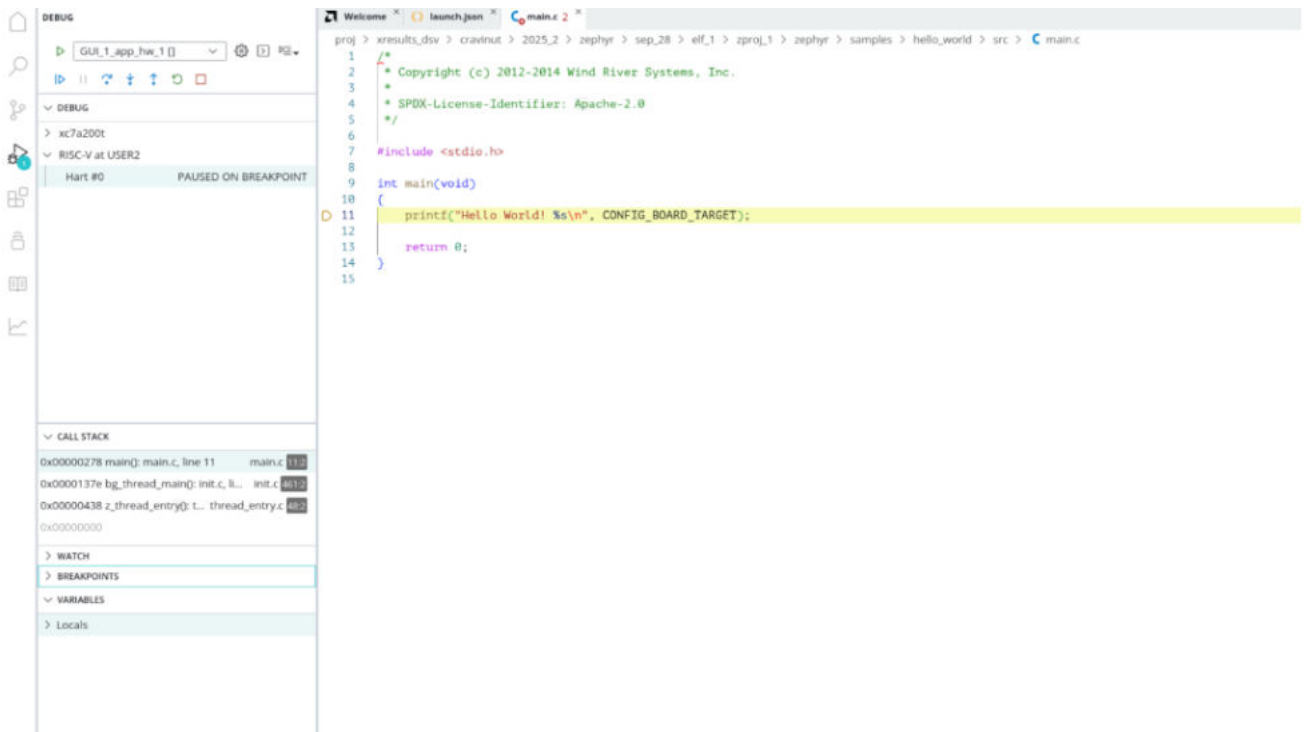
如果应用程序源文件路径已更改或 ELF 文件位置不在您的工作空间内，那么您需要重新映射源文件的原始路径和源代码的目标路径。

图 45：添加路径映射



启动调试会话。这样会在调试透视图中加载 `main.c` 文件。

图 46：在调试透视图中启动 `main.c`



## PS Trace

Vitis Unified IDE 提供处理器系统 (PS) 追踪支持，以便进行调试分析。PS Trace 记录处理器执行流，因此您可以检查仅使用断点和步进难以观察的运行时行为。

PS Trace 在调试会话期间于定义的追踪窗口中捕获执行数据。您可以使用捕获的追踪来调查控制流、分析运行时行为并支持根因分析。

PS Trace 根据目标和版本支持情况支持多种捕获模式，包括片上和片外追踪。

### OpenCSD 设置

Vitis Unified IDE 使用 OpenCSD 对追踪数据进行解码，以便在追踪分析视图中显示。如果您的环境要求显式指定 OpenCSD 路径，请在运行 PS Trace 之前在 IDE 设置中进行配置。

要配置 OpenCSD：

1. 使用您的环境支持的首选方法在系统上安装 OpenCSD。
2. 启动 Vitis Unified IDE。
3. 打开“Settings”（设置）。
4. 指定“OpenCSD installation path”（OpenCSD 安装路径）。
5. 保存设置。

### 执行 PS Trace

要运行 PS Trace：

1. 为您的应用创建或打开一个调试启动配置。
2. 在启动配置选项中启用 PS Trace。
3. 根据需要配置追踪存储器和输出设置。
4. 启动调试会话。

PS Trace 捕获使用的是环形缓冲器。缓冲器大小决定保留多少执行历史记录。当捕获超出缓冲器容量时，较新的数据会替换较旧的数据。

### 定义追踪窗口

PS Trace 捕获通常使用“trace start”（追踪开始）和“trace stop”（追踪停止）断点来定义要追踪的代码区域。在您希望开始追踪捕获的位置放置一个“trace start”断点，并在追踪捕获应结束的位置放置一个“trace stop”断点。

当执行到达追踪开始断点时，捕获开始。当执行到达追踪停止断点时，捕获结束并开始追踪处理。

### 分析 PS Trace 数据

捕获和处理完成后：

1. 在 Vitis Unified IDE 中打开生成的追踪摘要文件。
2. 使用“Trace File”（追踪文件）视图检查已解码的执行和控制流。
3. 如果需要，请配置源路径映射，以将追踪到的指令解析到本地源文件。
4. 浏览追踪输出以诊断问题并分析行为。

## PS Trace 和调试会话行为

PS Trace 扩充了标准调试工作流程。启用 PS Trace 时，断点和步进仍然可用。PS Trace 设置适用于当前活动的调试会话。

## 开始调试

以下章节描述了启动调试过程的步骤。

## 调试独立应用组件

本主题描述了如何使用调试器来调试独立应用。

1. 转至 Flow Navigator 并选择应用组件。
2. 打开“Debug Settings”（调试设置）并参阅 [启动配置](#) 以了解有关为应用组件创建新配置的信息。
3. 请参阅 [创建新的目标连接](#) 以了解有关设置目标连接的信息。
4. 单击“Debug”开始调试。

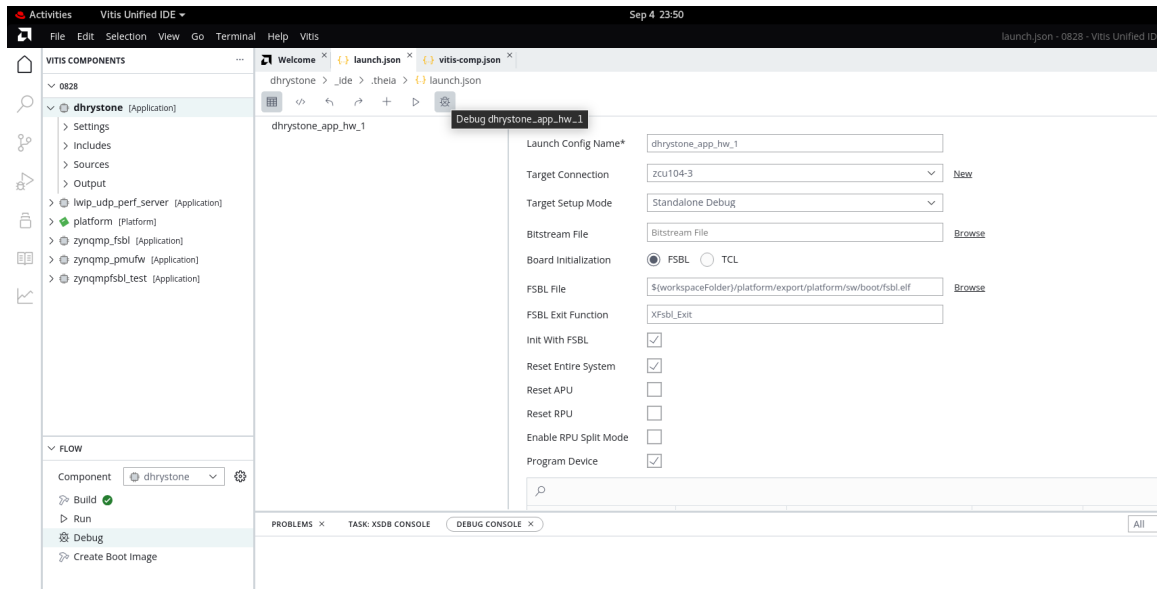
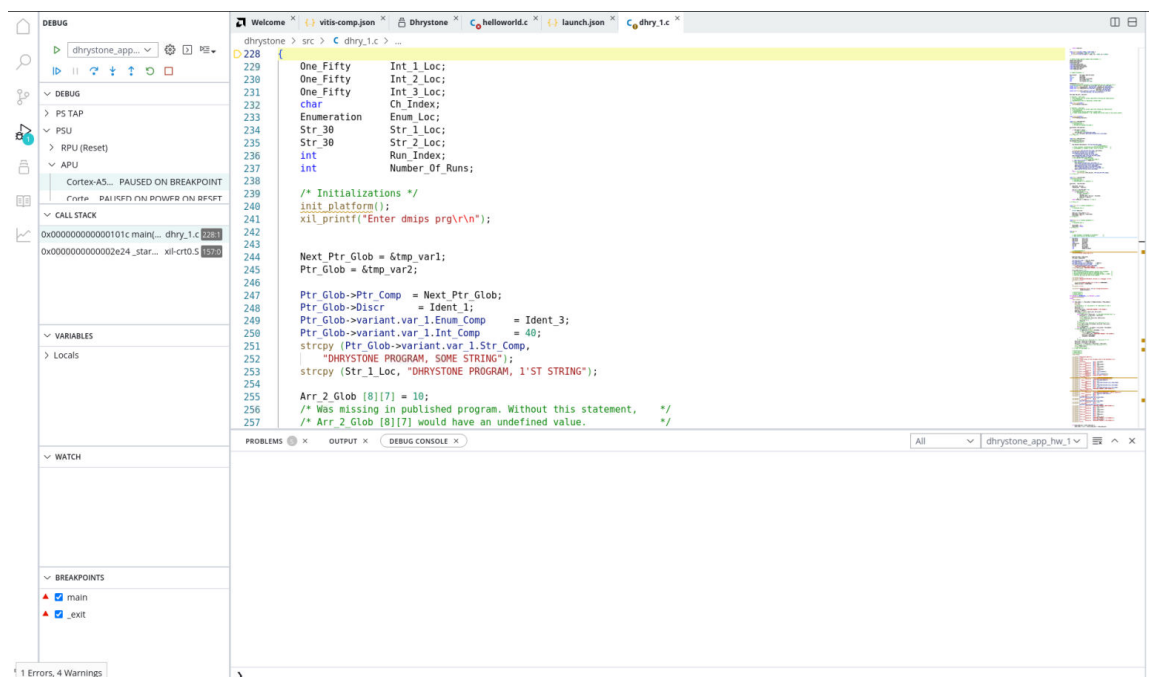


图 47: “Debug” 视图

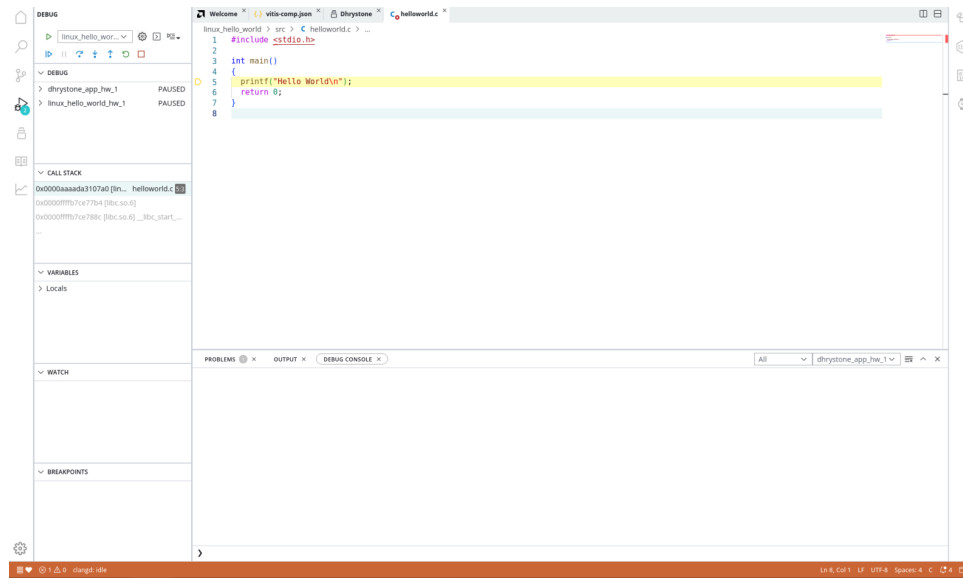


## 调试 Linux 应用组件

以下步骤解释了如何调试 Linux 应用组件。

1. 转至“Flow Navigator”并选择要调试的 Linux 应用组件。
2. 打开“Debug Settings”（调试设置），参阅 [启动配置](#) 以了解有关启动和编辑配置文件的信息。
3. 请参阅 [创建新的目标连接](#) 以了解有关设置目标连接的信息。
4. 单击“Debug”开始调试。

图 48: “Debug” 视图



## 在 QEMU 上调试独立应用组件

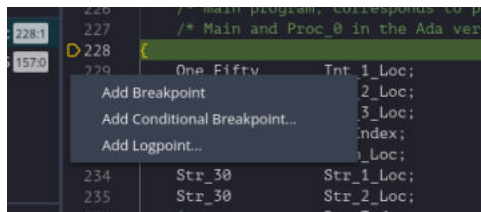
AMD Vitis™ Unified IDE 尚未支持该功能特性。如果您的工程有此需求，请使用传统 AMD Vitis™ IDE。请参阅 2023.1 文档，以获取有关如何在传统 IDE 中使用此功能特性的详细信息。

## 设置条件断点

该功能特性有助于您在调试过程中设置条件断点

要添加条件断点，请执行以下步骤。

1. 右键单击行号前的左侧边距。
2. 选中“Add Conditional Breakpoint”（添加条件断点）。



3. 输入求值结果为布尔值的任意表达式，并按 Enter 键以添加条件断点。

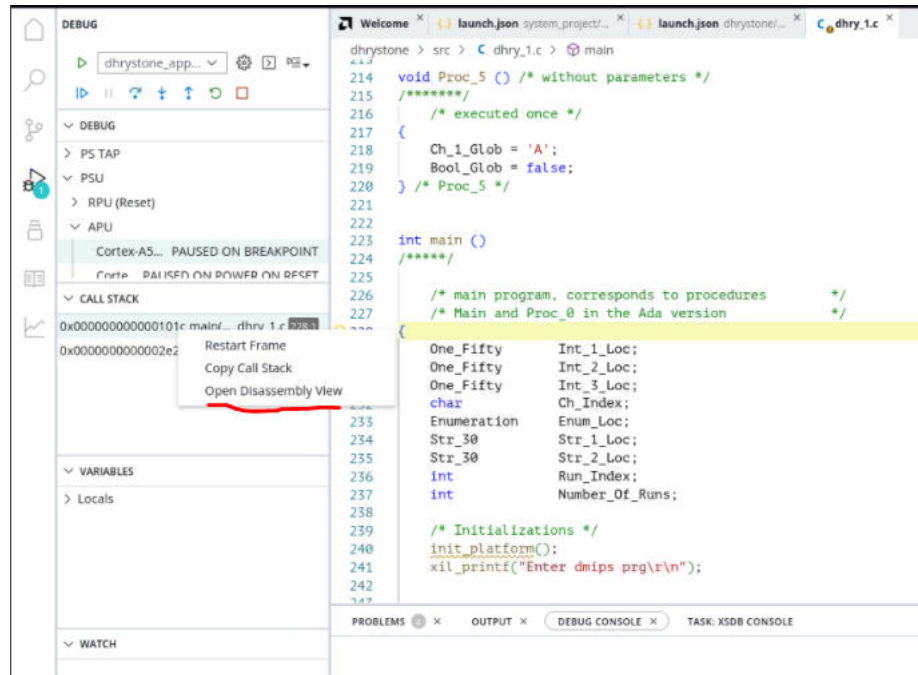
要将正常断点更改为条件断点，请遵循以下步骤进行操作。

- 右键单击现有断点
- 选择“Edit Breakpoint”（编辑断点）。
- 添加表达式以将其更改为条件断点。

## 查看反汇编代码

要查看反汇编代码，请在调试会话启动后右键单击栈视图中的函数栈。

图 49：查看反汇编代码



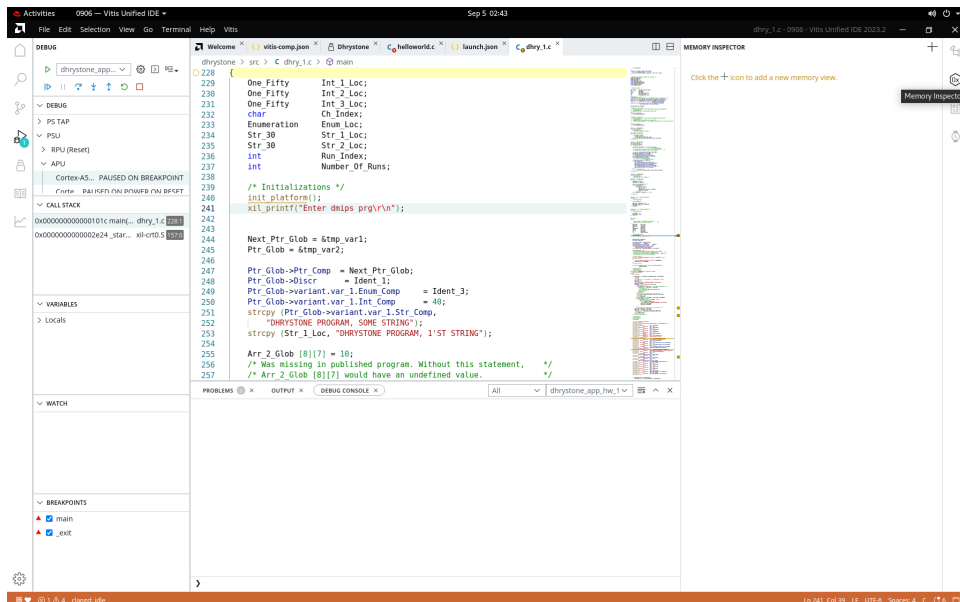
下一步，选中“Open Disassembly”（打开反汇编）以查看显示的反汇编代码。

## 查看存储器

### 查看特定存储器地址的值

请遵循以下步骤查看特定存储器地址的值：

1. 开始调试会话后，单击“Memory Inspector”（存储器检查器）即可查看位于右上角的存储器信息。

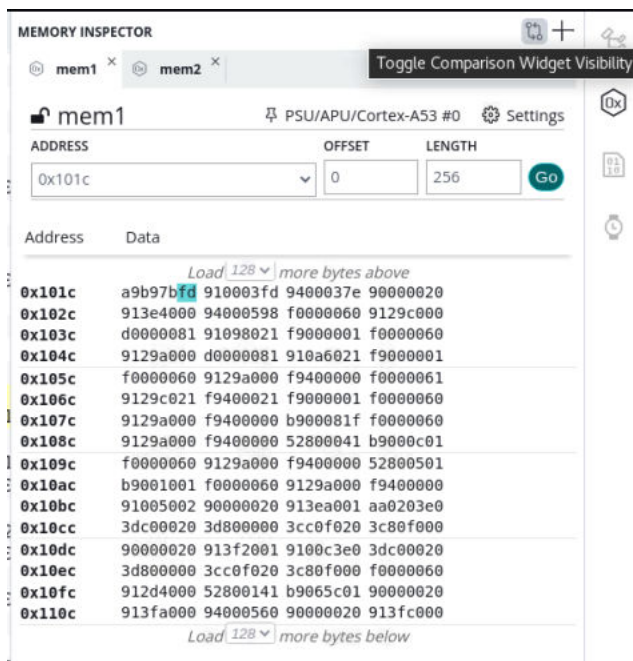


2. 单击 “+” 图标即可添加新的存储器视图。
3. 设置存储器地址、偏移和长度。单击 “Go” （执行）。

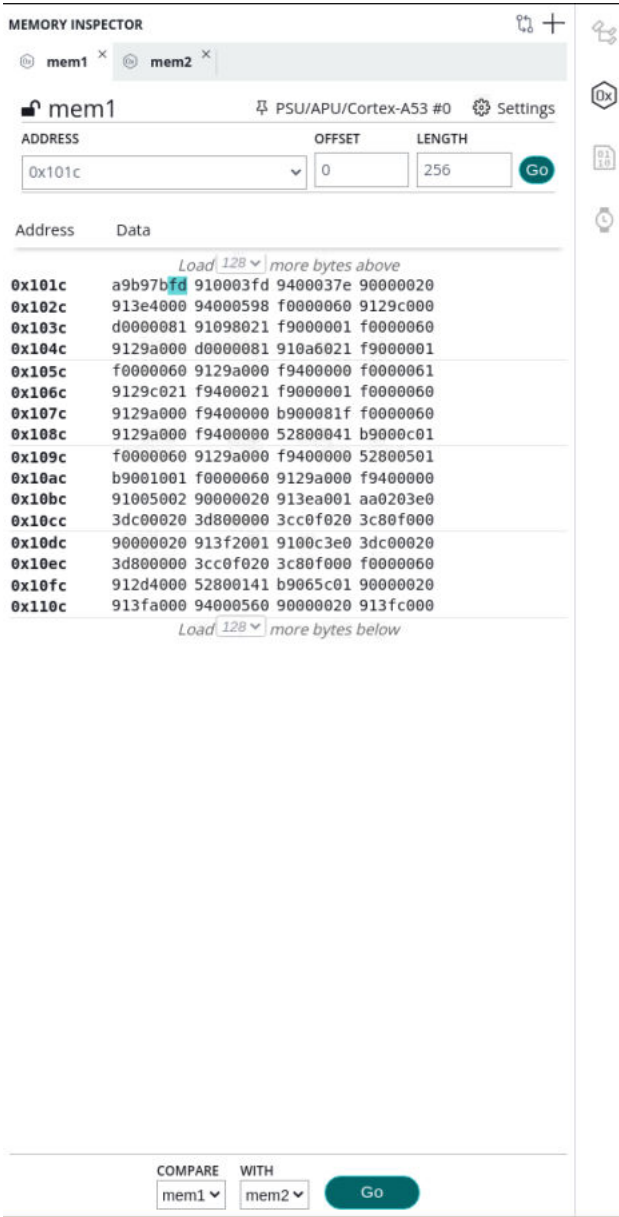
## 比较两个地址的存储器值

请遵循以下步骤来比较存储器地址的值：

1. 请参阅 [查看特定存储器地址的值](#) 以了解有关创建两个存储器检查器的信息。设置两个检查器的差异。



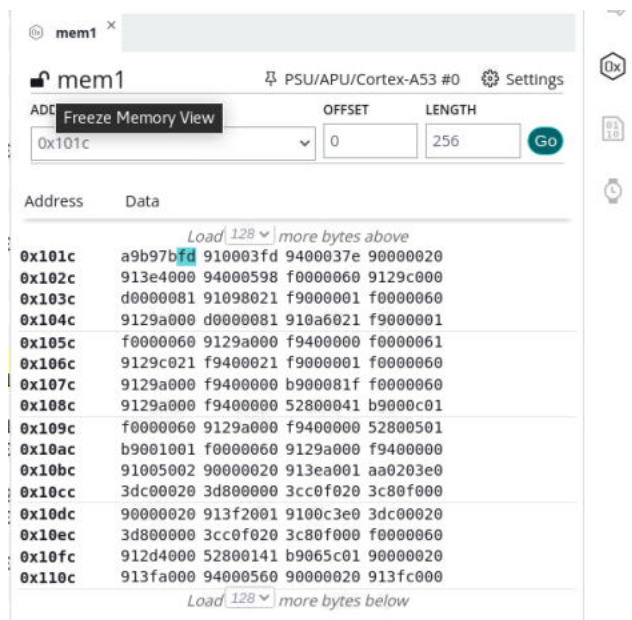
2. 单击 “Toggle Comparison Widget Visibility” （切换比较小组件可视性）以比较差异。
3. 选中要比较的存储器，然后单击 “Go” （执行）。



冻结存储器值

此功能特性会保留一份存储器内容的快照，直至关闭 Vitis 工具为止。您可通过对比存储的值或者使用此信息来调试问题。要使用此功能，请遵循以下步骤进行操作：

- 1. 请参阅 [查看特定存储器地址的值](#) 以了解有关创建新的存储器检查器的信息。
- 2. 单击锁定图标以冻结存储器视图。

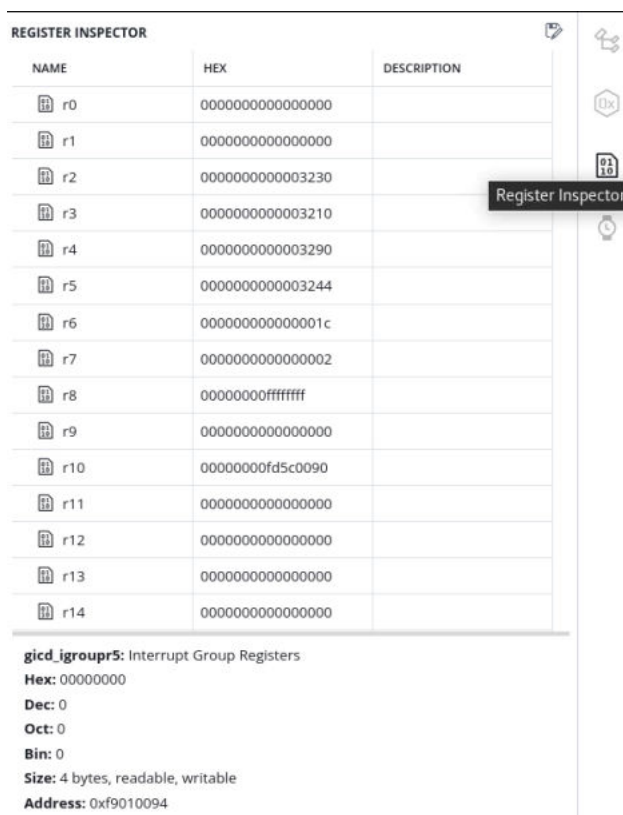


## 查看寄存器

“Registers Inspector”（寄存器检查器）会列出所有寄存器，包括通用寄存器、系统寄存器和 IP 寄存器。例如，对于 AMD Zynq™ 器件，在“Debug”（调试）视图中选中 Cortex®-A9 目标时，“Registers”（寄存器）视图会显示所有处理器和协处理器寄存器。选中 APU 目标时，“Registers”视图会显示系统寄存器和 IOU 寄存器。

要查看寄存器值，打开调试视图后，请单击“Register Inspector”按钮。您也可从“View” → “Register Inspector”（视图 > 寄存器检查器）打开“Register Inspector”。

图 50: “Register Inspector” 窗口



| NAME | HEX              | DESCRIPTION |
|------|------------------|-------------|
| r0   | 0000000000000000 |             |
| r1   | 0000000000000000 |             |
| r2   | 0000000000003230 |             |
| r3   | 0000000000003210 |             |
| r4   | 0000000000003290 |             |
| r5   | 0000000000003244 |             |
| r6   | 00000000000001c  |             |
| r7   | 000000000000002  |             |
| r8   | 00000000ffffff   |             |
| r9   | 0000000000000000 |             |
| r10  | 00000000fd5c0090 |             |
| r11  | 0000000000000000 |             |
| r12  | 0000000000000000 |             |
| r13  | 0000000000000000 |             |
| r14  | 0000000000000000 |             |

**gicd\_igroupr5:** Interrupt Group Registers  
**Hex:** 00000000  
**Dec:** 0  
**Oct:** 0  
**Bin:** 0  
**Size:** 4 bytes, readable, writable  
**Address:** 0xf9010094

您可在调试期间修改可编辑的字段值。您也可单击对应的寄存器名称以查看详细信息。

## 从 Vitis IDE 导出寄存器

该功能支持您更轻松地将目标处理器中存在的寄存器导出至文本文件并读取所有寄存器值。

1. 选择应用组件，并启动调试会话。欲知详情，请参阅 [开始调试](#)。
2. 打开“Register Inspector”（寄存器检查器）视图。欲知详情，请参阅 [查看寄存器](#)。
3. 单击“Export Registers”（导出寄存器）按钮即可导出寄存器。

| REGISTER INSPECTOR |                  |             | Export Registers |
|--------------------|------------------|-------------|------------------|
| NAME               | HEX              | DESCRIPTION |                  |
| r0                 | 0000000000000000 |             |                  |
| r1                 | 0000000000000000 |             |                  |
| r2                 | 0000000000003230 |             |                  |
| r3                 | 0000000000003210 |             |                  |
| r4                 | 0000000000003290 |             |                  |
| r5                 | 0000000000003244 |             |                  |
| r6                 | 00000000000001c  |             |                  |
| r7                 | 0000000000000002 |             |                  |
| r8                 | 00000000ffffff   |             |                  |
| r9                 | 0000000000000000 |             |                  |
| r10                | 00000000fd5c0090 |             |                  |
| r11                | 0000000000000000 |             |                  |
| r12                | 0000000000000000 |             |                  |
| r13                | 0000000000000000 |             |                  |
| r14                | 0000000000000000 |             |                  |

4. 在下一屏幕添加以下信息：

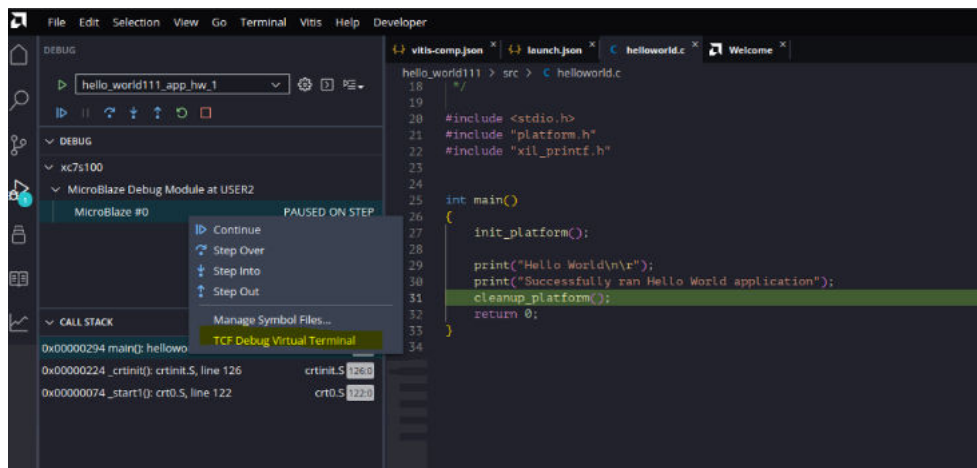
- “Location”（位置）：提供寄存器转储文件的保存位置。
- “Select registers/groups to export”（选择要导出的寄存器/组）：选择要转储的寄存器列表。取消勾选您不需要的所有寄存器。

5. 单击“OK”（确定）即可将寄存器转储至您在上一步中指定的位置。

## 使用虚拟 UART 终端

针对 MDM 终端支持 TCF 虚拟 UART 终端。右键单击 MicroBlaze™ 核或 MicroBlaze™ V 核，并选择“TCF Debug Virtual Terminal”（TCF 调试虚拟终端）。

图 51：TCF Debug Virtual Terminal

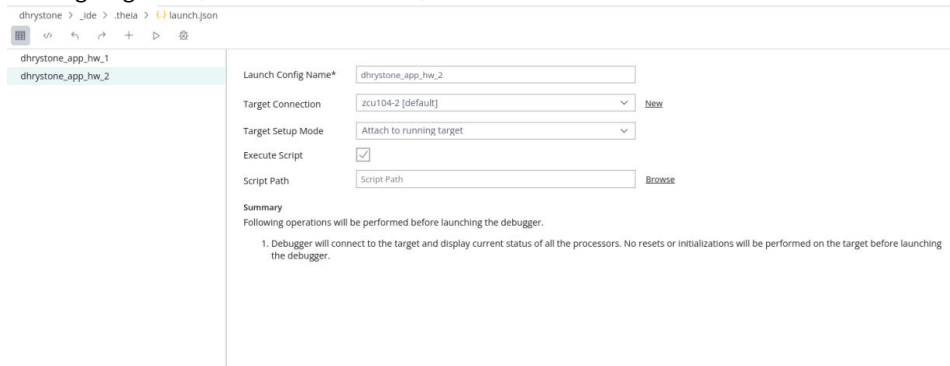


## 调试已在目标器件上运行的应用组件

调试器允许对已在目标器件上运行的应用进行调试。例如，独立应用组件或从闪存器件启动的 Linux 内核。以下提供了连接到目标上运行的应用组件的步骤。

**注释：**调试器不会修改目标器件上的处理器状态，只是连接到这些处理器而已。您可以中止这些处理器，并从当前 PC 进行调试。

1. 创建到主机的目标连接，该主机连接到硬件开发板。如果硬件开发板连接到正在运行 Vitis Unified IDE 的机器，则可跳过此步骤。您可在涉及远程开发板和远程连接的后续步骤中使用默认的“Local connection”（本地连接）。
  - a. 请参阅 [创建新的目标连接](#) 以了解有关创建目标连接的信息。
2. 请参阅 [启动配置](#) 以了解有关启动调试配置的信息。将“Target Setup Mode”（目标设置模式）设置为“Attach to running target”（连接到运行中的目标）并选中目标连接。



3. 遵循 [开始调试](#) 开始调试应用。

## 源代码重映射

当您编译启用了调试信息的应用时（例如，使用 `-g`），编译器会在 ELF 调试信息中记录源文件的绝对路径。调试器使用这些记录的路径将程序地址映射到源代码行，并在调试会话期间在编辑器中打开相应的源文件。

如果调试器无法在记录的路径中找到源文件，就无法正确打开文件或正确绑定断点。通常下列情况下会发生此问题：在您构建 ELF 之后移动源代码树、调试在另一台机器上构建的应用，或者跨操作系统调试时。源代码重映射通过将 ELF 中记录的原始源路径映射到当前本地存在相同源文件的路径来解决此问题。

### 何时使用源代码重映射

当 ELF 中记录的源路径在您的系统上不存在时，请使用源代码重映射。常见场景包括调试在其他机器上构建的 ELF、在不重新构建源代码的情况下调试预构建镜像、在构建后移动源代码树，或者在“Call Stack”（调用栈）视图显示的文件路径在您的系统上无效时进行调试。

### 重要注释

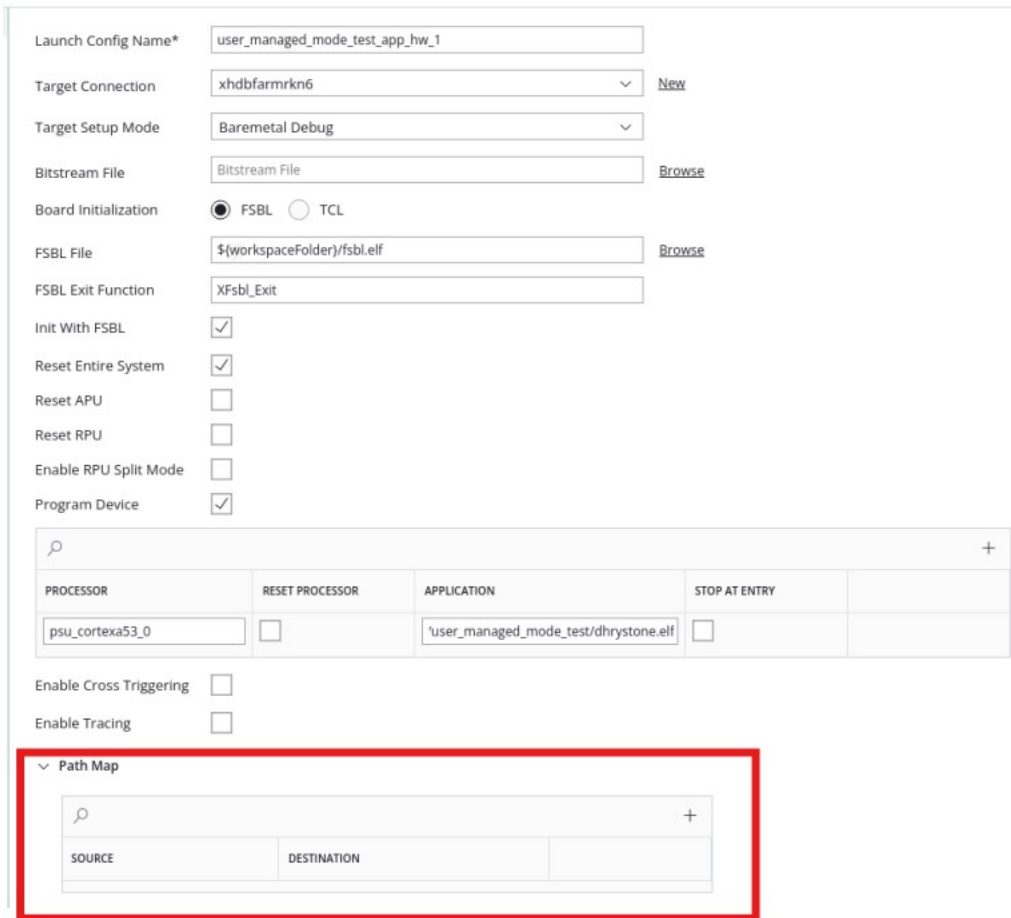
目标源文件必须与用于构建 ELF 的源文件一致。如果源文件不同，调试器可能会显示错误的行映射，或将断点绑定到意外的位置。如果构建记录了多个源代码根目录，您必须为每个不同的源代码根目录添加一个映射。

### 源代码重映射的工作方式

在调试期间，调试器会读取存储在 ELF 调试信息中的原始源文件路径。对于您定义的每个路径映射，调试器都会将记录的路径前缀替换为您提供的目标路径前缀。随后，调试器在解析栈帧或显示源代码时，会从目标路径加载源文件。

## 在 Vitis Unified IDE 中添加路径映射

1. 转至调试启动配置，并展开“Path Map”（路径映射）。



Launch Config Name\*

Target Connection  [New](#)

Target Setup Mode

Bitstream File  [Browse](#)

Board Initialization ☒ FSBL ☐ TCL

FSBL File  [Browse](#)

FSBL Exit Function

Init With FSBL ☒

Reset Entire System ☒

Reset APU ☐

Reset RPU ☐

Enable RPU Split Mode ☐

Program Device ☒

| PROCESSOR       | RESET PROCESSOR          | APPLICATION                            | STOP AT ENTRY            |
|-----------------|--------------------------|----------------------------------------|--------------------------|
| psu_cortexa53_0 | <input type="checkbox"/> | 'user_managed_mode_test/dhrystone.elf' | <input type="checkbox"/> |

Enable Cross Triggering ☐

Enable Tracing ☐

▼ Path Map

| SOURCE | DESTINATION |
|--------|-------------|
|        |             |

2. 单击“+”添加源文件映射。
  - a. “SOURCE”（源）：表示源文件的原始路径。在“CALL STACK”（调用栈）视图中，您可悬停在源文件名称上以查看此路径。
  - b. “DESTINATION”（目标）：源代码的当前文件路径。
3. 单击“OK”（确定）并启动调试。

提示：在“Call Statck”视图中将鼠标悬停在源文件名称上，即可查看 ELF 中记录的源路径。

### 映射示例

如果 ELF 是使用位于 C:\testapp 下的源文件构建的，请使用 C:\testapp 作为 SOURCE 路径。如果相同的源文件现在位于 /scratch/source 下，请使用 /scratch/source 作为 DESTINATION 路径。此映射会指示调试器从 /scratch/source 加载源文件。

### 验证映射

在断点处停止执行并打开“Call Stack”视图。选择一个栈帧，并确认编辑器打开了正确的本地源文件。如果某些文件仍无法解析，请为构建期间记录的其他源代码根目录添加额外映射。

## 故障排除

如果添加路径映射后断点仍未绑定，则目标源文件很可能与用于构建 ELF 的源文件不一致。如果某些文件可以解析而其他文件不能，则构建很可能记录了多个源代码根目录，并且需要多个映射。如果调试器仍然无法定位源文件，请验证 SOURCE 路径是否与 ELF 中记录的路径前缀匹配。

## 在硬件仿真器上调试应用 (QEMU)

**注释：** Vitis Unified IDE 不支持该功能。如果您要访问该功能，请改用传统 Vitis IDE（在 2024.2 版及更低版本内可用）并参阅 2024.2 文档。

## 在系统工程下同时运行和调试多个应用组件

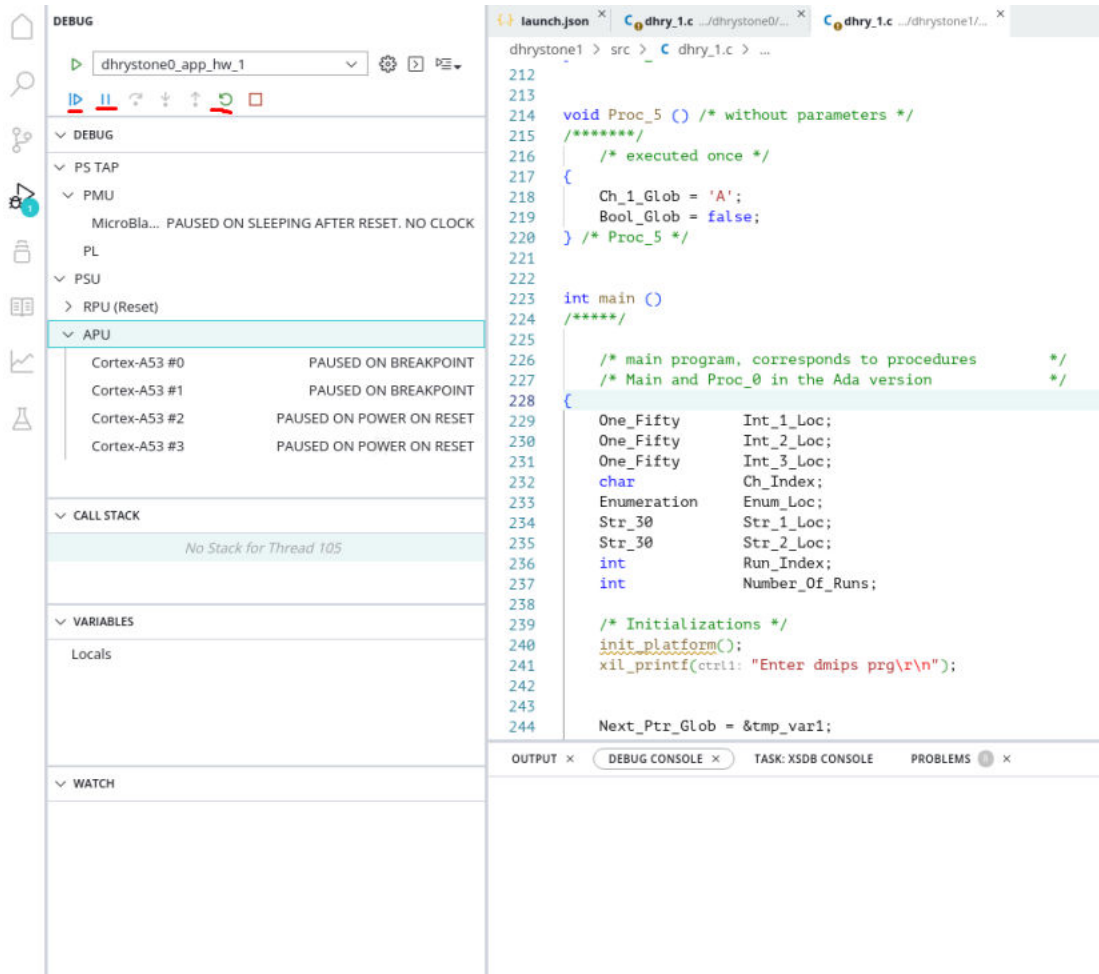
系统工程下的每个应用组件均可单独运行。系统工程下的应用组件也可一起启动。Vitis Unified IDE 可逐一下载和启动每个应用组件。在调试模式下，所有应用都会停止于 `main()`。以下步骤详述了在系统工程下同时运行或调试多个应用组件的步骤。

1. 在“Component”（组件）视图中选中系统工程，转至“Flow Navigator”以开始运行或调试启动配置。

**注释：** 如需了解有关启动新配置的信息，请参阅 [启动配置](#)，如需了解有关设置目标连接的信息，请参阅 [创建新的目标连接](#)。复查配置页面中的应用、处理器和目标设置。

2. 单击“debug”（调试）或“run”（运行）。

**注释：** 选择与相应应用程序对应的核，以执行单步跳过、单步进入和单步跳出操作。您也可以选择父级节点来启动、暂停或恢复所有核。



## 在远程开发板上调试

以下描述了在远程开发板上调试的步骤。

### 1. 设置远程系统环境：

- 以非默认端口（例如，3122）运行 `hw_server` 即可启用远程连接。使用以下命令在端口 3122 上启动 `hw_server`：

```
the hw_server -s TCP::3122
```

- 请确保正确连接到您的开发板。
- 在主机 `cmd` 窗口中，检查 IP 地址：

```
Windows IP Configuration

Ethernet adapter Local Area Connection:

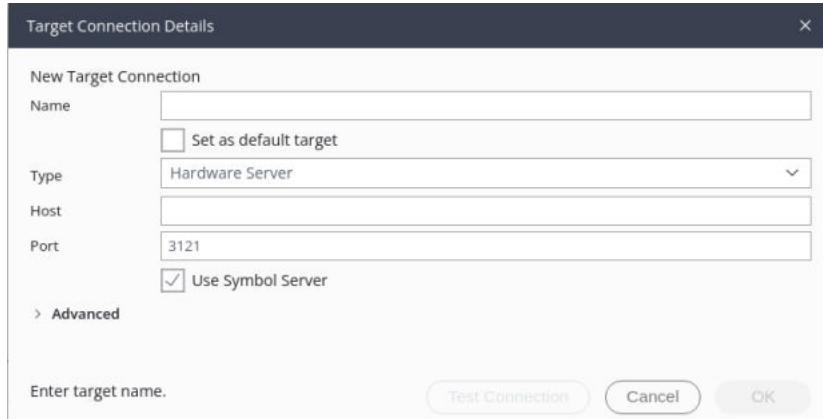
    Connection-specific DNS Suffix  . : xilinx.com xlnx.xilinx.com
    Link-local IPv6 Address . . . . . : fe80::1562:9f81:b0c9:9a65%11
    IPv4 Address. . . . . : 172.23.19.161
```

Remote  
System IP

### 2. 设置本地系统以便执行远程调试：

- 启动 Vitis Unified IDE。

- b. 选择要调试的应用组件。
- c. 转至“Flow Navigator”并单击“Open Launch”（打开启动）配置。欲知详情，请参阅 [启动配置](#)。
- d. 对于“Target Connection”（目标连接）字段，请单击“New”（新建）创建新的目标连接。



- e. 在“New Target Connection” Wizard（新建目标连接向导）中，为连接到目标的远程主机添加必要的详细信息。
- f. 在“Target Name”（目标名称）中，输入目标的名称。
- g. 在“Host”（主机）字段中，输入主机的 IP 地址或名称。
- h. 在“Port”（端口）字段中，输入硬件服务器的启动端口，例如，3122。
- i. 选择“Use Symbol Server”（使用符号服务器）以确保源代码视图在远程调试应用期间可用。符号服务器则充当硬件服务器与 Vitis Unified IDE 之间的中介。
- j. 单击“OK”（确定）。
- k. 在以下截屏中，有两条连接可用。remote\_zc702\_1 是远程连接。



- l. 单击“debug”（调试）按钮。

**注释：**如需了解有关创建目标连接的信息，请参阅 [目标连接](#)。

## 操作系统感知调试

Vitis Unified IDE 当前不支持此功能特性。如果您的工程有此需求，请使用传统 Vitis IDE。请参阅 2023.1 文档，以获取有关如何在传统 IDE 中使用此功能特性的详细信息。

## Xen 感知调试

Xen 是一个开源虚拟机管理器，允许在一台物理机上运行多个虚拟机。Xen 感知调试是指专为调试在 Xen 虚拟机管理器上运行的软件而设计的调试技术与工具。启用 Xen 感知调试支持后，您即可执行 Xen 感知调试。

请遵循以下步骤启用 Xen 感知调试。

1. 请参阅《PetaLinux 工具文档：参考指南》([UG1144](#)) 准备环境，以便启动预构建的 Linux 作为 dom0。
2. 启动 Vitis Unified IDE。

3. 创建并构建应用。
4. 启动“Debug”（调试）配置。
5. 单击“New Launch Configuration”（新建启动配置）。
6. 选择“Target Connection”（目标连接），将目标模式设置为“Attach to running process”（连接到运行中的进程）。
7. 单击“Debug”按钮开始调试会话。
8. 右键单击 APU Cortex A72 #0，然后单击“Debug”视图中的“Manage symbol files”（管理符号文件）。
9. 单击“+”按钮添加符号。选择在第一步的 PetaLinux 工程中生成的 `xen-syms` 符号文件，启用以下所有选项，然后单击“Ok”（确定）按钮。该步骤会映射 Xen 虚拟机管理器的符号。
10. 右键单击 VCPU #0，然后单击“Manage Symbol files”（管理符号文件）。为 DOM0 添加来自第一步中的 PetaLinux 工程的 `vmlinux` 符号文件。在此步骤中，请勿启用“OS Awareness”（操作系统感知）选项。
11. 选择 VCPU #0，并在“Debug”视图中选择“Pause”（暂停）。
12. 检查“Call Stack”（调用栈）视图，确保所有符号文件的映射。使用单步跳入和单步跳过选项进行进一步调试。

## 调试自我重定位程序

自我重定位程序会在运行时对自己的代码节和数据节进行重定位，以帮助您加载和重定位应用程序。ELF 文件中提供的调试信息并未指定程序节的重定位位置。因此，您必须为调试器提供相对地址，以便调试器对程序节进行重定位。您可以通过 Vitis Unified IDE 或 CLI 来完成此操作。

要在 Vitis Unified IDE 中执行该过程，请更新您要调试的应用的 `launch.json` 文件。选中“Is Self Relocating Application”（是自我重定位应用），并提供相对地址，如以下截屏所示。

图 52：调试自我重定位程序

| PROCESSOR       | RESET PROCESSOR                     | APPLICATION                             | STOP AT ENTRY            | IS SELF RELOCATING APPLICATION      | RELATIVE ADDRESS |
|-----------------|-------------------------------------|-----------------------------------------|--------------------------|-------------------------------------|------------------|
| psv_cortexa72_0 | <input checked="" type="checkbox"/> | sceFolder/dhrystone/build/dhrystone.elf | <input type="checkbox"/> | <input checked="" type="checkbox"/> |                  |

使用以下命令，通过提供程序节驻留位置的地址，对 XSDB 执行该过程：`memmap -reloc <addr> -file <path-to-elf>`。

## 使用硬件仿真器调试应用工程（命令行流程）

**注释：** Vitis Unified IDE 不支持该功能。如果您要使用该功能，请改用传统 Vitis IDE 并参阅 2023.1 文档。

## 在 Visual Studio Code 中设置 Vitis 调试器扩展

本主题介绍如何为 Visual Studio Code 安装 Vitis 调试器扩展、配置与目标的目标机通信框架 (TCF) 连接，以及启动调试会话。



**重要提示！** 在 Vitis 2026.1 中，适用于 Visual Studio Code 的 Vitis 调试器扩展是在**内部**软件交付里程碑发布的。该功能已针对内部使用完成验证，不建议用于量产客户流程。如需量产支持的调试体验，请使用 Vitis Unified IDE。

### 准备工作

- 安装 Vitis 2026.1，并选择调试器组件。
- 在主机上安装 Visual Studio Code。
- 构建您要调试的应用。确保可执行文件 (.elf) 以及必需的任何符号文件在主机上均可用。
- 对于裸机目标，请准备好 PDI 或比特流、FSBL 以及器件初始化脚本。请参阅本主题末尾的 [裸机目标](#)。
- 确认与目标硬件或用于硬件仿真的 QEMU 实例之间具有物理连接或网络连接。

### 安装 Vitis 调试器扩展

1. 启动 Vitis 安装程序。
2. 在组件列表中选择适用于 Visual Studio Code 的 Vitis 调试器扩展。
3. 完成安装。安装程序会部署该扩展，使其与 Vitis 调试基础架构集成。
4. 打开 Visual Studio Code，并确认 Vitis 调试器扩展显示在“Extensions”（扩展）视图中。

### 配置目标机通信框架连接

该扩展支持下表列出的三种 TCF 连接类型。请选择与您的目标匹配的类型。

表 9：目标通信框架连接类型

| 连接类型         | 目标                 | 注释                                                       |
|--------------|--------------------|----------------------------------------------------------|
| hw_server    | 物理硬件上的裸机应用         | 连接到运行 hw_server 的开发板。需要大量的 targetSetup 配置。请参阅本主题末尾的裸机目标。 |
| Linux TCF 代理 | 物理硬件上的嵌入式 Linux 应用 | 连接到目标操作系统上运行的 TCF 代理。                                    |
| QEMU         | 硬件仿真               | 连接到主机或远程计算机上运行的 QEMU 实例。                                 |

要配置连接，请执行以下操作：

1. 在 Visual Studio Code 中，打开包含待调试的应用源代码的工作空间。
2. 打开扩展设置，并找到“Target Communication Framework”（目标通信框架）配置。
3. 为您的目标选择连接类型：hw\_server、Linux TCF Agent 或 QEMU。
4. 输入所选连接类型的主机名或 IP 地址以及端口。

5. 保存配置。

### 启动调试会话

1. 在 Visual Studio Code 中，打开“Debug”（调试）视图。
2. 选择以 Vitis 调试器为目标的启动配置。
3. 指定应用可执行文件 (.elf) 的路径。
4. 启动调试会话。该扩展通过已配置的 TCF 连接来连接到目标，并在程序入口点或第一个断点处停止。
5. 使用标准的 Visual Studio Code 调试控件来设置断点、单步执行、检查寄存器和变量、浏览调用栈以及切换线程。

**注释：**调试会话设置仅适用于当前活动会话。该扩展不会修改工程文件或应用二进制文件。

### 裸机目标

通过 Visual Studio Code 扩展进行裸机调试需要大量 `targetSetup` 配置，包括 PDI 或比特流、FSBL 以及器件初始化。您必须通过扩展设置手动提供并维护这些工件。

对于依赖硬件设置、目标编程或工程集成调试的裸机工作流程，请改用 Vitis Unified IDE。Vitis Unified IDE 将目标配置、编程和调试会话作为单一集成流程进行管理。

## 在 Visual Studio Code 中使用 Vitis 调试器扩展

适用于 Visual Studio Code 的 Vitis 调试器扩展可让您在 Visual Studio Code 工作空间内，对面向受支持 AMD 平台的嵌入式 Linux 和裸机应用进行调试。该扩展通过“Target Communication Framework” (TCF)（目标通信框架）连接到目标，并通过 Visual Studio Code 标准调试接口提供调试操作，包括断点、步进控制、寄存器和变量检查、调用栈导航以及线程视图。



**重要提示！** 在 Vitis 2026.1 中，此功能是在内部软件交付里程碑交付的。该功能已针对内部使用完成验证，不建议用于量产客户流程。如需量产支持的调试体验，请使用 Vitis Unified IDE。

从 Vitis 安装程序安装该扩展，然后打开一个包含待调试的应用源代码的 Visual Studio Code 工作空间。根据目标，该工作空间可包含 Linux 或裸机应用源代码。

### 支持的工作流程

该扩展支持以下调试工作流程：

- 在受支持的 AMD 平台上进行嵌入式 Linux 应用调试。
- 裸机应用调试。裸机流程需要通过扩展设置来手动进行目标配置。请参阅以下 [裸机注意事项](#)。
- 通过 QEMU 进行硬件仿真调试。

### 目标通信框架连接类型

在开始调试会话之前，请通过扩展提供的“Target Communication Framework”设置来配置目标连接。该扩展支持三种 TCF 连接类型：

- `hw_server`：连接到运行 `hw_server` 的物理开发板，以进行裸机调试。
- Linux TCF Agent：连接到在目标操作系统上运行的 TCF 代理，以进行嵌入式 Linux 应用调试。
- QEMU：连接到 QEMU 实例，以进行基于硬件仿真的调试。

### 裸机注意事项

通过 Visual Studio Code 扩展进行裸机调试需要大量 `targetSetup` 配置，包括 PDI 或比特流、FSBL 以及器件初始化。您必须通过扩展设置手动提供并维护这些工件。

对于依赖硬件设置、目标编程或工程集成调试的裸机工作流程，请改用 Vitis Unified IDE。Vitis Unified IDE 将目标配置、编程和调试会话作为单一集成流程进行管理。

### Visual Studio Code 和调试会话行为

Vitis 调试器扩展独立于 Vitis Unified IDE 用户界面运行。在调试会话期间，您可以直接从 Visual Studio Code 界面控制所有调试操作。

该扩展不会修改工程文件或应用二进制文件。调试会话设置仅适用于当前活动会话，不会影响其他调试工作流程。

### 设置调试会话

如需了解有关安装扩展、配置 TCF 连接以及启动调试会话的分步过程，请参阅 [在 Visual Studio Code 中设置 Vitis 调试器扩展](#)。

---

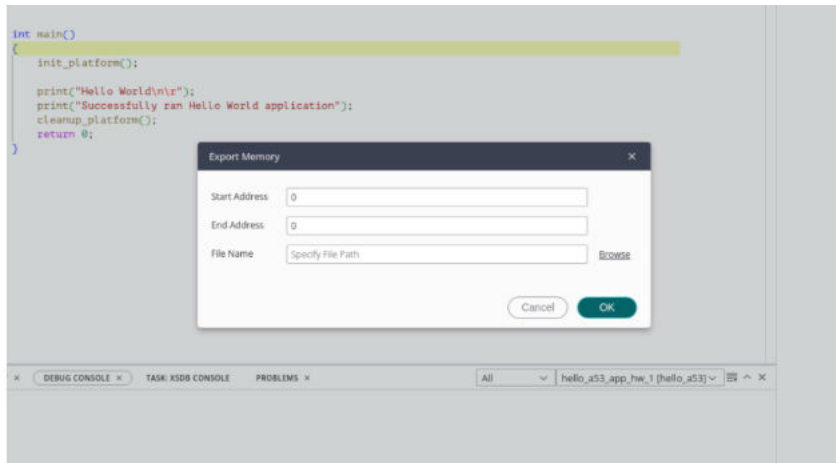
## 导出存储器函数

在 Vitis Unified IDE 中，您可以在调试会话期间将存储器节导出到 s 记录文件。此过程使您能够在运行时检查存储器内容。在活动的调试会话期间，请遵循以下步骤来导出存储器的某一部分。

1. 开启调试会话时，请选择右侧的“Memory Inspector”（存储器检查器）视图。



2. 选中“Memory Inspector”视图中的“Export Memory”（导出存储器）图标。这样会显示“Export Memory Wizard”（存储器导出向导）。



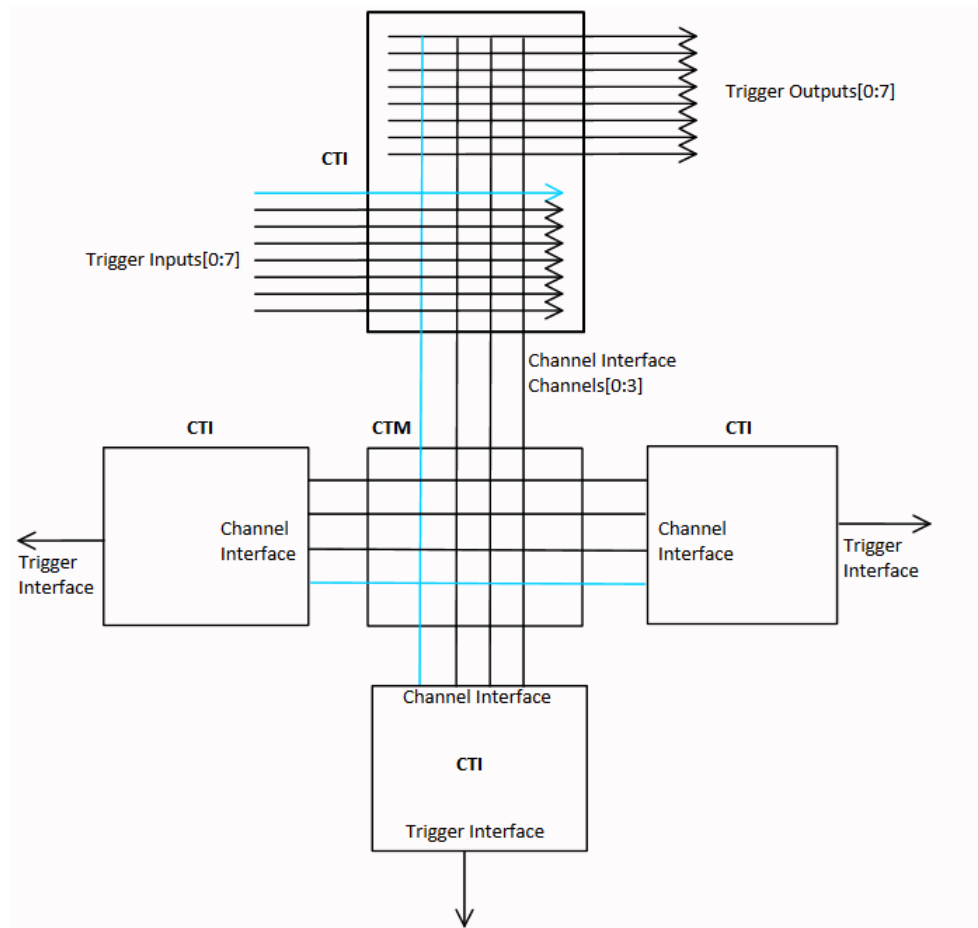
3. 输入您首选的起始地址、结束地址和输出文件路径。

## 交叉触发

来自 Arm 的嵌入式交叉触发 (ECT) 模块支持交叉触发。ECT 为 SoC 中的多个子系统提供了一种机制，用于通过交换调试触发器来彼此进行交互。ECT 包含 2 个模块：

- 交叉触发接口 (CTI)：CTI 将触发器请求加以组合并映射，然后将其作为通道事件广播至 ECT 上的所有其他接口。当 CTI 接收到通道事件时，它会将其映射到触发输出上。这样即可支持子系统彼此进行交叉触发。
- 交叉触发矩阵 (CTM)：CTM 用于控制通道事件的分布。它可提供通道接口，用于连接到 CTI 或 CTM。CTM 支持连接多个 ECT。

图 53：通用设置中的 CTI 和 CTM



单个 CTM 与多条通道一起即可组成一个事件广播网络。

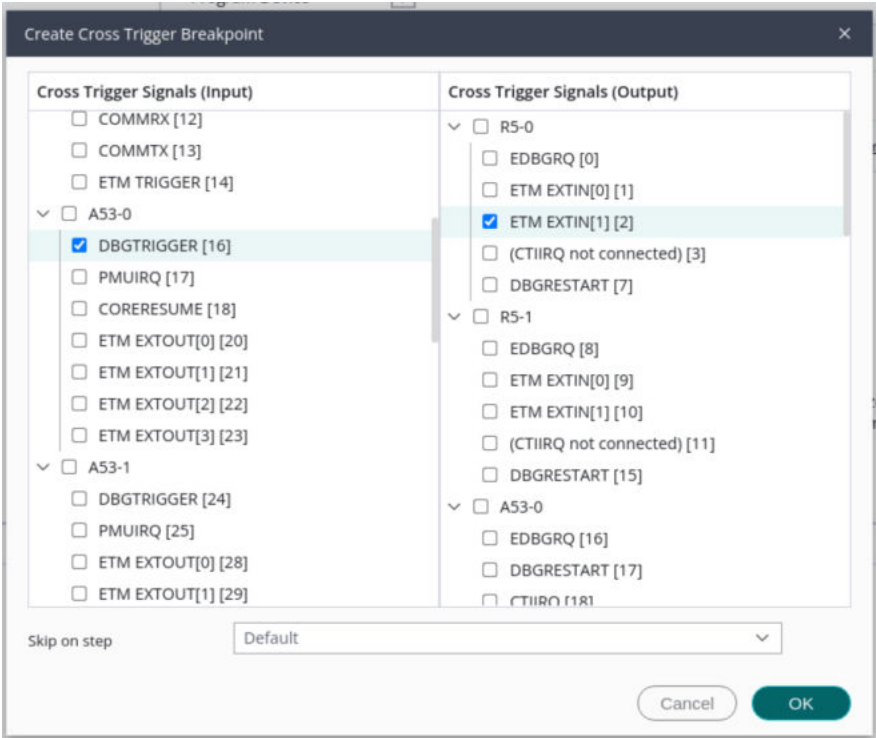
- 每个 CTI 都会监听一条或多条通道的事件。
- 每个 CTI 都会将接收到的事件映射到触发器，并将该触发器发送到一个或多个 CoreSight 组件，且这些组件均连接到 CTI。
- 每个 CTI 还会将来自连接的 CoreSight 组件的触发器组合在一起并进行映射，然后通过一条或多条通道将其作为事件进行广播。
- 您可配置 CTI，以通过其寄存器接口来监听特定通道的事件。
- CTI 可以将触发器作为事件广播到特定通道。

在以上示例中有 4 条通道。顶部的 CTI 会将触发输入 0 上的触发器事件传播到通道 0。您可配置其他 CTI 以监听该通道的事件，并通过触发输出来广播这些事件。触发输出会对连接到 CTI 的组件进行调试。CTI 还支持通道门控，因此，无需禁用通道到触发器 I/O 映射，即可直接关闭某些通道。

## 启用交叉触发

您可以使用“Launch Configurations”（启动配置）页面在目标上创建、编辑或移除交叉触发断点，也可以应用断点。要启用交叉触发，请执行以下操作。

- 1. 启动 Vitis Unified IDE。
- 2. 创建并构建独立应用组件。或者您可选择现有应用组件。
- 3. 转至 “Flow Navigator” 并选择应用组件。
- 4. 请参阅 [启动配置](#) 和 [创建新的目标连接](#) 以了解有关启动调试配置和设置目标连接的信息。
- 5. 在启动配置页面中启用 “Enable Cross Triggering” （启用交叉触发）复选框。
- 6. 单击 “Add Item” （添加项）以创建新断点。



您可在此屏幕上设置输入和输出信号，或者设置 “skip on step” （跳过步骤）的模式。

Zynq 器件中的交叉触发

在 Zynq 器件中，ECT 有 4 条广播通道、4 个 CTI 和 1 个 CTM。其中 1 个 CTI 连接到 ETB/TPIU，1 个连接到 FTM，每个 Cortex-A9 核也各连接 1 个 CTI。下表显示了每个 CTI 的触发输入和触发输出连接。

**注释：**下表中的连接均为硬连线的连接。

表 10: Zynq 器件中的 CTI 触发端口

| CTI 触发端口           | 信号       |
|--------------------|----------|
| CTI 连接到 ETB 和 TPIU |          |
| 触发输入 2             | ETB 已满   |
| 触发输入 3             | ETB 采集完成 |
| 触发输入 4             | ITM 触发   |
| 触发输出 0             | ETB 刷新   |
| 触发输出 1             | ETB 触发   |

表 10: Zynq 器件中的 CTI 触发端口 (续)

| CTI 触发端口   | 信号          |
|------------|-------------|
| 触发输出 2     | TPIU 刷新     |
| 触发输出 3     | TPIU 触发     |
| FTM CTI    |             |
| 触发输入 0     | FTM 触发      |
| 触发输入 1     | FTM 触发      |
| 触发输入 2     | FTM 触发      |
| 触发输入 3     | FTM 触发      |
| 触发输出 0     | FTM 触发      |
| 触发输出 1     | FTM 触发      |
| 触发输出 2     | FTM 触发      |
| 触发输出 3     | FTM 触发      |
| CPU0/1 CTI |             |
| 触发输入 0     | CPU DBGACK  |
| 触发输入 1     | CPU PMU IRQ |
| 触发输入 2     | PTM EXT     |
| 触发输入 3     | PTM EXT     |
| 触发输入 4     | CPU COMMTX  |
| 触发输入 5     | CPU COMMTX  |
| 触发输入 6     | PTM TRIGGER |
| 触发输出 0     | CPU 调试请求    |
| 触发输出 1     | PTM EXT     |
| 触发输出 2     | PTM EXT     |
| 触发输出 3     | PTM EXT     |
| 触发输出 4     | PTM EXT     |
| 触发输出 7     | CPU 重新启动请求  |

## 在 Zynq UltraScale+ MPSoC 中进行交叉触发

在 Zynq UltraScale+ MPSoC 中，ECT 有 4 条广播通道、9 个 CTI 和 1 个 CTM。下表显示了每个 CTI 的触发输入和触发输出连接。这些是硬连线连接。如需了解更多详情，请参阅《Zynq UltraScale+ 器件技术参考手册》(UG1085)。

表 11: Zynq UltraScale+ MPSoC 中的 CTI 触发端口

| CTI 触发端口              | 信号            |
|-----------------------|---------------|
| CTI 0 (soc_debug_fpd) |               |
| IN 0                  | ETF 1 FULL    |
| IN 1                  | ETF 1 ACQCOMP |
| IN 2                  | ETF 2 FULL    |
| IN 3                  | ETF 2 ACQCOMP |
| IN 4                  | ETR FULL      |

表 11: Zynq UltraScale+ MPSoC 中的 CTI 触发端口 (续)

| CTI 触发端口                     | 信号                |
|------------------------------|-------------------|
| IN 5                         | ETR ACQCOMP       |
| IN 6                         | -                 |
| IN 7                         | -                 |
| OUT 0                        | ETF 1 FLUSHIN     |
| OUT 1                        | ETF 1 TRIGIN      |
| OUT 2                        | ETF 2 FLUSHIN     |
| OUT 3                        | ETF 2 TRIGIN      |
| OUT 4                        | ETR FLUSHIN       |
| OUT 5                        | ETR TRIGIN        |
| OUT 6                        | TPIU FLUSHIN      |
| OUT 7                        | TPIU TRIGIN       |
| <b>CTI 1 (soc_debug_fpd)</b> |                   |
| IN 0                         | FTM               |
| IN 1                         | FTM               |
| IN 2                         | FTM               |
| IN 3                         | FTM               |
| IN 4                         | STM TRIGOUTSPTE   |
| IN 5                         | STM TRIGOUTSW     |
| IN 6                         | STM TRIGOUTHETE   |
| IN 7                         | STM ASYNCOUT      |
| OUT 0                        | FTM               |
| OUT 1                        | FTM               |
| OUT 2                        | FTM               |
| OUT 3                        | FTM               |
| OUT 4                        | STM HWEVENTS      |
| OUT 5                        | STM HWEVENTS      |
| OUT 6                        | -                 |
| OUT 7                        | HALT SYSTEM TIMER |
| <b>CTI 2 (soc_debug_fpd)</b> |                   |
| IN 0                         | ATM 0             |
| IN 1                         | ATM 1             |
| IN 2                         | -                 |
| IN 3                         | -                 |
| IN 4                         | -                 |
| IN 5                         | -                 |
| IN 6                         | -                 |
| IN 7                         | -                 |
| OUT 0                        | ATM 0             |
| OUT 1                        | ATM 1             |

表 11: Zynq UltraScale+ MPSoC 中的 CTI 触发端口 (续)

| CTI 触发端口                   | 信号              |
|----------------------------|-----------------|
| OUT 2                      | -               |
| OUT 3                      | -               |
| OUT 4                      | -               |
| OUT 5                      | -               |
| OUT 6                      | -               |
| OUT 7                      | 图像调试开始          |
| <b>CTI 0 和 1 (RPU)</b>     |                 |
| IN 0                       | DBGTRIGGER      |
| IN 1                       | PMUIRQ          |
| IN 2                       | ETMEXTOUT[0]    |
| IN 3                       | ETMEXTOUT[1]    |
| IN 4                       | COMMRX          |
| IN 5                       | COMMTX          |
| IN 6                       | ETM TRIGGER     |
| IN 7                       | -               |
| OUT 0                      | EDBGRQ          |
| OUT 1                      | ETMEXTIN[0]     |
| OUT 2                      | ETMEXTIN[1]     |
| OUT 3                      | - (CTIIRQ, 未连接) |
| OUT 4                      | -               |
| OUT 5                      | -               |
| OUT 6                      | -               |
| OUT 7                      | DBGRESTART      |
| <b>CTI 0、1、2 和 3 (APU)</b> |                 |
| IN 0                       | DBGTRIGGER      |
| IN 1                       | PMUIRQ          |
| IN 2                       | -               |
| IN 3                       | -               |
| IN 4                       | ETMEXTOUT[0]    |
| IN 5                       | ETMEXTOUT[1]    |
| IN 6                       | ETMEXTOUT[2]    |
| IN 7                       | ETMEXTOUT[3]    |
| OUT 0                      | EDBGRQ          |
| OUT 1                      | DBGRESTART      |
| OUT 2                      | CTIIRQ          |
| OUT 3                      | -               |
| OUT 4                      | ETMEXTIN[0]     |
| OUT 5                      | ETMEXTIN[1]     |
| OUT 6                      | ETMEXTIN[2]     |

表 11: Zynq UltraScale+ MPSoC 中的 CTI 触发端口 (续)

| CTI 触发端口 | 信号          |
|----------|-------------|
| OUT 7    | ETMEXTIN[3] |

## Versal 器件中的交叉触发

在 Versal 器件中，ECT 有 4 条广播通道、12 个 CTI 和 1 个 CTM。下表显示了每个 CTI 的触发输入和触发输出连接。这些是硬连线连接。欲知详情，请参阅《Versal 自适应 SoC 技术参考手册》(AM011)。

表 12: Versal 器件中的 CTI 触发端口

| CTI 触发端口                                                                                            | 信号             |
|-----------------------------------------------------------------------------------------------------|----------------|
| <b>R5 CTI 0 和 1 (RPU)。XSDB ID = 0-7 (R5 #0), 8-15 (R5 #1)</b>                                       |                |
| IN 0                                                                                                | R5 DBGTRIGGER  |
| IN 1                                                                                                | R5 PMUIRQ      |
| IN 2                                                                                                | ETM EXTOUT[0]  |
| IN 3                                                                                                | ETM EXTOUT[1]  |
| IN 4                                                                                                | R5 COMMRX      |
| IN 5                                                                                                | R5 COMMTX      |
| IN 6                                                                                                | ETM TRIGGER    |
| IN 7                                                                                                | -              |
| OUT 0                                                                                               | R5 EDBGREQ     |
| OUT 1                                                                                               | ETM EXTIN[0]   |
| OUT 2                                                                                               | ETM EXTIN[1]   |
| OUT 3                                                                                               | -              |
| OUT 4                                                                                               | -              |
| OUT 5                                                                                               | -              |
| OUT 6                                                                                               | -              |
| OUT 7                                                                                               | R5 DBGRESTART  |
| <b>CTI 0、1、2 和 3 (APU)。XSDB ID = 16-23 (A72 #0), 24-31 (A72 #1), 32-39 (A72 #2), 40-47 (A72 #3)</b> |                |
| IN 0                                                                                                | A72 DBGTRIGGER |
| IN 1                                                                                                | A72 PMUIRQ     |
| IN 2                                                                                                | -              |
| IN 3                                                                                                | -              |
| IN 4                                                                                                | ETM EXTOUT[0]  |
| IN 5                                                                                                | ETM EXTOUT[1]  |
| IN 6                                                                                                | ETM EXTOUT[2]  |
| IN 7                                                                                                | ETM EXTOUT[3]  |
| OUT 0                                                                                               | A72 EDBGREQ    |
| OUT 1                                                                                               | A72 DBGRESTART |
| OUT 2                                                                                               | GIC PPI 24     |
| OUT 3                                                                                               | -              |

表 12: Versal 器件中的 CTI 触发端口 (续)

| CTI 触发端口                                      | 信号              |
|-----------------------------------------------|-----------------|
| OUT 4                                         | ETM EXTIN[0]    |
| OUT 5                                         | ETM EXTIN[1]    |
| OUT 6                                         | ETM EXTIN[2]    |
| OUT 7                                         | ETM EXTIN[3]    |
| <b>CTI p (pmc_debug)。XSDB ID = 48-55</b>      |                 |
| IN 0                                          | ATM TRIGOUT[0]  |
| IN 1                                          | -               |
| IN 2                                          | -               |
| IN 3                                          | -               |
| IN 4                                          | -               |
| IN 5                                          | -               |
| IN 6                                          | -               |
| IN 7                                          | -               |
| OUT 0                                         | ATM TRIGIN[0]   |
| OUT 1                                         | -               |
| OUT 2                                         | -               |
| OUT 3                                         | -               |
| OUT 4                                         | -               |
| OUT 5                                         | -               |
| OUT 6                                         | -               |
| OUT 7                                         | -               |
| <b>CTI 0d (soc_debug_lpd)。XSDB ID = 56-63</b> |                 |
| IN 0                                          | ATM0 TRIGOUT[0] |
| IN 1                                          | ATM0 TRIGOUT[1] |
| IN 2                                          | ATM0 TRIGOUT[2] |
| IN 3                                          | ATM0 TRIGOUT[3] |
| IN 4                                          | ATM0 TRIGOUT[4] |
| IN 5                                          | -               |
| IN 6                                          | -               |
| IN 7                                          | -               |
| OUT 0                                         | ATM0 TRIGIN[0]  |
| OUT 1                                         | ATM0 TRIGIN[1]  |
| OUT 2                                         | ATM0 TRIGIN[2]  |
| OUT 3                                         | ATM0 TRIGIN[3]  |
| OUT 4                                         | ATM0 TRIGIN[4]  |
| OUT 5                                         | 7               |
| OUT 6                                         | -               |
| OUT 7                                         | -               |

表 12: Versal 器件中的 CTI 触发端口 (续)

| CTI 触发端口                                      | 信号                  |
|-----------------------------------------------|---------------------|
| <b>CTI 1a (APU)。XSDB ID = 64-71</b>           |                     |
| IN 0                                          | ELA 1a CTTRIGOUT[0] |
| IN 1                                          | ELA 1a CTTRIGOUT[1] |
| IN 2                                          | ETF 1a FULL         |
| IN 3                                          | ETF 1a ACQCOMP      |
| IN 4                                          | -                   |
| IN 5                                          | -                   |
| IN 6                                          | -                   |
| IN 7                                          | -                   |
| OUT 0                                         | ELA 1a CTTRIGIN[0]  |
| OUT 1                                         | ELA 1a CTTRIGIN[1]  |
| OUT 2                                         | ETF 1a FLUSHIN      |
| OUT 3                                         | ETF 1a TRIGIN       |
| OUT 4                                         | PMUSNAPSHOT[0]      |
| OUT 5                                         | PMUSNAPSHOT[1]      |
| OUT 6                                         | -                   |
| OUT 7                                         | -                   |
| <b>CTI 1b (soc_debug_fpd)。XSDB ID = 72-79</b> |                     |
| IN 0                                          | STM TRIGOUTSPTE     |
| IN 1                                          | STM TRIGOUTSW       |
| IN 2                                          | STM TRIGOUTHETE     |
| IN 3                                          | STM ASYNCOUT        |
| IN 4                                          | ETF 1 FULL          |
| IN 5                                          | ETF 1 ACQCOMP       |
| IN 6                                          | ETR FULL            |
| IN 7                                          | ETF ACQCOMP         |
| OUT 0                                         | STM HWEVENTS        |
| OUT 1                                         | STM HWEVENTS        |
| OUT 2                                         | TPIU FLUSHIN        |
| OUT 3                                         | TPIU TRIGIN         |
| OUT 4                                         | ETF 1 FLUSHIN       |
| OUT 5                                         | ETF 1 TRIGIN        |
| OUT 6                                         | ETR FLUSHIN         |
| OUT 7                                         | ETR TRIGIN          |
| <b>CTI 1c (soc_debug_fpd)。XSDB ID = 80-87</b> |                     |
| IN 0                                          | pl_ps_trigger[0]    |
| IN 1                                          | pl_ps_trigger[1]    |
| IN 2                                          | pl_ps_trigger[2]    |
| IN 3                                          | pl_ps_trigger[3]    |

表 12: Versal 器件中的 CTI 触发端口 (续)

| CTI 触发端口                               | 信号                   |
|----------------------------------------|----------------------|
| IN 4                                   | -                    |
| IN 5                                   | -                    |
| IN 6                                   | -                    |
| IN 7                                   | -                    |
| OUT 0                                  | ps_pl_trigger[0]     |
| OUT 1                                  | ps_pl_trigger[1]     |
| OUT 2                                  | ps_pl_trigger[2]     |
| OUT 3                                  | ps_pl_trigger[3]     |
| OUT 4                                  | -                    |
| OUT 5                                  | -                    |
| OUT 6                                  | HALT System Timer    |
| OUT 7                                  | RESTART System Timer |
| CTI 1d (soc_debug_fpd)。XSDB ID = 88-95 |                      |
| IN 0                                   | ATM1 TRIGOUT[0]      |
| IN 1                                   | ATM1 TRIGOUT[1]      |
| IN 2                                   | ATM1 TRIGOUT[2]      |
| IN 3                                   | ATM1 TRIGOUT[3]      |
| IN 4                                   | ATM1 TRIGOUT[4]      |
| IN 5                                   | ATM1 TRIGOUT[5]      |
| IN 6                                   | ATM1 TRIGOUT[6]      |
| IN 7                                   | -                    |
| OUT 0                                  | ATM1 TRIGIN[0]       |
| OUT 1                                  | ATM1 TRIGIN[1]       |
| OUT 2                                  | ATM1 TRIGIN[2]       |
| OUT 3                                  | ATM1 TRIGIN[3]       |
| OUT 4                                  | ATM1 TRIGIN[4]       |
| OUT 5                                  | ATM1 TRIGIN[5]       |
| OUT 6                                  | ATM1 TRIGIN[6]       |
| OUT 7                                  | -                    |

用例

FPGA 到 CPU 触发

FPGA 到 CPU 触发是 Zynq 中最常见的交叉触发用例之一。

- 在 FPGA CTI 上有 4 项触发输入。您可配置这些输入以中止两个 CPU 中任何一个 CPU (EDBGRQ)。同样，当 CPU 中止 (DBGACK) 时，可触发 4 项 FPGA CTI 触发输出。
- FPGA 触发输入和输出可连接到 ILA 核。只要这两个 CPU 中的任何一个发生中止，其中一个 ILA 触发器即可中止一个或多个 CPU，并触发该 ILA 对其当前监控的信号执行捕获。

如需了解有关在 Vivado Design Suite 中设置到 FTM 的交叉触发的更多详细信息，请参阅《Vivado Design Suite 教程：嵌入式处理器硬件设计》(UG940) 中的“交叉触发设计”部分。

## PTM 到 CPU 触发

将追踪捕获与处理器状态同步。例如，ETB 已满事件可作为触发器来中止一个或多个 CPU。

## CPU 到 CPU 触发

您可使用交叉触发来同步 CPU 之间进入和退出调试状态的操作。例如，当 CPU0 中止时，该事件可触发 CPU1 调试请求，这样即可中止 CPU1。

## XSDB 交叉触发命令

XSDB 断点添加命令 (bpadd) 允许在不同组件之间进行交叉触发。

例如，以下命令可用于设置交叉触发，以便在核 0 停止时，停止 Zynq 核 1。

```
bpadd -ct-input 0 -ct-output 8
```

对于 Zynq，-ct-input 0 表示 CTI CPU0 TrigIn0。

CTI 的触发输入 0 连接到 CPU0，后者连接到 DBGACK（当该核中止时断言有效）。-ct-output 8 表示 CTI CPU1 TrigOut0，后者连接到 CPU 调试请求。将此管脚断言有效会导致该核中止运行。hw\_server 使用可用通道来设置这些管脚之间的交叉触发路径。当 core 0 中止时，会通过选定通道将此事件广播到 core 1，从而导致 core 1 中止运行。

使用以下命令，使 Zynq UltraScale+ MPSoC 在 A53 核 0 停止时中止 A53 核 1。

```
bpadd -ct-input 16 -ct-output 24
```

---

# 剖析/分析

## TCF 剖析

TCF 剖析器支持对独立应用和 Linux 应用进行剖析。构建应用时，TCF 剖析无需设置任何其他编译器标志。首先通过调试接口进行“Program Counter”（程序计数器）采样，随后方可基于采样结果通过 JTAG 进行独立应用剖析。禁用栈追踪时，它并不会更改程序执行流程，属于非侵入式操作。启用栈追踪时，由于调试器需要收集栈追踪信息，因此会减缓程序执行速度。

1. 启动 Vitis Unified IDE 并选中您要剖析的应用组件。
2. 转至 Flow Navigator 并创建调试启动配置。

**注释：**请参阅 [启动配置](#) 和 [创建新的目标连接](#) 以创建调试启动配置。

**注释：**关于目标连接，如果应用组件是 Linux 应用，请选中 TCF 代理。如果是独立应用，请选中硬件服务器。

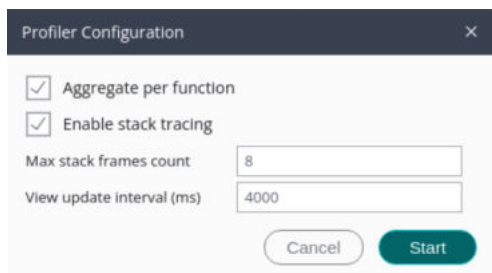
3. 完成配置后，单击“Debug”（调试）启动调试会话。

**注释：**请参阅 [开始调试](#) 以启动调试会话。

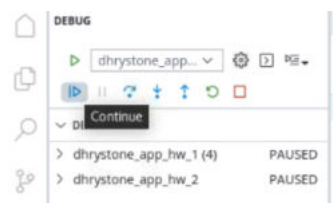
- 单击“TCF Profiler”（TCF 剖析器）按钮启动剖析视图。



- 单击“Start”（启动）按钮开始剖析。选择“Enable stack tracing”（启用栈追踪）选项以显示样本数据中每个地址的栈追踪。您还可以根据自己的要求设置帧数和更新时间间隔。
  - 指定“Max stack frames count”（最大栈帧数）以设置栈追踪视图中的最大帧数。
  - 对于时间间隔，请以毫秒为单位来指定“View update interval”（视图更新时间间隔）。请确保此处设置的时间间隔与为收集剖析样本所设的时间间隔不同。



- 单击“Start”执行剖析。
- 单击“Continue”（继续）即可自由运行该应用。



- 现在您可以检索应用组件的剖析信息。

The screenshot displays the Vitis Unified IDE interface. On the left, the 'Disassembly' window shows the source code for 'dhry\_1.c'. The code includes comments and function definitions for 'stop\_time', 'Proc\_3', and 'Proc\_1'. The 'TCF PROFILER' window on the right shows a table of functions with their addresses, exclusive and inclusive percentages, and line numbers. The 'Called From' and 'Child Calls' sections show the call stack for the selected function.

| ADDRESS  | % EXCLUSIVE | % INCLUSIVE | FUNCTION    | FILE         | LINE |
|----------|-------------|-------------|-------------|--------------|------|
| 00003bf0 | 91.7        | 91.7        | _exit       | _exit.c      | 16   |
| 00003418 | .000        | 91.7        |             |              | 0    |
| 00002da8 | .000        | 8.25        | _startup    | xil-crt0.S   | 68   |
| 0000101c | 1.56        | 8.24        | main        | dhry_1.c     | 228  |
| 00000e0c | 1.20        | 2.44        | Proc_1      | dhry_1.c     | 151  |
| 00001b00 | 1.99        | 1.99        | Proc_8      | dhry_2.c     | 121  |
| 00001d08 | .449        | .814        | Func_2      | dhry_2.c     | 170  |
| 00000da0 | .449        | .583        | Proc_3      | dhry_1.c     | 139  |
| 000019bc | .413        | .547        | Proc_6      | dhry_2.c     | 63   |
| 00001cb4 | .534        | .534        | Func_1      | dhry_2.c     | 146  |
| 00001ac4 | .401        | .401        | Proc_7      | dhry_2.c     | 103  |
| 00000f28 | .340        | .340        | Proc_2      | dhry_1.c     | 185  |
| 00000f9c | .340        | .340        | Proc_4      | dhry_1.c     | 205  |
| 00003540 | .267        | .267        | strcmp      |              | 0    |
| 00002ef0 | .109        | .219        | outbyte     | xil_io.h     | 137  |
| 00002174 | .000        | .146        | xil_vprintf | xil_printf.c | 272  |
| 0000198c | .134        | .134        | Func_3      | dhry_2.c     | 45   |
| 00002690 | .000        | .109        | xil_printf  | xil_printf.c | 259  |
| 00000ff8 | .061        | .061        | Proc_5      | dhry_1.c     | 218  |
| 00000cac | .000        | .012        | print_time  | dhry_1.c     | 76   |
| 00001e50 | .000        | .012        | outnum      | xil_printf.c | 110  |

**Called From**

|          |      |        |          |     |
|----------|------|--------|----------|-----|
| 00000e0c | 2.53 | Proc_1 | dhry_1.c | 151 |
|----------|------|--------|----------|-----|

**Child Calls**

|          |      |        |          |     |
|----------|------|--------|----------|-----|
| 00001ac4 | .416 | Proc_7 | dhry_2.c | 103 |
|----------|------|--------|----------|-----|

- 单击剖析视图中的函数即可在“Profiler”视图右下角显示“Called From”（调用自）和“Child Calls”（子调用）。
- 该视图支持函数名称与源代码之间的交叉探测。单击函数即可跳转至“Source Code”（源代码）视图中的源代码。

## gprof 剖析

**注释：** Vitis Unified IDE 不支持此功能特性。如果您要访问该功能，请切换到传统 Vitis IDE（在 2024.2 版及更低版本内可用）并参阅 2024.2 文档。

## 适用于 MicroBlaze 处理器的非侵入式剖析

Vitis Unified IDE 不支持该功能，但您可以使用 XSDB 来实现相同的效果。使用 XSDB 或 Python XSDB 工具为 MicroBlaze 和 MicroBlaze RISC V 运行非侵入式剖析。

XSDB 命令是 `mbprofile`，或者执行 `mbprofile -help` 以获取更多详细信息。

## 使用 STM 进行 FreeRTOS 分析

**注释：** Vitis Unified IDE 不支持此功能特性。如果您要访问该功能，请改用传统 Vitis IDE（在 2024.2 版及更低版本内可用）并参阅 2024.2 文档。

## 创建启动镜像

AMD FPGA 和系统级芯片 (SoC) 器件通常有多个硬件与软件二进制文件。这些二进制文件会启动 FPGA 或 SoC 进入其指定功能。二进制文件可包含 FPGA 比特流、固件镜像、启动加载程序、操作系统和用户选择的应用。这些文件能以非安全方式和安全方式来加载。

Bootgen 工具支持您将二进制文件整合在一起，并生成器件启动镜像。Bootgen 定义了多个属性、特性和参数作为创建启动镜像时的输入，以供 AMD 器件使用。

如需了解有关 Bootgen 实用工具的更多信息，请参阅《Bootgen 用户指南》(UG1283)。

有两种方法可用于创建启动镜像：从“Flow Navigator”创建或者从头开始创建。

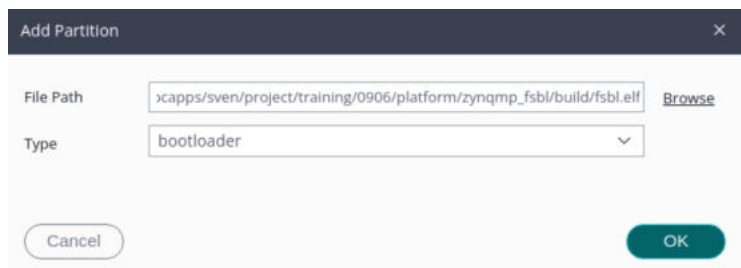
### 从头开始创建启动镜像

完成应用组件编译后，请遵循以下步骤来创建启动镜像。

根据您的设计，选择“Vitis”→“Create Boot Image”（Vitis > 创建启动镜像），然后选择器件。

### ZYNQ/ZYNQMP

1. 如果您没有现有的 BIF 文件，请选择“Create a new BIF file”（创建新的 BIF 文件）。
2. 单击“+”添加镜像分区。



3. 指定镜像，设置“Type”（类型），然后单击“OK”（确定）。

**注释：** 您可根据需求添加启动加载程序、FPGA 比特文件、bl31 和应用 ELF 文件。

**注释：** 您可修改镜像属性，方法是选中镜像（请参阅以下示例）并根据需求更改属性。

Input BIF

☒ Create a new BIF file ☐ Import existing BIF file

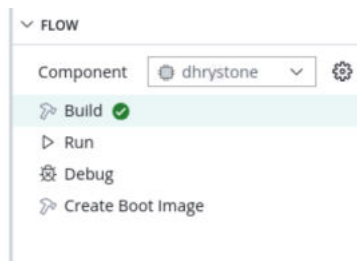
|                                   |   |   |
|-----------------------------------|---|---|
| +                                 | ↑ | ↓ |
| all                               |   |   |
| fsbl.elf [bootloader]             |   |   |
| dhystone.bif [datafile]           |   |   |
| edt_zcu104_wrapper.bit [datafile] |   |   |

|                    |                                                                |        |
|--------------------|----------------------------------------------------------------|--------|
| File path          | /group/bcapps/sven/project/training/0906/dhystone/dhystone.bif | Browse |
| Partition type     | datafile                                                       |        |
| Destination device | PS                                                             |        |
| Destination CPU    | a53-0                                                          |        |
| Authentication     | none                                                           |        |
| Presign            | Presign                                                        | Browse |
| Encryption         | none                                                           |        |
| AES Key File       | AES Key File                                                   | Browse |
| Checksum           | none                                                           |        |
| Alignment          | Alignment                                                      |        |
| Load               | Load                                                           |        |
| Offset             | Offset                                                         |        |
| Reserve            | Reserve                                                        |        |
| Startup            | Startup                                                        |        |
| Exception Level    | 0x0                                                            |        |

4. 指定“Output Bif File Path”（输出 BIF 文件路径）和“Output Image path”（输出镜像路径）。
5. 单击“Create Image”（创建镜像）。几秒钟后，log 日志会指示此镜像已就绪。

### 从 Flow Navigator 创建启动镜像（更快）

1. 在“Component”（组件）视图选择构建的应用组件。
2. 转至 Flow Navigator 并单击“Create Boot Image”（创建启动镜像）。



3. 创建启动镜像对话框会显示您的组件中必需的镜像。指定“Output Bif File Path”（输出 BIF 文件路径）和“Output Image path”（输出镜像路径）。

Create Zynq UltraScale+ Boot Image

Input BIF

☒ Create a new BIF file ☐ Import existing BIF file

the\_ROM\_image

- fsbl.elf [bootloader]
- edt\_zcu104\_wrapper.bit [datafile]
- dhrystone.elf [datafile]

Name: the\_ROM\_image

Use Encryption: ☐

Part Name:

Keysrc Encryption: efuse\_gry\_key

Operational Key: ☐

Use Authentication: ☐

PPK File:  [Browse](#)

PSK File:  [Browse](#)

SPK File:  [Browse](#)

SSK File:  [Browse](#)

SPK Signature:  [Browse](#)

PPK select: ZERO

SPK Id:

Use Bh auth: ☐

UDF Data:  [Browse](#)

Split: ☐

Boot Image Options

Output BIF File Path\*:  [Browse](#)

Output Image\*:  [Browse](#)

Extra Arguments:

[Cancel](#) [Preview BIF Changes](#) [Create Image](#)

- 单击“Create Image”（创建镜像）。几秒钟后，log 日志会指示创建的镜像已就绪。

## 闪存烧录

Program Flash 是用于对设计中的闪存执行烧录的工具。支持烧录的各类闪存如下：

- 支持非 Zynq 器件，例如，来自 Micron 和 Spansion 等品牌的 Parallel Flash (BPI) 和 Serial Flash (SPI)。
- 支持 Zynq 器件，例如 QSPI、NAND 和 NOR。您可使用各种不同配置的 QSPI，例如，QSPI SINGLE、QSPI DUAL PARALLEL 和 QSPI DUAL STACKED。
- 支持 Versal 器件，例如，QSPI、emmc 和 OSPI。您可使用各种不同配置的 QSPI，例如，QSPI SINGLE、QSPI DUAL PARALLEL 和 QSPI DUAL STACKED。

转至菜单中的“Vitis” → “Program Flash” Vitis，打开闪存烧录向导。

图 54：闪存烧录窗口

Program Flash Memory

Program Flash Memory via In-system Programmer.

Project: dhrystone

Connection: zcu104-2 [New](#)

Image File: Image File [Browse](#) [Search](#)

Offset: Offset

Flash Type: qspi-x4-single

Init File: /group/bcapps/sven/project/training/0906/platform/zynqmp\_fsbl/build/fsbl.elf [Browse](#)

☐ Blank check after erase

☐ Verify after flash

Image file is not specified

[Cancel](#) [Program](#)

“Program Flash Memory”（闪存烧录）页面上提供的选项如下所述：

- “Project”（工程）：用于选择您计划使用的系统工程。Vitis 工具会自动选择组件视图。
- “Connection”（连接）：用于选择到硬件服务器的连接。
- “Image File”（镜像文件）：用于选择要写入闪存的文件。
- Zynq 器件：
  - QSPI 闪存类型支持的文件格式为 BIN 格式或 MCS 格式。
  - NOR 和 NAND 类型支持的文件格式是 BIN 格式。
- 非 Zynq 器件：
  - 非 Zynq 器件中受支持的闪存部件类型为 BIT、ELF、SREC、MCS 和 BIN。
- “Offset”（偏移）：指定相对于闪存基地址的偏移量。Vitis 工具会在此偏移位置烧录文件。

**注释：**对于 MCS 文件，无需指定偏移。

- “Flash Type”（闪存类型）：选择闪存类型。
- Zynq 器件：
  - qspi\_single
  - qspi\_dual\_parallel
  - qspi\_dual\_stacked
  - nand\_8
  - nand\_16
  - nor
  - emmc

**注释：**emmc 闪存类型仅适用于 Zynq UltraScale+ MPSoC 和 Versal 器件。

- 非 Zynq 器件：

- 闪存类型下拉列表中会填充来自所连接的 FPGA 的信息。根据检测到的器件，在此对话框中会填充针对器件支持的所有闪存部件。如果不存在到硬件服务器的连接，则会在页面上显示如下错误消息。

```
"Could not retrieve Flash Part information. Please check hardware server connection"
```

**注释：**您可根据设计选择相应的部件。对于 AMD 开发板，可在相应的开发板用户指南中找到部件名称。

- “Init File”（初始化文件）：提供初始化文件路径。
- “Blank check after erase”（擦除后执行空白检查）：空白检查会验证擦除操作以及已擦除的区域是否为空白。
- “Verify after Flash”（烧录后验证）：Vitis 工具会读回闪存内容，并与烧录后的数据进行交叉比对。

## 多线缆和多器件支持

AMD Vitis™ 环境的这项特性允许您在单一 JTAG 链上使用多条 JTAG 线或多个器件来运行和调试应用工程、执行比特流/PDI 烧录以及闪存烧录。主要用例如下所示：

- 多线缆：此用例假定您有多块开发板连接到系统并且想要使用所有开发板开展工作。以下是多线缆目标连接视图。如需了解有关目标连接的信息，请参阅 [目标连接](#)。

| NAME                                                                                                                                                                                                                                                                                                               | ID CODE  | IR LENGTH | FPGA? | FREQUENCY |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----------|-------|-----------|
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>Xilinx VCK190 FT4232H 802044124743A (2)</li> <li>arm_dap</li> <li>xvc1902</li> </ul> </li> <li> <ul style="list-style-type: none"> <li>Digilent JTAG-SMT2NC 210308B0AEC1 (2)</li> <li>xczu9</li> <li>arm_dap</li> </ul> </li> </ul> | 6ba00477 | 4         |       | 15000000  |
|                                                                                                                                                                                                                                                                                                                    | 14ca8093 | 6         | Yes   |           |
|                                                                                                                                                                                                                                                                                                                    | 24738093 | 12        | Yes   | 15000000  |
|                                                                                                                                                                                                                                                                                                                    | 5ba00477 | 4         |       |           |

- 多器件：此用例假定您的某一条 JTAG 链中有多个器件，并且您想要使用所有器件开展工作。以下是多器件目标连接视图。如需了解有关目标连接的信息，请参阅 [目标连接](#)。

图 55：多器件目标连接视图

Name

Multi-device

☐ Set as default target

Type

Hardware Server

Host

xhdbfarmrk1

Port

3121

☐ Use Symbol Server

Advanced

☒ Automatically discover devices on JTAG chain

☐ Manual configuration of JTAG chain

| NAME                                     | ID CODE  | IR LENGTH | FPGA? | FREQUENCY |
|------------------------------------------|----------|-----------|-------|-----------|
| Platform Cable USB II 0000153f74cd01 (4) |          |           |       | 6000000   |
| arm_dap                                  | 4ba00477 | 4         |       |           |
| xc7z020                                  | 03727093 | 6         | Yes   |           |
| arm_dap                                  | 4ba00477 | 4         |       |           |
| xc7z045                                  | 03731093 | 6         | Yes   |           |

利用多线缆或多器件进行应用调试

- 导航到调试启动配置文件（请参阅 [启动配置](#) 以获取详细信息）。
- 单击“Device”（器件）字段旁的“Select”（选择）。

图 56：launch.json 选项卡

launch.json

dhrystone > \_ide > launch.json

dhrystone\_app\_hw\_1

Launch Config Name\*

dhrystone\_app\_hw\_1

Target Connection

multi-device

New

Target Setup Mode

Baremetal Debug

Device

Auto Detect

Select

Bitstream File

Bitstream File

Browse

Board Initialization

☒ FSBL ☐ TCL

FSBL File

\${workspaceFolder}/platform/export/platform/sw/boot/fsbl.elf

Browse

FSBL Exit Function

XFsbld\_Exit

Init With FSBL

☒

Reset Entire System

☒

Reset APU

☐

Reset RPU

☐

Enable RPU Split Mode

☐

Program Device

☒

Partial Bitstream

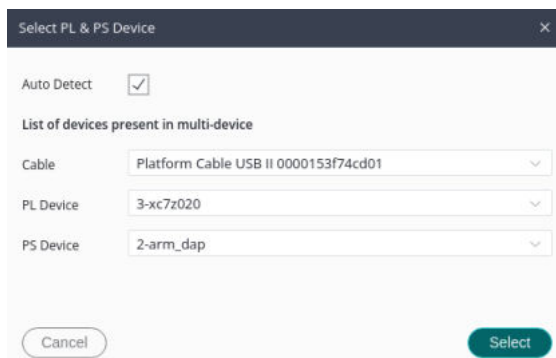
☐

Skip Revision Check

☐

- 取消选中“Auto Detect”（自动检测）

图 57：选择 PL 和 PS 器件屏幕



- 根据您的需求，在此对话框中选择“PS and PL device”（PS 和 PL 器件）。
- 单击“Select”（选择）和“Start”（开始）以开始调试。

#### 使用多器件或多线缆设置执行器件烧录

1. 转至“Vitis” → “” → “Program Device”（Vitis > 器件烧录）
2. 要从多线缆或多器件设置中选择器件，请单击“Device”（器件）字段右侧的“Select”（选择）按钮。
3. 从下拉菜单中选择相应的器件或线缆，然后单击“Program”（烧录）对 FPGA/PDI 文件进行烧录。

**注释：**如果器件状态为“Auto Detect”（自动检测），那么 Vitis 工具会自动选取列表中的首个器件或线缆。

**注释：**烧录闪存时，使用相同的方法选择 PS 和 PL 器件。

## 经身份验证的 JTAG 访问

“Authenticated JTAG Access”（经身份验证的 JTAG 访问）提供了一条安全的途径，用于控制和保护对器件的 JTAG 接口的访问。如果您启用该功能特性，那么您必须先成功完成身份验证，然后才能执行任何 JTAG 操作（如调试、烧录或器件检测）。

该功能特性有助于防止通过 JTAG 对器件进行未经授权的访问，即使有人可以对硬件进行物理访问也同样如此。

要访问经身份验证的 JTAG，请遵循以下步骤进行操作：

1. 在启动配置中选择名为“Authenticate JTAG”（JTAG 身份验证）的选项。

Launch Config Name\*

Target Connection  [New](#)

Target Setup Mode

Device  [Select](#)

PDI File  [Browse](#)

Reset Entire System ☒

Program Device ☒

Partial Bitstream ☐

Skip Revision Check ☐

Enable RPU Split Mode ☐

| PROCESSOR       | RESET PROCESSOR                     | APPLICATION                              | STOP AT ENTRY            | IS SELF RELOCATING APPLICATION      | RELATIVE ADDRESS |
|-----------------|-------------------------------------|------------------------------------------|--------------------------|-------------------------------------|------------------|
| psv_cortexa72_0 | <input checked="" type="checkbox"/> | Folder\hello_world\build\hello_world.elf | <input type="checkbox"/> | <input checked="" type="checkbox"/> | 0x0              |

Enable Cross Triggering ☐

Authenticated Jtag Options

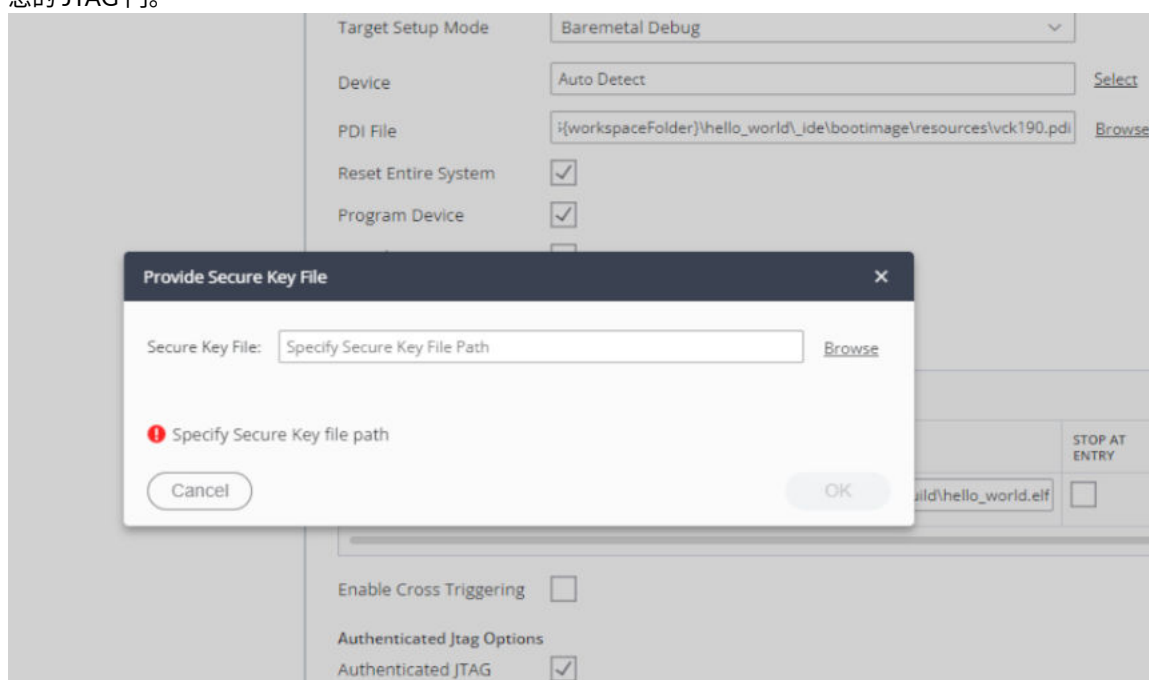
Authenticated JTAG ☒

Authenticated JTAG File  [Browse](#) [Create](#)

Enable Tracing ☐

> Path Map

2. 启用该选项后，选择“Create”（创建）按钮，并指定用于创建 JTAG 文件的路径。这样会生成解密文件用于打开您的 JTAG 门。



随后，会在“Authenticated JTAG File”（经身份验证的 JTAG 文件）字段中自动填充此文件。调试会话会自动使用该文件打开 JTAG 门以启用安全访问。

---

## XEN 感知调试

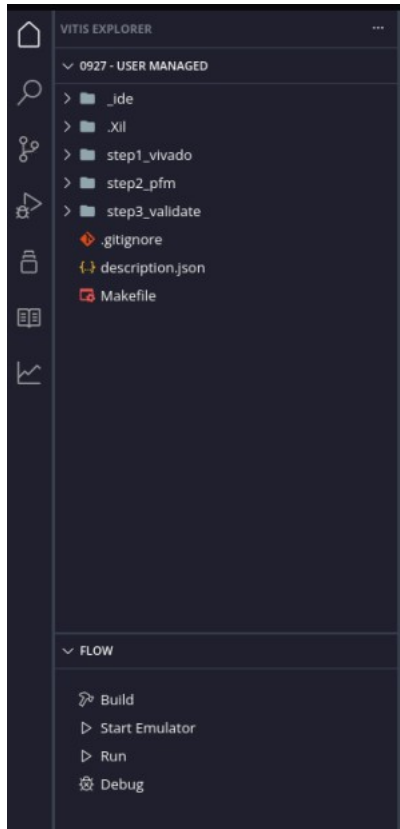
“XEN Aware Debug” 作为一项抢先体验功能可供使用。如需获取详细的指示信息，请参阅 [新功能特性预览](#) 页面。

# 用户管理模式

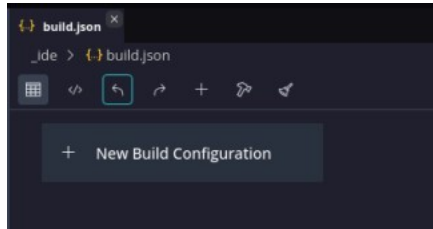
如果您倾向于以独有方式来管理自己的源代码层级和构建进程，例如，自行编写 Makefile 文件或 CMake 文件，那么您可使用“User Managed Mode”（用户管理模式）。该模式可为您提供增强的支持和调试能力，在您监督构建流程时尤其实用。此外，“User Managed Flow”简化了整个进程，能以对用户更友好的方式来无缝完成构建启动。如果您倾向于使用命令行流程来开发应用，那么“User managed flow”正是理想选择。此流程支持基于 Makefile 的工程。

找不到任何组件但在当前工作空间内检测到文件时，会自动激活“User Managed Mode”，以允许您直接管理和配置文件，并构建或调试基于 Makefile 的工程。

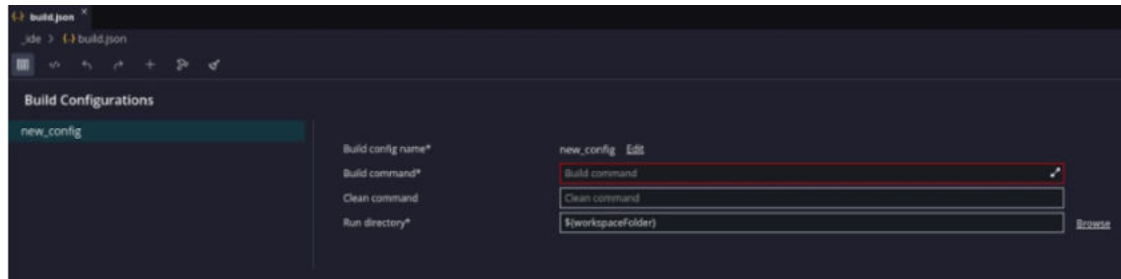
1. 启动 Vitis Unified IDE。
2. 打开一个没有 JSON 文件且包含额外文件夹的工作空间，以显示“User Managed Mode”屏幕。



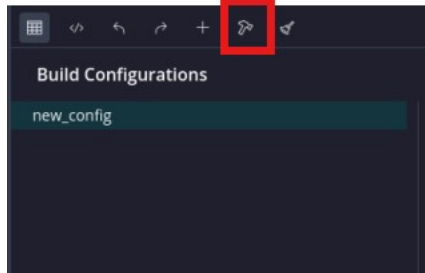
3. 创建构建配置并发起构建：
  - a. 将光标悬停在“Build”（构建）上以单击“Setting”（设置）按钮。



- b. 单击“+ New Build Configuration”（添加新的构建配置），或者也可以单击“+”按钮来为构建生成新的配置。
- c. 输入构建、清理和运行目录。



- d. 单击“Build”以构建该工程。



4. 构建完成后，按以下步骤创建运行/调试启动配置。
  - a. 将光标悬停在“Debug”（调试）上并单击“Setting”（设置）。
  - b. 当启动配置文件 `Launch.json` 出现时，单击“+ New Launch Configuration”（添加新的启动配置）或单击“+”按钮创建新的启动配置。对于嵌入式应用组件，Vitis 工具支持两种设计类型。
  - c. 嵌入式独立应用具有以下两个调试选项：
    - “Configure the Device and Start Debug”（配置器件并启动调试）：该选项用于调试或运行裸机应用组件。指定 XSA 文件并单击“Submit”（提交）以创建配置文件。等待至弹出“Launch Configuration”（启动配置）文件。请参阅 [启动配置](#) 以了解有关设置参考的信息。
    - “Attach to Running Target”（连接到运行中的目标）：如果在运行中的目标上执行调试，请选择该选项并单击“Submit”（提交）。您需要提供目标连接。
  - d. 嵌入式 Linux 应用具有以下两个调试选项：
    - i. “Transfer the Elf file and start debugging”（传输 Elf 文件并启动调试）  
如需运行或调试 Linux 应用，请选中该选项并提供 ELF。接下来，提供必要信息用于运行或调试。
    - ii. “Attaching to Running Processor on Linux Target”（连接 Linux 目标上运行中的处理器）  
要对已运行的 Linux 目标上的 Linux 应用进行调试，请选中该选项并单击“Submit”（提交）。提供目标连接和可执行文件用于运行或调试。

## 设置用户工具链

Vitis 工具支持用户管理模式下的多个工具链版本。

1. 在 Linux 中打开终端，或者打开 Windows 命令提示符。
2. 在命令行中，输入以下命令。

对于 Windows：

```
set VITIS_IDE_USER_TOOLCHAIN=<custom_tool_chain_bin_directory_path>
```

对于 Linux：

```
export VITIS_IDE_USER_TOOLCHAIN=<custom_tool_chain_bin_directory_path>
```

您可从终端或“CMD”窗口启动 Vitis。

# Vitis 实用工具

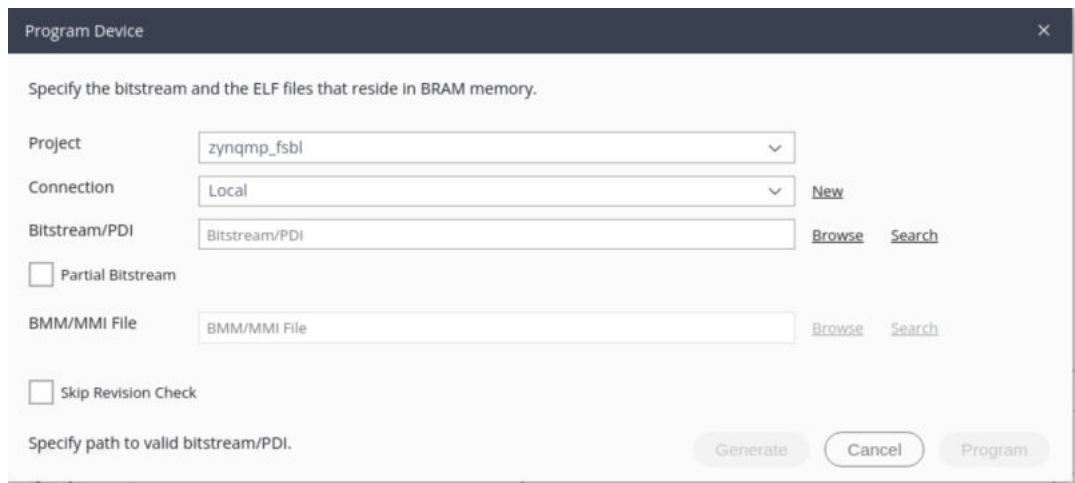
## 软件调试器

启动控制台窗口以便与 XSDB 进行交互。如需了解有关 XSDB 的更多信息，请参阅《软件调试器参考指南》(UG1725)。

## 器件烧录

您可在“Program Device”（器件烧录）屏幕中使用比特流来烧录 FPGA。

图 58：器件烧录



下表列出了“Program Device”（器件烧录）页面上可用的选项：

- “Project”（工程）：用于选择您要使用的系统工程。
- “Connection”（连接）：如果开发板位于远端，请选择硬件或硬件服务器。
- “Bitstream/PDI File”（比特流/PDI 文件）：用于指定比特流/PDI 文件
- “Partial Bitstream”（部分比特流）：表明这是部分比特流文件。
- “BMM/MMI file”（BMM/MMI 文件）：仅适用于 MicroBlaze 设计。这些文件可存储 MicroBlaze 的块 RAM 名称和位置信息。Vivado 会生成这些文件。

- “Skip Revision Check”（跳过版本检查）：跳过版本检查。
- “Generate”（生成）：将 ELF 文件与比特流文件拼接在一起并生成最终比特流文件。
- “Program”（烧录）：对 FPGA 进行烧录。

## Vitis 终端

您可从“Terminal”→“New Terminal”（终端 > 新建终端）创建新的终端。此终端是来自当前工作环境的分支。使用此终端来运行任意 AMD 独立实用工具（Bootgen、Program Flash、Program Device 等）。

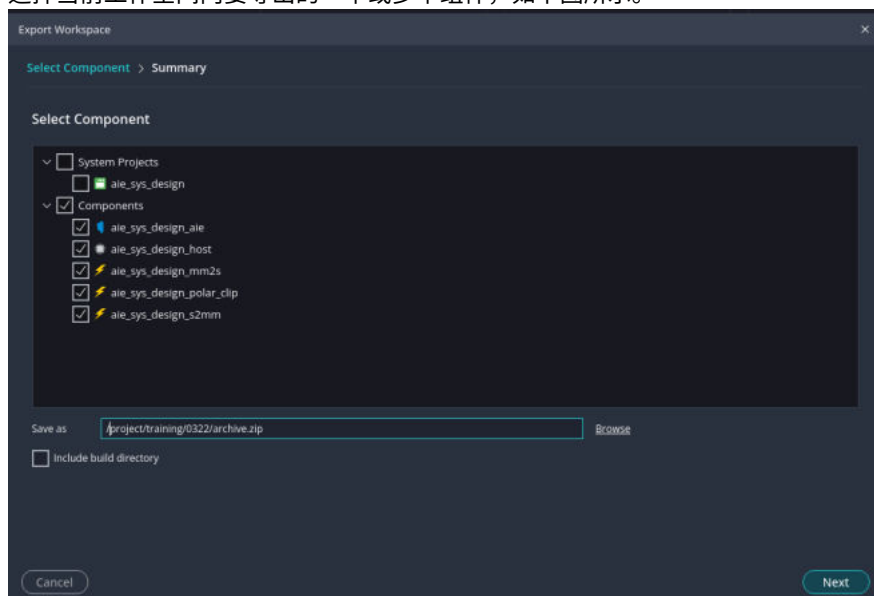
## 工程导出和导入

Vitis Unified IDE 提供了一种简化方法，用于在您的工作空间中导出或导入一个或多个 AMD Vitis™ Unified IDE 工程或组件。

### 导出 Vitis 组件

导出组件时，Vitis 会将组件存档为一个 ZIP 文件。此 ZIP 文件包含导入另一个工作空间所需的所有相关文件。

1. 要导出组件，请从主菜单中依次选择“File”→“Export”（文件 > 导出）。
2. 选择当前工作空间内要导出的一个或多个组件，如下图所示。



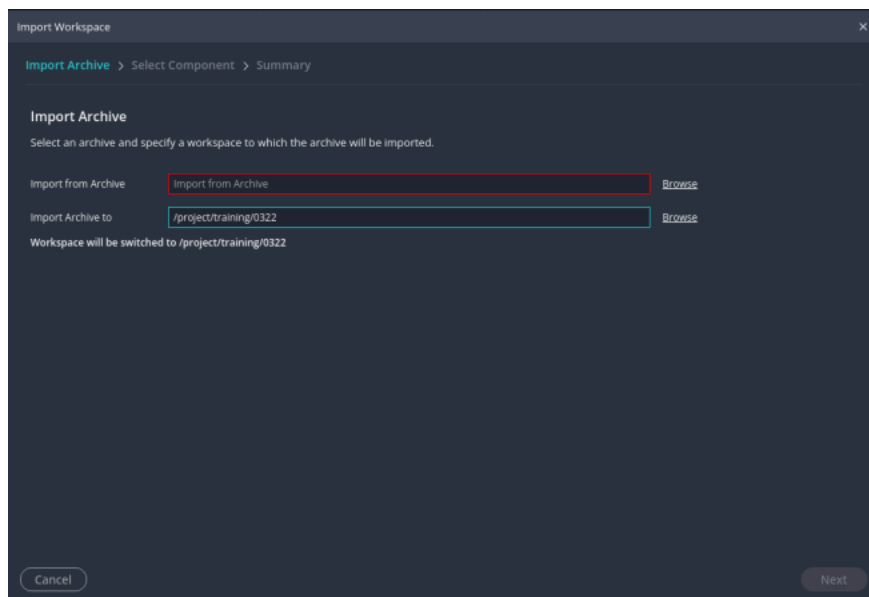
3. 选择您要导出的组件。
4. 单击“Browse”（浏览），选择存档文件的保存位置。然后单击“Next”（下一步）。
5. 请复查导出的组件的汇总信息，然后单击“Finish”（完成）。

**注释：** Vitis 工具会将您选择的 IDE 组件存档到指定的文件和位置。您可将这些存档导入位于不同工作空间下、不同计算机上或者不同用户所拥有的 Vitis Unified IDE 内。

### 导入 Vitis 组件

1. 要导入工程，请从顶部菜单中选择“File” → “Import”（文件 > 导入）。

这样会打开“Vitis Import Workspace” Wizard（Vitis 导入工作空间向导），您可在其中选择存档文件并指定新的工作空间，如下图所示。



2. 选择要导入的存档文件并指定新工作空间，然后单击“Next”。
3. 在此页面中选择您要导入工作空间的组件，然后单击“Next”。
4. 请复查导入的组件的汇总信息，然后单击“Finish”。

## 生成设备树

Vitis 工具从 XSA 中提取系统设备树 (SDT)，并使用它生成 Linux 设备树二进制对象 (DTB)。Vitis Unified IDE 支持两种流程。如果您希望平台创建步骤自动生成 DTB，请使用 Vitis CLI 流程。当您完全控制设备树生成时，请使用 SDT 和 Lopper 流程，例如自定义 Linux 域或在平台流程之外应用叠加。

### 使用 Vitis CLI 生成 DTB

设置 `generate_dtb=True` 后，Vitis CLI 会在平台创建过程中生成 DTB。此流程将 DTB 生成集成到标准平台组件流程中，因此您无需单独运行 `sdtgen` 或 `Lopper`。

开始之前，请确保您拥有 XSA 文件、Linux 操作系统目标、目标 CPU，以及一个可选的自定义 DTSI 目录，以便在需要时注入自定义设备树片段。

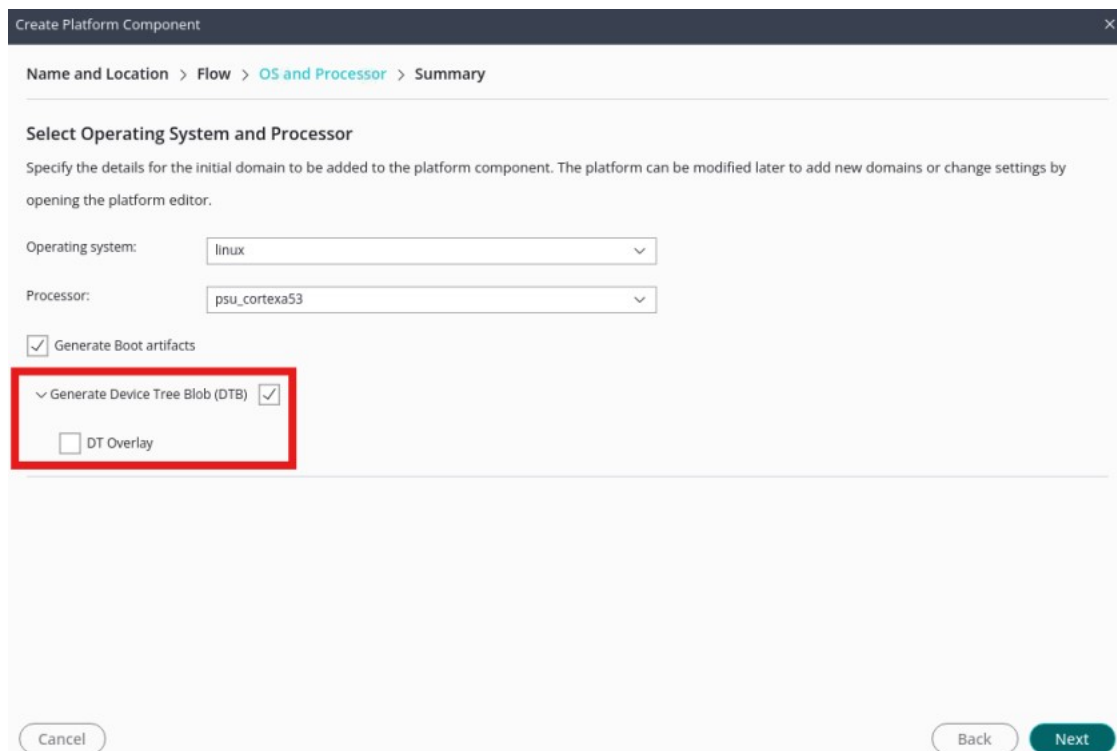
要生成 DTB，请启动 Vitis CLI Python 客户端，创建一个指向 DTSI 目录并配置叠加和 ZOCL 行为的高级选项字典，然后使用 `generate_dtb=True` 创建平台组件。以下示例为 `psv_cortexa72` 创建了一个启用 DTB 生成的平台：

```
adv_options = client.create_advanced_options_dict(
    user_dtsi=dtsi_dir,
    dt_overlay="0",
    dt_zocl="1",
)

platform = client.create_platform_component(
    name="pfm",
    hw_design=curr_dir + "/" + xsa_fixed,
    os="linux",
    cpu="psv_cortexa72",
    domain_name="linux_xrt",
    generate_dtb=True,
    advanced_options=adv_options,
)
```

创建平台组件后，Vitis 工具会将 DTB 写入生成的平台输出目录。您还可以在“Create Platform Component” Wizard（创建平台组件向导）中，通过在“OS and Processor”（操作系统和处理器）页面上选择“Generate Device Tree Blob (DTB)”（生成设备树二进制对象）来启用此选项。

图 59：生成设备树



### 使用 SDT 和 Lopper 生成 DTB（高级）

此流程将 SDT 生成与 DTB 生成分离，并让您能够直接控制每个阶段。当 Vitis CLI 流程未公开您所需的自定义功能时，请使用此流程。

首先，使用 `sdtgen` 从 XSA 生成 SDT。以下命令会提取 VCK190 设计的 SDT，并将输出写入 `sdt_out` 目录：

```
sdtgen -dir sdt_out -xsa design_1_wrapper.xsa -board_dts versal-vck190-  
rev1.1  
sdtgen generate_sdt
```

此步骤会生成 `system-top.dts`，供 Lopper 在下一步中使用。

接下来，通过导出 Lopper 传递给 `dtc` 的设备树编译器标志来配置 Lopper：

```
export LOPPER_DTC_FLAGS="-b 0 -@"
```

最后，结合 Linux 域配置在 SDT 上运行 Lopper 以生成 DTB。

```
lopper -f --enhanced \  
-i lop-a72-imux.dts \  
./linux_ws/platform/hw/sdt/system-top.dts \  
system.dtb \  
-- gen_domain_dts psv_cortexa72_0 linux_dt
```

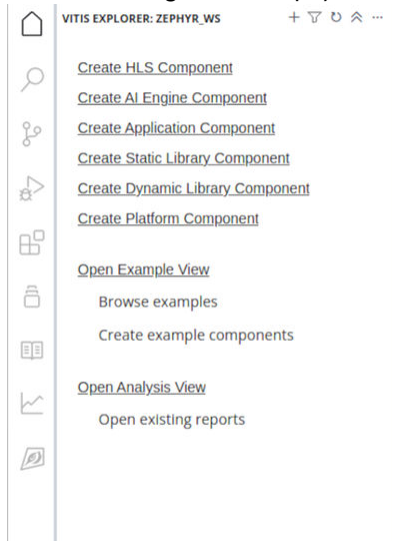
**注释：**DTB 必须与 XSA 中描述的硬件相匹配。传递给平台组件或 Lopper 的 CPU 和域名称必须与 SDT 中的条目完全匹配。自定义 DTSI 片段必须与生成的 SDT 保持兼容。否则，Lopper 或 `dtc` 在构建 DTB 时会报告合并错误。

# Zephyr IDE 扩展

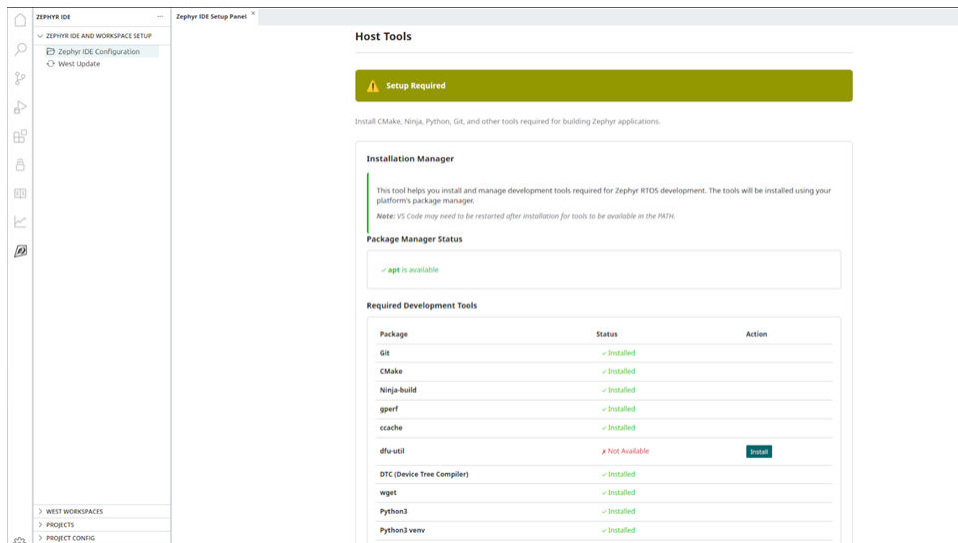
Vitis Unified IDE 支持在 MicroBlaze V 处理器上使用 Zephyr IDE 扩展执行 Zephyr 应用工作流程。在 2026.1 中，您可以使用该扩展设置 Zephyr 工作空间、添加工程、构建应用，并生成 Zephyr ELF 文件。然后，您可以使用 Vitis Unified IDE 创建启动配置，并以用户管理模式启动调试会话。如需了解有关 Zephyr 工作空间、工具链和工程设置的信息，请参阅 Zephyr IDE 扩展文档。如需了解有关创建启动配置并启动用户管理调试会话的 Vitis Unified IDE 步骤，请参阅本主题。在 2026.1 中已预安装 Zephyr IDE 扩展。您无需从 Open VSX Registry 安装它。

开始之前，请先完成以下步骤。

1. 单击左侧工具栏底部的“Zephyr IDE Extension”（Zephyr IDE 扩展）。这样会在 IDE 左侧打开“Zephyr IDE Extension Navigator”（Zephyr IDE 扩展导航器）。



2. 安装主机工具。选择“Zephyr IDE Configuration” → “Host Tools”（Zephyr IDE 配置 > 主机工具）。



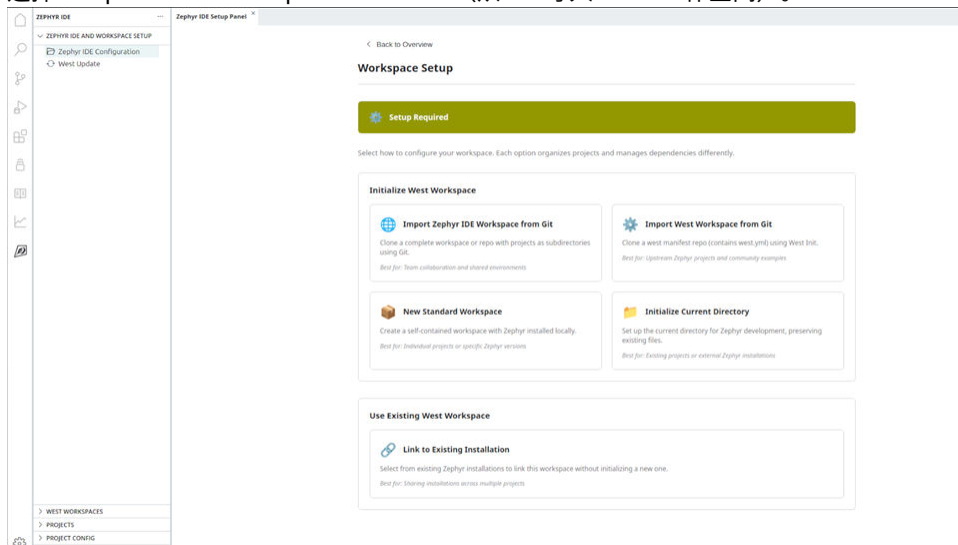
- 验证所有主机工具都已安装。如果某个工具未安装，请单击该开发工具旁边的“Install”（安装）。

**注释：**您必须提供 sudo 访问权限。如果不可行，请在 IDE 外部安装该依赖关系，然后重新加载 Vitis。

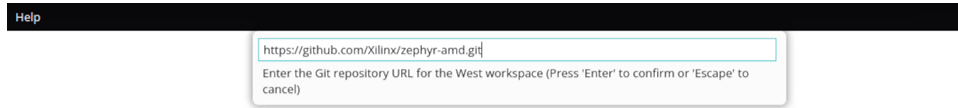
**注释：**除非已安装所有依赖关系，否则您无法继续在 Zephyr IDE 中进行开发。

Zephyr 工作空间有多种配置选项。最可靠的方法是从 [AMD Zephyr 仓库](#) 克隆 Zephyr Project 的 AMD 复刻。

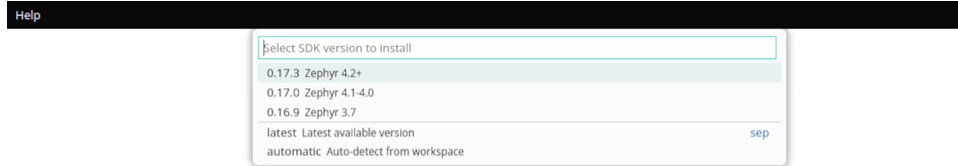
- 选择“Zephyr IDE Configuration” → “Workspace”（Zephyr IDE 配置 > 工作空间）。
- 选择“Import West Workspace from Git”（从 Git 导入 West 工作空间）。



- 当系统提示输入 Zephyr Git 仓库 URL 时，请输入 <https://github.com/Xilinx/zephyr-amd.git> 并按“Enter”。这样会开始工作空间初始化进程。



7. 工作空间初始化进程完成后，会出现提示。选择要安装的 Zephyr SDK 版本。选择 Zephyr SDK 4.2 或更高版本。



**注释：**将 Zephyr SDK 安装到 `~/.zephyr_ide/toolchains` 文件夹。请先确认这一点然后再继续，因为 Zephyr IDE 默认会在该位置查找 SDK。

要在 AMD 平台上创建 Zephyr 应用，必须先将系统设备树 (SDT) 转换为与 Zephyr 兼容的设备树。`west lopper` 工具会处理 SDT，并生成 Zephyr 构建系统所需的设备树输入。在创建或构建 Zephyr 应用之前，请安装 `west lopper` 命令并确保它在环境中可用。

8. 在 Vitis 中打开终端。选择“Terminal” → “New Terminal”（终端 > 新建终端）。
9. 激活 Python 虚拟环境。

```
source .venv/bin/activate
```

10. 切换到 `zephyr` 目录并安装 `west-lopper`。

```
cd zephyr
west lopper-install
```

Zephyr IDE 扩展现已在 Vitis 中设置完成。您可以创建、构建和调试 Zephyr 应用。

## 生成 Zephyr 设备树

1. 从 Vivado 导出的 XSA 文件生成系统设备树。使用 SDTGEN 工具运行以下命令，这些命令会生成 `system-top.dts` 文件。

```
set_dt_param -dir ./sdt_out
set_dt_param -xsa microblaze_v_preset_wrapper.xsa
generate_sdt
```

2. 激活 Python 虚拟环境以访问 `west lopper` 命令。

```
source .venv/bin/activate
```

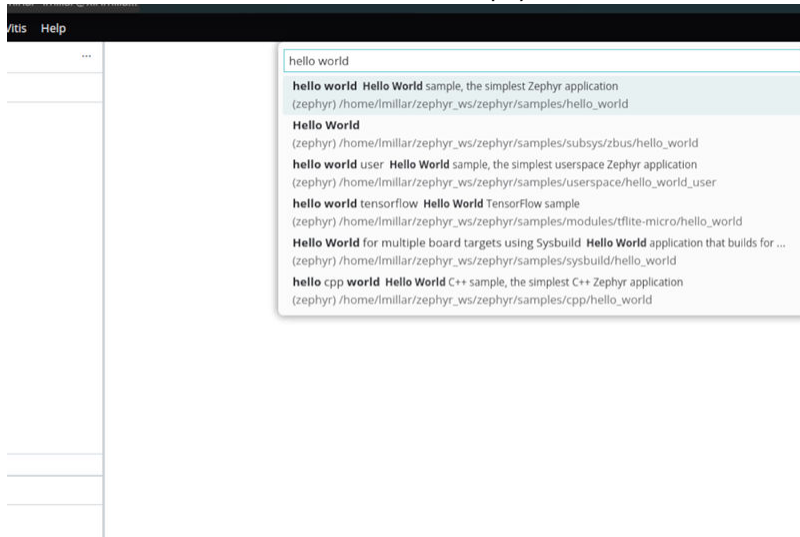
3. 使用 `west lopper` 命令将 SDT 转换为 Zephyr 设备树。提供 `system-top.dts` 作为输入。

```
LOPPER_DTC_FLAGS="-b 0 -@" west lopper-command -p microblaze_riscv_0 -s
design_1_wrapper/system-top.dts -w ~/zephyrproject/zephyr
```

**注释：**终端输出会显示 Zephyr 设备树路径：`/home/zephyr_ws/zephyr/boards/amd/mbv32/mbv32.dts`

## 为 AC701 创建 MicroBlaze V Hello World 应用

1. 打开“Projects”（工程）面板。此面板会显示工作空间中所有 Zephyr 工程的树视图。如果这是您的第一个工程，则该面板为空。
2. 单击“Projects”面板顶部的“Create Project From Template”（从模板创建工程）。该按钮图标显示为带加号的文件夹。
3. 在模板选择提示中，在 `/zephyr/samples/hello_world` 处选择“Hello World Sample”（Hello World 样本）。此选项会将 Hello World 样本应用从 Zephyr 源码树复制到您的工作空间中。



4. 指定工程名称和工程位置：
  - a. “Project Location in current workspace”（工程位置在当前工作空间内）：在您打开的工作空间文件夹中创建子目录（推荐）。
  - b. “Project in external directory”（工程位于外部目录内）：指定工作空间外部的绝对路径。
5. 单击“OK”（确定）并等待工程创建完成：
  - a. Zephyr IDE 会将 Hello World 样本文件复制到您的工程目录中。
  - b. “Projects”面板中会出现一个使用您所选名称的新工程条目。

6. 添加一个构建配置，用于指定目标开发板、构建目录以及构建专用设置（Kconfig 叠加层、设备树叠加层、CMake 实参）。在“Projects”面板中，单击工程名称旁边的箭头以展开工程。单击“Add Build”（添加构建）。
7. 单击工程名称旁边的“Add Build”（添加构建）图标（加号图标）。当系统提示选择其他开发板目录时，请选择“Zephyr Directory Only”（仅限 Zephyr 目录）。
8. 在“Board Picker”（开发板选择器）菜单中，选择“mbv32”。
9. 输入构建名称（可选）。
10. 选择“Debug”（调试）以进行构建优化，并接受预定义的调试标志。
11. 按“Enter”键。Zephyr IDE 会在您的工程中创建一个构建目录：

```
hello_world/build/mbv32/
```

“Projects”面板现在会显示新工程和构建配置。

要构建该应用，请将该构建设置为活动构建。在“Projects”面板中，验证在“Active Project”下是否显示 mbv32 构建。如果它未处于活动状态，请单击构建名称以选中它。

在“Active Project”（活动工程）面板中单击“Build”（构建）以开始构建。在终端中监控构建进度。

**注释：**要在 MicroBlaze V 上调试 Zephyr 应用，请参阅 [在 MicroBlaze V 上调试 Zephyr 应用程序](#)。

## 第三部分

# Vitis Python API

**AMD Vitis™ Unified IDE** 在开发环境中提供逻辑向导，以帮助您进行设计开发。这对开发新的处理器架构非常有用，对新手用户来说尤其如此。不过，要想提升灵活性和获取更多扩展功能，必须熟练掌握在工具中使用脚本的技能。开发定期运行的回归测试或运行一组常用命令时，脚本特别有用。AMD Vitis 命令行接口 (CLI) 是可脚本化的交互式命令行接口，与 Vitis Unified IDE 对接。与其他 AMD 工具一样，AMD Vitis™ CLI 的脚本编写语言同样基于 Python。

Vitis API 支持 Vitis 工程管理、配置、构建和调试，例如：

- 创建平台工程和域
- 创建系统和应用工程
- 配置和构建域/BSP 以及应用
- 管理存储库
- 在硬件目标上下载和运行应用
- 读写寄存器
- 设置断点和观察表达式

**注释：**您可从 Vitis 菜单查看所有 Python API：转至“Help” “Document” “Vitis Python API Documentation”（帮助 > 文档 > Vitis Python API 文档）。

## 在 Python ENV 中启用 Vitis API

在您的用户 Python 环境中启用 Vitis API 即可允许您在 Vitis 工具中运行通用 Python 脚本。

### Linux

打开 Linux 命令提示符并执行以下命令：

```
export XILINX_VITIS=<LOCAL_VITIS_INSTALLATION_DIRECTORY>/2025.2/Vitis
source <LOCAL_VITIS_INSTALLATION_DIRECTORY>/2025.2/Vitis/cli/examples/
customer-python-utils/setup_vitis_env.sh
```

使用以下命令在命令提示符中运行 Python：

```
$VITIS_INSTALL_PATH/2025.2/Vitis/tps/lnx64/python-3.13.0/bin/python
or Launch your local python using
./python
```

执行下列 Python 命令：

```
import vitis
client = vitis.create_client()
client.get_template()
```

图 60：Python 脚本

```
bash-4.4$ ./python --version
Python 3.13.0
bash-4.4$ export XILINX_VITIS=<LOCAL_VITIS_INSTALLATION_DIRECTORY>/2025.2/Vitis
bash-4.4$ source <LOCAL_VITIS_INSTALLATION_DIRECTORY>/2025.2/Vitis/cli/examples/customer_python_utils/setup_vitis_env.sh
=====
Xilinx Vitis environment configured.
=====
bash-4.4$ ./python
Python 3.13.0 (main, Oct 15 2025, 15:26:15) [GCC 8.5.0 20210514 (Red Hat 8.5.0-21)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import vitis
>>> cl=vitis.create_client()
Vitis Server started on port '35449'.
>>> cl.set_workspace("/tmp/test_custom_python")
True
>>> cl.get_workspace()
'/tmp/test_custom_python'
>>> cl.create_platform_component(name="plat1", hw_design="zcu102", cpu="psu_cortexa53_0")
Platform plat1 creation started.
SDT generated successfully.
[INFO]: Validating inputs
[INFO]: Starting domain creation
[INFO]: Generating Domain-specific DTS for psu_cortexa53_0
[INFO]: Validating inputs
[INFO]: Starting domain creation
[INFO]: Generating Domain-specific DTS for psu_cortexa53_0
[INFO]: Generating required metadata from domain specific DT
[INFO]: Successfully created Domain at /tmp/test_custom_python/plat1/psu_cortexa53_0/bsp
Domain added successfully.
[INFO]: Generating required metadata from domain specific DT
CMake Warning at libsrc/xilsecure/src/CMakeLists.txt:31 (message):
  XilSecure library requires xilmailbox library in client mode.
[INFO]: Successfully created Domain at /tmp/test_custom_python/plat1/zynqmp_fsbl/zynqmp_fsbl_bsp
Platform FSBL Boot domain added successfully.
Platform creation finished successfully.
Platform Quick Build initiated.
Initiating build for the plat1 component.
Generating Export directory
Generated platform metadata for creating application(s) based on platform plat1.
{'project_name': 'plat1', 'project_location': '/tmp/test_custom_python/plat1'}
```

要运行 Python 脚本，请执行以下命令：

```
&VITIS_INSTALL_PATH/2025.2/Vitis/tps/lrx64/python-3.13.0/bin/python -s
sample_testcase.py
```

## Windows

在 Windows 上打开 Windows 命令提示符并执行以下命令：

```
set XILINX_VITIS=<LOCAL_VITIS_INSTALLATION_DIRECTORY>\2025.2\Vitis
<LOCAL_VITIS_INSTALLATION_DIRECTORY>\2025.2\Vitis\cli\examples\customer_python_utils>setup_vitis_env.bat
```

要在命令提示符中运行 Python 脚本，请执行以下命令：

```
<vitis_install_dir>\2025.2\Vitis\tps\win64\python-3.13.0\python.exe
```

```
Python 3.13.0 (main, Nov 19 2024, 22:38:08) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import vitis
>>> client = vitis.create_client()
Vitis Server started on port '54276'.
>>> client.get_template()
'installed_examples/aie_system_examples/aie-ml_sys_design'
>>> |
```

要运行 Python 脚本，请执行以下命令：

```
<vitis_install_dir>\2025.2\Vitis\tps\win64\python-3.13.0\python.exe -s
sample_testcase.py
```

# Python API：用于在 Vitis 中创建和管理工程的命令行工具

Vitis Unified IDE 提供了一个直观的图形界面，用于组装和管理您的嵌入式软件工程。利用 Vitis Unified IDE，您可以创建和构建所有 Vitis 组件，包括平台、应用程序、HLS 设计和 AI 引擎。另外，Python API 提供了一种可脚本化的方法，支持您以编程方式创建和管理 Vitis 组件。

## Vitis CLI 命令



**重要提示！** 您可在以下安装文件夹内找到所有 Vitis CLI 命令的详细信息：<Vitis\_install>2025.2/Vitis/cli/api\_docs/build/html/vitis.html

## 执行 Python API

要执行 Python API，首先请在服务器与客户端之间建立连接。您可以在命令行模式下执行 Python API，也可以使用 Python 脚本来执行。AMD Vitis™ Unified IDE 支持以下两种 Python API 运行方法：

1. 在 CLI（命令行接口）模式下运行 Python API：仅支持在交互模式下以 CLI 模式执行 Python API。欲知详情，请参阅《Vitis 参考指南》(UG1702) 中的 [Vitis 交互式 Python Shell](#)。

```
vitis -i (This opens the Vitis interactive shell)
```

2. 在 Python 脚本中运行 Python API。Vitis Unified IDE 支持以批处理模式和交互式模式来执行 Python 脚本。

批处理模式：

```
In batch mode:
$ vitis -s <.py>
In Interactive mode:
$ vitis -i
(This opens the Python interactive shell)
vitis [1]: run <.py>
```

有些 Python API 有助于管理客户端生命周期、创建 Vitis 工作空间和管理 Vitis 工程流程。如需了解更多详情，请参阅 Vitis API CLI 文档。

表 13: Python API: 管理客户端

| Python API    | 描述                   | 示例                           |
|---------------|----------------------|------------------------------|
| create_client | 创建客户端实例。             | client=vitis.create_client() |
| dispose       | 关闭所有客户端连接并终止与服务器的连接。 | client.dispose()             |
| exit          | 关闭会话。                | exit()                       |

创建并构建组件和系统工程后，即可使用以下命令打开工作空间：

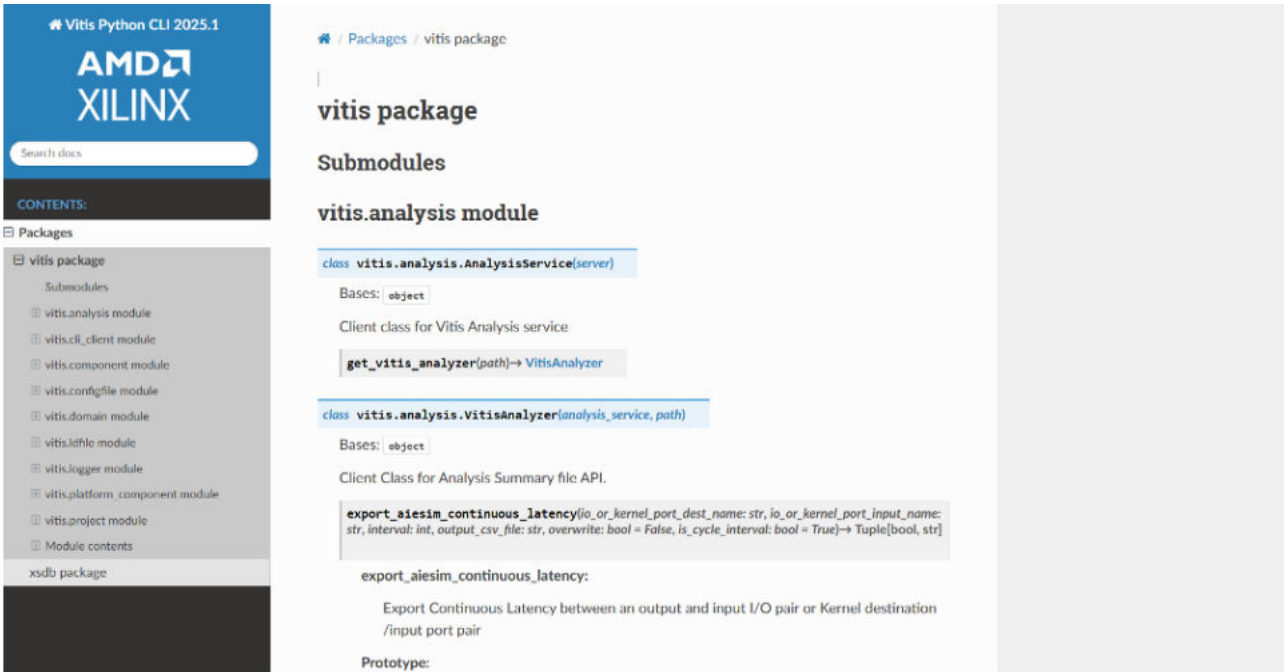
```
vitis -w <workspace_path>
```

如果您是通过 Python API 来构建或创建组件的，那么这些组件在打开时会呈现 done 状态。

Vitis 支持的所有 Python API

**重要提示！** 如需了解 Vitis 支持的所有 Python API，请参阅 <vitis\_installation\_path>/cli/api\_docs/build/html/vitis.html

图 61: Vitis 支持的 Python API 文档



Python 示例可在 <vitis\_installation\_path>/cli/examples 中找到

# 通过 Python API 管理 Vitis IDE 组件

运行脚本前，请为 Vitis Unified IDE 设置环境变量。请参阅《Vitis 统一软件平台文档：应用加速开发》(UG1393) 中的“设置 Vitis 工具环境”。Vitis Unified IDE 支持使用 Python API 来执行工程流程管理，包括创建和构建下列 Vitis 组件：

- 平台控制
- 应用组件
- 系统工程
- AIE
- HLS

## 平台

平台 Python API 支持创建、构建和修改平台。

表 14: Python API：平台

| Python API                | 描述      | Python API 示例                                                                                                                                                                                                 |
|---------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| create_platform_component | 创建新平台   | platform = client.create_platform_component(name = "platform", hw_design = "zcu102", cpu = "psu_cortexa53_0", os = "standalone", domain_name = "standalone_a53", architecture = "32-bit", compiler = "gcc" )  |
| add_domain                | 将域添加到平台 | platform.add_domain(cpu = "psu_cortexa53_0", os = "standalone", name = "a53_standalone", display_name = "a53_standalone", generate_dtb = True, dt_overlay = True, architecture = "64-bit", compiler = "gcc" ) |
| build                     | 生成平台    | platform.build()                                                                                                                                                                                              |

以下是用于创建和构建平台的 Python 脚本示例：

```
import vitis

client = vitis.create_client()
client.set_workspace(path="vitis_2025.2_ws")

platform = client.create_platform_component(name = "platform",hw_design = "zcu102",os = "standalone",cpu = "psu_cortexa53_0",domain_name = "standalone_psu_cortexa53_0",generate_dtb = False,architecture = "64-bit",compiler = "gcc")

platform = client.get_component(name="platform")
status = platform.build()

status = platform.build()
```

# 应用程序

创建平台后，可使用 Vitis 工具的 Python API 创建应用组件。此 API 还支持基于示例模板或带有导入源代码的空应用程序来创建应用程序。

| Python API           | 描述        | Python API 示例                                                                                                                                                                                                                            |
|----------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| create_app_component | 创建应用组件。   | app_comp = client.create_app_component(name = <comp_name>, platform = <platform>, domain = <domain>, template = <template>, cpu = <cpu>, os = <os>, sysroot_toolchain = <sysroot_path>, use_sysroot_toolchain = <boolean>)               |
| import_files         | 将文件导入应用组件 | status = app_comp.import_files(from_loc = <Location of src or config file(s) path>, files = ["file1",...]**, files_to_exclude = ["file1",...]**, dest_dir_in_cmp = <Dest location or config file(s) path>, is_skip_copy_sources = False) |
| build                | 生成应用。     | app_comp.build()                                                                                                                                                                                                                         |

以下 Python 脚本示例演示了如何使用“Hello World”模板来创建应用。

```
import vitis

client = vitis.create_client()
client.set_workspace(path="vitis_2025.2_ws")

comp = client.create_app_component(name="hello_world",platform = "/vitis_2025.2_ws/platform/export/platform/platform.xpfm",domain = "standalone_psu_cortexa53_0",template = "hello_world")

comp = client.get_component(name="hello_world")
comp.build()
```

# 系统工程

Vitis Unified IDE 支持通过 Python API 来创建和构建系统工程。创建平台组件和应用组件后，系统工程即告创建完成。然后，您可将应用组件添加到系统工程中并进行构建。您可以单独构建主机组件。当 Python API 触发系统工程构建时，Vitis 会构建与系统关联的所有组件。

表 15：Python API：系统工程

| Python API         | 描述                        | Python API 示例                                                                       |
|--------------------|---------------------------|-------------------------------------------------------------------------------------|
| create_sys_project | 为给定模板创建系统工程。 <sup>1</sup> | proj = client.create_sys_project(name = 'system_project', platform=<platform_path>) |
| add_component      | 将指定组件添加到给定系统工程中           | proj.add_component(name = "host_component")                                         |
| build              | 为给定构建目标发起系统工程的构建。         | proj.build(target='hw')                                                             |

注释：

- 1. 嵌入式安装程序不支持加速流程。

以下所示 Python 脚本用于创建系统工程并为 target=Hardware 执行构建，以导出 Vitis 元数据存档：

```
# Add package: Vitis Python CLI
import vitis

# Create a Vitis client object
client = vitis.create_client()

# Set Vitis Workspace
client.set_workspace(path="./workspace")

# Defining names for platform, application_component and
plat_name="vck190_embd"
comp_name="standalone_embd_app"

# Create and build platform component for vck190 for
standalone_psv_cortexa72
platform_obj=client.create_platform_component(name=plat_name,
hw_design="vck190", cpu="psv_cortexa72_0", os="standalone")
platform_obj.build()

# This returns the platform xpfm path
platform_xpfm=client.find_platform_in_repos(plat_name)

# Create and build application component
comp = client.create_app_component(name=comp_name, platform =
platform_xpfm, domain = "standalone_psv_cortexa72_0", template =
"hello_world")
comp.build()

# Create system project
sys_proj = client.create_sys_project(name="system_project",
platform=platform_xpfm, template="empty_accelerated_application")
# Add application component to the system project
sys_proj_comp = sys_proj.add_component(name="hello_world")
# Build system project
sys_proj_comp.build()

vitis.dispose()
```

在 <VITIS\_INSTALL\_DIR>/2025.2/cli/examples 中可找到更多 Python 命令示例。

**注释：**嵌入式安装程序不支持加速流程。

如需了解 Vitis 支持的所有 Python API 的更多详情，请访问以下链接：<vitis\_install\_path>cli/api-docs/build/html/vitis.html

## AI 引擎组件

Vitis 工具的 Python API 支持从头开始或者基于模板来创建和构建 AI 引擎组件。

| Python API           | 描述          | Python API 示例                                                                                                           |
|----------------------|-------------|-------------------------------------------------------------------------------------------------------------------------|
| create_aie_component | 创建 AIE 组件   | aie_comp = client.create_aie_component(name = <comp_name>, platform = <platform>, part = <part>, template = <template>) |
| add_cfg_file         | 将配置文件添加到组件中 | status = component.add_cfg_file(cfg_file = <file>)                                                                      |
| build                | 生成 AI 引擎组件  | aie_comp.build()                                                                                                        |

以下 Python 代码会在 Vitis 工具中创建和构建一个 AI 引擎组件。

```
import vitis

client = vitis.create_client()
client.set_workspace(path="vitis_2025.2_ws")

comp = client.create_aie_component(name="simple_aie_application", platform =
"/vitis_2025.2_ws/vck190_platform/export/vck190_platform/
vck190_platform.xpfm", template = "installed_aie_examples/simple")

comp = client.get_component(name="simple_aie_application")
comp.build(target="hw")
```

## HLS 组件

Vitis 工具的 Python API 支持从头开始或者基于模板来创建和构建 HLS 组件。

| Python API           | 描述               | Python API 示例                                                                                                                                       |
|----------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| create_hls_component | 创建 HLS 组件        | hls_comp = client.create_hls_component(name = <comp_name>, platform = <platform_name>, part = <part>, cfg_file = <cfg_file>, template = <template>) |
| add_cfg_file         | 将配置文件添加到 HLS 组件中 | status = component.add_cfg_file(cfg_file = <file>)                                                                                                  |
| run                  | 在 HLS 组件上运行指定的操作 | status = component.run(operation = <operation_type>)                                                                                                |

以下 Python 脚本会创建一个 HLS 空组件并修改其配置文件：

```
import vitis

client = vitis.create_client()
client.set_workspace(path="vitis_2025.2_ws")

comp = client.create_hls_component(name = "mm2s",cfg_file =
["hls_config.cfg"],template = "empty_hls_component")

cfg = client.get_config_file(path="/vitis_2025.2_ws/
simple_aie_application_system_project/hw_link/binary_container_1-link.cfg")

cfg = client.get_config_file(path="/vitis_2025.2_ws/mm2s/hls_config.cfg")

cfg.set_value(section="hls", key="syn.compile.clang_version", value="7")

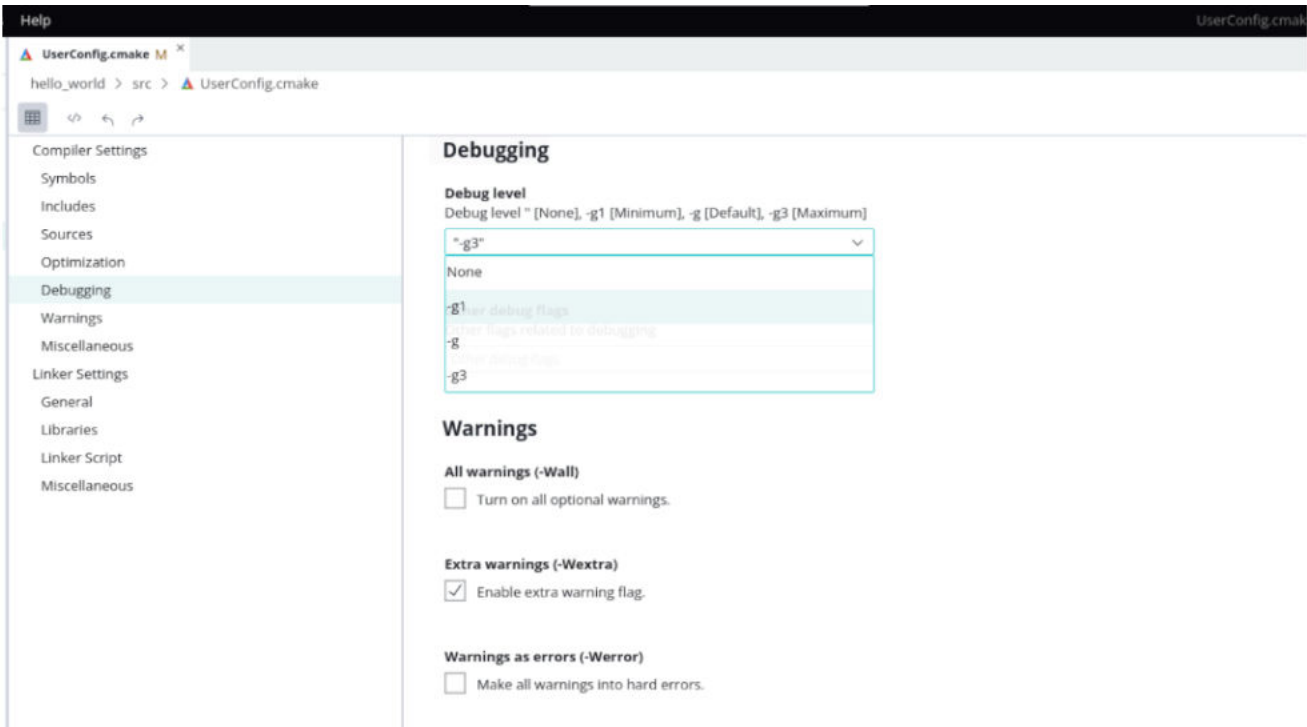
cfg.set_value(section="hls", key="package.output.format", value="rtl")
```

## 修改配置文件

虽然 Python API 可用于创建和构建 Vitis 组件，但它也可用于为每个组件修改配置文件。

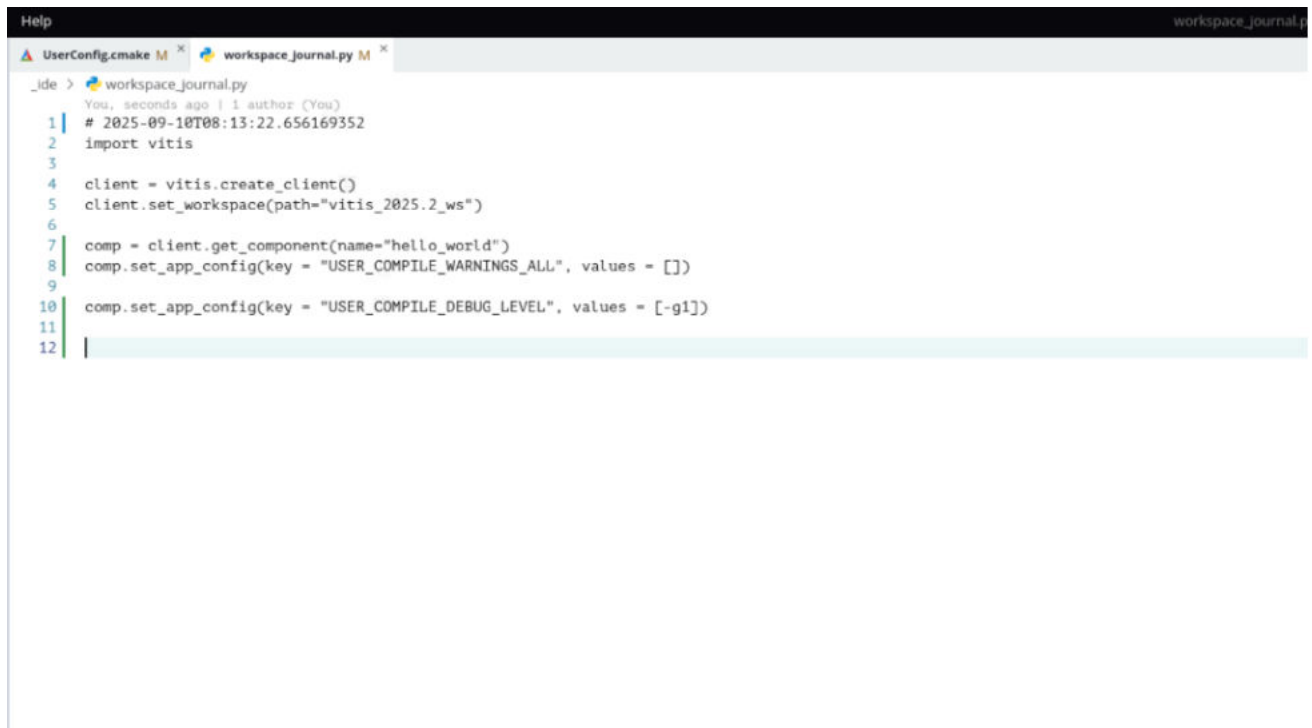
首先，导航到 UserConfig.cmake 文件。

图 62：UserConfig.cmake



在 workspace\_journal.py 文件中可看到等效的 Python API 命令。

图 63: workspace\_journal.py 文件

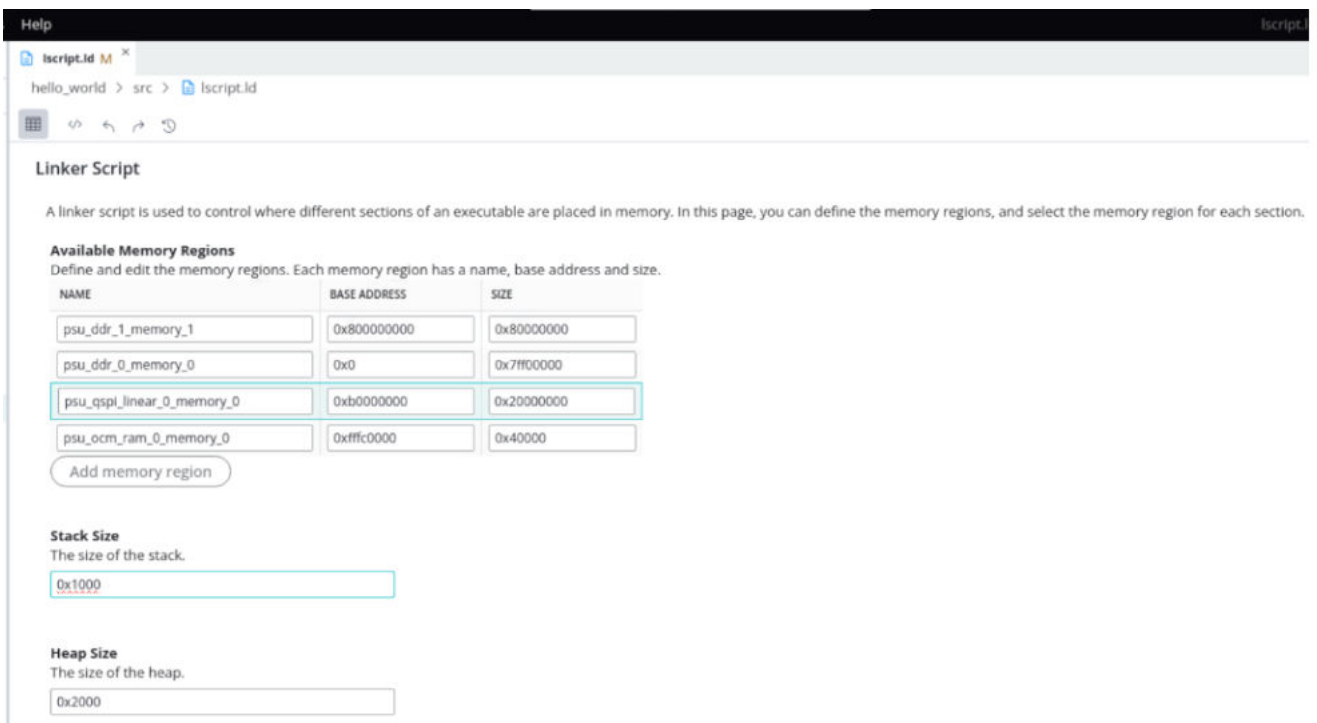


The screenshot shows a code editor with a file named `workspace_journal.py`. The code is as follows:

```
1 # 2025-09-10T08:13:22.656169352
2 import vitis
3
4 client = vitis.create_client()
5 client.set_workspace(path="vitis_2025.2_ws")
6
7 comp = client.get_component(name="hello_world")
8 comp.set_app_config(key = "USER_COMPILE_WARNINGS_ALL", values = [])
9
10 comp.set_app_config(key = "USER_COMPILE_DEBUG_LEVEL", values = [-g1])
11
12
```

接下来，编辑连接器脚本。

图 64: 连接器脚本



The screenshot shows the "Linker Script" configuration page in the Vitis IDE. The page title is "Linker Script". Below the title, there is a description: "A linker script is used to control where different sections of an executable are placed in memory. In this page, you can define the memory regions, and select the memory region for each section."

Below the description, there is a section titled "Available Memory Regions". It says: "Define and edit the memory regions. Each memory region has a name, base address and size."

| NAME                       | BASE ADDRESS | SIZE       |
|----------------------------|--------------|------------|
| psu_ddr_1_memory_1         | 0x80000000   | 0x80000000 |
| psu_ddr_0_memory_0         | 0x0          | 0x7ff00000 |
| psu_qspi_linear_0_memory_0 | 0xb0000000   | 0x20000000 |
| psu_ocm_ram_0_memory_0     | 0xfffc0000   | 0x40000    |

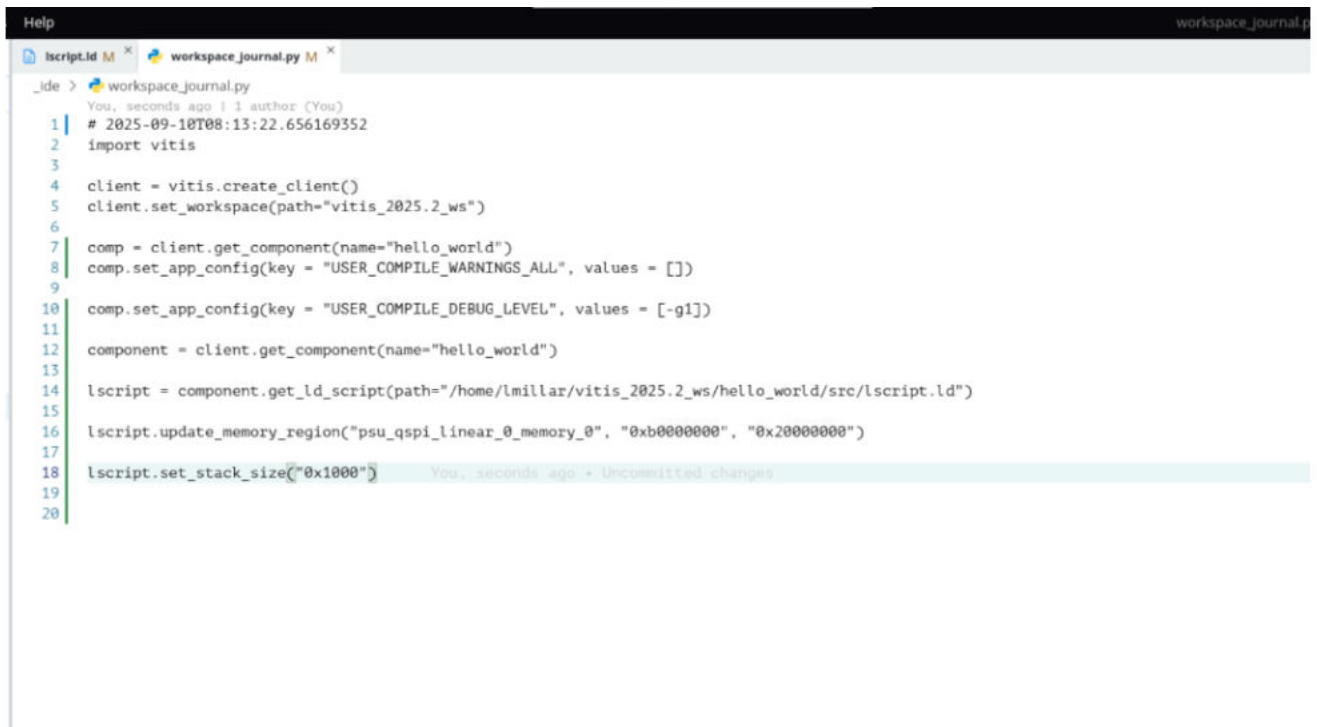
Below the table, there is a button labeled "Add memory region".

Below the table, there is a section titled "Stack Size". It says: "The size of the stack." Below this, there is a text input field containing "0x1000".

Below the stack size section, there is a section titled "Heap Size". It says: "The size of the heap." Below this, there is a text input field containing "0x2000".

工作空间日志文件已更新，其中包含用于更新存储器区域和设置栈大小的命令。

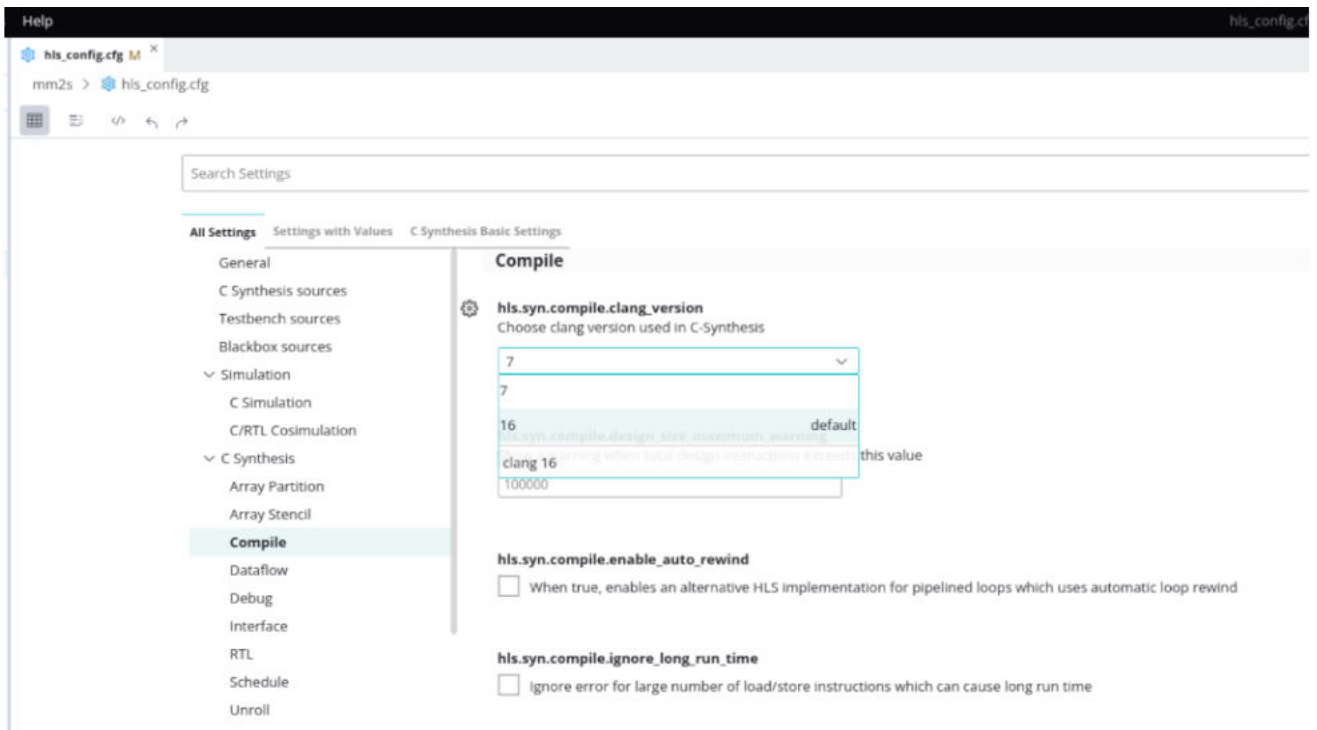
图 65：工作空间日志



```
Help workspace_journal.py
You, seconds ago | 1 author (You)
1 # 2025-09-10T08:13:22.656169352
2 import vitis
3
4 client = vitis.create_client()
5 client.set_workspace(path="vitis_2025.2_ws")
6
7 comp = client.get_component(name="hello_world")
8 comp.set_app_config(key = "USER_COMPILE_WARNINGS_ALL", values = [])
9
10 comp.set_app_config(key = "USER_COMPILE_DEBUG_LEVEL", values = [-g1])
11
12 component = client.get_component(name="hello_world")
13
14 lscript = component.get_ld_script(path="/home/lmillar/vitis_2025.2_ws/hello_world/src/lscript.ld")
15
16 lscript.update_memory_region("psu_qspi_linear_0_memory_0", "0xb0000000", "0x20000000")
17
18 lscript.set_stack_size("0x1000")
19
20
```

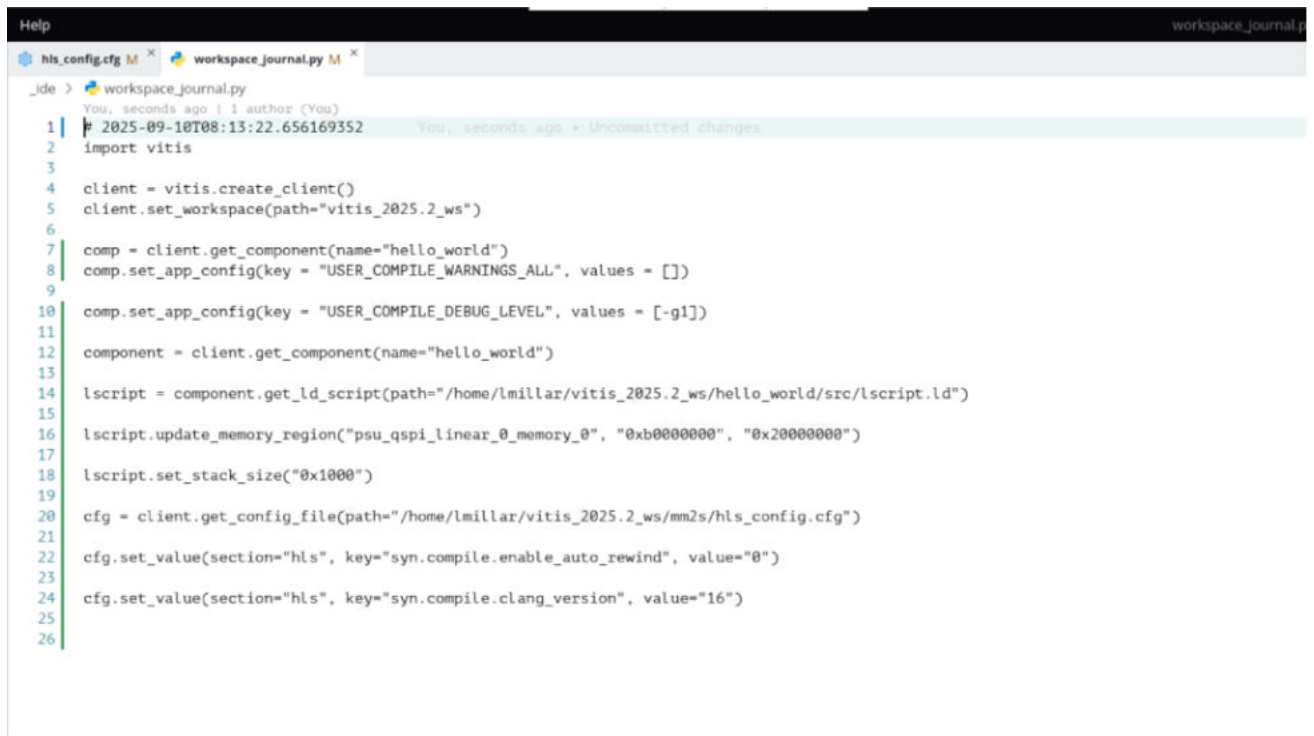
确认 HLS 配置设置。

图 66: HLS 配置



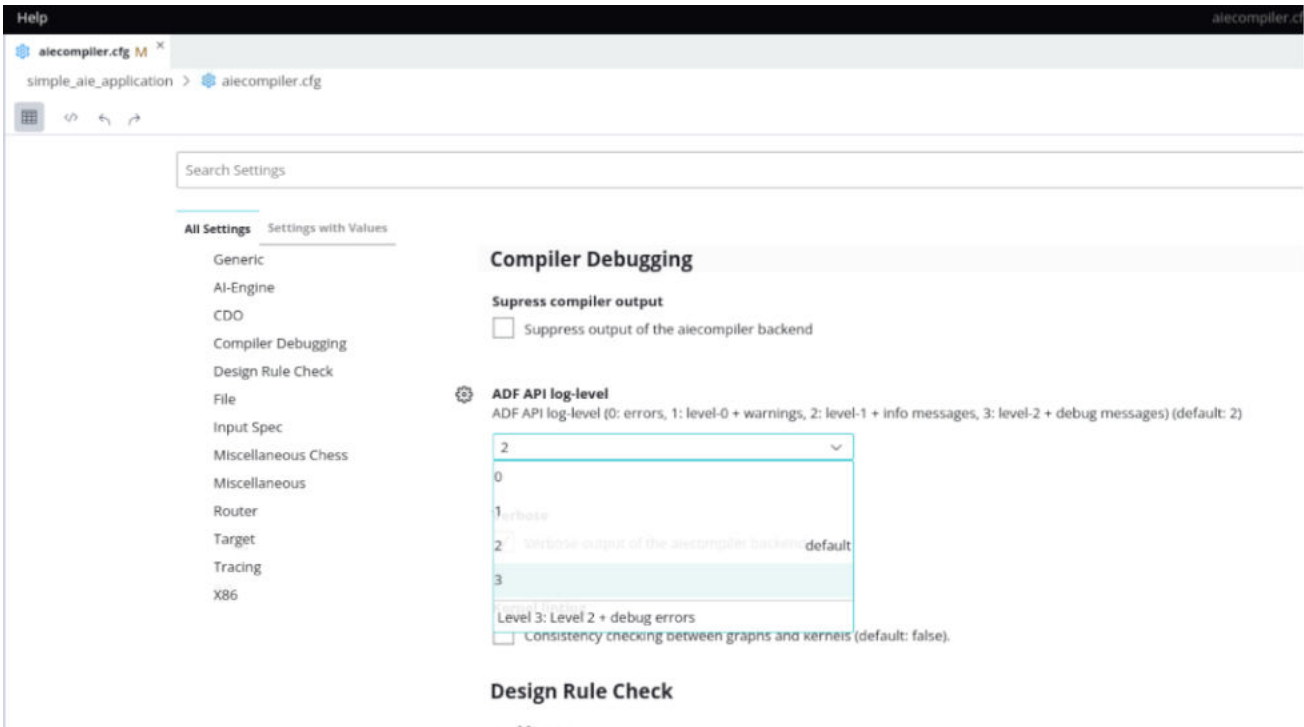
clang 版本和自动回绕设置是在 Python 文件中更新的。

图 67: Python 文件设置



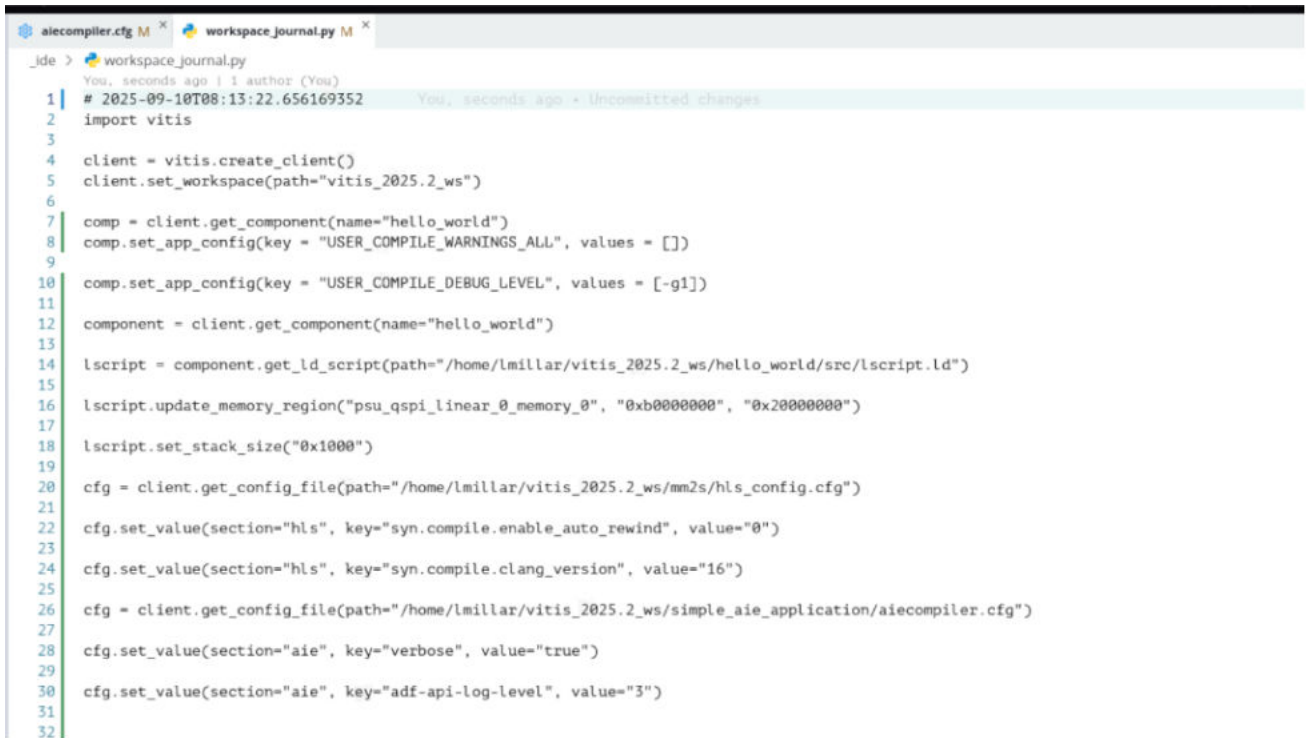
确认 AI 引擎配置。

图 68：AI 引擎配置



详细设置和 ADF 级别是在工作空间日志中设置的。

图 69：详细设置和 ADF 级别



```

1 | # 2025-09-10T08:13:22.656169352 You, seconds ago • Uncommitted changes
2 | import vitis
3 |
4 | client = vitis.create_client()
5 | client.set_workspace(path="vitis_2025.2_ws")
6 |
7 | comp = client.get_component(name="hello_world")
8 | comp.set_app_config(key = "USER_COMPILE_WARNINGS_ALL", values = [])
9 |
10 | comp.set_app_config(key = "USER_COMPILE_DEBUG_LEVEL", values = [-g1])
11 |
12 | component = client.get_component(name="hello_world")
13 |
14 | lscript = component.get_ld_script(path="/home/lmiller/vitis_2025.2_ws/hello_world/src/lscript.ld")
15 |
16 | lscript.update_memory_region("psu_qspi_linear_0_memory_0", "0xb0000000", "0x20000000")
17 |
18 | lscript.set_stack_size("0x1000")
19 |
20 | cfg = client.get_config_file(path="/home/lmiller/vitis_2025.2_ws/mm2s/hls_config.cfg")
21 |
22 | cfg.set_value(section="hls", key="syn.compile.enable_auto_rewind", value="0")
23 |
24 | cfg.set_value(section="hls", key="syn.compile.clang_version", value="16")
25 |
26 | cfg = client.get_config_file(path="/home/lmiller/vitis_2025.2_ws/simple_aie_application/aiecompiler.cfg")
27 |
28 | cfg.set_value(section="aie", key="verbose", value="true")
29 |
30 | cfg.set_value(section="aie", key="adf-api-log-level", value="3")
31 |
32 |

```

## 工作空间日志覆盖范围

从工作空间管理角度来看，工作空间 journal 日志会记录前几页中记述的所有用户操作。包括组件创建、构建、配置修改。具体来说，它会记录以下操作：

- 组件创建
- 组件构建
- 配置文件（UserConfig.cmake、lscript.ld、hls.config.cfg 和 aie\_compiler.cfg）修改

以下提供的工作空间 journal 日志文件示例中，记录了这些组件的创建操作以及对其配置文件进行的编辑操作。

```

import vitis

client = vitis.create_client()
client.set_workspace(path="vitis_2025.2_ws")

platform = client.create_platform_component(name =
"vck190_platform", hw_design = "/project_2/ext_platform-wrapper.xsa", os =
"aie_runtime", cpu = "ai_engine", domain_name =
"aie_runtime_ai_engine", generate_dtb = False)

platform = client.get_component(name="vck190_platform")
status = platform.build()

comp = client.create_aie_component(name="simple_aie_application", platform

```

```

= "/vitis_2025.2_ws/vck190_platform/export/vck190_platform/
vck190_platform.xpfm", template = "installed_aie_examples/simple")

comp = client.get_component(name="simple_aie_application")
comp.build(target="hw")

comp.build(target="hw")

comp.build(target="hw")

comp = client.create_hls_component(name = "mm2s",cfg_file =
["hls_config.cfg"],template = "empty_hls_component")

comp = client.create_hls_component(name = "s2mm",cfg_file =
["hls_config.cfg"],template = "empty_hls_component")

proj =
client.create_sys_project(name="simple_aie_application_system_project",
platform="/vitis_2025.2_ws/vck190_platform/export/vck190_platform/
vck190_platform.xpfm", template="empty_accelerated_application" ,
build_output_type="xsa")

proj = client.get_sys_project(name="simple_aie_application_system_project")

status = proj.add_container(name="binary_container_1")

proj = proj.add_component(name="mm2s", container_name="binary_container_1")

proj = proj.add_component(name="s2mm", container_name="binary_container_1")

proj = proj.add_component(name="simple_aie_application",
container_name="binary_container_1")

cfg = client.get_config_file(path="/vitis_2025.2_ws/
simple_aie_application_system_project/hw_link/binary_container_1-link.cfg")

cfg.add_lines(values=["param=compiler.addOutputTypes=hw-export"])

cfg = client.get_config_file(path="/vitis_2025.2_ws/simple_aie_application/
aiecompiler.cfg")

cfg.set_value(section="aie", key="event-trace", value="runtime")

cfg.set_value(section="aie", key="xlopt", value="2")

cfg = client.get_config_file(path="/vitis_2025.2_ws/mm2s/hls_config.cfg")

cfg.set_value(section="hls", key="syn.compile.clang_version", value="7")

cfg.set_value(section="hls", key="package.output.format", value="rtl")

vitis.dispose()

```

**注释：**要查看您的工作空间 journal 日志文件，请在 Vitis Unified IDE 中选择“Vitis” → “Workplace Journal”（Vitis > 工作空间日志）。

# Python Vitis 命令

使用以下命令启动 Vitis CLI。

```
vitis -i
```

```
client = vitis.create_client()  
help(client)
```

您可在以下位置找到 Vitis Python 命令示例：<VITIS\_INSTALL\_DIR>/2025.2/cli/examples。

您可在以下位置找到用于重新构建 Vitis 工作空间的所有 Vitis Python 命令：<vitis workspace>/\_ide/workspace\_journal.py。

# Python XSDB 命令

您可在 AMD Vitis™ CLI 中使用以下命令查看所有 Python XSDB API 的完整列表。

使用以下命令启动 Vitis CLI。

```
vitis -i
```

```
from xsdb import *  
session = start_debug_session()  
help("functions")
```

**注释：**您可在以下安装文件夹内找到所有 XSDB Python CLI 命令的详细信息：<Vitis\_install>2025.2/  
Vitis/cli/api\_docs/build/html/xsdb.html

# Python XSDB 使用示例

## 对会话对象执行的调试操作

在此模式下，您可创建会话对象，并使用相同会话对象对不同调试目标运行调试操作。

```
session=start_debug_session()
session.connect(url="TCP:xhdbfarmrkd11:3121")
session.targets(3)
# All subsequent commands are run on target 3, until the target is changed
with targets() function
session.dow("test.elf")
session.bpad(addr='main')
session.con()
session.targets(4)
# All subsequent commands are run on target 4
session.dow("foo.elf")
session.bpad(addr='foo')
session.con()
```

## 对目标对象执行的调试操作

您还可使用 targets() 函数返回的 “Target”（目标）对象，并且可对这些对象执行调试操作。示例如下：

```
session = start_debug_session()
session.connect(url="TCP:xhdbfarmrkd11:3121")
ta3 = session.targets(3)
ta4 = session.targets(4)
# Run debug commands using target objects
ta3.dow("test.elf")
ta4.dow("foo.elf")
bp1 = ta3.bpad(addr='main')
bp2 = ta4.bpad(addr='foo')
ta3.con()
ta4.con()
...
bp1.status()
bp2.status()
```

对于交互式使用，请使用命令和选项代替函数和实参，因为函数需要额外输入。XSDB 模块中的 interactive() 函数支持多条命令。示例如下：

```
Vitis-ng [1]: import xsdb
Vitis-ng [2]: xsdb.interactive()
% conn -host xhdbfarmrkb9
tcfchan#0
% ta
1 APU
   2 ARM Cortex-A9 MPCore #0 (Running)
   3 ARM Cortex-A9 MPCore #1 (Running)
4 xc7z020
% ta 2
```

```
<xsdpy._target.Target object at 0x7fb2652d3520>
% stop
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0xffffffff28 (Suspended)
% q
Vitis-ng [3]:
```

### 通过 JTAG 烧录 U-Boot

```
s = xsdb.start_debug_session()
# connecting to the target
s.connect(url="TCP:xhdbfarmrk7:3121")
# Disable Security gates to view PMU MB target
s.targets("--set", filter="name =~ PSU")
# By default, JTAG security gates are enabled
# This disables security gates for DAP, PLTAP and PMU.
s.mwr(0xffca0038, words=0x1ff)
time.sleep(0.5)
# Load and run PMU FW
s.targets("--set", filter="name =~ MicroBlaze PMU")
s.dow('pmufw.elf')
s.con()
time.sleep(0.5)
# Reset A53, load and run FSBL
s.targets("--set", filter="name =~ Cortex-A53 #0")
s.targets()
s.rst(type='processor')
s.dow('fsbl.elf')
s.con()
# Give FSBL time to run
time.sleep(5)
s.stop()
# Downloading the other Softwares like u-boot..etc using below command
s.dow('system.dtb', '-d', addr=0x100000)
s.dow('u-boot.elf')
s.dow('bl31.elf')
s.con()
time.sleep(5)
s.stop()
```

### 生成 SVF 文件

```
# Reset values of respective cores
core = 0
apu_reset_a53 = [0x380e, 0x340d, 0x2c0b, 0x1c07]
# Generate SVF file for linking DAP to the JTAG chain
# Next 2 steps are required only for Rev2.0 silicon and above.
s = xsdb.start_debug_session()
s.connect(url="TCP:xhdxxxx41x:3121")
svf = s.svf()
svf.config('--linkdap', scan_chain=[0x14738093, 12, 0x5ba00477, 4],
           device_index=1, out="dapcon.svf")
svf.generate()

svf = s.svf()
# Configure the SVF generation
svf.config(scan_chain=[0x14738093, 12, 0x5ba00477, 4],
           device_index=1, cpu_index=core, delay=10, out="fsbl_hello.svf")
# Record writing of bootloop and release of A53 core from reset
svf.mwr(0xffff0000, 0x14000000)
svf.mwr(0xfd1a0104, apu_reset_a53[core])
# Record stopping the core
```

```
svf.stop()
# Record downloading FSBL
svf.dow(file='zynq_fsbl.elf')
# Record executing FSBL
svf.con()
svf.delay(100000)
# Record some delay and then stopping the core
svf.stop()
# Record downloading the application
svf.dow(file='zynq_hello.elf')
# Record executing application
svf.con()
# Generate SVF
svf.generate()
```

### 使用 readjtaguart 函数

```
s = xsdb.start_debug_session()
# connecting to the target
s.connect(url="TCP:xhdbfarmrke9:3121")
# List available targets and select a target through its id.
# The targets are assigned IDs as they are discovered on the Jtag chain,
# so the IDs can change from session to session.
# For non-interactive usage, -filter option can be used to select a target,
# instead of selecting the target through its ID
s.targets("--set", filter="name =~ ARM Cortex-A9 MPCore #0")
s.targets()
s.rst()
s.loadhw(hw='zc702.xsa')
# run FSBL for ps7_init
s.dow('fsbl.elf')
s.con()
time.sleep(0.5)
s.stop()
# Download the application program
s.dow('test_jtag_uart.elf')
s.readjtaguart()
s.con()
s.readjtaguart('--stop') # after you are done
```

### 使用 readjtaguart\_file 函数

```
s = xsdb.start_debug_session()
# connecting to the target
s.connect(url="TCP:xhdbfarmrke9:3121")
# List available targets and select a target through its id.
# The targets are assigned IDs as they are discovered on the Jtag chain,
# so the IDs can change from session to session.
# For non-interactive usage, -filter option can be used to select a target,
# instead of selecting the target through its ID
s.targets("--set", filter="name =~ ARM Cortex-A9 MPCore #0")
s.targets()
s.rst()
s.loadhw(hw='zc702.xsa')
# run FSBL for ps7_init
s.dow('fsbl.elf')
s.con()
time.sleep(0.5)
s.stop()
```

```
# Download the application program
s.dow('test_jtag_uart.elf')
s.readjtaguart(file='streams.log', mode='w')
s.con()
s.readjtaguart('--stop') # after you are done
```

### 使用 jtag\_terminal 函数

```
s = xsdb.start_debug_session()
# connecting to the target
s.connect(url="TCP:xhdbfarmrke9:3121")
# List available targets and select a target through its id.
# The targets are assigned IDs as they are discovered on the Jtag chain,
# so the IDs can change from session to session.
# For non-interactive usage, -filter option can be used to select a target,
# instead of selecting the target through its ID
s.targets("--set", filter="name =~ ARM Cortex-A9 MPCore #0")
s.targets()
s.rst()
s.loadhw(hw='zc702.xsa')
# run FSBL for ps7_init
s.dow('fsbl.elf')
s.con()
time.sleep(0.5)
s.stop()
# Download the application program
s.dow('test_jtag_uart.elf')
s.jtagterminal()
s.con()
s.jtagterminal('--stop') # after you are done
```

### Performing standalone\_app\_debug:

```
Vitis [2]: import xsdb

Vitis [3]: s = xsdb.start_debug_session()

# connecting to the target
Vitis [4]: s.connect(url="TCP:xhdbfarmrke9:3121")
tcfchan#0
Out[4]: 'tcfchan#0'

# List available targets and select a target through its id.
# The targets are assigned IDs as they are discovered on the Jtag chain,
# so the IDs can change from session to session.
# For non-interactive usage, -filter option can be used to select a target,
# instead of selecting the target through its ID
Vitis [5]: s.targets()
1  APU
2  ARM Cortex-A9 MPCore #0 (Running)
3  ARM Cortex-A9 MPCore #1 (Running)
4  xc7z020

Vitis [6]: s.targets("--set", filter="name =~ APU")
Out[6]: <xsdb._target.Target at 0x7f01764c6e20>

Vitis [7]: s.targets()
1* APU
2  ARM Cortex-A9 MPCore #0 (Running)
3  ARM Cortex-A9 MPCore #1 (Running)
4  xc7z020
```

```

Vitis [8]: s.rst()

Vitis [9]: s.fpga(file='zc702.bit')
100%      3.86MB      1.66MB/s      00:02ETA

Vitis [10]: s.targets(2)
Out[10]: <xsdb._target.Target at 0x7f014d1f1610>

Vitis [11]: s.loadhw(hw='zc702.xsa')
INFO: [Hsi 55-2053] elapsed time for repository (/proj/xbuils/
HEAD_daily_latest/installs/lin64/Vitis/HEAD/data/embeddedsw) loading 1
seconds

# run FSBL for ps_init
Vitis [12]: s.dow('fsbl.elf')
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0xffffffff28 (Suspended)
Downloading Program -- fsbl.elf
    section, .text: 0x00000000 - 0x0000fc8b
    section, .handoff: 0x0000fc8c - 0x0000fcd7
    section, .init: 0x0000fcd8 - 0x0000fce3
    section, .fini: 0x0000fce4 - 0x0000fcef
    section, .rodata: 0x0000fcf0 - 0x00010043
    section, .data: 0x00010048 - 0x00013017
    section, .mmu_ttbl: 0x00014000 - 0x00017fff
    section, .init_array: 0x00018000 - 0x00018003
    section, .fini_array: 0x00018004 - 0x00018007
    section, .rsa_ac: 0x00018008 - 0x0001903f
    section, .bss: 0x00019040 - 0x0001b3ff
    section, .heap: 0x0001b400 - 0x0001d3ff
    section, .stack: 0xffff0000 - 0xffffd3ff

Successfully downloaded fsbl.elf
Setting PC to Program Start Address 0x00000000

Vitis [13]: s.con()

Info: ARM Cortex-A9 MPCore #0 (target 2) Running
Vitis [14]: s.stop()

Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0xfcac (Suspended)

# Download the application program
Vitis [15]: s.dow('test_jtag_uart.elf')
Downloading Program -- test_jtag_uart.elf
    section, .text: 0x00100000 - 0x00101503
    section, .init: 0x00101504 - 0x0010150f
    section, .fini: 0x00101510 - 0x0010151b
    section, .rodata: 0x0010151c - 0x001016cc
    section, .data: 0x001016d0 - 0x00101b3f
    section, .eh_frame: 0x00101b40 - 0x00101b43
    section, .mmu_ttbl: 0x00104000 - 0x00107fff
    section, .init_array: 0x00108000 - 0x00108003
    section, .fini_array: 0x00108004 - 0x00108007
    section, .bss: 0x00108008 - 0x0010802f
    section, .heap: 0x00108030 - 0x0010a02f
    section, .stack: 0x0010a030 - 0x0010d82f

Successfully downloaded test_jtag_uart.elf
Setting PC to Program Start Address 0x00100000

Vitis [16]: s.bpadd(addr='main')

```

```

Out[16]: <xsdb._breakpoint.Breakpoint at 0x7f016fe59af0>

Info: Breakpoint 0 status:
      target 2: {Address: 0x100564 Type: Hardware}
Info: Breakpoint 0 status:
      target 3: {At col 4: Undefined identifier main. Invalid expression}
Vitis [17]: s.con()

Info: ARM Cortex-A9 MPCore #0 (target 2) Running
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0x100570 (Breakpoint)
main() at ../src/helloworld.c: 57
57: int l_int_b = 20;
Vitis [18]: s.stp()

Info: ARM Cortex-A9 MPCore #0 (target 2) Running
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0x100578 (Step)
main() at ../src/helloworld.c: 58
58: int l_int_a = 40;
Vitis [19]: s.stp()

Info: ARM Cortex-A9 MPCore #0 (target 2) Running
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0x100580 (Step)
main() at ../src/helloworld.c: 60
60: init_platform();
Vitis [20]: s.rrd()
      r0: 00000000          r1: 00000000          r2:
00000001
      r3: 00000028          r4: 00000003          r5:
0000001e
      r6: 0000ffff          r7: f8f00000          r8:
0000767b
      r9: ffffffff          r10: 00000000          r11:
0010c02c
      r12: 00000000          sp: 0010c020          lr:
001007bc
      pc: 00100580          cpsr: 6000005f          usr
      fiq                    irq                    abt
      und                    svc                    mon
      vfp                    cp15                    Jazelle
gpv_qos301_cpu          gpv_qos301_dmac          gpv_qos301_iou
gpv_trustzone          l2cache                    mpcore

Vitis [21]: s.mrd(0xe000d000)
E000D000 : 800A0000

# Local variable value can be modified
Vitis [22]: s.locals()
l_int_b : 20
l_int_a : 40

Vitis [23]: s.locals(name='l_int_b', val=815)

Vitis [24]: s.locals(name='l_int_b')
l_int_b : 815

# Global variables and be displayed, and its value can be modified
Vitis [25]: s.print(expr='g_int_a')
g_int_a : 60

Vitis [26]: s.print(expr='g_int_a', val=9919)

Vitis [27]: s.print(expr='g_int_a')

```

```

g_int_a      : 9919

# Expressions can be evaluated and its value can be displayed
Vitis [28]: s.print('--add', expr='l_int_a + l_int_b')
l_int_a + l_int_b      : 855

# Step over a line of source code
Vitis [29]: s.nxt()

Info: ARM Cortex-A9 MPCore #0 (target 2) Running
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0x100584 (Step)
main() at ../src/helloworld.c: 61
61: test_function(l_int_a, l_int_b);

# View stack trace
Vitis [30]: s.bt()
0  0x100584 main()+32:../src/helloworld.c, line 61
1  0x1007bc _start()+88:xil-crt0.S, line 119
2  unknown-pc

# Set a breakpoint at exit and resume execution
Vitis [31]: s.bpadd(addr='&exit')
Out[29]: <xsdb._breakpoint.Breakpoint at 0x7f016fd2c5e0>

Info: Breakpoint 1 status:
      target 2: {Address: 0x1012ac Type: Hardware}
Info: Breakpoint 1 status:
      target 3: {At col 5: Undefined identifier exit. Invalid expression}
Vitis [32]: s.con()

Info: ARM Cortex-A9 MPCore #0 (target 2) Running
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0x1012ac (Breakpoint)

# View stack trace
Vitis [33]: s.bt()
0  0x1012ac exit()+0
1  0x71034f12

```

### Select\_target\_based\_on\_target\_properties:

```

Vitis [2]: import xsdb

Vitis [3]: s = xsdb.start_debug_session()

# connecting to the target
Vitis [4]: s.connect(url="TCP:xhdbfarmrke9:3121")
tcfchan#0
Out[4]: 'tcfchan#0'

# check the jtag targets connected, the IDs listed with jtag targets are
called node IDs
Vitis [5]: s.jtag_targets()
1  Digilent JTAG-SMT1 210203344713A ( is_pdi_programmable)
2  arm_dap (idcode 4ba00477 irlen 4 is_pdi_programmable)
3  xc7z020 (idcode 23727093 irlen 6 fpga is_pdi_programmable)

# check the targets connected, the IDs listed with targets are called
target IDs
Vitis [6]: s.targets()
1  APU
2  ARM Cortex-A9 MPCore #0 (Breakpoint)
3  ARM Cortex-A9 MPCore #1 (Running)

```

```

4 xc7z020

# check jtag target properties of 2nd device (2nd ARM DAP). Note the
target_ctx here.
Vitis [7]: s.jtag_targets('-t', filter="node_id == 2")
Out[7]:
{'jsn-JTAG-SMT1-210203344713A-4ba00477-0': {'target_ctx': 'jsn-JTAG-
SMT1-210203344713A-4ba00477-0',
  'level': 1,
  'node_id': 2,
  'is_open': 1,
  'is_active': 1,
  'is_current': 0,
  'name': 'arm_dap',
  'jtag_cable_name': 'Digilent JTAG-SMT1 210203344713A',
  'state': '',
  'jtag_cable_manufacturer': 'Digilent',
  'jtag_cable_product': 'JTAG-SMT1',
  'jtag_cable_serial': '210203344713A',
  'idcode': '4ba00477',
  'irlen': '4',
  'is_fpga': 0,
  'is_pdi_programmable': 0}}

# using the target context, select the targets associated with the JTAG
target (2nd ARM DAP - node id = 2)
Vitis [8]: s.targets(filter="jtag_device_ctx == jsn-JTAG-
SMT1-210203344713A-4ba0
...: 0477-0")
1 APU
2 ARM Cortex-A9 MPCore #0 (Breakpoint)
3 ARM Cortex-A9 MPCore #1 (Running)

```

### Debugging\_prog\_already\_running\_on\_target:

```

Vitis [1]: import xsdb

Vitis [2]: s = xsdb.start_debug_session()

# connecting to the target
Vitis [3]: s.connect(url="TCP:xhdbfarmrke9:3121")
tcfchan#0
Out[3]: 'tcfchan#0'

# Select the target on which the program is running and specify the symbol
file using the
# memmap command

Vitis [4]: s.target(2)
Out[4]: <xsdb._target.Target at 0x7f3167dd12e0>

Vitis [5]: s.memmap(file='test_jtag_uart.elf')

# When the symbol file is specified, the debugger maps the code on the
target to the symbol
# file. bt command can be used to see the back trace. Further debug is
possible, as shown in
# the first example

Vitis [6]: s.stop()
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0x10101c (Suspended)
sleep_A9() at sleep.c: 63

```

```

63: } while (tCur < tEnd);

Vitis [7]: s.backtrace()
0  0x10101c sleep_A9()+56:sleep.c, line 63
1  0x100608 test_function()+76:../src/helloworld.c, line 78
2  0x100590 main()+44:../src/helloworld.c, line 61
3  0x1007ac _start()+88:xil-crt0.S, line 119
4  unknown-pc

```

### Debug\_app\_on\_zu\_plus\_mpsoc:

```

Vitis [1]: import xsdb

Vitis [2]: s = xsdb.start_debug_session()

# connect to remote hw_server by specifying its url.
# If the hardware is connected to a local machine, -url option and the <url>
# are not needed. connect command returns the channel ID of the connection
Vitis [3]: s.connect(url="TCP:xhdbfarmrk5:3121")
tcfchan#0
Out[3]: 'tcfchan#0'

# List available targets and select a target through its id.
# The targets are assigned IDs as they are discovered on the Jtag chain,
# so the IDs can change from session to session.
Vitis [4]: s.targets()
1  PS TAP
2  PMU
3  PL
4  PSU
5  RPU (Reset)
6  Cortex-R5 #0 (RPU Reset)
7  Cortex-R5 #1 (RPU Reset)
8  APU (L2 Cache Reset)
9  Cortex-A53 #0 (APU Reset)
10 Cortex-A53 #1 (APU Reset)
11 Cortex-A53 #2 (APU Reset)
12 Cortex-A53 #3 (APU Reset)

Vitis [5]: s.targets("--set", filter="name =~ PSU")
Out[5]: <xsdb._target.Target at 0x7fe9ca93bac0>

Vitis [6]: s.ta()
1  PS TAP
2  PMU
3  PL
4* PSU
5  RPU (Reset)
6  Cortex-R5 #0 (RPU Reset)
7  Cortex-R5 #1 (RPU Reset)
8  APU (L2 Cache Reset)
9  Cortex-A53 #0 (APU Reset)
10 Cortex-A53 #1 (APU Reset)
11 Cortex-A53 #2 (APU Reset)
12 Cortex-A53 #3 (APU Reset)

Vitis [7]: s.rst(type='system')

# Configure the FPGA. When the active target is not a FPGA device,
# the first FPGA device is configured
Vitis [8]: s.fpga(file='zcu102.bit')

```

```

100%      3.86MB      1.66MB/s      00:02ETA

Vitis [9]: s.targets(10)
Out[9]: <xsdb._target.Target at 0x7fe9a99347f0>

Vitis [10]: s.rst(type='cores')

Info: Cortex-A53 #0 (target 9) Stopped at 0xffff0000 (Reset Catch)
Info: Cortex-A53 #1 (target 10) Stopped at 0xffff0000 (Reset Catch)
Info: Cortex-A53 #2 (target 11) Stopped at 0xffff0000 (Reset Catch)
Info: Cortex-A53 #3 (target 12) Stopped at 0xffff0000 (Reset Catch)

# run fsbl to initialize PS
Vitis [11]: s.dow('fsbl_a53.elf')
Downloading Program -- fsbl_a53.elf
    section, .text: 0xfffc0000 - 0xfffd65b7
    section, .note.gnu.build-id: 0xfffd65b8 - 0xfffd65db
    section, .init: 0xfffd6600 - 0xfffd6633
    section, .fini: 0xfffd6640 - 0xfffd6673
    section, .rodata: 0xfffd6680 - 0xfffd6b94
    section, .sys_cfg_data: 0xfffd6bc0 - 0xfffd732b
    section, .mmu_tlb10: 0xfffd8000 - 0xfffd800f
    section, .mmu_tlb11: 0xfffd9000 - 0xfffdafff
    section, .mmu_tlb12: 0xfffdb000 - 0xfffdffff
    section, .data: 0xfffdf000 - 0xfffe02c7
    section, .sbss: 0xfffe02c8 - 0xfffe02ff
    section, .bss: 0xfffe0300 - 0xfffe297f
    section, .heap: 0xfffe2980 - 0xfffe2d7f
    section, .stack: 0xfffe2d80 - 0xfffe4d7f
    section, .dup_data: 0xfffe4d80 - 0xfffe6047
    section, .handoff_params: 0xfffe9e00 - 0xfffe9e87
    section, .bitstream_buffer: 0xffff0040 - 0xfffffc3f

Successfully downloaded fsbl_a53.elf
Setting PC to Program Start Address 0xfffc0000

Vitis [12]: s.con()

Info: Cortex-A53 #1 (target 10) Running
Vitis [13]: s.stop()

Info: Cortex-A53 #1 (target 10) Stopped at 0xfffd2c7c (External Debug Request)

# run the application
Vitis [14]: s.dow('zcu102_hello.elf')
Downloading Program -- zcu102_hello.elf
    section, .text: 0x00000000 - 0x000020b3
    section, .init: 0x000020c0 - 0x000020f3
    section, .fini: 0x00002100 - 0x00002133
    section, .rodata: 0x00002140 - 0x00002360
    section, .rodata1: 0x00002361 - 0x0000237f
    section, .sdata2: 0x00002380 - 0x0000237f
    section, .sbss2: 0x00002380 - 0x0000237f
    section, .data: 0x00002380 - 0x00002bb7
    section, .data1: 0x00002bb8 - 0x00002bbf
    section, .note.gnu.build-id: 0x00002bc0 - 0x00002be3
    section, .ctors: 0x00002be4 - 0x00002bff
    section, .dtors: 0x00002c00 - 0x00002bff
    section, .eh_frame: 0x00002c00 - 0x00002c03
    section, .mmu_tlb10: 0x00003000 - 0x0000300f
    section, .mmu_tlb11: 0x00004000 - 0x00005fff
    section, .mmu_tlb12: 0x00006000 - 0x00009fff

```

```

section, .preinit_array: 0x0000a000 - 0x00009fff
section, .init_array: 0x0000a000 - 0x0000a007
section, .fini_array: 0x0000a008 - 0x0000a047
section, .sdata: 0x0000a048 - 0x0000a07f
section, .sbss: 0x0000a080 - 0x0000a07f
section, .tdata: 0x0000a080 - 0x0000a07f
section, .tbss: 0x0000a080 - 0x0000a07f
section, .bss: 0x0000a080 - 0x0000a0bf
section, .heap: 0x0000a0c0 - 0x0000c0bf
section, .stack: 0x0000c0c0 - 0x0000f0bf

Successfully downloaded zcu102_hello.elf
Setting PC to Program Start Address 0x00000000

Vitis [15]: s.bpadd(addr='main')
Out[15]: <xsdb._breakpoint.Breakpoint at 0x7fe9cae59cd0>

Info: Breakpoint 0 status:
target 6: {At col 4: Undefined identifier main. Invalid expression}
Info: Breakpoint 0 status:
target 7: {At col 4: Undefined identifier main. Invalid expression}
Info: Breakpoint 0 status:
target 9: {At col 4: Undefined identifier main. Invalid expression}
Info: Breakpoint 0 status:
target 10: {Address: 0xd00 Type: Hardware}
Info: Breakpoint 0 status:
target 11: {At col 4: Undefined identifier main. Invalid expression}
Info: Breakpoint 0 status:
target 12: {At col 4: Undefined identifier main. Invalid expression}
Vitis [16]: s.con()

Info: Cortex-A53 #1 (target 10) Running
Info: Cortex-A53 #1 (target 10) Stopped at 0xd08 (Breakpoint)
main() at ../src/helloworld.c: 29
29: int l_int_b = 20;
Vitis [17]: s.rrd()
r0: 00000000      r1: 00000000      r2: 00000000      r3:
00000004
r4: 0000000f      r5: ffffffff      r6: 0000001c      r7:
00000002
r8: ffffffff      r9: 00000000      r10: fd5c0090      r11:
00000000
r12: 00000000      r13: 00000000      r14: 00000000      r15:
00000000
r16: 00000000      r17: 00000000      r18: 00000000      r19:
00000000
r20: 00000000      r21: 00000000      r22: 00000000      r23:
00000000
r24: 00000000      r25: 00000000      r26: 00000000      r27:
00000000
r28: 00000000      r29: 0000e0a0      r30: 00000f34      sp:
0000e0a0
pc: 00000d08      cpsr: 600002cd      vfp
dbg              acpu_gic      sys

Vitis [18]: s.stp()

Info: Cortex-A53 #1 (target 10) Running
Info: Cortex-A53 #1 (target 10) Stopped at 0xd10 (Step)
main() at ../src/helloworld.c: 30
30: int l_int_a = 40;
Vitis [19]: s.stp()

```

```

Info: Cortex-A53 #1 (target 10) Running
Info: Cortex-A53 #1 (target 10) Stopped at 0xd18 (Step)
main() at ../src/helloworld.c: 32
32: init_platform();

# Local variable value can be modified
Vitis [20]: s.locals()
l_int_b      : 20
l_int_a      : 40

Vitis [21]: s.locals(name='l_int_b', val=815)

Vitis [22]: s.locals(name='l_int_b')
l_int_b      : 815

# Global variables and be displayed, and its value can be modified
Vitis [23]: s.print(expr='g_int_a')
g_int_a      : 60

Vitis [24]: s.print(expr='g_int_a', val=9919)

Vitis [25]: s.print(expr='g_int_a')
g_int_a      : 9919

# Expressions can be evaluated and its value can be displayed
Vitis [26]: s.print('--add', expr='l_int_a + l_int_b')
l_int_a + l_int_b : 855

# Step over a line of source code
Vitis [27]: s.nxt()

Info: Cortex-A53 #1 (target 10) Running
Info: Cortex-A53 #1 (target 10) Stopped at 0xd1c (Step)
main() at ../src/helloworld.c: 33
33: test_function(l_int_a, l_int_b);

# View stack trace
Vitis [28]: s.bt()
0  0xd1c main()+28:../src/helloworld.c, line 33
1  0xf34 _startup()+124:xil-crt0.S, line 157

# Set a breakpoint at exit and resume execution
Vitis [29]: s.bpadd(addr='&exit')
Info: Breakpoint 1 status:
      target 6: {At col 5: Undefined identifier exit. Invalid expression}
Info: Breakpoint 1 status:
      target 7: {At col 5: Undefined identifier exit. Invalid expression}
Info: Breakpoint 1 status:
      target 9: {At col 5: Undefined identifier exit. Invalid expression}
Info: Breakpoint 1 status:
      target 10: {Address: 0x1a90 Type: Hardware}
Info: Breakpoint 1 status:
      target 11: {At col 5: Undefined identifier exit. Invalid expression}
Info: Breakpoint 1 status:
      target 12: {At col 5: Undefined identifier exit. Invalid expression}
Out[29]: <xsdb._breakpoint.Breakpoint at 0x7fe9a35e9ca0>

Vitis [30]: s.con()

Info: Cortex-A53 #1 (target 10) Running

Vitis [31]: s.stop()

```

```
Info: Cortex-A53 #1 (target 10) Stopped at 0x192c (External Debug Request)
sleep_A53() at sleep.c: 71
71: } while (tCur < tEnd);

# View stack trace
Vitis [32]: s.bt()
0 0x192c sleep_A53()+44:sleep.c, line 71
1 0x192c sleep_A53()+44:sleep.c, line 70
```

## 第四部分

## XSDB 命令用法



**重要提示！** 如需了解有关 XSDB 命令用法的更多信息，请参阅《软件调试器参考指南》([UG1725](#))。

第五部分

# 从 XSCT 移植到 Python API

本版本包含下列更改：

- 从 2026.1 开始不再提供 XSCT 以供使用。建议您使用 Python API 进行工程创建，并使用基于 Python 的 XSDB 进行调试。如需了解有关 Python API 和 Python XSDB 的更多详细信息，请参阅 [第三部分：Vitis Python API](#) 小节。
- 您仍可使用 XSDB 执行调试，或者也可以将自己的脚本转换为 Python CLI。
- 您可以使用 Vitis Python API 进行工程管理，也可以使用基于 Python 的 XSDB 进行调试。您可复查 `workspace_journal.py` 文件，了解如何使用 Python API。请参阅“Vitis” → “Workplace Journal”（Vitis > 工作空间 Journal 日志）。
- 您可以使用 Python CLI 执行基于 Python 的 HSI 命令。

如需了解更多详细信息，请参阅 [第三部分：Vitis Python API](#) 和 [Vitis 脚本编制流程教程](#)。

## app

应用工程管理。

表 16：app - 应用工程管理

| XSCT 方法    | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                     | 对应 Python API                                                                                                                                                          | 实参                                                                                                                                                                                                                                                                                                                                        | 注释                                                                                                       |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| app create | <p><b>-name</b> &lt;application-name&gt; 表示要创建的应用的名称。</p> <p><b>-platform</b> &lt;platform-name&gt; 平台名称。使用“repo-platforms”可列出可用的预定义平台。</p> <p><b>-domain</b> 域的名称。使用“platform report &lt;platform-name&gt;”可列出平台中可用的系统配置。</p> <p><b>-hw</b> &lt;hw-spec&gt; 从 AMD Vivado™ (XSA) 导出的硬件规格文件。</p> <p><b>-sysproj</b> &lt;system-project&gt; 系统工程的名。使用“sysproj list”可查看工作空间内可用的系统工程。</p> <p><b>-proc</b> &lt;processor&gt; 为该核创建应用程序。</p> | <ol style="list-style-type: none"><li>1. <code>client.create_aie_component</code> (适用于 AIE 应用)</li><li>2. <code>client.create_app_component</code> (适用于主机组件)</li></ol> | <p>必需实参</p> <ol style="list-style-type: none"><li>1. <b>name</b> = &lt;comp_name&gt; AIE 组件名称。</li><li>2. <b>platform</b> = &lt;platform&gt; 或 <b>part</b> = &lt;part&gt; 将为此处指定的平台/部件创建组件。用户只需指定平台或部件之一即可。</li></ol> <p>可选实参</p> <ol style="list-style-type: none"><li>1. <b>template</b> = &lt;template&gt; 此处指定的模板适用于要创建的组件。</li></ol> | 在 XSCT 中，Vitis 工具会在创建组件（应用程序）时提及系统工程。在 Python CLI 中，Vitis 工具会首先创建组件，然后将其添加到系统工程中。这样即可将创建的组件灵活添加到各种系统工程中。 |

表 16: app - 应用工程管理 (续)

| XSCT 方法           | 实参                                                                                                                                                                                                                                                                                                                                                                                                      | 对应 Python API           | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                     | 注释                                                                                                          |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| app create<br>(续) | <p><b>-template</b> &lt;application template&gt; 模板应用的名<br/>称。默认值为 'Hello<br/>World'。使用 “repo<br/>-apps” 可列出可用的模板<br/>应用</p> <p><b>-os</b> &lt;os-name&gt; 操作系统<br/>类型。默认类型为独立<br/>(standalone)。</p> <p><b>-lang</b> &lt;programming<br/>language&gt; 可用编程语言<br/>为 c 或 c++。</p> <p><b>-arch</b> &lt;arch-type&gt; 处理器<br/>架构, &lt;arch-type&gt; 可设为<br/>32 位或 64 位。该选项用于<br/>使用 32/64 位工具链构建<br/>工程</p> |                         | <p>必需实参</p> <ol style="list-style-type: none"> <li><b>name</b> =<br/>&lt;comp_name&gt; 应用<br/>组件名。</li> <li><b>platform</b> =<br/>&lt;platform&gt; 将为此处<br/>指定的平台创建组<br/>件。对于裸机平台,<br/>用户可以在指定平台<br/>的同时指定域。</li> </ol> <p>可选实参</p> <ol style="list-style-type: none"> <li><b>template</b> =<br/>&lt;template&gt; 此处指定<br/>的模板适用于要创建<br/>的组件。</li> <li><b>domain</b> = &lt;domain&gt;<br/>当平台上有多个域<br/>时, 此项用于指定其<br/>中一个域。</li> </ol> |                                                                                                             |
| app remove        | <b>app_name</b>                                                                                                                                                                                                                                                                                                                                                                                         | client.delete_component | <b>name</b> = <comp_name><br>组件名称。                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                             |
| switch            | <b>app_name</b>                                                                                                                                                                                                                                                                                                                                                                                         | client.get_component    | <b>name</b> = <comp_name><br>组件名称。                                                                                                                                                                                                                                                                                                                                                                                                     | Get 命令会返回可用于运行<br>应用命令的应用对象。                                                                                |
| app list          | <p><b>-dict</b> 以 Tcl 词典格式列出<br/>工作空间的所有应用。如<br/>不使用该选项, 则以表格<br/>格式列出应用。<br/>(在 Python CLI 中不支持<br/>此实参)</p>                                                                                                                                                                                                                                                                                           | client.list_components  |                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                             |
| app build         | <p><b>-name</b> &lt;app-name&gt; 表示<br/>要构建的应用的名称。</p> <p><b>-all</b> 要构建的应用的名称。<br/>该选项用于构建所有应用<br/>工程。<br/>(在 Python CLI 中不支持<br/>-all)</p>                                                                                                                                                                                                                                                           | component.build         |                                                                                                                                                                                                                                                                                                                                                                                                                                        | 要构建组件, 组件对象是<br>必需的。例如:<br>component1 =<br>client.get_component('ai<br>e_component1')<br>component1.build() |
| app clean         | <b>-name</b> <app-name>                                                                                                                                                                                                                                                                                                                                                                                 | component.clean         | omponent.clean                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                             |
| app report        | <b>app_name</b>                                                                                                                                                                                                                                                                                                                                                                                         | component.report        |                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                             |

# bsp

配置裸机域的 BSP 设置。

表 17: bsp - 配置裸机域的 BSP 设置

| XSCT 方法    | 实参                                                                                                                     | 对应 Python API      | 实参                                                                                                                                                                                                                                                                                                                                            | 注释                                                       |
|------------|------------------------------------------------------------------------------------------------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| bsp config | <param> <value>                                                                                                        | set_config         | 必需实参： <ul style="list-style-type: none"> <li>· <b>选项</b><br/>= &lt;"lib"/"os"/"proc"&gt;<br/>有效选项为<br/>"lib"/"os"/"proc"。</li> <li>· <b>param_value</b><br/>= [key1:value1,<br/>key2:value2, **]<br/>要设置的键:值对的列表。使用<br/>list_param("lib"/"os"/<br/>"proc") 即可获取可配置参数的列表。</li> </ul> 可选实参：<br><b>lib_name</b> = <lib_name><br>选中 'lib' 选项时的库名。 |                                                          |
| getdrives  | <b>-dict</b> 以成对 <IP-name<br>driver:version> 形式返回结果。<br>(Python CLI 中对此不予支持)                                           | domain.get_drivers | 无                                                                                                                                                                                                                                                                                                                                             |                                                          |
| getlibs    | <b>-dict</b> 以成对 <lib-name<br>version> 形式返回结果                                                                          | domain.get_libs    |                                                                                                                                                                                                                                                                                                                                               |                                                          |
| getos      | <b>-dict</b> 以成对 <os-name<br>version> 形式返回结果                                                                           | domain.get_os      |                                                                                                                                                                                                                                                                                                                                               |                                                          |
| listparams | <b>-lib</b> <lib-name> 返回 BSP<br>中的库的可配置参数<br><b>-os</b> 返回 BSP 中操作系统的<br>可配置参数<br><b>-proc</b> 返回 BSP 中处理器的<br>可配置参数。 | domain.list_params | 必需实参<br><b>option</b> =<br><"lib"/"os"/"proc"><br>有效选项为<br>"lib"/"os"/"proc"<br>可选实参<br><b>lib_name</b> = <lib_name><br>选中 'lib' 选项时的库名                                                                                                                                                                                                       |                                                          |
| regenerate |                                                                                                                        | domain.regenerate  |                                                                                                                                                                                                                                                                                                                                               |                                                          |
| reload     |                                                                                                                        |                    |                                                                                                                                                                                                                                                                                                                                               | 在 Python CLI 中不受支持                                       |
| write      |                                                                                                                        |                    |                                                                                                                                                                                                                                                                                                                                               | 在 Python CLI 中不受支持                                       |
| removelib  |                                                                                                                        | domain.remove_lib  |                                                                                                                                                                                                                                                                                                                                               | 必需实参：<br><b>lib_name</b> = <lib_name><br>表示要添加到 BSP 设置的库 |

表 17: bsp - 配置裸机域的 BSP 设置 (续)

| XSCT 方法      | 实参                                                                                                                     | 对应 Python API      | 实参                                                                                                                                                                                                                                               | 注释                                |
|--------------|------------------------------------------------------------------------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| setdriver    | <b>-driver</b> <driver-name> 要分配给 IP 的驱动程序<br><b>-ip</b> <ip-name> 必须为其添加驱动程序的 IP 实例。<br><b>-ver</b> <version> 驱动程序版本。 | domain.update_path | 在下列实参中，针对 setdriver，option 应设置为 'DRIVER'<br><b>option</b> = <'OS'/'DRIVER'/'LIBRARY'> 有效选项包括 'OS'、'DRIVER' 和 'LIB'<br><b>name</b> = <OS/Driver/Library name> 操作系统/驱动程序/库的名称<br><b>new_path</b> = <OS/Driver/Library path> 要为提及的操作系统/驱动程序/库设置的新路径 | 在 Python CLI 中，会传递路径，而不是 version。 |
| setlib       | <b>-name</b> <lib-name> 请将此库添加到 BSP 设置中。这是默认选项，因此无需使用该选项即可直接指定 lib-name 作为实参。<br><b>-ver</b> <version> 库版本。            | domain.update_path | 在下列实参中，针对 setlib，option 应设为 'LIBRARY'<br><b>option</b> = <'OS'/'DRIVER'/'LIBRARY'> 有效选项包括 'OS'、'DRIVER' 和 'LIB'<br><b>name</b> = <OS/Driver/Library name> 操作系统/驱动程序/库的名称<br><b>new_path</b> = <OS/Driver/Library path> 要为提及的操作系统/驱动程序/库设置的新路径    | 在 Python CLI 中，会传递路径，而不是 version。 |
| setosversion | <b>-ver</b> <version> 操作系统版本。                                                                                          | domain.update_path | 在下列实参中，针对 setosversion，option 应设为 'OS'<br><b>option</b> = <'OS'/'DRIVER'/'LIBRARY'> 有效选项包括 'OS'、'DRIVER' 和 'LIB'<br><b>name</b> = <OS/Driver/Library name> 操作系统/驱动程序/库的名称<br><b>new_path</b> = <OS/Driver/Library path> 要为提及的操作系统/驱动程序/库设置的新路径   | 在 Python CLI 中，会传递路径，而不是 version。 |

# createdts

创建设备树。

表 18: **createdts** - 创建设备树

| XSCT 方法   | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 对应 Python API            | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 注释                  |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| createdts | <p><b>-platform-name</b> &lt;software-platform name&gt;<br/>表示要生成的软件平台的名称。</p> <p><b>-board</b> &lt;board name&gt;<br/>要生成的设备树的开发板名称。开发板名称可从如下位置获取：<br/>&lt;DTG Repo&gt;/device_tree/data/kernel_dtsi。</p> <p><b>-hw</b> &lt;handoff-file&gt;<br/>表示用于创建设备树的硬件描述文件。</p> <p><b>-out</b> &lt;output-directory&gt;<br/>表示需在其中创建软件平台的目录。如果未指定此选项，则工作空间为默认目录。</p> <p><b>-local-repo</b> &lt;directory location&gt;<br/>此处指定的目录位置中包含 Git 仓库的 bsp。如果不指定该选项，则从 Git 克隆设备树仓库。</p> <p><b>-git-url</b> &lt;Git URL&gt;<br/>要克隆的 dtg repo 的 Git URL。如需获取默认仓库，请单击<a href="#">此处</a>。</p> | client.create_platform() | <p>必需实参：</p> <p><b>name</b> = &lt;platform_name&gt; 平台名称。</p> <p><b>hw</b> = &lt;handoff_file&gt; 表示用于创建平台的硬件描述文件。</p> <p><b>emulation_xsa_path</b> = &lt;xsa_path&gt; 指向硬件仿真 XSA 文件的路径，该组件是在此文件中创建的。</p> <p><b>platform_xpfm_path</b> = &lt;xpfm_path&gt; 现有平台的 Xpfm 路径。</p> <p>可选实参：</p> <p><b>desc</b> = &lt;description&gt; 平台的描述。</p> <p><b>os</b> = &lt;os&gt; 该操作系统用于创建默认域。</p> <p><b>cpu</b> = &lt;processor&gt; 该处理器将用于创建默认域。</p> <p><b>domain_name</b> = &lt;domain_name&gt; 将在平台中创建的域的名称。</p> | 创建平台时会创建 DTS 和平台工程。 |

表 18: **createdts** - 创建设备树 (续)

| XSCT 方法          | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 对应 Python API                   | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 注释 |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| createdts<br>(续) | <p><b>-git-branch</b> &lt;Git Branch&gt;<br/>要检出的 Git 分支。默认选中 'xlnx_rel_v&lt;Vitis-release&gt;'。</p> <p><b>-zocl</b> 设置 zocl 标志以启用 zocl 驱动程序支持，默认设置为 False。仅当设计启用 PL 时，才应使用 zocl。仅限 master 和 xlnx_rel_v2021.2 分支才支持 zocl 属性。</p> <p><b>-overlay</b> 设置 overlay 标志以启用设备树叠加层支持，默认设置为 False。</p> <p><b>-dtsi</b> &lt;custom-dtsi-file list&gt;<br/>在设备树中包含 custom-dtsi 文件（如已指定）。文件路径必须采用列表格式。</p> <p><b>-compile</b> 指定该选项即可编译已生成的 dts 以创建 dtb。如果不指定该选项，则您可以手动使用 dts 来编译 dtb。</p> <p>例如，dtc -I dts -O dtb -o &lt;file_name&gt;.dtb &lt;file_name&gt;.dts</p> <p>编译 dts（设备树源文件）文件或 dtsi（设备树源文件 include 文件）文件。</p> <p>dtc -I dts -O dtb -f &lt;file_name&gt;.dts -o &lt;file_name&gt;.dtb</p> <p>将 dts（设备树源文件）转换为 dtb（设备树二进制对象）。</p> <p>dtc -I dtb -O dts -f &lt;file_name&gt;.dtb -o &lt;file_name&gt;.dts</p> <p>将 DTB（设备树二进制对象）转换为 DTS（设备树源文件）。</p> <p><b>-update</b> 设置 update 标志，启用现有设备树平台随新 xsa 一起更新的功能。</p> | client.create_platform()<br>(续) | <p><b>template</b> = &lt;template_name&gt;<br/>对于裸机平台，此模板用于创建域。默认值为 "Empty"</p> <p><b>no_boot_bsp</b> = &lt;bool&gt;<br/>将平台标记为构建时不生成启动组件。</p> <p><b>fsbl_target</b> = &lt;fsbl_target&gt; 对于此处指定的处理器类型，必须为其生成现有 FSBL。</p> <p>该选项仅适用于 ZU+。默认值为 "psu_cortexa53_0"</p> <p><b>fsbl_path</b> = &lt;path&gt;<br/>自定义 FSBL 的 FSBL 路径。选中 no_boot_bsp 时，会使用该选项。</p> <p><b>pmufw_Elf</b> = &lt;path&gt;<br/>预构建的 fsbl.elf，将用作启动组件。选中 no_boot_bsp 时，会使用该选项。</p> |    |

# domain

创建、配置、列出和报告域。

表 19: domain - 创建、配置、列出和报告域

| XSCT 方法           | 实参                                                                                                                                                                                                                                                                                                                                                            | 对应 Python API       | 实参                                                                                                                                                                                                                                                                                                                                                       | 注释                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| domain create     | <b>-name</b> <domain-name> 域的名称。<br><b>-display-name</b> <display_name> 域的报告中所显示的名称。<br><b>-desc</b> <description> 有关该域的简要描述。<br><b>-proc</b> <processor> 将用于创建域的处理器核。对于 SMP Linux，此处可以是处理器核的 Tcl 列表。<br><b>-arch</b> <processor architecture> 32 位或 64 位。该选项仅对 A53 处理器有效。<br><b>-os</b> <os> 操作系统类型。默认类型为独立 (standalone)。                                    | platform.add_domain | 必需实参：<br><b>cpu</b> = <cpu_core> 将用于创建域的处理器核。对于 SMP Linux，此处可以是处理器核的列表                                                                                                                                                                                                                                                                                   |                        |
| domain create (续) | <b>-support-app</b> <app-name> 创建一个域，其中所含 BSP 设置全部是使用 <app-name> 指定的应用所需的设置。该选项仅适用于独立域。“repo -apps”命令可用于列出可用应用的列表。<br><b>-auto-generate-linux</b> 自动生成 Linux 工件。<br><b>-sd-dir</b> <location> 对于采用 Linux 作为操作系统的域，创建 PetaLinux 工程时，请使用该目录中预构建的 Linux 镜像。该选项仅适用于 Linux 域。<br><b>-sysroot</b> <sysroot-dir> 将此 Linux sysroot 目录添加到平台中。此 sysroot 目录会在构建应用程序期间使用。 |                     | 可选实参：<br><b>os</b> = <os> 操作系统类型。默认操作系统设为独立 (standalone)。<br><b>name</b> = <domain_name> 要添加的域的名称。<br><b>display_name</b> = <display_name> 域的显示名称<br><b>support_app</b> = ["app1"...]**创建一个域，其中所含 BSP 设置全部是使用 <app-name> 指定的应用所需的设置。该选项仅适用于独立域。<br><b>sd_dir</b> = <location> 对于采用 Linux 作为操作系统的域，创建 PetaLinux 工程时，请使用该目录中预构建的 Linux 镜像。该选项仅适用于 Linux 域。 |                        |
| domain active     | <b>domain-name</b>                                                                                                                                                                                                                                                                                                                                            | platform.get_domain | 必需实参：<br><b>name</b> = <domain_name> 域的名称                                                                                                                                                                                                                                                                                                                | Get 命令会返回可用于运行域命令的域对象。 |

表 19: domain - 创建、配置、列出和报告域 (续)

| XSCT 方法              | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 对应 Python API                                                                                                                                                                                                      | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 注释 |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| domain<br>config     | <p><b>-display-name</b> &lt;display name&gt; 域的显示名称。</p> <p><b>-desc</b> &lt;description&gt; 有关该域的简要描述。</p> <p><b>-sd-dir</b> &lt;location&gt; 对于采用 Linux 作为操作系统的域，创建 PetaLinux 工程时，请使用该目录中预构建的 Linux 镜像。该选项仅适用于 Linux 域。</p> <p><b>-generate-bif</b> 为该域生成标准 BIF。域报告会显示生成的 BIF 的位置。该选项仅适用于 Linux 域。</p> <p><b>-sw-repo</b> &lt;repositories-list&gt; 存储库列表，生成该域时使用此列表选取驱动程序和库等软件组件。存储库列表需为软件存储库路径的 Tcl 列表。</p> <p><b>-mss</b> &lt;mss-file&gt; 使用 &lt;mss-file&gt; 指定的 mss，代替为域生成 mss 文件。</p> <p><b>-readme</b> &lt;file-name&gt; 为域添加 README 自述文件，其中包含启动指令等内容。</p>                                                                                                                                                                              | <ol style="list-style-type: none"> <li>1. update_name</li> <li>2. add_custom_dtb</li> <li>3. set_sd_dir</li> <li>4. add_boot_di</li> <li>5. add_bif</li> <li>6. add_qemu_args</li> <li>7. add_qemu_data</li> </ol> | <p>必需实参：</p> <ol style="list-style-type: none"> <li>1. <b>name</b> = &lt;new_name&gt; 域的新显示名称</li> <li>2. <b>path</b> = &lt;dtb_file&gt; 自定义 DTB 文件的路径。</li> <li>3. <b>path</b> = &lt;path&gt; 预构建的 Linux 镜像目录的路径</li> <li>4. <b>boot_dir</b> = &lt;boot_dir&gt; 要添加的启动目录</li> <li>5. <b>path</b> = &lt;file_path&gt; 要添加的 Bif 文件</li> <li>6. <b>qemu_option</b> = &lt;"PS"/"PMC"/"PMU"&gt; 有效的 qemu 选项为 "PS"、"PMC" 或 "PMU"。<br/><b>file_name</b> = &lt;file_name&gt; 此文件中会列出所有 pmu/pmc/ps qemu 实参。</li> <li>7. <b>data_dir</b> = &lt;data_dir&gt; 此目录包含 qemu-args 中提供的所有文件</li> </ol> |    |
| domain<br>config (续) | <p><b>-inc-path</b> &lt;include-path&gt; 额外的 include 路径，构建为该域创建的应用时，需添加此路径。</p> <p><b>-lib-path</b> &lt;library-path&gt; 额外的库搜索路径，在为该域创建的应用的连接器设置中需添加此路径。</p> <p><b>-sysroot</b> &lt;sysroot-dir&gt; 需将此 Linux sysroot 目录添加到平台中。此 sysroot 目录会在构建应用期间使用。</p> <p><b>-boot</b> &lt;boot-dir&gt; 此目录用于在构建 Linux 镜像后生成组件。</p> <p><b>-bif</b> &lt;file-name&gt; 此 bif 文件用于为 Linux 启动创建启动镜像。</p> <p><b>-qemu-args</b> &lt;file-name&gt; 此文件中会列出所有 PS QEMU 实参。此文件用于启动 PS QEMU。</p> <p><b>-pmuqemu-args</b> &lt;file-name&gt; 此文件中会列出所有 PMC QEMU 实参。此文件用于启动 PMU QEMU。</p> <p><b>-pmcqemu-args</b> &lt;file-name&gt; 此文件中会列出所有 pmcqemu 实参。此文件用于启动 pmcqemu。</p> <p><b>-qemu-data</b> &lt;data-dir&gt; 此目录包含在 qemu-args 和 pmuqemu-args 选项中提供的文件名中列出的所有文件。</p> |                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |    |
| list                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | platform.list_domain                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |    |

表 19: domain - 创建、配置、列出和报告域 (续)

| XSCT 方法 | 实参          | 对应 Python API | 实参                                               | 注释 |
|---------|-------------|---------------|--------------------------------------------------|----|
| report  | domain-name | domain.report | 必需实参：<br><b>name</b> = <domain_name><br>要删除的域的名称 |    |
| remove  | domain-name |               |                                                  |    |

# getaddrmap

获取连接到处理器的 IP 地址范围。

# getperipherals

获取硬件设计中所有外设的列表。

# getprocessors

获取硬件设计中所有处理器的列表。

表 20: getprocessors - 获取硬件设计中所有处理器的列表

| XSCT 方法       | 实参  | 对应 Python API         | 实参                                                                                         | 注释 |
|---------------|-----|-----------------------|--------------------------------------------------------------------------------------------|----|
| getprocessors | xsa | get_processor_os_list | <b>xsa</b> = <xsa_path> 或 <b>platform</b> = <platform_name> 对应 XSA 或平台，<br>将从中抽取处理器操作系统列表。 |    |

# getws

获取 Vitis 工作空间。

表 21: getws - 获取 Vitis 工作空间

| XSCT 方法 | 实参             | 对应 Python API | 实参                                 |
|---------|----------------|---------------|------------------------------------|
| setws   | -switch <path> | set_workspace | <b>path</b> = <workspace_location> |

# Iscrip

创建连接器脚本。

表 22：Iscrip - 创建连接器脚本

| XSCT 方法        | 实参                                                                                                                                                                                                                                                                                                                      | 对应 Python API                                                                                                                | 实参                                                                                                                                                                              | 注释                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Iscrip memory  | <p><b>-app</b> &lt;application-name&gt; 来自工作空间的应用的名称。</p> <p><b>-supported-mem</b> 返回每一节对应的受支持的存储器区域。</p>                                                                                                                                                                                                               | Iscrip.get_memory_regions                                                                                                    |                                                                                                                                                                                 | <p><b>注释：</b>在 Python CLI 中，使用主机组件创建 Iscrip 对象。而在 XSCT 中，通过传递 application_name 来获取存储器区域。</p> <p><b>注释：</b>在 XSCT 中，不存在创建/更新存储器区域的规定。在 Python CLI 中，可使用以下命令完成此操作：</p> <p><b>Iscrip.add_memory_regionupdate_memory_region</b></p> <p>必需实参：</p> <p><b>name</b> = &lt;name&gt; 新存储器区域的名称。</p> <p><b>base_address</b> = &lt;base_address&gt; 存储器区域的基地址。</p> <p><b>size</b> = &lt;size&gt; 存储器区域的大小。</p> |
| Iscrip section | <p><b>-app</b> &lt;application-name&gt; 来自工作空间的应用的名称。</p> <p><b>-name</b> &lt;section-name&gt; 要编辑的节的名称。</p> <p><b>-mem</b> &lt;memory-region&gt; 表示用于该节的存储器区域的名称。</p> <p><b>-size</b> &lt;section-size&gt; 该节的大小。</p> <p><b>-add</b> 添加新的节。</p> <p><b>-type</b> 要添加的新的节的类型。</p> <p>受支持的类型有：CODE、DATA、STACK 和 HEAP。</p> | <ol style="list-style-type: none"><li>Iscrip.get_ld_sections (用于列示各节)</li><li>Iscrip.update_ld_section (用于更新 ld 节)</li></ol> | <ol style="list-style-type: none"><li>无</li><li>必需实参：<p><b>section</b> = &lt;section&gt; 现有存储器代码节的标识符。</p><p><b>region</b> = &lt;memory_region&gt; 代码节的更新后的存储器区域。</p></li></ol> |                                                                                                                                                                                                                                                                                                                                                                                                    |

表 22: Iscript - 创建连接器脚本 (续)

| XSCT 方法         | 实参                                                                                                                              | 对应 Python API                                                           | 实参 | 注释 |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|----|----|
| Iscrip def-mem  | <b>-app</b> <application-name> 来自工作空间的应用的名称。<br><b>-code</b> 返回默认代码存储器。<br><b>-data</b> 返回默认数据存储器。<br><b>-stack</b> 返回默认栈和堆存储器。 | Iscrip.get_stack_size<br>(用于获取栈大小)<br>Iscrip.get_heap_size<br>(用于获取堆大小) |    |    |
| Iscrip generate |                                                                                                                                 | Iscrip.regenerate                                                       |    |    |

platform

创建、配置、列出和报告平台。

表 23: platform - 创建、配置、列出和报告平台

| XSCT 方法 | 实参            | 对应 Python API                 | 实参                                        | 注释                       |
|---------|---------------|-------------------------------|-------------------------------------------|--------------------------|
| active  | platform-name | client.get_platform_component | 必需实参: <b>name</b> = <platform_name> 平台名称。 | Get 命令会返回可用于运行平台命令的平台对象。 |
| clean   |               | platform.clean                |                                           |                          |

表 23: platform - 创建、配置、列出和报告平台 (续)

| XSCT 方法 | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 对应 Python API                                                                                                                                                                                                                                                                                                                                                                                                                         | 实参                                                                                                                                                                                                                                                           | 注释 |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| config  | <p><b>-desc</b> &lt;description&gt;<br/>添加有关平台的简要描述。</p> <p><b>-updatehw</b> &lt;hw-spec&gt;<br/>更新平台以使用由 &lt;hw-spec&gt; 指定的新硬件规格文件。</p> <p><b>-samples</b> &lt;samples-dir&gt;<br/>将 &lt;samples-dir&gt; 中指定的应用模板包含到平台中。该选项仅适用于可加速的应用。“repo -apps &lt;platform-name&gt;”可用于列出可供给定平台名称使用的应用模板。</p> <p><b>-prebuilt-data</b> &lt;directory-name&gt;<br/>对于可扩展平台，目录名称中指定的预生成硬件数据用于构建不包含加速器的用户应用。这样即可缩短构建时间。</p> <p><b>-make-local</b><br/>将引用的软件组件转变为平台的本地组件。</p> <p><b>-fsbl-target</b> &lt;processor-type&gt;<br/>必须重新生成的现有 FSBL 的处理器类型。该选项仅适用于 ZU+。</p> <p><b>-create-boot-bsp</b><br/>为平台生成启动组件。</p> | <div><div>1.</div><div>update_desc</div></div> <div><div>2.</div><div>update_hw</div></div> <div><div>3.</div><div>不支持</div></div> <div><div>4.</div><div>不支持</div></div> <div><div>5.</div><div>不支持</div></div> <div><div>6.</div><div>retarget_fsbl</div></div> <div><div>7.</div><div>generate_boot_bsp</div></div> <div><div>8.</div><div>remove_boot_bsp<br/>(对于 9 和 10, remove_boot_bsp 使用 fsbl_elf 和 pmufw_elf 选项)</div></div> | <p>必需实参：</p> <p><b>desc</b> = &lt;description&gt; 平台的描述</p> <p>必需实参</p> <p><b>hw</b> = &lt;hw_spec&gt; 或 emulation_xsa_path = &lt;xsa_path&gt; 硬件规格文件或仿真 XSA 路径。</p> <p>可选实参</p> <p><b>target_processor</b> = &lt;target&gt; 对于此处指定的处理器，必须为其重新生成现有 FSBL。</p> |    |

表 23: platform - 创建、配置、列出和报告平台 (续)

| XSCT 方法       | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 对应 Python API         | 实参                                                                                                                                                                                                                                                                    | 注释 |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| config<br>(续) | <p><b>-remove-boot-bsp</b><br/>移除平台创建期间生成的所有启动组件。</p> <p><b>-fsbl-elf</b> &lt;fsbl.elf&gt;<br/>指定 “remove-boot-bsp” 选项时，要用作启动组件的预构建的 fsbl.elf。</p> <p><b>-pmufw-elf</b> &lt;pmufw.elf&gt;<br/>指定 “remove-boot-bsp” 选项时，要用作启动组件的预构建的 pmufw.elf。</p> <p><b>-extra-compiler-flags</b> &lt;param&gt; &lt;value&gt;<br/>使用提供的值为该参数设置额外的编译器标志。<br/>仅支持使用 FSBL 和 PMUFW 作为参数。如果不传递该值，则会返回现有值。</p> <p><b>-extra-linker-flags</b> &lt;param&gt; &lt;value&gt;<br/>以提供的值为参数设置额外的连接器标志。<br/>仅支持使用 FSBL 和 PMUFW 作为参数。如果不传递该值，则会返回现有值。</p> <p><b>-reset-user-defined-flags</b> &lt;param&gt;<br/>将额外的编译器和连接器标志复位。仅支持使用 FSBL 和 PMUFW 作为参数。</p> <p><b>-report</b> &lt;param&gt;<br/>返回设置到给定参数的额外编译器和连接器标志的列表。仅支持使用 FSBL 和 PMUFW 作为参数。</p> |                       | <p>可选实参</p> <p><b>target_processor</b> = &lt;processor&gt; 用于生成启动 bsp 的目标处理器。默认值为 zynqmp_fsbl</p> <p>可选实参</p> <p><b>fsbl_path</b> = &lt;path&gt; 预构建的 fsbl.elf，移除启动组件时，此文件将用作启动组件。</p> <p><b>pmufw_elf</b> = &lt;pmufw.elf&gt; 预构建的 pmufw.elf，移除启动组件时，此文件将用作启动组件。</p> |    |
| list          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | client.list_platforms |                                                                                                                                                                                                                                                                       |    |
| report        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | platform.report       |                                                                                                                                                                                                                                                                       |    |
| remove        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | platform-name         |                                                                                                                                                                                                                                                                       |    |
| read          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 在 Python CLI 中不受支持    |                                                                                                                                                                                                                                                                       |    |
| write         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 在 Python CLI 中不受支持    |                                                                                                                                                                                                                                                                       |    |

# repo

获取、设置或修改软件存储库。

表 24: repo - 获取、设置或修改软件存储库

| XSCT 方法               | 实参                    | 对应 Python API                                                     | 实参                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------|-----------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -set                  |                       | 对于示例仓库<br>add_local_example_repo<br>对于软件仓库<br>set_sw_repo         | 必需实参：<br><b>name</b> = <repo_name><br>要添加的本地仓库名称。<br>local_directory =<br><local_repo_directory_path> 本地仓库目录路径。<br>可选实参：<br><b>type</b> = <repo_type>** 仓库类型。<br>有效类型为 “SYS_PROJ”（对应系统工程）、“HLS” 和 “AIE”。默认类型为 “SYS_PROJ”。<br><b>display_name</b> =<br><repo_display_name>**<br>仓库显示名称。<br><b>description</b> = <"Description of the repo">**<br>本地仓库的描述。<br>必需实参：<br><b>level</b> = <'LOCAL'/'GLOBAL'><br>软件仓库将设为此处指定的层级。<br>LOCAL - 可用于当前工作空间。<br>GLOBAL - 可跨工作空间使用。<br><b>path</b> = <repo_path><br>要设置的软件仓库路径。路径可以是单一路径，也可以是路径列表。 |
| -get                  |                       | 1. 对于示例仓库<br>list_example_repos<br>2. 对于软件仓库<br>get_sw_repo       | 可选实参：<br><b>type</b> = <repo_type>**<br>有效类型为 “SYS_PROJ”（对应系统工程）、“HLS”、“AIE” 和 “EMBD_APP”（对应嵌入式应用）。默认类型为 “SYS_PROJ”。<br>必需实参：<br><b>level</b> = <'LOCAL'/'GLOBAL'> 软件仓库的级别。                                                                                                                                                                                                                                                                                                                                                          |
| -scan                 |                       | 1. 对于平台仓库<br>rescan_platform_repos<br>2. 对于软件仓库<br>rescan_sw_repo | 必需实参：<br><b>platform</b> = <platform_path> 用户可以提供单一平台路径作为字符串，或者也可以提供多条平台路径作为字符串列表。                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| -os                   |                       | 对于域操作系统，请使用 get_os()                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| -libs                 |                       | 对于域库，请使用 get_libs()                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| -drivers              |                       | 对于域驱动程序，请使用 get_drivers                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| -app                  |                       | 对于 -app 选项，请使用 dom.get_applicable_libs                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| -add-platforms        | <platforms directory> | add_platform_repos                                                | 必需实参：<br><b>platform</b> = <platform_path><br>一条或多条平台路径的字符串/列表。                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| -remove-platforms-dir | <platforms directory> | delete_platform_repos                                             | 必需实参：<br><b>platform</b> = <platform_path><br>一条或多条平台路径的字符串/列表。                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

# setws

设置 Vitis 工作空间。

表 25: **setws** - 设置 Vitis 工作空间

| XSCT 方法 | 实参        | 对应 Python API                                      | 实参                                          |
|---------|-----------|----------------------------------------------------|---------------------------------------------|
| setws   | workspace | 1. client.set_workspace<br>2. client.get_workspace | 必需实参：<br><b>path</b> = <workspace_location> |

# sysproj

系统工程管理。

表 26: **setws** - 设置 Vitis 工作空间

| XSDB 方法 | 对应 Python API            | 实参                                                                                                                                                                                                                     |
|---------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| build   | sys_proj.build           | 可选实参：<br><b>target</b> = <target name><br>受支持的目标之一 (sw_emu/hw_emu/hw)。<br>sw_emu (默认值)<br><b>comp_name</b> = <comp_name>**<br>要构建的组件。完整的系统工程 (默认)<br><b>build_comps</b> = <bool><br>如果尚未构建从属组件，则构建这些组件。<br>True (默认值)。 |
| clean   | sys_proj.clean           | 可选实参：<br><b>target</b> = <target name> - 受支持的目标之一<br>(sw_emu/hw_emu/hw)。<br><b>comp_name</b> =<comp_name>**<br>要构建的组件。完整的系统工程 (默认)                                                                                     |
| list    | client.list_sys_projects |                                                                                                                                                                                                                        |
| remove  | client.remove_sys_proj   | 必需实参：<br><b>name</b> = <proj_name> 表示要删除的系统工程。<br>(字符串)                                                                                                                                                                |
| report  | sys_proj.report          |                                                                                                                                                                                                                        |

# importprojects

```
import_projects:
    Import components and system_projects from the mentioned zip directory
    to the current workspace.
    (Get the list of existing components and projects from the zip using
    get_project_info(src) API.)

Prototype:
    sys_proj = client.import_projects(src = <source_zip_dir>,
                                     components = <component_names>,
                                     system_projects = <proj_names>,
                                     dest = <destination_dir>)

Required Arguments:
    src = <source_zip_dir>
        Full path of the zipped project.

Optional Arguments:
    components = <component_names>
        Names of components to be imported.
    system_projects = <proj_names>
        Names of system projects to be imported.
    dest = <destination_dir>
        Path at which the project is to be imported.
        Default location is current workspace.
```

# importsources

```
import_files:
    Import files to the platform.

Prototype:
    status = platform.import_files(from_loc = <Location of src or
                                     config file(s) path>,
                                   files = ["file1",...]**,
                                   dest_dir_in_cmp = <Dest location or
                                   config file(s) path>)

Required Arguments:
    from_loc = <Location of src or config file(s) path>
        From The location can be a directory/file, the path can be
        absolute/relative.

Optional Arguments:
    files = ["file1",...]**
        List of files to be imported from the given from_loc path.
        Whole folder is imported if files are not mentioned.
```

```
dest_dir_in_cmp = <Dest location or config file(s) path>  
    Destination folder name, created if directory doesn't exist.  
    Files are imported directly to the component folder if  
    destination folder is not given.
```

# library

```
library  
create_library_component:  
    Create a software library component.  
  
Behavior:  
· The command creates a static library when you select a bare-metal domain.  
· The command creates a dynamic library when you select a Linux domain (if  
  the installed release supports that flow).  
  
Domain selection determines the library type and generated artifact.
```

## 第六部分

## GNU 编译器工具

本部分包含以下章节：

- [概述](#)
- [编译器框架](#)
- [常用编译器用法和选项](#)
- [MicroBlaze 编译器的使用和选项](#)
- [Arm 编译器的使用和选项](#)
- [其他注释](#)

**注释：**从 2024.1 版开始，本用户指南支持 MicroBlaze™ 器件和 MicroBlaze™-V 器件。

# 概述

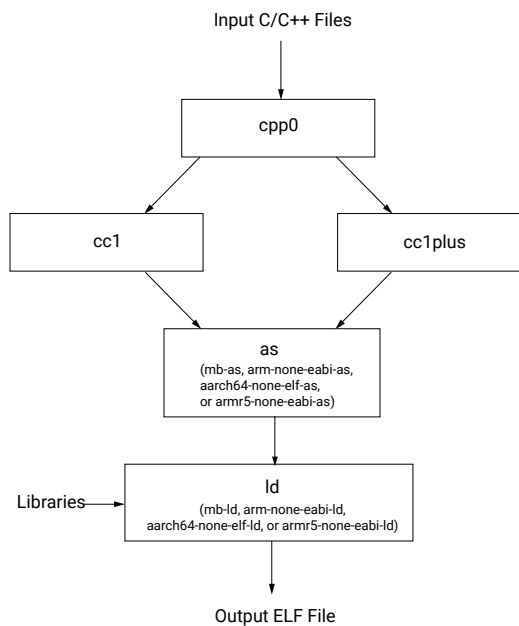
AMD Vivado™ Design Suite 包含 GNU 编译器集合 (GCC)，以供 MicroBlaze™ 和 MicroBlaze™-V 处理器以及 Arm® Cortex® A9、A53、A72 和 R5 处理器使用。

- Vivado GNU 工具支持 C 语言和 C++ 语言。
- MicroBlaze 和 MicroBlaze™-V GNU 工具包含 `mb-gcc` 编译器、`mb-g++` 编译器、`mb-as` 汇编程序和 `mb-ld` 连接器。
- Arm 处理器工具包括：
  - `arm-none-eabi-gcc` 编译器和 `arm-none-eabi-g++` 编译器
  - `arm-none-eabi-as` 汇编程序
  - 用于 Cortex A9 处理器的 `arm-none-eabi-ld` 连接器
  - 用于 Cortex-A53 和 Cortex-A72 处理器的 `aarch64-none-elf-*`
  - 用于 Cortex-R5F 处理器的 `armr5-none-eabi-*`
- 工具链还包含 C 语言库、数学库、GCC 库和 C++ 标准库。
- 编译器还使用常用二进制实用工具（称为 `binutils`），例如，汇编程序、连接器和对象转储。MicroBlaze 和 Arm 编译器工具使用基于 GNU 版本 2.31（在 2019.x 版本的源文件中）和基于 2.32 版本（在 2020.x 版本的源文件中）的 GNU `binutils`。

## 编译器框架

本节主要探讨 MicroBlaze™ 和 MicroBlaze™-V，以及 Cortex® A9、A53、A72 和 R5 处理器编译器的常用功能特性。下图显示了 GNU 工具流程。

图 70: GNU 工具流程



X13367-082021

GNU 编译器具有以下名称：

- MicroBlaze 和 MicroBlaze™-V: `mb-gcc`
- Cortex A9 核: `arm-none-eabi-gcc`
- Cortex-A53 和 Cortex-A72: `aarch64-none-elf-gcc`
- Cortex-R5F: `armr5-none-eabi-gcc`

GNU 编译器是封装器，可调用以下可执行文件：

- 预处理器 (`cpp0`)：这是编译器调用的首次直通。预处理器将所有宏都替换为源文件和头文件中的定义。
- 特定于机器和语言的编译器：此编译器用于处理第一阶段中输出并经过预处理的代码。特定于语言的编译器包括：
  - C 语言编译器 (`cc1`)：此编译器负责在输入 C 语言代码上完成的大部分优化操作，并负责生成汇编代码。
  - C++ 语言编译器 (`cc1plus`)：此编译器负责在输入 C++ 语言代码上完成的大部分优化操作，并负责生成汇编代码。

- 汇编程序：汇编代码在汇编语言中包含助记符。汇编程序执行如下操作：

- 将助记符转换为机器语言
- 对编译器生成的部分标签进行解析
- 创建对象文件，并将其传递到连接器。

汇编程序可执行文件命名方式如下：

- MicroBlaze 和 MicroBlaze™-V: `mb-as`
- Cortex A9 核: `arm-none-eabi-as`
- Cortex-A53 和 Cortex-A72: `aarch64-none-elf-as`
- Cortex-R5F: `armr5-none-eabi-as`
- 连接器：用于链接由汇编程序生成的所有对象文件。如果您在命令行中提供若干库，那么连接器可以通过链接汇编程序中的函数来解析部分未定义的引用。连接器可执行文件命名方式如下：

- MicroBlaze 和 MicroBlaze™-V: `mb-ld`
- Cortex A9 核: `arm-none-eabi-ld`
- Cortex-A53 和 Cortex-A72: `aarch64-none-elf-ld`
- Cortex-R5F: `armr5-none-eabi-ld`

如需获取每个可执行选项的详细信息，请参阅以下章节：

- [常用编译器选项：快捷参考](#)
- [连接器选项](#)
- [MicroBlaze 编译器的使用和选项](#)
- [MicroBlaze 连接器选项](#)
- [Arm 编译器的使用和选项](#)

**注释：**从此处开始，本章节中对 GCC 的引用均表示 MicroBlaze 和 MicroBlaze™-V 编译器 `mb-gcc`，对 G++ 的引用则表示 MicroBlaze C++ 编译器 `mb-g++`。

# 常用编译器用法和选项

## 用法

要使用 GNU 编译器，请输入以下命令：

```
<Compiler_Name> options files...
```

其中 <Compiler\_Name> 为以下编译器之一：

- MicroBlaze 和 MicroBlaze™-V: `mb-gcc`
- Cortex A9 核: `arm-none-eabi-gcc`
- Cortex-A53 和 Cortex-A72: `aarch64-none-elf-gcc`
- Cortex-R5F: `armr5-none-eabi-gcc`

要编译 C++ 程序，可使用 `mb-g++` 或 `arm-none-eabi-g++` 命令。

## 输入文件

编译器会取以下一个或多个文件作为输入：

- C 语言源文件
- C++ 语言源文件
- 汇编文件
- 对象文件
- 连接器脚本

**注释：** 这些文件均为可选。如果不指定这些文件，则使用连接器中嵌入的默认连接器脚本。默认脚本如下所述：

- MicroBlaze 和 MicroBlaze™-V: `mb-ld`
- Cortex A9 核: `arm-none-eabi-ld`
- Cortex-A53 和 Cortex-A72: `aarch64-none-elf-ld`
- Cortex-R5F: `armr5-none-eabi-ld`

在 [文件类型和扩展名](#) 中包含上述每一种类型的默认扩展名。除了上述文件外，编译器还会隐式引用库文件 `libc.a`、`libgcc.a`、`libm.a` 和 `libxil.a`。这些文件的默认位置为 Vivado 安装目录。使用 G++ 编译器时，还会引用 `libsupc++.a` 和 `libstdc++.a` 文件。这些文件分别对应的是 C++ 语言支持和 C++ 平台库。

# 输出文件

该编译器可生成以下文件作为输出：

- ELF 文件。
- 汇编文件。如果您使用 `-save-temps` 选项或 `-S` 选项，就会生成此文件。
- 对象文件。如果您使用 `-save-temps` 选项或 `-c` 选项，就会生成此文件。
- 采用 `.i` 文件或 `.ii` 文件形式的预处理器输出。如果您使用 `-save-temps` 选项，就会生成此文件。

# 文件类型和扩展名

GNU 编译器根据文件扩展名来决定文件类型。下表列出了有效扩展和对应文件类型。GCC 封装器会通过识别这些文件类型来调用相应的低级工具。

表 27：文件扩展名

| 扩展名  | 文件类型（方言）        |
|------|-----------------|
| .c   | C 语言文件          |
| .C   | C++ 文件          |
| .cXX | C++ 文件          |
| .cpp | C++ 文件          |
| .c++ | C++ 文件          |
| .cc  | C++ 文件          |
| .S   | 汇编文件，但可包含预处理器指令 |
| .s   | 不含预处理指令的汇编文件    |

# 库

下表列出了 `mb_gcc` 和 `arm-none-eabi-gcc` 编译器所需的库。

表 28：编译器使用的库

| 库        | 详细信息                                                       |
|----------|------------------------------------------------------------|
| libxil.a | 包含为 Vivado 工具开发的驱动程序、软件服务（例如，XilMFS）和初始化文件。                |
| libc.a   | 标准 C 语言库，包含 <code>strcmp</code> 和 <code>strlen</code> 等函数。 |

表 28：编译器使用的库 (续)

| 库           | 详细信息                                               |
|-------------|----------------------------------------------------|
| libgcc.a    | GCC 低级库，包含浮点和 64 位算术的硬件仿真例程。                       |
| libm.a      | 数学库，包含 cos 和 sine 等函数。                             |
| libsupc++.a | C++ 语言支持库，具有用于异常处理的例程、RTTI 等。                      |
| libstdc++.a | C++ 语言标准平台库。包含标准语言类，例如，用于串流 I/O、文件 I/O 和字符串处理的语言类。 |

这两种编译器都会自动链接各库。如果覆盖标准库，那么必须向编译器提供这些库的搜索路径。Vitis 会修改 `libxil.a` 文件，以添加驱动程序和库例程。

## 语言方言

GCC 编译器可识别 C 语言方言和 C++ 语言方言，并生成相应的代码。根据 GCC 规范，在源文件上 GCC 编译器或 G++ 编译器均可等效使用。根据您所使用的编译器以及源文件的扩展名来决定输入和输出文件上使用的方言。

使用 GCC 编译器时，程序的方言始终根据文件扩展名来决定，如 [文件类型和扩展名](#) 中所示。如果文件扩展名显示此文件为 C++ 源文件，则语言设为 C++。如果您在 `.c` 文件中编译 C 语言代码，那么即使使用 GCC 编译器，仍会重整函数名。

GCC 与 G++ 之间的主要差异在于 G++ 会将默认语言方言自动设置为 C++（与文件扩展名无关）。在链接期间，G++ 会自动拉取 C++ 支持库。这意味着即使您在 `.c` 文件中使用 G++ 编译器来编译 C 语言代码，G++ 仍会重整名称。

名称重整是 C++ 和支持符号重载的其他语言所特有的概念。当某个函数可以基于传入的实参执行不同操作并且可以返回不同的返回值时，将该函数称为重载函数。为支持此概念，C++ 编译器会在函数名称内对要调用的函数类型进行编码，以避免带有相同名称的函数包含多个定义。

如果您采用混合编译模式，请谨慎处理名称重整。当某些源文件包含 C 语言代码，而其他源文件包含 C++ 代码时，请务必谨慎处理。使用 GCC 编译某些文件、使用 G++ 编译其他文件时，务必注意。为防止 C 语言符号发生名称重整，请在符号声明中使用以下构造。

```
#ifdef __cplusplus
extern "C" {
#endif
int foo();
int morefoo();
#ifdef __cplusplus
}
#endif
```

请将这些声明包含在头文件中，并在所有源文件中使用。这样就会导致编译器在编译这些符号的定义或者引用时，使用 C 语言方言。

**注释：**所有 Vivado 驱动程序和库在其提供的所有头文件都遵循这些规范。使用 G++ 执行编译时，必须包含每个驱动程序和库中所记录的必要头文件。这样即可确保编译器能够将库符号识别为属于“C”类型的库符号。

使用编译器的任意变体执行编译时，可使用 `-x lang` 开关来强制文件使用某一特定方言。如需了解有关此开关的更多信息，请参阅 [GCC 文档](#)。

- 使用 GCC 编译器时，不会自动链入 `libstdc++.a` 和 `libsupc++.a`。

- 编译 C++ 程序时，请使用编译器的 G++ 变体。使用 G++ 可确保自动链接所有必需的支持库。
- 您还可以向 GCC 命令添加 `-lstdc++` 和 `-lsupc++` 选项。

如需了解有关如何为不同语言调用编译器的更多信息，请参阅 [GNU 在线文档](#)。

## 常用编译器选项：快捷参考

以下汇总信息列出了 MicroBlaze、MicroBlaze™-V 和 Arm 处理器的编译器常用的编译器选项。这些编译器选项区分大小写。

## 常规选项

- `-E`：仅执行预处理，不执行编译、汇编和链接。预处理后的输出会显示在标准输出器件上。
- `-S`：仅执行编译，不执行汇编和链接。生成 `.s` 文件。
- `-c`：仅执行编译和汇编；不执行链接。生成 `.o` 文件。
- `-g`：该选项用于向输出文件添加基于 DWARF2 的调试信息。GNU 调试器、`mb-gdb` 或 `arm-none-eabi-gdb` 都需要此信息。调试器可在源码级别和汇编级别提供调试。仅当输入为 C/C++ 源文件时，该选项才能添加调试信息。
- `-gstabs`：请将该选项用于在源码级别的汇编 (.S) 文件和汇编文件符号上添加基于 STABS 的调试信息。Vitis 为 GNU 汇编程序、`mb-as` 或 `arm-none-eabi-as` 提供此汇编程序选项。如果使用编译器 `mb-gcc` 或 `arm-none-eabi-gcc` 来对汇编文件进行编译，请给该选项添加 `-Wa` 前缀。
- `-On`：GNU 编译器可在不同级别提供优化。下表中的优化级别仅适用于 C 语言和 C++ 语言源文件。

表 29：适用于值 **n** 的优化

| n | 优化                |
|---|-------------------|
| 0 | 无优化。              |
| 1 | 中等优化。             |
| 2 | 完全优化              |
| 3 | 完全优化。尝试自动内联小型子程序。 |
| 5 | 针对大小进行优化。         |

**注释：**如果优化级别为 1 和更高级别，则会导致代码重排。调试代码时，请勿使用优化。通过 GDB 对优化后的程序进行调试时，结果可能不一致。

- `-v`：该选项采用详细模式执行编译器及其所有工具。该选项可以提供传递给所有工具的选项的完整描述。此描述可用于发现每个工具的默认选项。
- `-save-temps`：GNU 编译器提供一种机制，用于保存编译过程中生成的中间文件。编译器可存储以下文件：
  - 预处理器输出文件 `-input_file_name.i`（适用于 C 语言代码）和 `input_file_name.ii`（适用于 C++ 语言代码）
  - 编译器 (`cc1`) 输出文件，采用汇编格式 `-input_file_name.s`
  - 汇编程序输出文件，采用 ELF 格式 `-input_file_name.s`

编译器会将整个编译的默认输出保存为 `a.out`。

- `-o filename`：编译器会以 ELF 格式存储编译过程的默认输出，文件名为 `a.out`。您可以使用 `-o output_file_name` 更改默认名称。Vitis 会创建 ELF 格式的输出文件。
- `-Wp,<option>`、`-Wa,<option>` 和 `-Wl,<option>`：编译器 `mb-gcc` 或 `arm-none-eabi-gcc` 作为其他可执行文件的封装器使用，此类可执行文件包括预处理器、编译器 (`cc1`)、汇编程序和连接器等。您可以单独运行编译器的这些组件，或者也可以通过顶层编译器来运行。

有些选项对于某些工具是必需的，但对于顶层编译器则可能并不需要。要运行这些命令，请使用下表中的选项。

表 30：传递到顶层 GCC 编译器的工具专用选项

| 选项                              | 工具   | 示例                                                                                                                           |
|---------------------------------|------|------------------------------------------------------------------------------------------------------------------------------|
| <code>-Wp,&lt;option&gt;</code> | 预处理器 | <code>mb-gcc -Wp,-D -Wp, MYDEFINE ...</code><br><br>向预处理器发出信号，指示其使用 <code>-D MYDEFINE</code> 选项来定义 <code>MYDEFINE</code> 符号。 |
| <code>-Wa,&lt;option&gt;</code> | 汇编程序 | <code>mb-as -Wa, ...</code><br><br>向汇编程序发出信号，指示其将 MicroBlaze 和 MicroBlaze™ V 处理器定为目标。                                        |
| <code>-Wl,&lt;option&gt;</code> | 连接器  | <code>mb-gcc -Wl,-M ...</code><br><br>向连接器发出信号，指示其使用 <code>-M</code> 选项生成映射文件。                                               |

- `-help`：将该选项与任意 GNU 编译器搭配使用即可获取有关可用选项的更多信息。您也可以查询 GCC 手册。
- `-B directory`：向 C 语言运行时库搜索路径添加目录。
- `-L directory`：向库搜索路径添加目录。
- `-I directory`：向头文件搜索路径添加目录。
- `-l library`：搜索库中是否存在未定义的符号。

**注释：**编译器会向此命令行开关中所指示的库名称添加“`lib`”前缀。

## 库搜索选项

- `-l <library name>`：默认情况下，编译器仅搜索标准库，如 `libc`、`libm` 和 `libx11`。您也可以创建自己的库。您可以指定库的名称以及位置，以供编译器查找这些函数的定义。编译器会给您提供的库名称添加 `lib` 前缀。

编译器依赖于您提供选项的顺序，对于 `-l` 命令行开关尤其如此。请仅在命令行中的所有源代码之后提供此开关。

例如，如果您自行创建的库名为 `libproject.a`，您可使用以下命令来包含源自该库的函数：

```
Compiler Source_Files -L${LIBDIR} -l project
```



**重要提示！** 如果您在源文件之前提供库标志 `-l library_name`，那么编译器会找不到从任意源文件调用的函数。这是因为，编译器搜索仅限单向操作，并且不保留可用库的列表。

- `-L <lib_directory>`：该选项指定用于从中搜索各库的目录。编译器具有默认库搜索路径，用于从中查找标准库。您可使用 `-L` 选项在编译器搜索路径中包含其他目录。

## 头文件搜索选项

- `-I <directory_name>`：该选项用于先在 `/<dir_name>` 目录内搜索头文件，然后再搜索标准路径中的头文件。

## 默认搜索路径

编译器会搜索某些路径以查找库和头文件，这些编译器包括对应 MicroBlaze 和 MicroBlaze™-V 的 `mb-gcc`、对应 Cortex A9 的 `arm-none-eabi-gcc`、对应 Cortex-R5F 的 `armr5-none-eabi-gcc` 以及对应 Cortex-A53 Cortex®-A72 的 `aarch64-none-elf-gcc`。

### 库搜索过程

编译器按如下顺序搜索库：

1. 使用 `-L <dir_name>` 选项将目录传递给编译器。
2. 编译器会搜索以下库：

```
$XILINX_VITIS/gnu/<processor>/<platform>/<processor-lib>/usr/lib
```

### 头文件搜索过程

编译器按如下顺序搜索头文件：

1. 使用 `-I <dir_name>` 选项将目录传递给编译器。
2. 编译器会搜索以下头文件：

```
$XILINX_VITIS/gnu/<processor>/<platform>/<processor-lib>/usr/include
```

### 初始化文件搜索过程

编译器按如下顺序搜索初始化文件：

1. 使用 `-B <dir_name>` 选项将目录传递给编译器。
2. 编译器会搜索以下文件：

```
$XILINX_VITIS/gnu/<processor>/<platform>/<processor-lib>/usr/lib
```

其中：

- 对于 MicroBlaze 处理器和 MicroBlaze V 处理器，<processor> 为 microblaze，对于 A9，此项为 aarch32，对于 A53 为 aarch64，对于 R5 则为 armv5。
- 对于 MicroBlaze 和 MicroBlaze™ V，<processor-lib> 为 microblazeeb-xilinx-elf，对于 A9，此项为 aarch32-xilinx-eabi，对于 A53 为 aarch64-none/aarch64-xilinx-elf，对于 R5 处理器则为 gcc-arm-none-eabi/armv5-xilinx-eabi。

**注释：** platform 对于 64 位 Linux 指定 `lin64`，对于 Windows Cygwin 则指定 `nt`。

## 连接器选项

- `-defsym _STACK_SIZE=value`：此连接器选项用于修改栈的存储器分配总量。`_STACK_SIZE` 变量是分配的空间总量。`_STACK_SIZE` 变量默认值为 100 个字或 400 字节。如果您的程序栈与堆组合在一起所需字节数超过 400 字节，请增大 `_STACK_SIZE` 的值。该值以字节为单位。

在某些情况下，程序可能需要更大的栈。如果程序需要的栈大小比可用大小更大，就会导致代码执行出错。

**注释：** 对于与 AMD 提供的 C 语言运行时 (CRT) 相链接的程序，要求栈大小至少为 16 字节 (0x0010)。

- `-defsym _HEAP_SIZE=value`：以变量 `_HEAP_SIZE` 来为堆分配存储器总量。`_HEAP_SIZE` 默认值为 0。

动态存储器分配例会使用堆。如果您的程序以此方式来使用堆，那么您必须为 `_HEAP_SIZE` 提供合理的值。

对于高级用户：您可从 IP integrator 直接生成连接器脚本。

## 存储器布局

MicroBlaze、MicroBlaze™-V 以及 Arm Cortex A9 和 R5 处理器使用 32 位逻辑地址，可以对系统中范围在 0x0 到 0xFFFFFFFF 内的任意存储器进行寻址。预留存储器与 I/O 存储器共同构成此地址范围。Arm Cortex-A53 和 Cortex-A72 处理器使用 64 位逻辑地址。

### 预留存储器

硬件和软件编程环境定义了预留存储器，尤其是包含中断向量位置和操作系统级例程的存储器。下表列出了 MicroBlaze、MicroBlaze V 和 Arm 处理器的预留存储器位置（由处理器硬件定义）。如需了解有关这些存储器位置的更多信息，请参阅对应的处理器参考手册。

如需了解有关 Arm Cortex A9 存储器映射的信息，请参阅《Zynq 7000 SoC 技术参考手册》(UG585)。如需了解 Cortex-R5F、Cortex-A53 和 Cortex-A72 存储器映射的相关信息，请参阅《Zynq UltraScale+ 器件技术参考手册》(UG1085)。

**注释：** 除这些为硬件使用而预留的存储器外，软件环境还可以预留其他存储器。欲知详情，请参阅特定软件平台的使用手册。

表 31：硬件预留存储器位置

| 处理器家族                      | 预留存储器      | 预留用途                      | 默认文本起始地址 |
|----------------------------|------------|---------------------------|----------|
| MicroBlaze 和 MicroBlaze™-V | 0x0 - 0x4F | 复位向量、中断向量、异常向量和和其他预留向量位置。 | 0x50     |

## I/O 存储器

I/O 存储器即用于与处理器总线上的存储器映射外设进行通信的地址。这些地址是硬件平台规范的一部分。

## 用户和程序存储器

用户和程序存储器表示在可执行文件执行期间所需的所有存储器。

- 按照约定，这包括用于存储指令、只读数据、读写数据、程序栈和程序堆的存储器。
- 这些节可以存储在系统中的任意可寻址存储器中。
- 默认情况下，编译器会生成代码和数据，这些代码和数据从 [预留存储器](#) 的表中所列的地址开始，并占用连续的存储器位置。这是程序最常用的存储器布局。
- 您可通过在连接器中定义 `_TEXT_START_ADDR` 符号（用于 MicroBlaze 和 MicroBlaze V）和 `START_ADDR` 符号（用于 Arm）来修改程序的起始位置。

在特殊情况下，您可以使用连接器命令语言跨多个存储器对 ELF 文件的各部分进行分区。欲知详情，请参阅 [连接器脚本](#)。以下列出了可能需要更改可执行文件的存储器映射的各种情况。

- 跨多个较小的存储器对大型代码段进行分区时
- 将频繁执行的节重新映射到快速存储器
- 将只读区段映射到非易失性闪存

您可以任意方式对可执行文件进行分区，不受任何限制。分区可以在输出节级别完成，或者甚至可以在个别函数和数据级别完成。生成的 ELF 可以不连续，即在存储器映射中可能存在“孔隙”。请确保不使用已记录的预留位置。

如果您是高级用户，可以修改预留存储器位置的默认二进制数据。在此情况下，您必须替换默认启动文件和来自连接器的存储器映射。

## 对象文件节

您可以通过串联来自对象文件（.o 文件）的各输入节来创建可执行文件。默认情况下，编译器会跨标准节和明确定义的节创建代码。每个节都根据其关联的含义和用途来命名。请参阅下表中的对象文件的各标准节。

除了这些节外，您还可以创建自己的自定义节，并将其分配到您所选的存储器。

**注释：**通常您不会对下列保留的节进行修改：`.init`、`.fini`、`.ctors`、`.dtors`、`.got`、`.got2` 和 `.eh_frame`。

表 32：对象文件或可执行文件的各节布局

| 节                    | 描述          |
|----------------------|-------------|
| <code>.text</code>   | 文本节         |
| <code>.rodata</code> | 只读数据节       |
| <code>.sdata2</code> | 小型只读数据节     |
| <code>.sbss2</code>  | 小型只读未初始化数据节 |
| <code>.data</code>   | 读/写数据节      |
| <code>.sdata</code>  | 小型读/写数据节    |

表 32：对象文件或可执行文件的各节布局 (续)

| 节      | 描述        |
|--------|-----------|
| .sbss  | 小型未初始化数据节 |
| .bss   | 未初始化数据节   |
| .heap  | 程序堆存储器节   |
| .stack | 程序栈存储器节   |

- **.text**：对象文件的该节包含可执行程序指令。本节包含 **x**（可执行）、**r**（只读）和 **i**（已初始化）标志。您可将其分配到已初始化的只读存储器 (ROM)，该 ROM 可从处理器指令总线进行寻址。
- **.rodata**：本节包含只读数据。本节包含 **r**（只读）和 **i**（已初始化）标志。与 **.text** 节类似，您同样可将本节分配到已初始化的只读存储器，并且可从处理器数据总线对其进行寻址。
- **.sdata2**：本节与 **.rodata** 节类似。它包含大小小于 8 字节的小型只读数据。本节中的所有数据均可通过引用只读小型数据锚点来访问。这样即可确保本节的所有内容均可使用单一指令来访问。您可通过给编译器添加 **-G** 选项来更改进入本节的数据的大小。本节包含 **r**（只读）和 **i**（已初始化）标志。
- **.data**：本节包含读/写数据，并具有 **w**（读/写）标志和 **i**（已初始化）标志。您必须将其初始化并映射到随机存取存储器 (RAM)。不得将其映射到 ROM。
- **.sdata**：本节包含大小小于 8 字节的小型读/写数据。您可通过 **-G** 选项来更改进入本节的数据的大小。本节中的所有数据均可通过读写小型数据锚点来访问。这样即可确保本节的所有内容均可使用单一指令来访问。本节包含 **w**（读写）和 **i**（已初始化）标志。您必须将其映射到已初始化的 RAM。
- **.sbss2**：本节包含大小小于 8 字节且未初始化的小型只读数据。您可通过 **-G** 选项来更改进入本节的数据的大小。本节包含 **r**（读取）标志。您可将其映射到 ROM。
- **.sbss**：本节包含大小小于 8 字节且未初始化的小型数据。您可通过 **-G** 选项来更改进入本节的数据的大小。本节包含 **w**（读写）标志。您必须将其映射到 RAM。
- **.bss**：本节包含未初始化的数据。本节包含 **w**（读写）标志。您必须将其映射到 RAM。
- **.heap**：本节包含未初始化的数据，这些数据用作为全局程序堆。动态存储器分配例程会从本节分配存储器。您必须将本节映射到 RAM。
- **.stack**：本节包含未初始化的数据，这些数据用作为程序栈。您必须将本节映射到 RAM。本节通常紧随在 **.heap** 节之后。在某些版本的连接器中，**.stack** 节和 **.heap** 节可能显示为合并在一起，组成名为 **.bss\_stack** 的节。
- **.init**：本节包含语言初始化代码，其所含标志与 **.text** 相同。您必须将其映射到已初始化的 ROM。
- **.fini**：本节包含语言清理代码，其所含标志与 **.text** 相同。您必须将其映射到已初始化的 ROM。
- **.ctors**：本节包含在程序启动时必须调用的函数的列表，所含标志与 **.data** 相同。您必须将其映射到已初始化的 RAM。
- **.dtors**：本节包含在程序结束时必须调用的函数的列表，所含标志与 **.data** 相同。您必须将其映射到已初始化的 RAM。
- **.got2/.got**：本节包含指向程序数据的指针，所含标志与 **.data** 相同。您必须将其映射到已初始化的 RAM。
- **.eh\_frame**：本节包含帧展开信息，用于异常处理。其所含标志与 **.rodata** 相同。您可将其映射到已初始化的 ROM。
- **.tbss**：本节包含未初始化的线程本地数据，这些数据构成程序存储器镜像。本节所含标志与 **.bss** 相同。您必须将其映射到 RAM。

- `.tdata`：本节包含已初始化的线程本地数据，这些数据构成程序存储器镜像。您必须将其映射到已初始化的 RAM。
- `.gcc_except_table`：本节包含特定于语言的数据。您必须将本节映射到已初始化的 RAM。
- `.jcr`：本节包含注册已编译的 Java 类所需的信息。其中内容特定于编译器，供编译器初始化函数使用。您必须将本节映射到已初始化的 RAM。
- `.fixup`：本节包含执行修复所需的必要信息，例如，修复页表和修复记录表。您必须将本节映射到已初始化的 RAM。

## 连接器脚本

连接器实用工具使用连接器脚本中指定的命令，将程序分配到不同的存储器块中。它描述了所有输入对象文件中的所有节到可执行文件中的输出节之间的映射。该实用工具会将各节映射到系统中的存储器。如果您不想更改将程序内容连续分配到存储器的默认行为，则无需使用连接器脚本。连接器中提供了一个默认连接器脚本，用于对各节内容进行连续布局。

您可以选择只修改程序的起始地址。在 MicroBlaze 处理器和 MicroBlaze V 处理器上定义连接器符号 `_TEXT_START_ADDR`，或者在 Arm 处理器上定义 `START_ADDR`。请参阅以下代码作为示例：

```
mb-gcc <input files and flags> -Wl,-defsym _Wl,_TEXT_START_ADDR=0x100
mb-ld <.o files> -defsym _TEXT_START_ADDR=0x100
```

连接器使用来自 `$XILINX-/gnu///<processor_name>/lib/ldscripts` 区域的以下默认脚本，其中，`<procname>=microblaze`，`<processor_name>=microblaze`，`<platform>=lin` 或 `nt`。

- 当以下情况都不适用时，默认使用 `elf32<procname>.xo`
- 当使用 `-n` 选项调用连接器时，默认使用 `elf32<procname>.xno`
- 当使用 `-N` 选项调用连接器时，默认使用 `elf32<procname>.xbno`
- 当使用 `-r` 选项调用连接器时，默认使用 `elf32<procname>.xro`
- 当使用 `-Ur` 选项调用连接器时，默认使用 `elf32<procname>.xuo`

要使用连接器脚本，请在 GCC 命令行上提供该脚本。为编译器使用命令行选项 `-T <script>`。请参阅以下代码以获取详细信息：

```
compiler -T <linker_script> <Other Options and Input Files>
```

如果连接器自行执行，则请按如下方式包含连接器脚本。

```
linker -T <linker_script> <Other Options and Input Files>
```

这样即可告知 GCC 使用您的连接器脚本来代替默认内置连接器脚本。您可以右键单击应用组件并选择“Reset Link Script”（复位链接脚本）来为程序生成连接器脚本。

连接器脚本通过 C 语言代码中的各节属性来向特定存储器分配特定变量或函数。连接器脚本还可向存储器内的各节分配特定对象文件。要了解 GNU 连接器脚本的这些功能特性与其他特点，请参阅 [binutils 手册](#) 中的 GNU 连接器章节。要获取由 MicroBlaze 处理器和 MicroBlaze™-V 处理器连接器脚本所分配的输入节的具体列表，请参阅 [MicroBlaze 连接器脚本的各节](#)。

# MicroBlaze 编译器的使用和选项

MicroBlaze™ GNU 编译器衍生自标准 GNU 源文件，作为编译器的 AMD 移植版本。以下各小节介绍了 MicroBlaze 编译器特有的功能特性与选项。使用 MicroBlaze 编译器执行编译时，预处理器会自动提供 `__MICROBLAZE__` 定义。您可在任意条件代码中使用此定义。

## MicroBlaze 编译器

适用于 MicroBlaze 软核处理器的 `mb-gcc` 编译器引入了全新的选项，并对 GNU 编译器工具支持的某些选项进行了修改。本章呈现了新选项和经过修改的选项。

## 处理器特性选择选项

- `-mcpu=vX.YY.Z`：该选项会指示编译器为 MicroBlaze 硬件版本 `vX.YY.Z` 生成代码。要为给定处理器获取正确的代码，请将此开关搭配处理器的硬件版本一起使用。
  - mcpu 开关行为因版本而异，如下所述：
    - 低于 `v3.00.a` 的版本：使用 3 阶处理器流水线模式。不禁止将可能引发异常的指令移入延迟时隙。
    - `v3.00.a` 和 `v4.00.a`：使用 3 阶处理器流水线模型。禁止将可能引发异常的指令移入延迟时隙。
    - `v5.00.a` 及更高版本：使用 5 阶处理器流水线模型。不禁止将可能引发异常的指令移入延迟时隙。
- `-mlittle-endian/-mbig-endian`：使用这些选项为编译代码选择目标机器的字节序。
  - 这些选项会根据此开关来设置二进制对象文件的字节序。
  - GCC 驱动程序将开关传递给子工具（`as`、`cc1`、`cc1plus`、`ld`）以在子工具中设置对应的字节序。
  - 默认值为 `-mbig-endian`。
  - 您不能将混合字节序的对象文件链接在一起。
- `-mno-xl-soft-mul`：该选项允许将硬件乘法指令用于 32 位乘法。
  - MicroBlaze 处理器具有开启或关闭硬件乘法器资源使用的选项。
  - 在 MicroBlaze 处理器上启用硬件乘法器选项时，使用该选项。
  - 使用硬件乘法器可以改善应用性能。
  - 使用此开关时，编译器会自动定义 C 语言预处理器定义 `HAVE_HW_MUL`。这样您即可编写为硬件量身定制的 C 语言代码或汇编程序代码。

- `-mxl-multiply-high`：MicroBlaze 处理器具有启用指令以计算 32x32 位乘法的高 32 位的选项。
  - 该选项会告知编译器使用高位乘法指令。
  - 使用此开关时，编译器会自动定义 C 语言预处理器定义 `HAVE_HW_MUL_HIGH`。这样您即可编写为硬件量身定制的 C 语言代码或汇编程序代码。
- `-mno-xl-multiply-high`：该选项会告知编译器不得使用这些高位乘法指令。该选项为默认选项。
- `-mxl-soft-mul`：该选项告知编译器，在 MicroBlaze 处理器上不存在硬件乘法单元。每个 32 位乘法运算都会替换为对软件仿真 `routine__mulsi3` 的调用。该选项为默认选项。
- `-mno-xl-soft-div`：您可以在 MicroBlaze 中例化硬件除法单元。
  - 当存在除法单元时，该选项会告知编译器，在编译程序中可以使用硬件除法指令。
  - 如果程序包含大量除法运算，该选项可以改善程序性能。
  - 使用此开关时，编译器会自动定义 C 语言预处理器定义 `HAVE_HW_DIV`。这样您即可编写为硬件量身定制的 C 语言代码或汇编程序代码。
- `-mxl-soft-div`：该选项告知编译器，在目标 MicroBlaze 硬件上不存在硬件除法单元。  
该选项为默认选项。编译器会将所有 32 位除法替换为调用对应的软件仿真例程（`__divsi3`、`__udivsi3`）。
- `-mxl-barrel-shift`：使用该选项来配置 MicroBlaze 处理器的桶式移位功能。
  - 默认选项假定不存在任何桶式移位器，并且编译器使用加法和乘法运算来执行操作数移位。
  - 启用桶式移位可以显著加速应用，使用浮点库时尤其如此。
  - 使用此开关时，编译器会自动定义 C 语言预处理器定义 `HAVE_HW_BSHIFT`。这样您即可编写为硬件量身定制的 C 语言代码或汇编程序代码。
- `-mno-xl-barrel-shift`：该选项告知编译器不使用硬件桶式移位指令。该选项为默认选项。
- `-mxl-pattern-compare`：该选项用于激活在编译器内使用模式比较指令的功能。
  - 使用模式比较指令可以在程序内加速布尔值运算。
  - 模式比较操作还允许在字符串处理例程上对字节长度数据进行操作，而不是对字节长度数据进行操作。示例包括：`strcpy`、`strlen` 和 `strcmp`。
  - 在极为依赖字符串处理例程的程序上，能够显著提升速度。
  - 使用此脚本时，编译器会自动定义 C 语言预处理器定义 `HAVE_HW_PCMP`。这样您即可编写为硬件量身定制的 C 语言代码或汇编程序代码。
- `-mno-xl-pattern-compare`：该选项会告知编译器不使用模式比较指令。这是默认方式。
- `-mhard-float`：该选项会在编译器内开启单精度浮点指令（`fadd`、`frsub`、`fmul` 和 `fdiv`）的使用。
  - 该选项还使用 `fcmp.p` 指令，其中 `p` 是谓词条件，例如，`le`、`ge`、`lt`、`gt`、`eq` 或 `ne`。
  - 当在硬件中启用 FPU 时，这些指令由 MicroBlaze 在本地进行解码和执行。
  - 使用此脚本时，编译器会自动定义 C 语言预处理器定义 `HAVE_HW_FPU`。这样您即可编写为硬件量身定制的 C 语言代码或汇编程序代码。
- `-msoft-float`：该选项会告知编译器，使用软件仿真来执行浮点算术。该选项为默认选项。
- `-mxl-float-convert`：该选项可以在编译器中开启单精度浮点转换指令（`fint` 和 `flt`）的使用。当在硬件中启用 FPU 并启用这些指令时，MicroBlaze 能原生解码和执行这些指令。

- `-mxl-float-sqrt`：该选项会在编译器内开启单精度浮点平方根指令 (`fsqrt`) 的使用。当在硬件中启用 FPU 并启用这些指令时，MicroBlaze 能原生解码和执行这些指令。

## 常规程序选项

- `-msmall-divides`：如果不存在硬件除法器，那么该选项可生成专为小型除法优化的代码。对于分子和分母介于 0 到 15（含）之间的有符号整数除法，此开关可以提供基于表格查找的超快除法。启用硬件除法器后，此开关不起作用。
- `-mxl-gp-opt`：如果您的程序包含地址，并且地址中最高有效位部分（上 16 位）中有非零位，那么加载和存储操作需要 2 条指令。

MicroBlaze 处理器 ABI 可以提供 2 个全局小型数据区域，每个区域可以包含最多 64 KB 的数据。您可以使用小型数据区域锚点与 16 位即时值来访问这些区域内的任何存储器位置。这意味着只需一条指令即可完成对小型数据区域的加载或存储。此优化可使用 `-mxl-gp-opt` 命令行参数开启。您可以在这些区域中存储小于阈值的变量。您可以用更少的指令来对这些变量进行寻址。Vitis 会在链接阶段计算地址。



**重要提示！** 如果使用该选项，请将其提供给程序构建进程的编译命令和链接命令。未一致使用此开关可能导致编译、链接或运行时错误。

- `-mno-clearbss`：该选项可用于对仿真中使用的程序进行编译。

根据 C 语言标准，未初始化的全局变量会在 `.bss` 节中进行分配。程序开始执行时，变量的值为 0。在程序开始执行时，C 语言启动文件会运行循环，以 0 值填充 `.bss` 节。对于 C 语言代码中值为 0 的全局变量，优化编译器也会将这些变量分配到 `.bss` 节。

在仿真环境中，以上两种语言特性可能都属于并非必要的开销。部分仿真器会将整个存储器自动归零。即使在正常环境中，您也可以编写不依赖于初始为 0 的全局变量的 C 语言代码。此开关可用于这些场景。它会导致 C 语言启动文件不以 0 值来初始化 `.bss` 节。它还会在内部强制禁止编译器在 `.bss` 节内分配初始为 0 值的全局变量，而是改为将其移至 `.data` 节。该选项可以改善您的应用的启动时间。请谨慎使用该选项，确保满足以下条件之一：

- 不使用依赖于初始化为 0 的全局变量的代码。
- 仿真平台会执行存储器清零。
- `-mxl-stack-check`：通过该选项，您可以检查程序运行时是否发生栈溢出。

编译器会在每个函数的序言中插入代码，并将栈指针值与可用存储器进行对比。如果栈指针超出可用内存，那么程序会跳转到子例程 `_stack_overflow_exit`。该子例程会将 `variable _stack_overflow_error` 的值设置为 1。

您可以通过在源代码中提供 `_stack_overflow_exit` 函数充当栈上溢处理程序，以覆盖标准栈溢出处理程序。

## 应用执行模式

- `-xl-mode-executable`：这是用于通过 `mb-gcc` 编译程序的默认模式。`mb-gcc` 的命令行无需使用此模式。它使用的是启动文件 `crt0.o`。
- `xl-mode-bootstrap`：该选项用于启动加载程序所加载的应用程序。通常，启动加载程序驻留在非易失性存储器中，该存储器映射到处理器复位向量。如果该启动加载程序加载正常的可执行文件，那么应用程序复位向量会覆盖这些复位向量。在此情况下发生处理器复位时，不会首先执行启动加载程序来重新加载该应用程序。

为避免出现此情况，请使用该编译器标志来对启动加载程序所加载的应用进行编译。这样在发生处理器复位时，控制权就会交给启动加载程序而不是应用程序。

此模式使用 `crt2.o` 作为启动文件。对于部署场景不同于上述场景的应用程序，请勿使用此开关。

- `-xl-mode-novectors`：该选项用于无需任何 MicroBlaze 向量的应用。这通常指的是独立应用，此类应用不使用任何处理器复位、中断或异常功能。由于使用此开关会消除向量的指令，因此可以减少代码。此模式使用 `crt3.o` 作为启动文件。



**重要提示！** 请勿在命令行上使用多种执行模式。

## 位置无关代码

MicroBlaze 的 GNU 编译器支持 `-fPIC` 和 `-fpic` 开关。

- 这些开关支持在编译器内生成位置无关代码 (PIC)。
- Linux 仅将此功能特性用于 MicroBlaze，以实现共享库和可重定位的可执行文件。
- 该方案使用全局偏移表 (GOT) 来重新定位生成的代码中的所有数据访问。
- 它还使用过程链接表 (PLT) 来调用共享库中的函数。
- 这是基于 GNU 的平台中的标准约定，用于生成可重定位代码和基于共享库进行动态链接。

## MicroBlaze 应用二进制接口

MicroBlaze 的 GNU 编译器使用《MicroBlaze 处理器参考指南》(UG081) 中定义的应用二进制接口 (ABI)。请参阅 ABI 文档以了解寄存器和栈使用规范以及编译器所使用的标准存储器模型描述。

## MicroBlaze 汇编程序

适用于 MicroBlaze 软核处理器的 `mb-as` 汇编程序支持的选项集与标准 GNU 编译器工具支持的选项集相同。它支持的汇编程序指令集与标准 GNU 汇编程序支持的指令集也相同。

`mb-as` 汇编程序支持 MicroBlaze 机器指令集 (`imm` 指令除外) 中的所有操作码。当使用较大的即时值时，`mb-as` 汇编程序会生成 `imm` 指令。您无需使用 `imm` 指令来编写代码。如需了解有关 MicroBlaze 指令集的更多信息，请参阅《MicroBlaze 处理器参考指南》(UG081)。

`mb-as` 汇编程序要求具有即时操作数的所有 MicroBlaze 指令都将该即时操作数指定为常量或标签。如果该指令需要 PC 相关操作数，那么 `mb-as` 汇编程序会按需计算该操作数并包含 `imm` 指令。

例如，如果 `branch immediate if equal (beqi)` 指令需要 PC 相关操作数，请使用以下指令。

```
beqi r3, mytargetlabel
```

其中，`mytargetlabel` 是目标指令的标签。`mb-as` 汇编程序会按 `mytargetlabel - PC` 来计算指令的即时值。

如果此即时值大于 16 位，那么 `mb-as` 汇编程序会自动插入一条 `imm` 指令。如果在编译时 `mytargetlabel` 的值未知，那么 `mb-as` 汇编程序始终会插入一条 `imm` 指令。连接器的 `relax` 选项可用于移除任何不必要的 `imm` 指令。

同样，如果指令需要较大的常量作为操作数，那么您需要按原样使用此操作数，即，不使用 `imm` 指令。例如，以下代码会向寄存器 `r3` 的内容添加常量 200,000，并将结果存储在寄存器 `r4` 中：

```
addi r4, r3, 200000
```

`mb-as` 汇编程序可以识别出此操作数需要一条 `imm` 指令，并自动插入该指令。

除了标准 MicroBlaze 指令集外，`mb-as` 汇编程序还支持某些伪操作码，以简化汇编编程任务。下表列出了受支持的伪操作码。

表 33：GNU 汇编程序支持的伪操作码

| 伪操作码                        | 解释                                                        |
|-----------------------------|-----------------------------------------------------------|
| <code>nop</code>            | 无操作。替换为以下指令： <code>or R0, R0, R0</code>                   |
| <code>la Rd, Ra, Imm</code> | 替换为以下指令： <code>addik Rd, Ra, imm; = Rd = Ra + Imm;</code> |
| <code>not Rd, Ra</code>     | 替换为以下指令： <code>xori Rd, Ra, -1</code>                     |
| <code>neg Rd, Ra</code>     | 替换为以下指令： <code>rsub Rd, Ra, R0</code>                     |
| <code>sub Rd, Ra, Rb</code> | 替换为以下指令： <code>rsub Rd, Rb, Ra</code>                     |

## MicroBlaze 连接器选项

适用于 MicroBlaze 软核处理器的 `mb-ld` 连接器可以在 GNU 编译器工具支持的选项的基础上提供额外选项。

- `-defsym _TEXT_START_ADDR=value`：默认情况下，输出代码的文本部分以基地址 `0x28` 开头。使用 `-defsym _TEXT_START_ADDR` 选项可覆盖此设置。如果将该选项提供给 `mb-gcc` 编译器，那么输出代码的文本部分将从给定的值开始。

如果您当前使用编译器设置的默认起始地址，则不必使用 `-defsym _TEXT_START_ADDR`。

这是连接器选项，在单独调用连接器时使用。如果在 `mb-gcc` 流程中调用连接器，则必须使用以下选项：

```
-Wl,-defsym -Wl,_TEXT_START_ADDR=value
```

- `-relax`：这是连接器选项，用于移除由汇编程序生成的所有不需要的 `imm` 指令。汇编程序在无法决定立即要执行的下一条指令的值时，会为每一条指令生成 `imm` 指令。其中大部分指令并不需要 `imm` 指令。提供 `-relax` 命令行选项时，连接器即可移除这些指令。

汇编程序只有在自行调用连接器时才需要该选项。通过 `mb-gcc` 编译器调用连接器时，系统会自动提供该选项。

- `-N`：该选项是 MicroBlaze 程序所必需的，用于将文本节和数据节设置为可读和可写。它不会对数据段执行页对齐。顶层 GCC 编译器在调用连接器时会自动包含该选项。如果您打算在不使用 GCC 的情况下调用连接器，请使用此选项。

MicroBlaze 连接器使用连接器脚本来向存储器分配各节。请参阅下文以获取详细信息。

## MicroBlaze 连接器脚本的各节

下表列出了 MicroBlaze 连接器脚本分配的各输入节。

表 34：各节的名称和描述

| 节                     | 描述                               |
|-----------------------|----------------------------------|
| .vectors.reset        | 复位向量代码。                          |
| .vectors.sw_exception | 软件异常向量代码。                        |
| .vectors.interrupt    | 硬件中断向量代码。                        |
| .vectors.hw_exception | 硬件异常向量代码。                        |
| .text                 | 来自函数和全局汇编语句中的代码的程序指令。            |
| .rodata               | 只读变量。                            |
| .sdata2               | 含初始值的小型只读静态变量和全局变量。              |
| .data                 | 含初始值的静态变量和全局变量。由启动代码初始化为 0。      |
| .sdata                | 含初始值的小型静态变量和全局变量。                |
| .sbss2                | 不含初始值的小型只读静态变量和全局变量。由启动代码初始化为 0。 |
| .sbss                 | 不含初始值的小型静态变量和全局变量。由启动代码初始化为 0。   |
| .bss                  | 不含初始值的静态变量和全局变量。由启动代码初始化为 0。     |
| .heap                 | 为堆定义的存储器节。                       |
| .stack                | 为栈定义的存储器节。                       |

## 编写或自定义连接器脚本的技巧

当您自行编写或自定义连接器脚本时，请谨记以下要点：

- 请确保按 MicroBlaze 硬件中定义的方式将不同向量节分配到相应的存储器中。
- 在 .bss 节中，为堆和栈分配空间。将 `_stack` 变量设置为位于此区域的 `_STACK_SIZE` 位置之后的位置，并将 `_heap_start` 变量设置为位于 `_STACK_SIZE` 位置之后的下一个位置。由于无需为硬件和仿真初始化堆和栈，请先确定 .bss 和 COMMON 的定义，再定义 `_bss_end` 变量。

**注释：** .bss 节边界不包含堆或栈。

- 请确保 `_SDATA_START__`、`_SDATA_END__`、`SDATA2_START`、`_SDATA2_END__`、`_SBSS2_START__`、`_SBSS2_END__`、`_bss_start`、`_bss_end`、`_sbss_start` 和 `_sbss_end` 变量分别定义为 `sdata`、`sdata2`、`sbss2`、`bss` 和 `sbss` 节的开始和结束。
- ANSI C 要求您在启动时初始化所有未初始化的存储器。对于堆栈则无此要求。提供的标准 CRT 假定仅有一个 .bss 节，并且此节初始化为 0。如有多个 .bss 节，则此 CRT 无效。您需自行编写 CRT 以初始化所有 .bss 节。

## 启动文件

编译器在生成可执行文件时，会在最终链接命令中包含预编译的启动文件和结束文件。启动文件用于在执行应用代码之前，设置语言 and 平台环境。启动文件通常用于执行以下操作：

- 按需设置任意复位向量、中断向量和异常向量。
- 设置栈指针、小型数据锚点和其他寄存器。请参阅下表以获取详细信息。
- 将 BSS 存储器区域清零。
- 调用语言初始化函数，例如，C++ 语言构造函数。
- 对硬件子系统进行初始化。例如，要剖析程序，请初始化剖析定时器。
- 为 main 过程设置实参，并调用此过程。

编译器会使用结束文件来包含程序结束后必须执行的代码。结束文件通常会执行以下操作：

- 调用语言清理函数，例如，C++ 语言析构函数。
- 对硬件子系统进行反初始化。例如，剖析程序期间，清理剖析子系统。

下表列出了 C 语言运行时文件中的寄存器名称、值和描述。

表 35：C 语言运行时文件中的寄存器初始化

| 寄存器   | 值          | 描述                                                |
|-------|------------|---------------------------------------------------|
| r1    | _stack-16  | 栈指针寄存器经初始化后，指向栈区域底部，且初始负偏移为 16 字节。这 16 字节可用于传入实参。 |
| r2    | _SDA2_BASE | _SDA2_BASE_ 是只读小型数据锚点地址。                          |
| r13   | _SDA_BASE_ | _SDA_BASE_ 是读写小型数据锚点地址。                           |
| 其他寄存器 | 未定义        | 其他寄存器不含已定义的值。                                     |

以下小节旨在描述用于各种应用模式的初始化文件。此信息面向想要更改或者了解其应用的启动代码的高级用户。

对于 MicroBlaze，C 语言运行时初始化分为 2 个不同阶段。第一个阶段主要负责设置向量，随后它将调用第二个阶段初始化。它还基于不同应用模式提供出口存根。

### 第一阶段初始化文件

- `crt0.o`：通过使用此初始化文件，各程序无需使用任何启动加载程序或调试存根，即可在独立模式下执行。此文件会填充复位、中断、异常和硬件异常向量，并调用第二阶段启动例程 `_crtinit`。从 `_crtinit` 返回时，它会通过在 `_exit` 标签内无限循环，以结束该程序。
- `crt1.o`：通过使用此初始化文件，即可以软件侵入式方式对应用进行调试。它会填充除断点和复位向量外的所有向量，并将控制转交给第二阶段 `_crtinit` 启动例程。
- `crt2.o`：通过使用此初始化文件，即可使用启动加载程序来加载可执行文件。它会填充除复位向量外的所有向量，并将控制转交给第二阶段 `_crtinit` 启动例程。从 `_crtinit` 返回时，它会通过在 `_exit` 标签处无限循环，以结束该程序。由于不填充复位向量，因此发生处理器复位时，会将控制转交给启动加载程序。启动加载程序即可重新加载并重新启动该程序。

- `crt3.o`：当可执行文件不使用任何向量并且希望减少代码量时，即可采用此初始化文件。它仅填充复位向量，并将控制转交给第二阶段 `_crtinit` 启动例程。从 `_crtinit` 返回时，它会通过在 `_exit` 标签处无限循环，以结束该程序。由于不填充其他向量，GNU 链接机制不会拉入任意中断和异常处理相关的例程。这有助于节省代码空间。

## 第二阶段初始化文件

根据 C 语言标准规范，所有全局变量和静态变量都必须初始化为 0。

- 初始化所有全局与静态变量是上述所有 CRT 的共同要求。
- 独立例程 `_crtinit` 会对程序的 `.bss` 部分中的存储器进行初始化。`_crtinit` 例程也是用于调用 `main` 过程的封装器。
- 调用 `main` 过程前，它可能调用其他初始化函数。`_crtinit` 例程由如下所述启动文件来提供。
- `crtinit.o`：

这是默认第二阶段 C 语言启动文件，用于执行以下步骤：

1. 将 `.bss` 部分清零。
2. 调用 `_program_init`。
3. 调用构造函数 (`_init`)。
4. 为 `main` 设置实参，并调用 `main`。
5. 调用析构函数 (`_fini`)。
6. 调用 `_program_clean` 并返回。

- `pgcrtinit.o`：剖析进程会使用第二阶段启动文件。此文件执行的步骤如下：

1. 将 `.bss` 部分清零。
2. 调用 `_program_init`。
3. 调用 `_profile_init` 以初始化剖析库。
4. 调用构造函数 (`_init`)。
5. 为 `main` 设置实参，并调用 `main`。
6. 调用析构函数 (`_fini`)。
7. 调用 `_profile_clean` 以清理剖析库。
8. 调用 `_program_clean`，然后返回。

- `sim-crtinit.o`：当编译器使用 `-mno-clearbss` 开关时，会使用此第二阶段启动文件。此文件执行的步骤如下：

1. 调用 `_program_init`。
2. 调用构造函数 (`_init`)。
3. 为 `main` 设置实参，并调用 `main`。
4. 调用析构函数 (`_fini`)。
5. 调用 `_program_clean`，然后返回。

- `sim-pgcrtinit.o`：使用 `-mno-clearbss` 开关进行剖析期间，此第二阶段启动文件会按顺序执行以下步骤：

1. 调用 `_program_init`。
2. 调用 `_profile_init` 以初始化剖析库。
3. 调用构造函数 (`_init`)。
4. 为 `main` 设置实参，并调用 `main`。
5. 调用析构函数 (`_fini`)。
6. 调用 `_profile_clean` 以清理剖析库。
7. 调用 `_program_clean`，然后返回。

## 其他文件

编译器还会使用某些标准开始文件和结束文件来提供 C++ 语言支持。

这些文件包括 `crti.o`、`crtbegin.o`、`crtend.o` 和 `crtfn.o`。这些文件均为标准编译器文件，可为 `.init`、`.fini`、`.ctors` 和 `.dtors` 部分提供内容。

## 修改启动文件

Vivado 会以预编译形式和源文件形式来分发初始化文件。

预编译的对象文件位于编译器库目录内。

MicroBlaze GNU 编译器的初始化文件的源文件位于 `<AMD_>/Vitis/<version>/data/embeddedsw/lib/microblaze/src/` 内。

`<AMD_>` 是 Vivado 安装路径，`<version>` 是 Vitis 软件平台的发布版本。

要满足自定义启动文件要求，请从源码区域中提取这些文件，并将其包含在您的应用源文件内。或者，您也可以将这些文件汇编到 `.o` 文件中，并将其置于公用区域内。要引用新创建的对象文件而不是标准文件，请在调用 `mb-gcc` 时，使用 `-B directory -name` 命令行选项。

为避免使用默认启动文件，请在最终编译行上使用 `-nostartfiles`。

**注释：**源码不包含杂项编译器标准 CRT 文件，如 `crti.o` 和 `crtbegin.o`。在安装中，这些文件可按原样直接使用。请将其引入您的最终链接命令。

## 减少 C 语言程序的启动代码

如果应用对于 C 语言程序的代码量具有严格的要求，那么建议删除所有开销的来源。下文描述了如何在 C 语言程序中减少调用不必要的 C++ 构造函数或析构函数代码的开销。通过如下修改可节省约 220 字节的代码空间：

1. 请按先前章节中所述方式，遵循从安装区域创建启动文件的自定义副本的说明进行操作。具体来说，就是复制适合您的应用的特定版本的 `crtfn.s` 和 `xcertinit.s`。如果在应用上进行自举和剖析，请从安装区域复制 `crt2.s` 和 `pg-crtinit.s`。
2. 修改 `pg-crtinit.s`，移除以下行：

```
brlid r15, __init
/* Invoke language initialization functions */
nop
```

和

```
brlid r15, __fini
/* Invoke language cleanup functions */
nop
```

这样即可避免引用通常进行构造函数和析构函数处理时需要拉取的额外代码，从而减少代码量。

- 3. 将这些文件编译为 .o 文件，并将其置于您所选的目录下。您也可以将这些文件包含在自己的应用程序源代码内。
- 4. 将 -nostartfiles 开关添加到编译器中。如果您选择在特定文件夹内对文件进行汇编，则请添加 -B 目录开关。
- 5. 编译您的应用。

如果您的应用以不同方式来执行，那么您必须基于 [启动文件](#) 中的描述来选取相应的 CRT 文件。

# 编译器库

mb-gcc 编译器需要使用 GNU C 标准库和 GNU 数学库。Vivado 随附有这些库的预编译版本。MicroBlaze 的 CPU 驱动程序会根据 MicroBlaze 的硬件配置来复制正确版本的预编译库。如果您需要手动选择要使用的库版本，请打开以下文件夹：

```
$XILINX_/gnu/microblaze/<platform>/microblaze-xilinx-elf/lib
```

用于库编译的编译器标志与配置决定了文件名编码。例如，libc\_m\_bs.a 表示使用编译器中启用的硬件乘法器和桶式移位器进行编译的 C 语言库。

下表显示了当前所使用的编码以及此编码所指定的库配置。

表 36：编译器标志所对应的编码库文件名

| 编码  | 描述          |
|-----|-------------|
| _bs | 专为桶式移位器而配置。 |
| _m  | 专为硬件乘法器而配置。 |
| _p  | 专为模式比较器而配置。 |

特别值得注意的是数学库文件 (libm\*.a)。C 语言标准要求常用的数学库函数（例如，sin() 和 cos()）使用双精度浮点算术。双精度浮点可能无法充分利用 MicroBlaze 上的可选单精度浮点功能。

newlib 数学库具有替代版本，可使用单精度算术来实现这些数学函数。这些单精度库可能可直接使用 MicroBlaze 处理器硬件浮点单元 (FPU)，因而性能更好。

如果您的应用无需使用标准精度，并且您想要提升性能，那么可以手动更改链入的库的版本。

默认情况下，CPU 驱动程序会将库的双精度版本 (libm\*\_fpd.a) 复制到您的 IP integrator 工程中。

要获取单精度版本，您可以创建自定义 CPU 驱动程序，以便改为复制对应的 libm\*\_fps.a 库。请将对应的 libm\*\_fps.a 文件作为 libm.a 复制到您的处理器库文件夹（例如，microblaze\_0/lib 中）。

复制完要使用的库之后，请重新构建应用软件工程。

## 线程安全性

MicroBlaze 处理器的 C 语言库和 Vivado 的 C 语言数学运算库不支持多线程环境。常用 C 语言库函数（如 `printf()`、`scanf()`、`malloc()` 和 `free()`）并不具有线程安全性，在运行时会在系统中导致不可恢复的错误。在多线程环境内使用 Vivado 库时，请使用适当的互斥机制。

## 命令行实参

MicroBlaze 处理器程序不接受命令行实参。C 语言运行时例程会初始化命令行实参 `argc` 和 `argv`。

## 中断处理程序

中断处理程序使用的编译方法不同于常规子例程调用。除了保存非易失性寄存器外，中断处理程序还必须保存所使用的易失性寄存器。发生中断时，中断处理程序还会存储机器状态寄存器 (RMSR) 的值。

- `interrupt_handler`：为了区分中断处理程序与子例程，`mb-gcc` 会在代码声明中查找属性 (`interrupt_handler`)。以下是该属性的定义：

```
void function_name () __attribute__((interrupt_handler));
```

**注释：**中断处理程序的属性应仅存在于原型中，不得出现在定义中。

中断处理程序还可调用其他函数，这些函数可使用易失性寄存器。为了在易失性寄存器中保留正确的值，如果中断处理程序是非叶函数，那么它会保存所有易失性寄存器。

**注释：**非叶函数是指能调用其他子例程的函数。

中断处理程序在微处理器软件规范 (MSS) 文件内进行定义。这些定义会将属性自动添加到中断处理程序函数中。中断处理程序使用 `rtid` 指令返回至已中断的函数。

- `save_volatiles`：MicroBlaze 编译器提供了 `save_volatiles` 属性，该属性与 `interrupt_handler` 属性类似，但使用 `rtid` 代替 `rtid` 来执行返回。该属性会为非叶函数保存所有易失性寄存器，对于叶函数，则仅保存已使用的易失性寄存器。

```
void function_name () __attribute__((save_volatiles));
```

- `fast_interrupt`：MicroBlaze 编译器可提供 `fast_interrupt` 属性，该属性与 `interrupt_handler` 属性类似。出现快速中断时，MicroBlaze 会跳至中断例程地址，而不是跳至固定地址 `0x10`。

不同于普通中断，C 语言函数使用 `fast_interrupt` 属性时，MicroBlaze 仅保存最少量的寄存器。

```
void function_name () __attribute__((fast_interrupt));
```

表 37：属性的使用

| 属性                | 函数                                                                                       |
|-------------------|------------------------------------------------------------------------------------------|
| interrupt_handler | 该属性不仅保存非易失性寄存器，还会保存机器状态寄存器和所有易失性寄存器。rtid 是从中断处理程序返回的。如果中断处理程序函数是叶函数，那么该属性只保存函数使用的易失性寄存器。 |
| save_volatiles    | 该属性与 interrupt_handler 类似，但它使用 rtsd 代替 rtid 返回到被中断的函数。                                   |
| fast_interrupt    | 该属性与 interrupt_handler 类似，但它会直接跳至中断例程地址，而不是跳至固定地址 0x10。                                  |

# Arm 编译器的使用和选项

1. Arm®Cortex®-A9 目标可使用 arm-none-eabi 工具链来编译
2. Arm®Cortex-A53 目标可使用 aarch64-none-elf 工具链来编译
3. Arm®Cortex-R5F 目标可使用 armr5-none-eabi 工具链来编译

Arm® Cortex® A9 目标可使用 arm-none-eabi 工具链来编译。arm-none-eabi 工具链包含完整的 GNU 工具链，包括以下所有组件：

- 常用启动代码序列
- GNU 二进制实用工具 (binutils)
- GNU C 语言编译器 (GCC)
- GNU C++ 语言编译器 (G++)
- GNU C++ 语言运行时库 (libstdc++)
- GNU 调试器 (GDB)
- Newlib C 语言库

---

## 用法

**注释：**Cortex®-R5F 和 A53 工具链的使用方式较为相似。

### 编译

```
arm-none-eabi-gcc -c file1.c -I<include_path> -o file1.o
arm-none-eabi-gcc -c file2.c -I<include_path> -o file2.o
```

### 链接

```
arm-none-eabi-gcc -Wl,-T -Wl,ldscript.ld -L<libxil.a path> -o
"App.elf" file1.o file2.o -Wl,--start-group,-lxil,-lgcc,-lc,--end-group
```

如需获取上述命令中所用标志的描述，请使用以下命令参阅编译器帮助信息：

- --help
- -v --help
- --target-help

### 编译器选项

在 [GNU 网站](#) 上可找到可使用 Arm® 相关标志来应用的其他 GNU 编译器选项。您可以在上述步骤中使用这些标志。以上链接中列出了所有 Arm® GCC 编译器选项。但实际支持取决于所用的目标（在此例中，即 Arm Cortex-A9、Cortex-A53, Cortex®-A72 和 Cortex-R5F）和编译器工具链。

## 其他注释

---

### C++ 语言代码大小

GCC 工具链与最新开源 C++ 标准库 (`libstdc++-v3`) 相结合所生成的代码和数据片段比等效的 C 语言程序所生成的代码和数据片段要大得多。此开销中大部分来自于异常处理和运行时类型信息的代码和数据。部分 C++ 应用不需要这些特性。

高级用户可使用 `-fno-exceptions` 和/或 `-fno-rtti` 开关来移除开销并优化大小。请参阅 [GCC 文档](#) 以了解有关可用编译器选项及其影响的具体信息。

C++ 程序由于具有更为复杂的语言特性和库例程，因此具有更密集的动态存储器要求（堆和栈大小）。许多 C++ 库例程都能请求从堆分配存储器。请复查您的 C++ 程序堆栈大小要求，确保可满足这些要求。

---

### C++ 标准库

C++ 标准用于定义 C++ 标准库。其中部分平台特性在默认 AMD Vivado™ 软件平台上不可用。例如，此平台仅支持少数明确定义的 `STDIN/STDOUT` 串流。同样，语言环境函数、线程安全性和其他此类特性可能都不受支持。

**注释：**C++ 标准库对于多线程环境不适用。常用 C++ 特性（如 `new` 和 `delete`）均不具备线程安全性。在操作系统环境中使用 C++ 标准库时请谨慎处理。

如需了解有关 GNU C++ 标准库的更多信息，请参阅 GNU 网站上提供的文档。

---

### 位置无关代码（可重定位代码）

MicroBlaze™ 处理器编译器支持使用 `-fPIC` 开关来生成位置无关代码。虽然 AMD 编译器支持这两项功能特性，但是 Vivado 仅提供独立平台，不支持其他库和工具。调试器无法在运行时解读可重定位代码并执行正确的重定位。AMD 库、启动文件和其他工具不支持位置无关代码功能。第三方操作系统供应商可能会在其发行版和工具中使用这些功能作为标准功能。

---

## 其他开关和功能特性

AMD Vivado 编译器和平台并不支持所有开关和功能特性。例如，`-fprofile-arcs`。部分功能特性本质上是实验性功能（根据开源 GCC 的定义），如使用不当，可能生成错误代码。如需了解有关特定功能特性的更多信息，请参阅[GCC 文档](#)。

## 第七部分

# 嵌入式设计教程

以下特定于硬件的嵌入式设计教程可供嵌入式软件设计师使用。

- 《Zynq 7000 SoC：嵌入式设计教程》([UG1165](#))
- 《Zynq UltraScale+ MPSoC：嵌入式设计教程》([UG1209](#))
- 《Versal 自适应 SoC 嵌入式设计教程》([UG1305](#))

要了解有关软件功能特性的更多信息，请参阅《Vitis 统一软件平台教程登陆页面》([UG1605](#))。

## 第八部分

# 驱动程序和库

驱动程序和库均位于 AMD Wiki 上。您可以通过以下链接来访问这些驱动程序和库：

- [裸机驱动程序和库](#)
- [Linux 驱动程序](#)

## 多实例 IP 的 Tcl 生成调用

在平台和 BSP 生成期间，Vitis Unified IDE 会执行驱动程序 Tcl 代码，以从硬件设计生成软件工件。IDE 会调用驱动程序定义的 `generate` 过程，以便驱动程序输出配置数据和实例特定的输出。

当 IDE 处理硬件设计时，它会将所有绑定到同一驱动程序的 IP 实例分组，并为每个驱动程序组选择一个 IP 实例作为代表实例。Vitis Unified IDE 对每个驱动程序组只调用一次 `generate` 过程，并且仅将代表驱动程序句柄传递给该过程。IDE 不会为每个 IP 实例分别调用 `generate`，也不会提供所有实例句柄的列表。由于 IDE 在不同运行之间可能会选择不同的代表实例，驱动程序不得依赖实例名或实例排序。

### 对驱动程序作者的影响

设计可包含共享同一驱动程序的多个 IP 核，驱动程序作者必须确保该驱动程序能够处理所有此类实例。由于 Vitis Unified IDE 对每个驱动程序组只调用一次 `generate` 过程，因此驱动程序必须将此调用视为组级入口点，而不是实例特定的回调函数。

如果驱动程序只初始化 `generate` 过程传递的代表实例，则该驱动程序可能会跳过绑定到同一驱动程序的其他实例。只初始化代表实例的驱动程序会在平台或 BSP 工件中遗漏实例特定的配置数据或输出。

### 必需的驱动程序设计模式

正确的 `generate` 过程必须为绑定到该驱动程序的每个 IP 实例输出内容，而不只是为代表实例输出。HSI Tcl 库为此提供了两种互补机制：

- 高层次帮助程序。 `::hsi::utils::define_include_file`、`::hsi::utils::define_config_file` 和 `::hsi::utils::define_canonical_xpars` 帮助程序各自接收驱动程序句柄以及属性名列表，并自动迭代每个同组实例，输出相应的 `#define` 宏和配置表条目。 `embeddedsdsw` 树中的大多数量产驱动程序只使用这些帮助程序。
- 自定义逻辑 - 手动迭代。当高层次帮助程序未覆盖驱动程序需要输出的内容时，例如驱动程序必须根据多个属性为每个实例计算一个值，请使用 `::hsi::utils::get_common_driver_ips` 检索绑定到该驱动程序的完整 IP 实例集，然后使用 `foreach` 迭代该集合，并使用 `common::get_property` 读取实例属性。使用 `::hsi::utils::get_driver_param_name` 为每个实例构造唯一的 `XPAR_*` 宏名称。

无论驱动程序使用哪种机制，`generate` 过程都绝不能依赖代表句柄的实例名或实例排序。

## Tcl 模式示例

以下示例展示了如何将这两种机制应用于名为 `xGpio` 的驱动程序。高层次帮助程序在内部处理多实例发现和迭代；手动模式则展示当帮助程序未覆盖所需输出时应如何处理。

推荐的高层次模式在对每个帮助程序进行一次调用时，会为绑定到该驱动程序的每个实例输出 `xparameters.h`、规范的 `XPAR_*` 定义以及 `xgpio-g.c` 配置表：

```
proc generate {drv_handle} {
    ::hsi::utils::define_include_file      $drv_handle "xparameters.h"
    "XGpio" \
        "NUM_INSTANCES" "C_BASEADDR" "C_HIGHADDR" "DEVICE_ID" \
        "C_INTERRUPT_PRESENT" "C_IS_DUAL"

    ::hsi::utils::define_config_file      $drv_handle "xgpio-g.c"
    "XGpio" \
        "DEVICE_ID" "C_BASEADDR" "C_INTERRUPT_PRESENT" "C_IS_DUAL"

    ::hsi::utils::define_canonical_xpars $drv_handle "xparameters.h"
    "Gpio" \
        "C_BASEADDR" "C_HIGHADDR" "DEVICE_ID" "C_INTERRUPT_PRESENT"
    "C_IS_DUAL"
}
```

当驱动程序必须输出高层次帮助程序未生成的内容时，使用 `::hsi::utils::get_common_driver_ips` 枚举绑定到该驱动程序的每个 IP 实例，然后进行迭代。以下模式会打开一个 `include` 文件，遍历完整实例集，读取每个实例的属性，并为每个实例输出唯一的宏：

```
proc generate_custom_params {drv_handle file_name} {
    set file_handle [::hsi::utils::open_include_file $file_name]

    # Retrieve every IP instance that binds to this driver.
    set ips [::hsi::utils::get_common_driver_ips $drv_handle]

    foreach ip $ips {
        set ip_name [common::get_property NAME $ip]
        set value [common::get_property CONFIG.C_BASEADDR $ip]

        # Emit a unique XPAR_* macro per instance.
        puts $file_handle \
            "\#define [::hsi::utils::get_driver_param_name $ip"
        \ "CUSTOM_PARAM\" ] $value"

        puts "generate: processed $ip_name"
    }
    close $file_handle
}
```

这两种模式都将 `generate` 视为驱动程序级入口点：高层次帮助程序在内部对每个同组实例进行迭代，而手动模式则使用 `foreach` 进行显式迭代。在这两种情况下，驱动程序都不会依赖代表实例句柄作为唯一实例。

## 示例场景

有个设计例化 `sentinel_0`、`sentinel_1` 和 `sentinel_2`，它们都绑定到同一驱动程序。在生成期间，Vitis Unified IDE 对该驱动程序组只调用一次驱动程序的 `generate` 过程，并传递单个代表句柄。使用上述高层次帮助程序时，对 `::hsi::utils::define_config_file` 的一次调用会在配置表中输出三行，分别对应 `sentinel_0`、`sentinel_1` 和 `sentinel_2`；而 `::hsi::utils::define_include_file` 会在 `xparameters.h` 中为每个实例输出对应的 `#define` 块。驱动程序不得依赖多次 `generate` 调用。

## 验证

要验证驱动程序行为是否正确，请创建一个设计，其中包含多个 IP 实例，并且这些 IP 实例绑定到同一驱动程序，然后在 Vitis Unified IDE 中运行平台或 BSP 生成。在 `xparameters.h` 输出中，确认 `XPAR_<DRV>_NUM_INSTANCES` 与设计中的实例数匹配，并且每个实例都存在一个 `XPAR_<INSTANCE>_*` 块。在 `x<drv>_g.c` 输出中，确认配置表为每个实例包含一行。如果针对单一实例出现多个条目，则说明驱动程序绕过高层次帮助程序并且迭代不正确，请切换到上文所示的帮助程序模式，或使用 `::hsi::utils::get_common_driver_ips` 添加显式实例发现。

# 附加资源与法律声明

## 查找其他文档

### 技术信息门户网站

AMD 技术信息门户网站是旨在使用您的网页浏览器提供健全的文档搜索和导航的在线工具。要访问该技术信息门户网站，请转至 <https://docs.amd.com>。

**注释：**单击链接将打开英语版本，但您可从下拉列表中选择简体中文版本（如可用）。请注意，简体中文版本可能比英语版本旧。

### Documentation Navigator

Documentation Navigator (DocNav) 是预安装的工具，支持访问 AMD 自适应计算文档、视频和支持资源，您可在其中通过筛选和搜索来查找信息。要打开 DocNav，请执行以下操作：

- 在 AMD Vivado™ IDE 中，依次选择“Help” → “Documentation and Tutorials”（帮助 > 文档和教程）。
- 在 Windows 上，单击“Start”（开始）按钮，然后选择“AMD Adaptive SoC and FPGA Tools” → “DocNav”（AMD 自适应 SoC 与 FPGA 工具 > DocNav）。
- 在 Linux 命令提示中输入 `docnav`。

**注释：**如需了解有关 DocNav 的更多信息，请参阅《Documentation Navigator 用户指南》(UG968)。

**注释：**您无法从 DocNav 访问简体中文版本。请使用设计中心网页。

### 设计中心

AMD 设计中心提供了根据设计任务和其他主题整理的文档链接，您可使用这些链接来了解关键概念以及获取常见问题的解答。要访问设计中心，请执行以下操作：

- 在 DocNav 中，单击“Design Hubs View”选项卡。
- 转至[设计中心](#)网页。

## 支持资源

如需获取答复记录、技术文档、下载以及论坛等支持资源，请访问[技术支持](#)。

# 参考资料

以下技术文档是非常实用的补充资料，可配合本指南一起使用：

- 1. 《Vitis 统一软件平台文档登陆页面》(UG1416)
- 2. 《Vitis 软件平台版本说明》(UG1742)
- 3. 《embeddedsoc 组件移植指南：基于系统设备树的构建流程》(UG1647)
- 4. 《Zynq 7000 SoC：嵌入式设计教程》(UG1165)
- 5. 《Zynq UltraScale+ MPSoC：嵌入式设计教程》(UG1209)
- 6. 《Versal 自适应 SoC 嵌入式设计教程》(UG1305)
- 7. 《Zynq UltraScale+ 器件技术参考手册》(UG1085)
- 8. 《Versal 自适应 SoC 技术参考手册》(AM011)
- 9. 《Vivado Design Suite 教程：嵌入式处理器硬件设计》(UG940)
- 10. 《Bootgen 用户指南》(UG1283)
- 11. 《Zynq 7000 SoC 技术参考手册》(UG585)
- 12. 《Zynq UltraScale+ 器件技术参考手册》(UG1085)
- 13. 《MicroBlaze 处理器参考指南》(UG081)
- 14. 《Vitis 统一软件平台教程登陆页面》(UG1605)
- 15. 《PetaLinux 工具文档：参考指南》(UG1144)
- 16. 《Zynq UltraScale+ 器件技术参考手册》(UG1085)
- 17. 《使用 Vitis 进行嵌入式设计开发》(UG1701)
- 18. 《软件调试器参考指南》(UG1725)
- 19. 《Vivado Design Suite 用户指南：版本说明、安装和许可》(UG973)
- 20. 《Vitis 统一软件平台文档：应用加速开发》(UG1393)
- 21. 《Vitis 软件平台版本说明》(UG1742)
- 22. 《Vitis 参考指南》(UG1702)

# 修订历史

下表列出了本文档的修订历史。

| 章节                       | 修订综述                                                                          |
|--------------------------|-------------------------------------------------------------------------------|
| 2026 年 7 月 31 日 2026.1 版 |                                                                               |
| 基于 XSA 创建平台组件            | 更新 “User DTS” 选项说明，以阐明其同时支持 Linux 和独立平台创建，并添加一个 Python API 示例，用于包含自定义外设 DTSI。 |

| 章节                                                                          | 修订综述                                                                                                                                         |
|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 第五部分：从 XSCT 移植到 Python API                                                  | 更新 XSCT 到 Python API 移植主题，说明在 2026.1 中已不再提供 XSCT 以供使用，并指引用户使用 Python API、基于 Python 的 XSDB 以及用于 HSI 命令的 Python CLI。                           |
| PS Trace                                                                    | 重写相关内容，以阐明捕获模式覆盖范围，简化 OpenCSD 设置，并新增有关定义追踪窗口以及 PS Trace 在调试会话期间行为的章节。                                                                        |
| 在 Visual Studio Code 中使用 Vitis 调试器扩展 和 在 Visual Studio Code 中设置 Vitis 调试器扩展 | 添加适用于 Visual Studio Code 的 Vitis 调试器扩展（内部里程碑），包括受支持的工作流程、三种 TCF 连接类型（hw_server、Linux TCF Agent、QEMU），以及设置、配置和裸机过程。                           |
| 创建应用组件、为现有平台添加域 和 Vitis Unified IDE 启动选项                                    | 添加新的 <code>--wait</code> 启动选项、添加域时的多架构（32 位/64 位）处理器选择，以及创建应用组件时的 MicroBlaze V Linux 域选择。                                                    |
| 生成设备树                                                                       | 验证在“Generating Device Tree”中用于生成 Linux 设备树二进制对象的 SDT 和 Lopper 命令。                                                                            |
| 创建库工程、在应用工程中使用自定义库 和 library                                                | 扩展库组件工作流程，除现有适用于裸机 Arm、MicroBlaze 和 RISC-V 的静态库（.a）外，还包含适用于 Linux 域的动态库（.so），并通过 <code>create_library_component</code> 添加 CLI/Python API 指南。 |
| Zephyr IDE 扩展                                                               | 添加用于 MicroBlaze V 的 Zephyr IDE 扩展流程：安装该扩展、将 Zephyr 应用构建为 ELF，并从 Vitis Unified IDE 启动配置启动用户管理的调试会话。                                           |
| Vitis Unified IDE 中的 AI 辅助 和 AI 辅助的用例                                       | 介绍“AI Chat”（AI 聊天），包括设置步骤、用户责任免责声明以及三个基于任务的用例（诊断构建失败、更新工程配置、解释 IDE 生成的报告）。                                                                   |
| 多实例 IP 的 Tcl 生成调用                                                           | 新增关于多实例 IP 的主题，描述驱动程序生成过程契约以及使用 <code>::hsi::utils</code> 帮助程序所需的多实例迭代模式。                                                                    |

## 请阅读：重要法律声明

本文档所示信息仅做参考，其中可能包含不准确的技术信息、疏漏和印刷错误。受诸多原因影响，此处所含信息可能发生更改，也可能无法准确呈现，这些原因包括但不限于产品和路线图变更、组件和主板版本更改、新增模型和/或产品发布、不同制造商之间存在的差异、软件更改、BIOS 刷新、固件升级等。任何计算机系统均存在安全性漏洞风险，无法彻底阻止也无法缓解这类风险。AMD 没有任何义务来更新或者以任何其他方式纠正或修改这些信息。但 AMD 保留随时修改这些信息和更改文档内容的权利，AMD 没有任何义务将此类修改或更改通知任何人。此处信息“按原样”提供。AMD 对于本文档内容不作任何陈述或保证，并且对于这些可能出现的任何不准确、错误或疏漏问题不承担任何责任。对于有关任何暗示的非侵权、适销性及适合特定用途的保证，AMD 特此声明不承担任何责任。无论在任何情况下，对于任何人因使用此处包含的任何信息而形成的依赖或者引发的任何直接、间接、特殊或其他后果性损害，AMD 概不负责，即使 AMD 已明确获悉存在发生此类损害的可能性也是如此。

### 关于与汽车相关用途的免责声明

如将汽车产品（部件号中含“XA”字样）用于部署安全气囊或用于影响车辆控制的应用（“安全应用”），除非有符合 ISO 26262 汽车安全标准的安全概念或冗余特性（“安全设计”），否则不在质保范围内。客户应在使用或分销任何包含产品的系统之前为了安全的目的全面地测试此类系统。在未采用安全设计的条件下将产品用于安全应用的所有风险，由客户自行承担，并且仅在适用的法律法规对产品责任另有规定的情况下，适用该等法律法规的规定。

## 版权声明

© Copyright 2019 - 2026 AMD 公司，版权所有。AMD、AMD 箭头徽标 Alveo、UltraScale、UltraScale+、Versal、Vitis、Vivado、Zynq 及其组合均为 Advanced Micro Devices, Inc. 的商标。“AMBA”、“AMBA Designer”、“Arm”、“ARM1176JZ-S”、“CoreSight”、“Cortex”、“PrimeCell”、“Mali”和“MPCore”为 Arm Limited 在美国和/或其他国家或地区的商标。“PCI”、“PCIe”和“PCI Express”均为 PCI-SIG 拥有的商标，且经授权使用。此出版物中所使用的其他产品名称仅用于标识目的，可能是其各自所属公司的商标。